Inside X68000 (1992)

English translation from the Japanese original

Contents

”Inside X68000 Masahiko Katano”

SOFTBANK (in the logo at the bottom right corner).

(Blank page — inside cover; the scan is solid black with no text.)

(Blank page in the original.)

(Blank page in the original.)

”Inside X68000”

”Masanori Kato”

Introduction

The X68000 first appeared in 1987, riding on the graphics theme of Gradius. At that time in Japan, the dominance of the PC-9801 was overwhelmingly strong, and other manufacturers also switched to 80X86 + MS-DOS, resulting in the term ”personal computer” shifting from personal users to signify PCs used in companies. When there was talk of Sharp releasing a 16-bit version of the X1, many said, ”Just stick with the 86-series machine” or ”Why not just stay with the 98 series?” Such voices became prevalent, and personal users were gradually settling into personal computers.

The X68000, appearing before such users, greatly changed the perception of personal computers. Within its sleek vertical slim design, which was unprecedented, was a CPU desired by personal users. The X68000, equipped with 1 MB of standard memory, up to a maximum of 12 MB, plenty of memory space, a 65536 color graphics mode, 768 x 512 dot bitmap mode for text, sprites, FM sound source, ADPCM, hardware-like techniques such as 5-inch FDD, 3D scope, image data importing, a hard-disk interface as standard, etc., had incredible specifications for the price of approximately 400,000 yen, making it an exceptional value.

On the other hand, what may have once been observed as a business tool, the PC, became something that was purely pursued for operational use, leading to the expansion of its field. When considering presenting thoughts and ideas as images, and the ability to respond as flexibly as possible, it was natural to come up with the new term ”personal workstation” as a separate category from ”personal computers.”

This ”personal workstation,” responding to individual expression and creativity, enabling realization and further development, serves as a platform for continued innovation over the next five years. For those contemplating using the X68000 up until its limits, this book serves as a guidance resource.

February 1992

Yoshinori Yamada

The Lineage of X68000

After the debut of the first generation, we organized the X68000 series in chronological order. When talking about the X68000, it's not just the CPU or clock speed that evolved, nor just the high degree of compatibility in terms of software and hardware; the best thing about it might be its expandable, high-performance adaptability for hard disk drives (HDD), SCSI, and coprocessors. Traditionally, one could add units and optional boards to fit into the available empty space. This collectibility, starting from the initial model (plain), has become part of its charm.

In 1987, the first genuine X68000 was released, followed by the X68000 ACE/ACE-HD later in the year with an internal hard disk capacity of up to 20 MB and a 3.5-inch HDD. Further improvements in internal HDD capacity, up to 40 MB, began in 1989, including the addition of the EXPERT, followed by the PRO series, which aimed at standardizing high-end models. The PRO series retained the traditional X68000 design but underwent significant changes to suit business purposes. Naturally, the software was thoroughly revised. For example, the expansion slot modified from 2 slots to 4, the mouse port was slightly smaller in size, and the keyboard was changed from a traditional microswitch type to a mechanical key type. However, due to limited usage, the DOS terminals were omitted. Assembling became easier, and it maintained a low price despite these enhancements.

In 1990, the bread and butter model, the X68000, welcomed new additions. Series like the EXPERT and PRO underwent further refinements such as SCSI changes to the HDD interface, with the SUPER-HD version sporting 80 MB HDDs, helping enhance the family with five new models in one year. As the year progressed, the SUPER-HD with incremented HDD types and a more diversified SUPER-SE emerged, making the EXPERT the foundational lineage continuing.

1991 saw a leap where the initially unchanged 10 MHz clock frequency was boosted to 16 MHz, improving internal memory bandwidth and supporting further add-ons with coprocessor boards (XVI). The transition within SUPER models to a SCSI HDD interface, coupled with a clock frequency boost, integrated the innovative elements of the XVI models, and GUI speeds were noticeably improved. These updates and higher connectivity have set a high standard for compatibility among upgraded and basic models.

Regarding installed memory, early models started with 1MB, but the 2MB assumption was that there would be no software issues in running and everything would work as intended. (Along with improvement in memory, we hope this cleared up any worries).

Figure 1: X68000 Series Lineage

Top row: Model name Bottom row: Model number

1987

- (No label) CZ-600CE

1988

- ACE CZ-601C

1989

- EXPERT CZ-602C

1990

- EXPERT II CZ-603C
- HDD Interface SCSI 1c
    - SUPER CZ-604C
- 20MB HDD Built-in
    - ACE-HD CZ-611C
- 40MB HDD Built-in

- EXPERT-HD CZ-612C
- EXPERT I-HD CZ-613C

- 80MB HDD Built-in SCSI 1c
    - SUPER-HD CZ-623C
- 40MB HDD Built-in

- PRO CZ-652C
- PRO-HD CZ-622C

- 40MB HDD Built-in

- PRO II CZ-653C
- PRO II-HD CZ-663C

1991

- CPU Clock 16MHz

- XVI CZ-634C
- XVI-HD CZ-644C

1992

- Compact XVI CZ-674C

Additional info at the top of the chart: **Horizontal

- Number of expansion slots: 4
- 3D scope terminal removed**

In this book, I/O allocation charts, block diagrams, etc., are explained for all initial machines (non-residents), and the operation check of the sample program was also performed with an initial machine expanded with a copro board, SCSI interface board, etc. Therefore, I will present the configuration of the system actually used here.

● System #1 X 68000 (initial machine) + internal expansion (1 MB) + expansion memory board (2 MB) + copro board (CZ-6BPI) + 40 MB SASI hard disk (Caravelle/H540S) + color image unit (CZ-6VTI)

● System #2 X 68000 (initial machine) + internal expansion (1 MB) + SCSI interface board (CZ-6BS1) + 100 MB SCSI hard disk (I-O DATA/TX-100) We use a CZ-600 DE for both drives.

● About Sample Programs

The assembler logic is easy to see and has a smooth structure, so we decided to write the sample program mainly using C language. The C compiler for X68000 is published in the

Sharp Journal's ”XC” and the freeware GCC. To keep it simple, we originally intended to proceed with XC, but GCC is more systemically generated and has fewer bugs, so we used it as it is, in computer communications and BBS, ”C magazine” or ”Oh! X” magazine's attached disk, widely available. To make it suitable, we adopted GCC because non-standard ones like ccc can be used.

In this book, the program is created assuming gcc as the standard. We have confirmed it operates with XC as well just in case, but please don't forget to add the following line if you use XC, `#define volatile 0`.

Also, all sample programs use interrupt, making it easier to check the status stack when the interrupt ends. The X68000 interrupt is effective. The program is created with the goal of operation verification, but interrupts are convenient to separate, so we chose this method. To create something practically usable, you might want to avoid excessive interrupts as much as possible.

Sample Program Table of Contents

CONTENTS

Memory Map

DMA

CONTENTS

CONTENTS

RTC

CONTENTS

CONTENTS

• SCSI

◉ 1 Overview of SCSI

◉ 2 Overview of X 68000 SCSI Interface

◉ 3 SPC (SCSI Protocol Controller)

◉ 4 SPC Transfer Modes 485

◉ 5 SPC Commands 486

Page 17

COVER DESIGN: Masaki KATSUMATA

Memory Map

The X68000 CPU, which is the 68000, does not have an I/O space like the 80X86, but it only has a 16M memory space. Here, we will explain how the memory space is allocated in the X68000.

1. Memory Map

The memory map of the X68000 is shown in Figure 1 on page 20. Of the 16MB memory space that the CPU possesses, the area from address 0 to BFFFFF, which is 12MB, is the main memory area. Beyond address C00000, it encompasses the areas of VRAM or I/O for graphics and text screens and the IPL-ROM.

2. IPL Image

When the CPU called 68000 is reset, it reads the initial values of the SSP (Supervisor Stack Pointer) and PC (Program Counter) from addresses 000000 and 000004, respectively, and starts operations. In the case of the X68000, the side with address 0 is the main memory area, so you can perform initialization there.

X68000 Memory Map

$0000000

IPL Image

- Only at Reset start $FF0000 Same contents

$0100000

Main Unit Internal Memory (1MB) Standard Equipment

$100000

Main Unit Internal Memory (1MB) Standard Option (depends on model)

$200000

Expansion Memory (max 10MB)

$C00000

Graphics VRAM

$D00000

Text VRAM

$E00000

System I/O User I/O SRAM Unused (128KB)

$F00000

CGROM (768KB)

$FC0000

(Reserve) IPL-ROM (128KB)

$FFFFFF

$E80000

bit 15 bit 0 CRT Controller

```
2000 Video Controller
4000 DMA Controller
6000 MFP
8000 RTC
```

A000 Printer C000 Sound Port $E90000

```
2000 ADPCM
4000 FDC
6000 HDC
8000 SCC
```

A000 8255 C000 Joystick

$E80000 Sprite Registers

$EB8000

Sprite VRAM

$EC0000

User I/O (64KB) (Used with expanded board)

$ED0000

SRAM (16KB)

$ED4000

Reserve(48KB)

$EDFFFF

Page 20

Note: Memory addresses and specific hardware terms have been left unchanged as they are crucial technical details.

Memory Map

Otherwise, the CPU will read unstable data from DRAM right after the reset, causing a crash. Therefore, the X 68000 switches the 64 KB region from $000000 to $00FFFF to the DRAM region after power is applied and then reset, making the IPL-ROM region $FF0000-$FFFFFF visible (readable from somewhere). When the ROM of the same region is read, the $FF0000-$FFFFFF region is changed back to DRAM to be accessed. This mechanism is implemented to work, for example, when the reset is pressured upon the power-on switch, and the RESET instruction is executed, preventing it from reading IPL-ROM contents from the 0 address.

•3 Main Memory

The X 68000 can accommodate a maximum of 12 MB of main memory. From the 0th to the 1MB regions, all initial models are equipped with standard memory. From $100000 to $1FFFFF, it is up to 1 MB, either as standard equipment or optional, but it is built into the main unit to be checked for specific configurations.

The option mechanism for the over $200000 segment, except for the XVI model, installs the additional memory board into an expansion slot. The XVI model is internally equipped to check up to 8 MB.

•4 Graphics VRAM

The graphics VRAM extends up to 2 MB from C$0000-$DFFFFF, but the actual memory is 512 KB. The X 68000 graphics screen displays three types of 16-color mode, 256-color mode, and 65536-color mode. In either case, 1 word (4 bits) occupies the region. In the case of 16-color and 256-color modes, only 4 bits/8 bits out of 1 word are used at any one time. Thus, the actual usable memory is up to 512 KB.

Even without forests, memory space has the maximum screen size, which is equivalent to 1024x1024 dots.

•5 Text VRAM 512 K bytes of text VRAM are implemented. The text screen has a configuration of 4 planes of 1024x1024 dots, so it takes up 512 K bytes of memory space even though it does not have a meaningless bit like the graphics screen.

•6 System I/O Region The system I/O region includes memory for CRT and FD, FM sound source, peripheral control devices, and sprite use.

•7 User I/O, SRAM As an area that can be used by user original interface boards, 64 K bytes of $EC0000~$ECFFFF are allocated. This area can be freely used by the user and can be accessed in user mode. SRAM retains its contents even when powered off, as it is battery-backed memory. The memory of the size to be mounted and the initial value of the screen color, etc., are used for purposes such as saving data for system setup.

8 CGROM

CGROM (Character Generator ROM) is a memory where patterns of alphanumeric and Kanji characters are written. Since the text screen of X68000 is bitmap-based, it can display any text pattern, which allows various graphical displays. The CGROM contains six types of character patterns including 8x8, 8x16, 12x12, 12x24 size dot matrix alphanumeric characters, and 16x16, 24x24 size dot matrix Kanji characters.

9 IPL-ROM

IPL-ROM is a ROM that writes a program to be executed immediately after the CPU reset. On the X68000, it contains basic I/O subroutines (IOCS). Initially, the Human 68K version used the IOCS in this ROM; however, it now loads the polished IOCS into RAM to use them. Therefore, IPL-ROM is only used for the basic initialization of peripheral devices and for the startup process from FD or HD.

(Blank page in the original.)

DMA

DMA, which performs data transfer without involving the CPU, supports high-speed data transfers that are difficult to respond to using interrupts or independent data transfer processing from CPU operations. Here, we will explain how to handle DMA.

1 Overview

A DMAC (Direct Memory Access Controller) is an IC that performs memory and I/O data transfers without involving the CPU. This is where the name ”direct” comes from. Usually, the CPU responds to interrupt request signals from devices and disconnects the line (bus) it is currently executing to respond to the request. However, DMAC can disconnect the bus electrically and execute the appropriate operations to release the bus from the requesting device. Furthermore, DMAC stops the CPU operations that were interrupted

by the request. Utilizing this functionality, DMAC carries out data transfers independent of program execution by the CPU (of course, the CPU must first be programmed for how to perform data transfers, transmission, reception, etc.).

Figure 26-1 shows an example of data transfer from I/O or memory by DMAC. When an I/O data transfer request occurs, DMAC issues a bus release request to the CPU, performs the data transfer operation, and then cancels the bus release request to return control to the CPU. Although this operation involves hardware, it does not affect software program execution except for the temporary delay in CPU operation.

Figure 1 Overview of DMAC Operation

Bus Request
(REQ)

1. Transfer Request

- Input Device Request
  ↓
- (ACK) Input Acknowledge

CPU

DMAC
↓ Bus Acquisition ↓ Memory
I/O

- Bus Request
  ↓
- (ACK) Bus Acknowledge

CPU

DMAC (ACK)
↓

Memory

Data Transfer
Bus Release

2. Transfer Execution

- DMA Transfer Execution
  (Dual port access I/O ↔ Memory Transfer)

I/O

The operation of the DMAC involves transferring data without the application being aware of it, which is similar to data transfer by slicing, but in software-based transfers, the time required for the CPU to determine where to insert and delete data from memory or to handle system housekeeping prevents such immediate actions. However, with DMA, data transfer is requested and initiated only when a request is made, and as soon as the current cycle ends, the transfer can start immediately. This makes the initiation period three times shorter with DMA, allowing for high-speed data transfer. In the X68000, FD, HD, and ADPCM use DMA to achieve high-speed data transfers.

2 DMAC Channel Assignment

Figure 2 illustrates the channel assignment of the DMAC for X68000. The DMAC (HD 63450) used in the X68000 has four channels, and channels #0, #1, and #3 are assigned to FD, HD, and ADPCM, respectively. The remaining channel #2 is not used, and signals such as REQ (DMA transfer request signal), ACK (acknowledge signal), and PCL (channel input signal) are connected to the expansion slot. This channel can also be used for memory-to-memory transfers and other purposes.

Figure 2 X68000 DMAC Channel Assignment Diagram

[Diagram]

- DMAC (HD63450)

```
- Channel #0
  - REQ0, ACK0, PCL0
  - Floppy Disk Controller
  - External Sync Signal (if not performing supervision, keep Low)
- Channel #1
  - REQ1, ACK1, PCL1
  - Hard Disk Controller
  - External Sync Signal (keep High 5V if not performing supervision)
- Channel #2
  - REQ2, ACK2, PCL2
  - Expansion Slot
- Channel #3
   - REQ3, ACK3, PCL3
   - ADPCM Control Circuit
- DONE, DTC
```

- All channels can be used in dual address mode.
- Channels #0, #1, and #3 are set to cycle steal mode for external sync requests.
- Channel #2 is unused (can also be used for memory-to-memory transfers).

3. DMAC Register List

Figure 3 shows the register list of the DMAC, the HD 63450, adopted in the X 68000. Each channel has 17 registers (GCR is the control register for the entire DMAC and is only for Channel 3 in address space 0x840FF). Among these registers, CER is for read-only (cycling read), and the other registers can perform both read and write operations.

The following registers are used for specifying transfer addresses, or addresses of the transfer source and destination: MAR and DAR, respectively. MTC specifies the number of words to be transferred. MAR is used for specifying memory I/O addresses, while DAR is for specifying I/O addresses.

Additionally, BAR and BTC are used when utilizing the multiple block transfer function. Other registers will be explained later.

4. DMAC Operating Modes

The HD 63450 has many operating modes. There are two ways to transfer data: once per transfer request from the source to the destination, or in bursts where the transfer occurs all at once after storing up requests. In burst mode, data is returned to the CPU in blocks, compared to a cycle steal mode which transfers data while allowing the CPU to access the bus between each transfer.

The X 68000 has three operating modes: 0, 1, and 3. Each channel's intended use is determined, and some settings are fixed; however, different modes can be set for each channel. In this section, all modes of operation will be described to cover all possibilities.

Note that DMAC has four types of transfers: memory I/O, I/O memory, memory memory, and I/O I/O. To avoid confusion, this section will explain the operation focusing on I/O memory transfers.

Diagram 3 DMAC Register List

Channel #0

Channel #1

Channel #2

Channel #3

CSR: Channel Status Register CER: Channel Error Register DCR: Device Control Register OCR: Operation Control Register SCR: Sequence Control Register CCR: Channel Control Register MTC: Memory Transfer Counter MAR: Memory Address Register DAR: Device Address Register BTC: Base Transfer Counter BAR: Base Address Register NIV: Normal Interrupt Vector EIV: Error Interrupt Vector MFC: Memory Function Code CPR: Channel Priority Register DFC: Device Function Code BFC: Base Function Code GCR: General Control Register

GCR is only for Channel #3

4 • 1

1 Operand Transfer Mode

Examining the operation of the DMAC and the data flow on the bus, the data are once fetched into the DMAC. In dual address mode, where the DMAC performs both read and write operations, the DMAC generates memory addresses and can transfer data directly from the I/O to memory. In single address mode, they can be divided. Each action is illustrated in Figures 4 and 5.

Figure 4: Dual Address Mode (I/O-Memory transfer) (Memory-Memory)

DAR Device Address MAR Memory Address MTC Transfer Count

Data read out

Data write

DAR Device Address MAR Memory Address MTC Transfer Count

Address Update

Register Update

DAR Device Address Update MAR Address Update MTC MTC - 1

Figure 5: Single Address Mode (I/O-Memory)

Dual address mode allows the DMAC to read data from an I/O device and write it to memory by providing an address to the I/O device like the CPU does. In dual address mode, the memory address for the I/O device and memory address to transfer data are specified by DAR (Device Address Register), MAR (Memory Address Register), and MTC (Memory Transfer Counter). We call this operation dual address mode, where the prior and subsequent addresses are used.

Upon completing a transfer cycle, DAR and MAR addresses are set so updates (increment/decrement) are performed. MTC counts down to 0 to end data transfers. To surrounding hardware, dual address mode looks like data movement occurs in sequences from I/O to memory, enabling I/O-I/O transfer.

In single address mode, the DMAC outputs an ACK signal to the I/O for data transmission without issuing the memory address. Data transfers to I/O are stored in memory directly. Data write operation complete with one action, reducing overall cycles and enabling transfers like dual address mode.

*Note: DAR is not used.

... more high-speed data transfer.

However, in single address mode, when the I/O width to memory is 8 bits while the memory width is 16 bits, using DMA transfer alone would require splitting the 16-bit data bus into two 8-bit transfers, which will need a fine-tuning, hence handled by hardware. (In dual address mode, DMA handles this process.)

Also, for the same reason, memory-to-memory transfer is not performed in single address mode.

On the X68000, each channel uses dual address mode for memory-to-memory transfer.

4.2 1-Block Transfer Mode

The transfer mode for the HD 63450 is shown in Figure 6.

In a 1-block transfer mode, by focusing on the occurrence of transfer requests, there are three modes of operation handled automatically by the DMAC after generating transfer requests from an external source: auto-request mode, single edge mode (cycle steal mode), and burst mode. Additionally, after starting the transfer in auto-request mode, the first block is transferred in auto-request, but subsequent blocks follow external transfer requests.

Furthermore, these transfer modes are classified in greater detail based on the usage differences.

■ Figure 6: 1-Block Transfer Modes

| Transfer Mode | Operation Overview | | External Request Transfer | Cycle Steal Mode Without Hold | Starts at the edge of the request; releases the bus after the transfer, and waits for the next request | | External Request Transfer | Cycle Steal Mode With Hold | Starts at the edge of the request; releases the bus after a certain period during the transfer, and waits for the next request | | External Request Transfer | Burst Mode | Starts at the level of the request; continues transfer until REQ becomes low | | Auto Request | Maximum Speed | Transfers to the end while holding the bus | | Auto Request | Defined Speed | Releases bus at intervals defined by GCR | | Auto Request + External | After transfer start, the first block is in auto-request; subsequent blocks are transferred based on external requests |

4-2 External Request Transfer Mode

The external request transfer mode can be further classified into three types: non-hold cycle steal mode, hold cycle steal mode, and burst mode. The operational timing diagrams for each are shown in Figure 7.

Non-Hold Cycle Steal Mode Non-hold cycle steal mode is the most general transfer mode. On systems like the X68000, FD, HD, ADPCM systems, any of these utilize this mode. When

the REQ (transfer request) signal is triggered to go from high to low, if a transfer request is not generated at the end of the transfer, it is returned to the CPU.

(A) Non-Hold Cycle Steal Mode Operation by External Transfer Request

[Diagram showing Non-Hold Cycle Steal Mode]

- REQ
- Bus Activity: CPU Activity
- DMA Transfer

(B) Hold Cycle Steal Mode

Hold Cycle Steal Mode

- REQ
- Bus Refresh Activity: CPU Activity
- DMA Transfer
- SI: Sampling Interval (2 BT + BR#'s clock)
    - $S I \leq A \leq 2 \times SI$

(C) Burst Mode

Burst Mode

- REQ
- Bus Activity
- DMA Transfer

Page 33

If the transfer ends when the hold is enabled, the CPU will not return to the bus immediately but rather will watch the situation for a while. If a request comes in during that time, you won't need to exchange the bus between the CPU and DMA, making it efficient. However, it will consume time just to return when there is no request. In burst mode, the REQ signal is level-sensitive, allowing continuous transfer as long as the REQ signal is low. It is suitable for applications where large transfer sizes are to be fetched at once.

4.2 Auto Request Mode

In auto request mode, transfer starts when the transfer start bit in the internal register of the DMAC is set to '1' by the CPU. Since the transfer request is issued by itself, this mode is also called the automatic (self) request mode. There are two types of operating speeds in auto request mode: maximum speed and limited speed. The timing diagrams of their operations are shown in the figure. In maximum speed, transfer is executed as soon as the previous transfer ends, and the CPU returns to the bus, thus increasing the transfer speed; however, if large data is transferred at the maximum speed, the issue arises where the CPU will stop functioning for a long time.

To counter this, there is a limited speed mode. For limited speed, DMAC periodically returns the bus.

● Fig. 8 Operation in DMA by Auto Request Mode

(A) Maximum Speed

CPU Operation -- DMA Transfer -- DMA Transfer -- DMA Transfer -- CPU Operation
| |----------------|----------------|-----------------| <--- Bus released ---> <------------ Cycle 1 ------------><-
----- Cycle 2 ------->

(B) Limited Speed

CPU Operation -- DMA Transfer -- DMA Transfer -- CPU Operation
| |----------------|-----------------| <--- Bus released ---> <------- Interval A -------><------ Interval B ---->

A: Auto request interval (2 BT4 clocks)
B: Sampling interval (2 BT5 clocks)

p. 34

4-3 Multiple Block Transfer Mode

Normally, when a DMA transfer is performed, the controller executes the transfer as configured (one block) and completes the action, then the DMA waits while the CPU sets the next configuration. For multiple block transfers, the DMA checks the transfer status as recognized by the interrupt status of the CPU-DMAC, and it takes a while to set the new transfer address values. There might be idle time while performing DMA transfers, and it can occur when multiple block transfers are needed. To cope with this, the HD63450 in the X68000 supports mechanisms for transferring multiple blocks without interruption. However, to use these mechanisms, it is necessary to change the configuration values MAR and MTC sequentially; unless the DAR setup value is cleared, all transfer operations continue. In such cases (increment/decrement), transfers will be performed continuously.

The HD63450 supports three methods for transferring multiple blocks: Continuation Mode, Array Chain, and Link Array Chain. The next sections will describe these methods.

4-3-1 Continuation Mode

In Continuation Mode, while DMAC is performing the transfer, the next memory address, transfer counter values, function code, and other relevant details are set in the BAR, BTC, and BFC registers. Upon completion of the one block transfer, DMAC reads the values set in MAR, MTC, and MFC, and immediately begins the next transfer. At this point, the CPU sets the address and count values for the next transfer.

4-3-2 Array Chain Mode and Link Array Chain Mode

In Array Chain Mode and Link Array Chain Mode, the memory addresses and transfer count data are set in a table (Transfer Description Table) in memory. DMAC reads the address values sequentially from this table.

Page 35

If set to BAR, DMAC will perform multiple block transfers by reading repeatedly in a mode where this transfer operation is carried out. In sequential operation mode, the CPU needs to be reset every time one block is transferred, but in both array chain mode and link array chain mode, the mode is considered to be more resilient in operation.

The difference in the size of the array chain and link array chain is in the transfer block information table configuration and operational conditions. The structure of each transfer information table is shown in Figure 9 below.

Figure 9: Transfer Information Table for Array Chain and Link Array Chain Mode

DMAC

- DAR (Device Address)
- MAR (Set from the transfer table)
- MTC (Set from the transfer table)
- BAR (Table start address)
- BTC (Transfer block count)
    - BTC is not used

Array Chain

DMAC

DAR (Device Address) MAR (Set from the transfer table) MTC (Set from the transfer table) BAR (Table start address) BTC

- Memory Address #1
- Transfer Count #1
- Link Address #1
- ...
- Memory Address #2
- Transfer Count #2
- Link Address #2
- ...
- Memory Address #3
- Transfer Count #3
- '0' (End Marker)

(16-bit memory)

1 block transfer data ⬆⬇ 1 block transfer data

Link Array Chain

(16-bit memory)

Note: The structure of transferred information might need additional context to understand fully given its technical nature.

In array chaining mode, blocks of transfer information are arranged at addresses connected by each piece of transfer information, and the number of blocks transferred to DMAC's BTC is specified (i.e., the number of the transfer information table). Thus, when one block worth of transfer is completed, the BTC count is reduced, and it comes to a halt when it becomes 0. In link array chaining mode, after one block of transfer information table, the address of the next transfer information table (link address) is written. Using this link address, DMAC gets the next transfer information and proceeds with the transfer. Therefore, when the link address becomes 0, the transfer is completed. In link array chaining mode, BTC is not used. Link array chaining mode is more flexible than array chaining mode, as you need not arrange the transfer information table continuously at addresses.

Please refer to Figure 10 for a summary of the differences between array chaining mode and link array chaining mode.

Figure 10: Comparison between Array Chaining Mode and Link Array Chaining Mode

Transfer Mode	Array Chaining Mode	Link Array Chaining Mode
BAR Content	Transfer information start address	Same as left
Content of BTC	Number of transfer blocks	[Not used]
Content of Transfer Information Table	Transfer address, number of transfers	Transfer address, number of transfers, li
Transfer Completion Condition	BTC = 0	Link Address = 0

5. DMAC Registers Contents

DMAC, HD63450, has many registers, but each of them is not that difficult to set. Here, let's explain the contents of the registers of DMAC and understand the specific setting methods. When explaining the registers and their bit arrangement, as with X68000 and

db.x, they are conveniently displayed, and even with separate registers, they can be read out in 1-word units. Hence, the unit is word.

Page 37

These registers are accessed during a read operation, usually in word units. However, it may not be in word units for write operations. For example, if you try to set the ”1” in the status register's STR with the CCR register, it will result in a timing error. It is recommended to access these registers carefully while paying close attention to their individual usage.

5-1 CSR, CER

The bit configurations for CSR (Channel Status Register) and CER (Channel Error Register) are shown in Table 11.

CSR is used to indicate the current channel status of the PCL line status, while CER is used to indicate specific error details in the event of an error occurrence.

Previously mentioned bits of CSR, other than ACT and PCS, get ”1”. However, the bits for error data write in remain ”1” until reset. Especially, if COC, BTC, NDT, ERR, ACT bits are set to ”1,” subsequent transmission operations cannot be performed. (The controller becomes a timing error). Hence, make sure to check and reset regularly before usage.

5-1-1 COC (Channel Operation Complete)

The COC bit becomes ”1” when a channel operation is completed. It must be cleared (set to ”0”) before restarting that channel's operation. Failure to do so may result in a timing error.

5-1-2 BTC (Block Transfer Complete)

The BTC register, also known as block transfer counting register, may cause confusion due to its name. The actual BTC register referred to here is indicated by the BTC bit.

The BTC bit is set when a block transfer operation is complete (when setting CCR or CNT bit to ”1”, MTC is cleared to ”0”). This indicates the completion of data transfer amounting to one block in continuous operation mode.

The BTC bit must also be cleared before restarting the continuous transfer operation before setting CNT bit back to ”1”.

DMA

● Fig. 11 CSR: Channel Status Register CER: Channel Error Register (+$00)

CSR (Channel Status Register) ---- CER (Channel Error Register)

| COC | BTC | NDT | ERR | ACT | DIT | PCT | PCS | bit 15 8 7 6 5 4 3 2 1 0

```
        '0'         ERROR CODE
```

CER (Channel Error Register)

00000: No error 00001: Configuration error 00010: Operation timing error 00011: (Unused) 001rr: Address error 010rr: Bus error 011rr: Count error 10000: External forced abort 10001: Software forced abort

```
rr = 01: Memory address / memory counter
   = 10: Device address
   = 11: Base address / base counter
```

PCL Line Status

```
0: PCL = "Low"
1: PCL = "High"
```

PCL Transition

```
0: No falling edge of PCL (High→Low change) has occurred
1: A falling edge ( // ) has occurred
```

DONE Input Latching

```
0: No DONE input
1: A DONE input occurred while the OCR BTD bit was '1'
```

Channel Active

```
0: Channel inactive
1: Channel active (operating)
```

Error Bit

```
0: No error
1: Error occurred (error details are placed in the ERROR CODE bits)
```

Normal Device Termination

```
0: Not a device stop by DONE signal
1: Normal device stop by DONE signal
```

Block Transfer Complete

```
0: Block transfer not complete
1: Block transfer complete
```

Channel Operation Complete

```
0: Channel operation not complete
1: Channel operation complete
```

...it will result in a timing error.

5-03 NDT (Normal Device Termination)

In HD 63450, the DONE signal pin is provided to indicate that the I/O device has completed all data transfers for the DMA request. The NDT (Normal Device Termination) bit indicates that the DMA transfer is completed with this signal. The NDT bit must also be cleared before starting another DMA transfer, otherwise, a timing error will occur.

5-04 ERR (Error)

The bit will be set to '1' if there is some kind of error. At this time, the content of the error is set in the CER register. The ERR bit must also be cleared before starting another transfer, otherwise, a timing error will occur.

5-05 ACT (Channel Active)

The ACT bit indicates that the channel is active. When the STR (Start) bit in the CCR is set and the transfer operation starts, the bit becomes '1', and when the transfer operation ends, it becomes '0'. It is similar to the COC bit that indicates the end of channel operation and is cleared to '0' by software, but the difference is that the ACT bit only becomes '1' when the channel is active.

5-06 DIT (DONE Input Transition)

When a multi-block transfer mode with DONE is selected (set the OCR or BTD bit to '1'), if a transfer stop by DONE input occurs, it will be '1'.

DMA

5.1.7 PCT (PCL Transition)

The HD63450 provides a PCL pin as a general-purpose input/output line for each channel (the function of this pin is determined by the DCR PCL bit and the DCR DTYP bit). The PCT bit is set to '1' when a change from High to Low occurs, regardless of how this signal pin is programmed. In the X68000, the external video vertical sync signal is connected to the PCL of channel #0, and the ADPCM DMA request signal is connected to the PCL of channel #3.

5.1.8 PCS (PCL Line Status)

The PCS bit reads the state of the PCL pin as it is. It does not matter how the PCL pin is programmed. If the PCL pin is High it is read as '1', and if it is Low it is read as '0'.

5.1.9 ERROR CODE

When the ERR bit of the CSR is set, the CER contains data indicating the content of the error. The respective error statuses and their causes are shown below.

1) Configuration Error

```
- When the CNT (continuous operation indication) bit is set during chain mode.
- When, in Single Address Mode, the device port size (specified by the DCR DPS bit) and
↪  the operand size (specified by the OCR SIZE bit) do not match.
- When, in Dual Address Mode, the external transfer request (OCR REQG bit) = '10' or
↪  '11', the device port size is set to 16 bits, and the operand size is set to 8 bits.
- When undefined values are set in any of the DCR, OCR, or SCR bits.
- When, in Dual Address Mode, the OCR SIZE bit is set to '11' while the device port size
↪  is set to anything other than 8 bits.
```

2) Operation Timing Error

- In chain mode, when neither STR bit (CCR register) nor ACT bit (CSR register) is set and the CNT bit is set.
- When one of the bits in CSR (COC, BTC, NDT, ERR, ACT) is '1' and the STR bit is set.
- When both STR bit and ACT bit are '1' (indicating that the channel is in operation) and data is written into DCR, OCR, SCR, CCR, MAR, DAR, MTC, MFC, DFC.
- When the BTC bit is '1' and the CNT bit is set.

3) Address Error

- When attempting to send to an unintended address in Word Down or Word Parent (an error occurs if accessing an area that is not actually being accessed).
- When resetting DMA or CS during a DMA bus cycle or setting IACK bit to Low (this will not occur except in case of hardware malfunction on X68000).

4) Bus Error

- When a bus error occurs while DMA bus is in use.

5) Counter Error

- When setting MTC register to 0 other than in chain mode and setting STR bit (when attempting to transfer 0 bytes).
- When setting BTC to 0 in array chain mode and setting STR bit.
- When MTC is loaded from memory during chain mode or continue mode (synchronous operation in chain mode) or BTC (continuous operation).

6) Forced Termination

- When a PCL abort input signal is programmed, and both STR and ACT bits are '1'.

7) Software Abort

- When the STR bit and ACT bit are '1' and the CCR register SAB (Software Abort) bit is set.

5.2 DCR, OCR

DCR (Device Control Register) and OCR (Operation Control Register) bit configurations are shown in Figure 12 on page 44. DCR is used to set the types of I/O devices connected to the DMAC or the function of PCI pins, while OCR is used to set the transfer mode of the DMAC.

5.2.1 XRM (External Request Mode)

XRM is used to set the transfer when an external request is used. This setting is valid when the REQG bit of the OCR register is either '10' or '11'. The Human 68K uses '10' (hold cycle steal mode) for channels #0, 1, and 3.

5.2.2 DTYP (Device Type)

This sets the type of I/O access method connected to the DMAC. Setting '00' or '01' is for DMA transfer, '10' or '11' are for single address modes. '00' and '01' are for 68000 signal devices that can be used as is, for devices that can read/write at the same time as the CPU. Memory is divided into these types. For example, the X68000 has only one type, internal memory I/O, but usually '00' is set. '01' is for devices around the 68000 that take the motor's B signal bit on the CPU bus, like serial address mode. This DMAC PCL line is an input terminal to operate the E clock of the 68000, whether to match the X68000 timing is ignored (PCL bit configuration is irrelevant). 6800 families are older devices, so it is rare for them to be used at this point.

'10' and '11' say the transfer times between I/O devices and DMAC are different, and send a signal for ACK when DMAC finishes I/O side transfer during '10' settings.

The page number is noted as 43.

Figure 12 DCR: Device Control Register OCR: Operation Control Register DCR (Device Control Register) OCR (Operation Control Register) bit 15 bit 0 | XRM | DTYP | DPS | '0' | PCL | DIR | BTD | SIZE | CHAIN | REQ |

- Request Generation Method

```
- 00: Auto request limited to initial
- 01: Auto request not limited to initial
- 10: External bus request (by REQ line)
- 11: First transfer is auto request, subsequent are external bus transfer
```

- Chaining Operation

```
- 00: No chaining
- 01: (Not in use)
- 10: Array chaining
- 11: Link array chaining
```

- Operand Byte Size

```
- 00: Byte (8bit)
- 01: Word (16bit)
- 10: Long word (32bit)
- 11: Byte transfer with size specified
```

- Direction

```
- 0: Memory device < MAR → DAR
- 1: Device memory / DAR → MAR
```

- Done-Attached Multiple Block Transfer

```
- 0: Normal transfer
- 1: Forced transfer if there's DONE input, proceed to next block
```

- Peripheral Control Line

```
- 00: Status input
- 01: Input with interrupt
- 10: External start input
- 11: ABORT (forced end) input
```

- Device Port Size

```
- 0: 8-bit port
- 1: 16-bit port
```

- Device Type

```
- 00: 68000 bus type
- 01: 68000 VAX type
- 10: Device with ACK
- 11: Device with ACK & READY
```

- External Request Mode

```
- 00: Burst transfer mode
- 01: (Undefined)
- 10: Cycle-stealed without hold signal
- 11: Cycle-stealed with hold signal
```

Note: The bit fields, such as ”XRM”, ”DTYP”, etc., are placeholders for actual register bits and should be referred to in the appropriate technical documentation for exact meanings and configurations.

DPS (Device Port Size)

Whether the connected I/O is an 8-bit port or a 16-bit port is decided here. In Dual Address Mode, it is decided using DAR (Device Address Register) if devices are unable to access the 16-bit board or you wish to restrict them to 8-bit access.

If DPS indicates '0' (set as 8-bit port), make sure the bit is changed to set DAR; otherwise, note that the bit is altered during data transmission. Refer to Figure 13 on page 46 for more details. Memory will appear as 16-bit port increments.

The X68000 system defaults FD, HD, ADPCM to an 8-bit port. To operate in a non-standard scenario, set it as usual to the 16-bit port board, forwarding within a 256-color/10-color graphical interface.

When DPS is designated as '0' (8-bit port), DAR changes. Usually, within the X68000 graphical interface or similar, it operates to access lower and upper bits individually as needed during forwarding. In such situations, conversion and forwarding methods differ based on the DPS '0' setting for DMA forwarding.

PCL (Peripheral Control Line)

Beyond when DCR and DTYP bits are '01' (6800 bus type), the functionality of DMAC's PCL pin can be specified.

Typically, when set to '00' and '01', PCL pin acts as a data input pin. When bit '01' is set, PCL pin fluctuations occur, generating a change from High to Low.

During this configuration, the PCL pin function is specified.

● Fig. 13 Transfer when the device port size is 8 bits

```
                bit 15        8  7        bit 0
            +0    'A'              'B'
```

Data in memory +2 'C' 'D'

```
            +4
            +6

                bit 15        8  7        bit 0
```

Device address +0 'A' (DAR) initial +2 'B' value is odd +4 'C'

```
            +6                     'D'

                bit 15        8  7        bit 0
```

Device address +0 'A' (DAR) initial +2 'B' value is even +4 'C' +6 'D'

interrupt. However, whether or not the interrupt was caused by this PCL pin can be determined by reading the CSR within the interrupt handling routine and checking that the PCT bit is set. When set to '10', the PCL pin operates as an output signal indicating to the outside that the channel has become active. The PCL pin is normally at High level, but after the channel becomes active it goes Low for only four clock cycles. When set to '11', the PCL pin operates as a forced-termination (ABORT) input signal pin for the DMA transfer. If the DMA transfer is terminated by this signal it is treated as an error, and the CSR

DMA

When ERR bit is '1', the CER is set to $10 (External Force Stop).

5-25 DIR (Direction)

This configures the direction of data transfer for the DMA. Setting this bit to '0' will transfer data from memory to I/O, and setting it to '1' will transfer data from I/O to memory. In dual address mode, transfers occur from the address shown in MAR (Memory Address Register) to the address shown in DAR (Device Address Register) when '0', and vice versa when set to '1'.

5-26 BTD (DONE attached multiple block transfer)

The HD 63450 has a feature for transferring multiple blocks. When using the DONE input to interrupt and proceed to transfer the next block forcefully, it has a feature to transfer multiple blocks if DONE is high. When this bit is set to '1', this mode is selected.

5-27 SIZE (Operand Size)

This bit sets whether the data transfer units are byte (8-bit), word (16-bit), or long word (32-bit). However, when the operand size is set to 8 bits (1 byte), DMAC's usage efficiency is increased and it may perform burst operations to transfer multiple data together, regardless of the SIZE. Therefore, the actual operation of the bus might not conform exactly to the specified SIZE.

For example, if the SIZE is set to 8 bits and the DPS (Data Path Size) is set to 8 bits, consider the cases where transferring more than 2 bytes results in an odd number of memory accesses. In such cases, DMAC performs two consecutive memory accesses, taking two transfer cycles for each CPU read or write access. If the transition is carried out when there's no burst activity, it will read/write memory data twice from I/O, storing data later. During a burst, memory accesses are completed after two cycles and data is written automatically. Conversely, data transfer from I/O to memory will force one word into memory after two consecutive reads, synchronizing the written data to memory with a single memory access later.

Note: This text refers to technical descriptions and settings related to Direct Memory Access (DMA) functionality in HD 63450, a DMA controller.

When the address on the side performing the word access (in the previous example, the memory) ends in an odd number, or when the address on the side performing the word access does not change (set by SCR on MAC or DAC), no pack operation is performed, and transfer is carried out in byte units according to the SIZE setting. The setting of the SIZE bit to '11' is similar to the case mentioned earlier for 8-bit boards, and in such a setting, data transfer between memory and I/O is prohibited, allowing access to memory in one-byte I/O units. On the X68000 DOS, Human 68K, channels #0, #1, and #3 have the SIZE bit.

■ Figure 14 Example of DMAC Pack Operation No pack Pack Operation (Diagrams)

No pack Pack Operation

Do not write to memory Write as 16-bit units

Page 48

Set this to '11'.

5-○8 CHAIN (Chaining Operation)

As noted earlier, HD63450 supports multi-block transfers, allowing the DMAC itself to perform array chaining operations and linked list chaining operations while reading the transfer address table from memory.

The CHAIN bit is a bit that determines whether or not to perform this chaining operation, and if it is set, chaining operations are executed. In the case of '00,' chaining operations are not executed. In the case of '10,' only array chaining operations are performed, and in the case of '11,' linked list chaining operations are performed.

5-○9 REQG (Request Generation Method)

This is a bit for selecting the automatic request mode, the external request mode, etc., which are transfer modes for one-block transfers. '00' and '01' are for automatic requests and do not generate transfer request signals from external sources and are used only for memory-to-memory transfers. In the case of '01,' as it is the fastest mode, it finishes the transfer in the same way as a bus while taking passes but performs a comparatively fractional transfer with the set ratio in the GCR for '10.'

'10' responds to an REQ signal from external I/O and executes the transfer in external demand mode, while '11' starts the channel operation and the first transfer is in automatic request mode, followed by external demand mode.

There are combinations of DTYP, DPS, SIZE, REQG bits that cannot be set. Please refer to the diagram on page 50, which summarizes the modes supported by the DMAC.

(Note: Some technical terms and abbreviations have been retained as is to preserve the original meaning and context.)

Table:

Address Mode (DTYP)	Device Port Size (DPS)	Transfer Request Generation Method (REQG)	Operand
Dual Address Mode (DTYP='00' or '01')	8 bit	'00', '01', '10', '11'	O
Dual Address Mode (DTYP='00' or '01')	16 bit	'00', '01'	O
Dual Address Mode (DTYP='00' or '01')	16 bit	'10', '11'	O
Single Address Mode (DTYP='10' or '11')	8 bit	'00', '01', '10', '11'	O
Single Address Mode (DTYP='10' or '11')	16 bit	'00', '01', '10', '11'	X

O: Configurable X: Not configurable DTYP, DPS, DCR: Corresponding bits in the Device Control Register REQG, SIZE: Corresponding bits in the Operational Control Register

Section 5.3 SCR, CCR

SCR (Sequence Control Register) and CCR (Channel Control Register) bits layout is shown in Figure 16. SCR is used for initiation/termination control of transfer requests and transfer destinations, masking interruptions, etc. CCR is used for channel operation control.

Section 5.4 MAC (Memory Address Counter)

MAC defines the value of MAR (Memory Address Register) when DMA transfer occurs. If MAC is '00', MAR will not change. If it is '01', MAR will increment with each transfer, and if it is '10', MAR will decrement (count down).

For single address mode, this change matches the operand size, but for dual address mode, it depends on the DCR and DPS bits and the OCR SIZE bits.

Figure 17 on page 52 shows the summary of data transfer forms and whether a burst transfer is only determined by MAC or not, upon transferring each operand. When the operand size is in byte units, no issue arises as no weird actions should occur in typical DMA transfer helper operations. Thus, no need to keep an eye on CPU systems when DMA helpers raise transfer speeds via cameras involved (it is necessary to know more when the transfer resumes after termination).

This is the end of the text.

DMA

● Fig. 16 SCR: Sequence Control Register CCR: Channel Control Register (+$06)

SCR (Sequence Control Register) ---- CCR (Channel Control Register)

bit 15 8 7 bit 0 | '0' | MAC | DAC | | STR | CNT | HLT | SAB | INT | '0' |

Interrupt Enable

```
0: Disable interrupts
1: Enable interrupts
```

Software Abort 1: Stop channel operation

Halt Operation 1: Temporarily halt channel operation

Continue Operation

```
0: No continued operation
1: // (continued operation)
```

Start Operation 1: Start operation

Device Address Register Count

```
00: Device Address Register value does not change
01: Device Address Register value increments each time a transfer occurs
10: // (decrements)
11: (Unused)
```

Memory Address Register Count

```
00: Memory Address Register value does not change
01: Memory Address Register value increments each time a transfer occurs
10: // (decrements)
11: (Unused)
```

5.3.2 DAC (Device Address Register Count)

This bit specifies the increment/decrement of the DAR (Device Address Register), which specifies the device-side address in dual address mode.

5.3.3 STR (Start Operation)

This is the bit that indicates the start of DMA transfer. It is normally '0'; setting this bit to '1' initiates DMA transfer.

■Diagram - 17 Operation in Dual Address Mode

Device Space Size (DPS)	Operation Size	Memory Access (Size) (Times)	Device Space Access (Size) (Times)	Address
		8 bit MODE		
Byte (SIZE=00')	Byte	Word x 1	Byte x 2	±2
Word (SIZE=01')	Word	Word x 1	Byte x 4	±2
Long Word (SIZE=10')	Long Word	Word x 2	Byte x 4	±4*
Packing Byte (SIZE=11')	Byte	Byte x 1	Byte x 1	±2*
		16 bit MODE		
Byte (SIZE=00')	Byte	Word x 1	Word x 1	±2
Word (SIZE=01')	Word	Word x 1	Word x 2	±2
Long Word (SIZE=10')	Long Word	Word x 2	Word x 2	±4

*1: When packing transfer is performed, regardless of device or word size. *2: When using the SAB pin during transfer, the device address count is multiplied by 2.

Data transfer begins. Writing '0' to the STR bit does not stop the operation. To forcefully terminate, use the SAB pin and set the HLT bit to '1' for a temporary stop. When setting the STR bit to '1', access to the DAC register should be done in bytes. When accessing word or long word, the operation changes to single word operation. In the Human68K, the db.x command should be executed in a read/write mode with word units, so DMA start cannot be performed. Please be careful.

5-3-4 CNT (Continue Operation)

This bit is used to perform continuous operations including multiple block data transfer. When the STR bit or CSR ACT bit becomes '1' (during data transfer), after setting the next transfer address and sending the block transfer codes to BAR, BTC, and BFC registers, setting the CNT bit to '1' will allow continuous operation.

When neither STR nor ACT bits are '1', setting the CNT bit to '1' will become an operation timing error. Also, if OCR's CHAIN bit is set in chain mode ('10' or '11'), setting CNT bit to '1' will result in a configuration error.

DMA

5.3.5 HLT (Halt Operation)

Setting the HLT bit to '1' during a transfer operation temporarily halts the transfer operation. Returning the HLT bit to '0' resumes the halted transfer operation. Even if the operation is interrupted by the HLT bit during an external request transfer, the next request is sensed and acted upon when it occurs. However, in burst transfer mode, after the HLT bit returns to '0', the I/O device must keep asserting the REQ signal until the first transfer starts.

5.3.6 SAB (Software Abort)

Setting this to '1' forcibly terminates the transfer operation. At this time, the ERR bit within the CSR becomes '1', and $11 (software forced abort) is set in the CER. Since the SAB bit is automatically cleared when the ERR bit becomes '1', the SAB bit is always read as '0'.

5.3.7 INT (Interrupt Enable)

This specifies whether or not to interrupt the CPU when the channel operation ends or an error occurs. When set to '1', interrupts are generated; when set to '0', interrupts are not generated. The conditions for an interrupt to occur are: when INT is '1' and any of the COC, BTC, ERR, NDT, or PCT bits of the CSR register is '1'. However, PCT becomes an interrupt source only when the DCR is programmed for PCL-bit interrupt-enabled status input. The interrupt vector number at the time of the interrupt is specified by the NIV (Normal Interrupt Vector) register or the EIV (Error Interrupt Vector) register. The EIV is used when the ERR bit is '1', and the NIV value is used in all other cases.

5.4 CPR

The bit layout of the CPR (Channel Priority Register) is shown in Fig. 18 on page 54.

☒.....18 Channel Priority Register (+$2D)

bit 7 | bit 0 '0' | CP

- Channel Priority

```
- 00: Highest priority
- 01: Second highest priority
- 10: ...
- 11: Lowest priority
```

CPR determines the priority (priority order) between the 4 channels that DMAC has. The priority is highest when '00', and lowest when '11'. When multiple channel demands occur simultaneously, the channel with lower priority is the one that is served. It is possible to set the same priority on multiple channels. In this case, the lower priority one will be served in a round-robin order while other channels are serviced.

5◉5 MFC, DFC, BFC

MFC (Memory Function Code), DFC (Device Function Code), BFC (Base Function Code) indicates bits allocated to indicate functions at access time and are displayed in bit position 19. The 68000 CPU outputs this state signal externally in the form of a 3-bit status signal, called the function code when accessing memory I/O. This status signal shows whether the access is at the user mode level or the super user mode level, and whether it is data access, program read-out, direct retrieval, and so forth.

In the case of X68000, when Human 68k's operating environment accesses low-priority VRAM or I/O outside the user area, the X68000 uses this function code to inform of that condition.

DMAC CPU can output this function code using the function code function. The register that holds addresses from DMAC is called MAR (Memory Address Register), DAR (Device Address Register), and BFC (Base Function Code). Therefore, it's possible to specify three types of register function codes. When MAR is accessed, MFC is used, DFC is used when DAR is accessed, and BFC is specified when the BFC is accessed during continuous operation.

● Fig. 19 MFC/DFC/BFC: Function Code Register (+$29/+$31/+$39)

bit 7	bit 0
'0'	FC2 FC1 FC0

```
└ Function Code
   000: (Not used)
   001: User Data
   010: User Program
   011: (Not used)
   100: (Not used)
   101: Supervisor Data
   110: Supervisor Program
   111: Interrupt Acknowledge
```

```
In normal use on the X68000, the function code should be set to '101',
```

i.e. Supervisor Data (data access in the supervisor state).

● 5.6 GCR

```
The bit layout of GCR (General Control Register) is shown in Fig. 20 on
```

page 56. GCR controls how the bus is held when transferring at a limited speed.

● 5.6.1 BT (Burst Time)

```
Sets, in clock counts, the period at which DMA transfer requests are
```

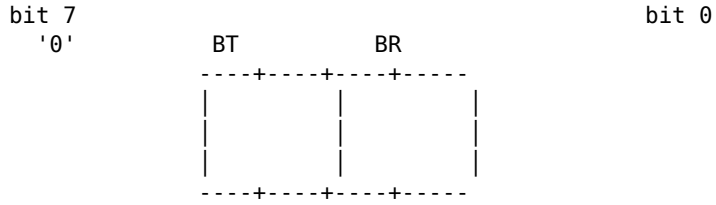
generated (the auto-request interval) when operating at a limited speed. The auto-request interval is 2^(BT+4) clocks.

● 5.6.2 BR (Burst Width Ratio)

```
The bits that determine the bus usage ratio when operating at a limited
```

speed. The DMAC monitors the BGACK (Bus Grant Acknowledge) signal output by the CPU, and sets the limited speed so that the period during which a device other than the CPU (on the X68000 only the DMAC exists) uses the bus becomes 2^(-(BT+1)) of the entire cycle.

Figure ... 20 General Control Register (+$FF)



Band Width Ratio (Bus Occupancy Rate)

```
00: 50.00%
01: 25.00%
10: 12.50%
11:  6.25%
```

Burst Time (Number of DMA clock cycles per burst)

```
00: 16 clocks
01: 32 clocks
10: 64 clocks
11: 128 clocks
```

- When BT and BR mode is set to ”Auto-Request”, it operates only when the two bits are set to '00'.

This setting is for the transfer. Now, let's assume we set the BT bit to '00' and the BR bit to '01'. At this time, the auto-request inter-burst interval is 16 clocks and the bus occupancy rate is 25 percent. So, if DMAC is performing the sampling to keep track of the bus usage at this occupancy rate for a period of 2^8 + 2^4 + 8 clocks, then the sampling period in this case is 64 clocks.

During the 64 clocks period, DMAC measures the period when a device other than the CPU is using the bus ignoring the BGACK signal. If this period is 16 clocks or less, the next 64 clocks period starts, and within the same restricted speed, DMA transfer requests occur. If it is after the 16-clock period and the bus to be used has been allocated, then DMA transfer request at a restricted speed will not be generated during the next 64 clocks period. This operation, when viewed in a longer period, maintains a bus occupancy rate of about 25 percent for devices other than the CPU.

In restricted speed bus usage, note that it is not the usage rate of the channel to be transferred but the usage rate during which devices other than the CPU use the bus as will be calculated. Be careful with the calculation during the time to use the bus under restricted speed. If there are multiple channels and one channel is set to a sufficiently high-speed mode, the channel set to restricted speed will not be transferred until other channels' DMA transfers are completed.

Page 56

Human 68K Initial Setting Values

The DMAC setting values by Human 68K are shown in Figure 21, so please refer to them when initializing the DMAC yourself. When the Human 68K initializes the DMAC, it does not perform initialization at once but does it when using each channel. Therefore, be careful of peculiar values when reading the registers of unused or once-used channels (hard disk or channel #3 (ADPCM)) from the floppy disk.

Channels #0 and #1 disable DMAC from causing an interrupt, and vector #0 is entered into vector $0F. FD and HD have the controller LSI generate an interrupt, and DMAC does not generate any interrupts.

Vector $0F interrupt is also called the initial interrupt vector number, which is reserved in the 68000 system as an interrupt generated by an I/O device or similar that has not completed initialization (after HD 63450 is reset, NIV = EIV and vector #0 are set to $0F).

Figure.....21 Human 68K Setting Values

Register	Channel #0	Channel #1	Channel #2	Channel #3
DCR	$80	$08	$08	$80
OCR	$B2	$B2	$00	$32
SCR	$04	$04	$00	$04
CCR	$00	$00	$00	$08
NIV	$0F	$0F	$68	$6A
EIV	$0F	$0F	$09	$6B
CPR	$00	$02	$03	$01

*Vector number 0

(Page 57)

Sample Program

As a sample program to operate the DMAC, we created one that clears the text screen and transfers short graphic images to the screen.

The sample program can be compiled with GCC or XC. XC has different rules, but considering that GCC is frequently used for free software, the sample is made for GCC. Because XC cannot use volatile, please compile after commenting out define macros at the end of the title using /* */, or after deleting the word volatile in the list. If you use GCC, be sure not to delete volatile, as it will be optimized and may not work. Delete it only if necessary.

Below is the batch file used to create the sample program.

- For GCC: gcc -O -fomit-frame-pointer -finline-functions -fstrength-reduce %1 %2 %3 %4 %5 baslib.a icoslib.a doslib.a
- For XC: cc %1 %2 %3 %4 %5 /W /Y

§ 1 Clearing the Text Screen with the DMAC

This section describes a program that clears the text screen using the DMAC. After entering supervisor mode with SUPER(0);, write 0 to the top address of the text VRAM.

The DMA transfers from the MAR (source address) to the DAR (destination address). By incrementing the DAR as it writes to the text VRAM top address, it programs to write 0 to the entire text VRAM. In this sample, we write 0 to the top address, so it clears the text screen.

The operand size is set to LONGWORD (32 bits). The text screen size is 256K.

DMA

Due to the byte size, MTC is only 16-bit (64 K bytes), so operands are transferred once by transferring 64 Kx4=256 K bytes in long words.

● List 1 Text Screen Clearing by DMAC

```
/*
 * List 1: Text Screen Clearing by DMA Controller
 *
 * Because volatile is not supported by XC,
 * please insert the following line to invalidate volatile
 *
 * #define volatile
 */

#include <doslib.h>

struct DMAREG {
    unsigned char  csr;
    unsigned char  cer;
    unsigned short spare1;
    unsigned char  dcr;
    unsigned char  ocr;
    unsigned char  scr;
    unsigned char  ccr;
    unsigned short spare2;
    unsigned short mtc;
    unsigned char  *mar;
    unsigned long  spare3;
    unsigned char  *dar;
    unsigned short spare4;
    unsigned short btc;
    unsigned char  *bar;
    unsigned long  spare5;
```

```
    unsigned char  spare6;
    unsigned char  niv;
    unsigned char  spare7;
    unsigned char  eiv;
    unsigned char  spare8;
    unsigned char  mfc;
    unsigned short spare9;
    unsigned char  spare10;
```

};

unsigned char cpr; unsigned short spare11; unsigned char spare12; unsigned char dfc; unsigned long spare13; unsigned short spare14; unsigned char spare15; unsigned char bfc; unsigned long spare16; unsigned char spare17; unsigned char gcr; };

volatile struct DMAREG *dma; void main(); void dma_setup(); void dma_start(); void wait_complete(); void clear_flag();

void main() {

```
    SUPER(0);
    *(unsigned int *)0xe00000 = 0;
    dma = (struct DMAREG *)0xe84080;    /* Use channel #2 */
    clear_flag();                       /* Clear CSR flag */
    dma_setup();                        /* Initialize DMA controller */
    dma_start();                        /* Start transfer */
    wait_complete();                    /* Wait for transfer completion */
    clear_flag();                       /* Clear flag */
```

}

void dma_setup() {

```
    dma->dcr = 0x08;
    dma->ocr = 0x21;
    dma->scr = 0x01;
    dma->ccr = 0x00;
    dma->cpr = 0x03;
    dma->bfc = 0x05;
    dma->dfc = 0x05;
```

}

Heading: DMA

Code: dma->mtc = 0xffff; dma->mar = (unsigned char *)0xe00000; dma->dar = (unsigned char *)0xe00000; }

void dma_start() { dma->ccr |= 0x80; }

void wait_complete() {

```
    while(!(dma->csr & 0x90))
        ;
```

}

void clear_flag() { dma->csr = 0xff; }

Section: ◆2 Transferring rectangular areas to graphic VRAM (Part 1)

Using array chain mode, where non-continuous areas can be transferred at once, we created a program that transfers rectangular areas to the graphics screen (List 2). After writing a gradation pattern to the screen in 65536 color mode, data is read sequentially into a buffer from the upper priority. This buffer holds transfer information to transfer data to a rectangular area. Transfer information is created one per horizontal line, and the dot

number in the vertical direction is divided and added as transfer information table. The destination address is sequentially changed so that rectangular areas appear to move on the screen.

● List 2: Transferring rectangular areas of graphical VRAM (Array Chain Mode)

```
/*
* List 2: Transferring rectangular areas of graphical screen in Array Chain Mode
*
* Since volatile is not supported in XC,
* please disable volatile by adding the following line
*
* #define volatile
*/
```

#include <doslib.h>

struct DMAREG {

```
    unsigned char csr;
    unsigned char cer;
    unsigned short spare1;
    unsigned char dcr;
    unsigned char ocr;
    unsigned char scr;
    unsigned char tcr;
    unsigned short spare2;
    unsigned short mtc;
    unsigned char *mar;
    unsigned long spare3;
    unsigned char *dar;
    unsigned short spare4;
    unsigned short btc;
    unsigned char *bar;
    unsigned long spare5;
    unsigned char spare6;
    unsigned char niv;
    unsigned char spare7;
    unsigned char eiv;
    unsigned char spare8;
    unsigned char mfc;
    unsigned short spare9;
    unsigned char spare10;
    unsigned char cpr;
    unsigned short spare11;
    unsigned char spare12;
    unsigned char dfc;
```

};

Note: This is a code snippet dealing with graphics VRAM and DMA registers.

unsigned long sparel3; unsigned short sparel4; unsigned char sparel5; unsigned char bfc; unsigned long sparel6; unsigned char sparel7; unsigned char gcr; };

struct XFR_INF {

```
    unsigned short *adrs;
    unsigned short length;
```

} xfr_inf[512]; unsigned short databuf[256*256]; volatile struct DMAREG *dma; unsigned short src_data;

void main(); void init_screen(); void dma_box(); void dma_setup(); void dma_start(); void wait_complete(); void clear_flag();

void main() {

```
    int i;
    screen (1,3,1,1);
    SUPER(0);
    init_screen();
    for (i = 0; i<255; i+=4)
        dma_box(databuf, 255-i, i, 511-i, i+256, 0xffff);
```

}

void init_screen() {

```
    unsigned short *vram, *buf;
    unsigned int i, h, s, v;
    vram = (unsigned short *)0xc00000;
    for (i=0; i<512*512; i++) {
        ...
s = i & 0x1f;
v = (i >> 5) & 0x1f;
h = ((i >> 10) % 0x20);
```

*vram++ = hsv(h, s, v); }

```
 vram = (unsigned short *)0xc00000;
 buf = databuf;
```

for (i=0; i<256*256; i++) *buf++ = *vram++; }

void dma_box(buf, x1, y1, x2, y2, col)

```
   unsigned short *buf;
   unsigned int x1, y1, x2, y2, col;
```

{

```
   int i, xlen, ylen;
   unsigned short *sadrs;
   xlen = x2-x1;
   ylen = y2-y1;
   src_data = col;
   sadrs = (unsigned short *)0xc00000;
   sadrs += 512*y1+x1;
   for(i=0; i <= ylen; i++, sadrs+=512) {
       xfr_inf[i].adrs = sadrs;
       xfr_inf[i].length = xlen;
   }
   dma = (struct DMAREG *)0xe84080;
   clear_flag();
   dma_setup(buf, ylen+1);
   dma_start();
   wait_complete();
   clear_flag();
```

}

void dma_setup(bufadrs, links)

```
   unsigned short *bufadrs;
   unsigned int links;
```

{

```
   dma->dcr = 0x08;
   dma->ocr = 0x99;
   dma->scr = 0x05;
```

}

Graphics VRAM Rectangular Area Transfer (Part 2)

This is listing 3, which modifies the rectangular area transfer described in section 7-2 to use link array chaining mode. Comparing it with listing 2, you will understand the differences between array chaining mode and link array chaining mode.

● List 3. Transferring rectangular areas to graphic VRAM (Link Array Chain Mode)

```
/*
 * List 3: Transferring rectangular areas of the graphics screen in Link Array Chain Mode
 *
 * Since volatile is not supported in XC,
 * please disable volatile by including the following line
 *
 * #define volatile
 */
```

#include <doslib.h>

struct DMAREG {

```
    unsigned char csr;
    unsigned char cer;
    unsigned short spare1;
    unsigned char dcr;
    unsigned char ocr;
    unsigned char scr;
    unsigned char ccr;
    unsigned short spare2;
    unsigned short mtc;
    unsigned char *mar;
    unsigned long spare3;
    unsigned char *dar;
    unsigned short spare4;
    unsigned short btc;
    unsigned char *bar;
    unsigned long spare5;
    unsigned char spare6;
    unsigned char niv;
    unsigned char spare7;
    unsigned char eiv;
    unsigned char spare8;
    unsigned char mfc;
    unsigned short spare9;
    unsigned char spare10;
    unsigned char cpr;
    unsigned short spare11;
    unsigned char spare12;
    unsigned char dfc;
    unsigned long spare13;
    unsigned short spare14;
```

};

unsigned char spare15; unsigned char bfc; unsigned long spare16; unsigned char spare17; unsigned char gcr; };

struct XFR_INF {

```
    unsigned short *adrs;
    unsigned short length;
    struct XFR_INF *link;
```

} xfr_inf[512]; unsigned short databuf[256*256]; volatile struct DMAREG *dma; unsigned short src_data;

void main(); void init_screen(); void dma_box(); void dma_setup(); void dma_start(); void wait_complete(); void clear_flag();

void main() {

```
    int i;
    screen (1, 3, 1, 1);
    SUPER(0);
    init_screen();
    for(i = 0; i < 255; i += 4)
        dma_box(databuf, 255 - i, i, 511 - i, i + 256, 0xffff);
```

}

void init_screen() {

```
    unsigned short *vram, *buf;
    unsigned int i, h, s, v;
    vram = (unsigned short *)0xc00000;
    for (i = 0; i < 512*512; i++) {
        s = i & 0x1f;
        v = (i >> 5) & 0x1f;
        h = ((i >> 10) % 0xc0);
    }
```

}

The code defines a data structure for handling Direct Memory Access (DMA) operations and contains some functions related to screen initialization and DMA operations.

*vram++ = hsv(h, s, v); }

```
vram = (unsigned short *)0xc00000;
buf = databuf;
```

for (i=0; i<256*256; i++) *buf++ = *vram++;

}

void dma_box(buf, x1, y1, x2, y2, col) unsigned short *buf; unsigned int x1, y1, x2, y2, col;
{

```
    int i, xlen, ylen;
    unsigned short *sadrs;
    xlen = x2 - x1;
    ylen = y2 - y1;
    src_data = col;
    sadrs = (unsigned short *)0xc00000;
    sadrs += 512 * y1 + x1;
    for (i = 0; i < ylen; i++, sadrs += 512) {
        xfr_inf[i].adrs = sadrs;
        xfr_inf[i].length = xlen;
        xfr_inf[i].link = &xfr_inf[i + 1];
    }
    xfr_inf[i - 1].link = 0;
    dma = (struct DMAREG *)0xe84080;
    clear_flag();
    dma_setup(buf);
    dma_start();
    wait_complete();
    clear_flag();
```

}

void dma_setup(bufadrs) unsigned short *bufadrs; {

```
    dma->dcr = 0x08;
    dma->ocr = 0x9d;
    dma->scr = 0x05;
    dma->ccr = 0x00;
    dma->cpr = 0x03;
```

}

dma->mfc = 0x05; dma->dfc = 0x05; dma->bfc = 0x05; dma->dar = (unsigned char *)bufadrs; dma->bar = (unsigned char *)xfr_inf; }

void dma_start() { dma->ccr |= 0x80; }

void wait_complete() {

```
  while(!(dma->csr & 0x90))
  ;
```

}

void clear_flag() { dma->csr = 0xff; }

(Blank page in the original.)

Interrupts

X68000 notifies most system state changes and service requests from LSI through interrupts. In this section, we will explain the overview of interrupt operation and a list of interrupt vectors in Human 68K.

Interrupt Systems and Level Assignment

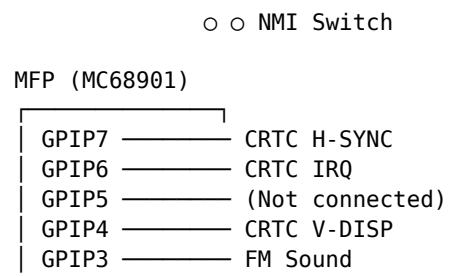
1. Interrupt Systems and Level Assignment

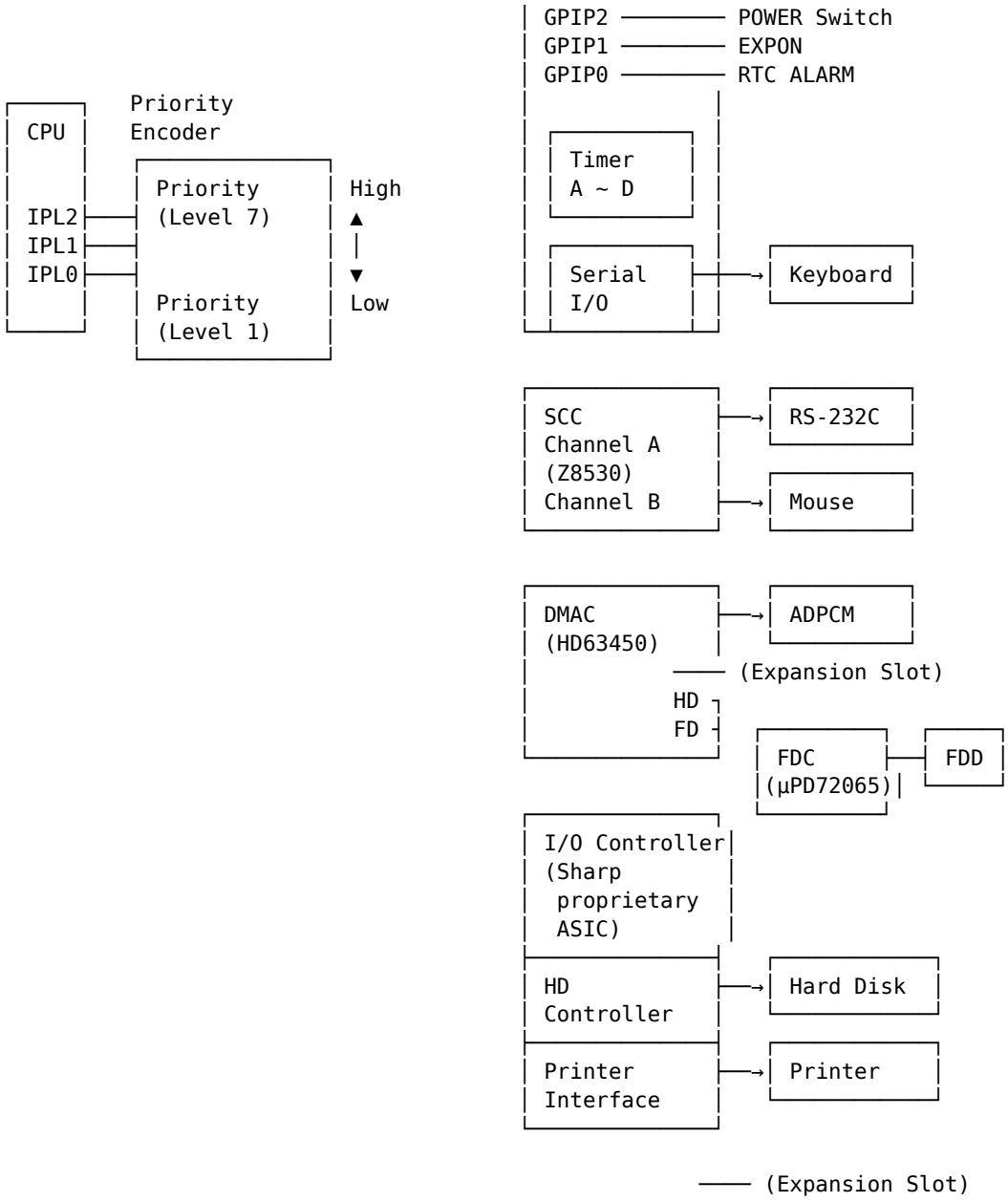
The interrupt system diagram of the X68000 is shown on page 72. For the X68000 CPU, there are 7 levels of interrupt priority, ranging from level 1 to level 7. When an interrupt is requested, the CPU identifies this through 3 interrupt priority level signals: IPL0, IPL1, and IPL2. If a level 0 interrupt is recognized during a state display instruction, it means there is no higher priority than level 1. Among the 7 levels of interrupts, the one with lower priority is level 1, and the one with higher priority is level 7. The status register of the CPU has a 3-bit interrupt mask bit called the status bit, and the interrupts lower than this level are masked. When the CPU receives an interrupt request, it reads the interrupt mask level from the status register, and interrupts with a priority higher than the current mask level will be sharp-interrupted and processed. Only level 7 interrupts are not masked by the status bit of the status register. Additionally, level 7 interrupts are also called NMIs (Non-Maskable Interrupts).

The X68000 maps these 7 levels of interrupt as follows:

Page 71

● Fig. 1 Interrupt System Diagram





Level 7 (NMI)

- NMI switch on the main body

Level 6

```
 * MFP (Multi-Function Peripheral)
     * [CRTC, FM sound source, timer, keyboard, etc.]
```

Level 5

- SCC (Serial Communication Controller)
    - [RS-232C, mouse]

Level 4

- Expansion slot

Level 3

- DMAC (DMA Controller)
    - [ADPCM, FD, HD]

Level 2

- Expansion slot

Level 1

- I/O Controller LSI
  - [FD, HD, printer]

2. Interrupt Operation

The outline of the interrupt operation of the 68000 is shown in Figure 2 on page 74. When a peripheral device issues an interrupt request, it decodes the external interrupt priority order, and notifies the CPU of one of the three IPL signals (0, 2, 4) corresponding to its level. 2 The CPU acknowledges the interrupt but outputs an interrupt acknowledgment signal to the lower part of the address bus (A1 to A3). At the same time, it indicates that it is in the H state by setting the function code (FC0 to FC2) to H level, showing that it is in the interrupt acknowledgment cycle and reading the interrupt vector from the peripheral device. 3 The peripheral device puts the interrupt vector on the lower part of the data bus and indicates that it is valid by asserting DTACK signal to the CPU. 4 The CPU reads the vector, starts interrupt processing, and switches to the interrupt handling routine.

This is how the peripheral device specifies an acknowledgment vector. CPU reads the value of the vector and uses the default vectors stored at $19 - $1F (corresponding to Levels 1 - 7).

For the 68K Human system, not only does the Level 7 NMI use the vector $1F, but all peripheral devices use autovector to output vector numbers.

● Figure 2 Interrupt Operation of the 68000

CPU Peripheral Device

_____

| | All ”High” | FC2 | FC1 | FC0 3 | A3 | A2 | A1 |
| IPL2 | IPL1 INT | IPL0 1 | | |_____

Interrupt Vector

|---------->
Peripheral Device (Auto Vector) | | | | CPU Peripheral Device

_____

| | DTACK 5 | | D0~D7 4 | ----------> Vector Table | | | | | | | | CPU Peripheral Device

_____

| |
| VPA 6 | | | | | | | |

◆3 Exception Vector

Figure 3 shows the settings and usage of the exception vectors on the 68000 and Human 68K. Among these vectors, $00~$3F are reserved by the manufacturer (Motorola) who designed the CPU and are prohibited from being used as interrupt vectors by peripheral devices. Human 68K assigns $40~$4F to MFP, $50~$5F to SCC, $60~$63 to I/O controller, and $64~$6B to DMAC.

Note: The symbols like $40~$4F denote hexadecimal values.

Figure 3: Vector Assignment Examples

Vector Number (Decimal)	Vector (Hexadecimal)	Vector Address (Hexadecimal)	Vector Function Assignment
0	$00	S000000	SSP Value After Reset
1	$01	S000004	//
2	$02	S000008	Bus Error
3	$03	S00000C	Address Error
4	$04	S000010	Illegal Instruction
5	$05	S000014	Division by Zero
6	$06	S000018	CHK Instruction
7	$07	S00001C	TRAP Instruction
8	$08	S000020	Privilege Violation
9	$09	S000024	Trace
10	$0A	S000028	Line 1010 Emulator
11	$0B	S00002C	Line 1111 Emulator
12	$0C	S000030	
13	$0D	S000034	(Reserved for Future Use)
14	$0E	S000038	Initial COPROCESSOR Instruction
15	$0F	S00003C	Initial Unassigned Instruction
16-23	$10-$17	S000040-S00005C	(Future Use)
24	$18	S000060	Spurious Interrupt
25	$19	S000064	Level 1 Interrupt (Auto-Vector)
26	$1A	S000068	// 2 //
27	$1B	S00006C	// 3 //
28	$1C	S000070	// 4 //
29	$1D	S000074	// 5 //
30	$1E	S000078	// 6 //
31	$1F	S00007C	// 7 //
32-39	$20-$27	S000080-S00009C	TRAP 0 - TRAP 7 Instructions
40	$28	S0000A0	TRAP 8 Instruction
41	$29	S0000A4	// 9 //
42	$2A	S0000A8	// A //
43	$2B	S0000AC	// B //
44	$2C	S0000B0	// C //
45	$2D	S0000B4	// D //
46	$2E	S0000B8	// E //
47	$2F	S0000BC	// F //
48-63	$30-$3F	S0000C0-S0000FC	(Future Use)
64-79	$40-$4F	S000100-130C	User-Defined Vectors
80-95	$50-$5F	S000140-170C	
96-99	$60-$63	S000180-1C0C	
100-107	$64-$6B	S000190-1A0C	
108-255	$6C-$FF	S0001B0-3FC	(Future Use)

Any specific areas you want to dive deeper into?

4 Interrupt Vector Assignment Ports

It is cumbersome to identify the ports that set the interrupt vectors for each peripheral device, so they are summarized in Figure 4.

Figure .... 4 Interrupt Vector Assignment Ports

LSI	Address	bit 7	bit 0
MFP	$E88017	P	Interrupt factor changes *1
SCC	$E98003/7 (write register 2)		

LSI	Address	bit 7	bit 0
CH #0	$E84025		
	$E84027	P	
CH #1	$E84065		
	$E84067	P	
DMAC			
CH #2	$E840A5		P
	$E840A7	P	
CH #3	$E840E5		P
	$E840E7	P	
I/O Controller	$E9C003	P	Interrupt factor changes *2

P: Can be set freely

*1: 0000: GPIO0 0001: GPIO1 0010: GPIO2 0011: GPIO3 0100: Timer D 0101: Timer C 0110: GPIO4 0111: GPIO5 1000: Timer B 1001: Communication slot 1010: Communication buffer empty 1011: Communication error 1100: Communication buffer full 1101: Timer A 1110: GPIO6 1111: GPIO7

*2: 00: FDC 01: FDD 10: HD 11: Printer

MFP

The MFP, which integrates timer and general-purpose I/O, serial ports, etc. into a single chip, is used not only for keyboards but also for tasks such as periodic timer intervals, CRTC and FM sound source, RTC alarm signals, and capturing various statuses.

1 Overview

The MFP (Multifunction Peripheral MC68901) includes a counter/timer, serial port, and general-purpose I/O ports into a single LSI. Figure 1 on page 78 shows the block diagram within the X68000 and the connection state overview. The MFP has 4 timers, 1 channel serial port, a general-purpose I/O port with 8 bits, and is used in the X68000 for tasks such as discriminating the cause of the interrupt from the CRTC, power ON, and interfacing with the keyboard.

2 Assignment of MFP Functions

Let's see how the various functions of the MFP are assigned within the X68000. It will make it easier to understand.

- « Diagram 1 MFP Internal Block Diagram »
- [Diagram text starting from the top left]
- Pin Connections:

```
- CLK
- XTAL1
- XTAL2
- Clock 4MHz
MC68901
```

- [Diagram text located centrally and vertically aligned with respective components]

```
- Timer A:
  - TAI
  - TAO
  (Not used)
- Timer B:
  - TBI
  - TBO
```

```
  (Not used)
- Timer C:
  - TCO
  - TDO
  (Not used)
- Timer D:
  - TDI
  - TDO
  (Not used)
```

- [Diagram text at the lower and middle left section]

```
GPIP (General-purpose I/O)
  - GPIP0
  - GPIP1
  - GPIP2
  - GPIP3
  - GPIP4
  - GPIP5
  - GPIP6
  - GPIP7
(Interrupt Input)
```

- [Diagram text located near the USART section]

```
- USART (Serial Port)
  - Reception Control: RC
  - Transmission Control: TC
  - Received Data: RR
  - Transmitted Data: TR
  - SI
  - SO
  (Serial Output)
- Keyboard
```

- [Text at the bottom]:

Among the four timers, Timer B is used as the clock for determining the communication speed with the keyboard via the serial port, so if the operation mode is changed, the keyboard will not function. Other timers do not have a specified hardware use.

In Human 68K, Timer C is used for timing operations such as monitoring the interval between pressing keys, pausing the FDD, etc., and Timer D Version 2.0 is used to assist multi-task functionality with minimized overhead.

The control line for Timer A input contains a V-DISP signal generated by the CRTC. This allows counting of the number of V-DISP signal changes, enabling the CPU to be interrupted after a fixed interval.

- Page number at the bottom right: 78

It is possible to perform measurements of synchronization or V-DISP signal phase timing by making a tear, as referred to above.

The real-time clock port of the MFP has several operation modes, but since the X 68000 system has the port connected to the keyboard, it uses the keyboard communication settings.

Of the eight general-purpose I/O ports, GPIP0 to GPIP7, those that are unused are often used as base input ports. Since GPIP5 is fixed to H level at the external terminal, inputting a tear will read it as '1'.

3 MFP Register List

The list of MFP registers is shown in Figure 2 on page 80.

MFP registers are allocated sequentially from $E88001 to $E8802F. Most of the registers are 8 bits in length, and only the lower byte is used for word access. The MFP registers are used for high-speed interrupts (GPIP control interrupt for $E88001 to $E88005, and preemption control interrupt for $E88007 to $E88017, timer control interrupt for $E88019 to $E88025, USART (serial port) control for $E88027 to $E8802F).

The MFP registers are divided into those with special features on the status/read/write some, but basically, the other bits are unused as indicated in the diagram. In the writing state, reading '0' will indicate a light end. In the initialized state, they are fixed to '0' but in other LSI, it's common practice to keep them at '0'.

4 GPIP (General-purpose I/O Port)

The bit configuration for GPIP control related registers is shown in figure 3 on page 81. GPIP control registers consist of GPIP, AER, DDR, 3 elements, summarized in one table.

MFP (MC68901) Register List

The X68000 maps the MFP at base $E88000; each register occupies the lower (odd) byte, so register N is at $E88000 + (2N + 1).

Addr	Reg	Name
$E88001	GPIP	General-Purpose I/O
$E88003	AER	Active Edge Register
$E88005	DDR	Data Direction Register
$E88007	IERA	Interrupt Enable A
$E88009	IERB	Interrupt Enable B
$E8800B	IPRA	Interrupt Pending A
$E8800D	IPRB	Interrupt Pending B
$E8800F	ISRA	Interrupt In-Service A
$E88011	ISRB	Interrupt In-Service B
$E88013	IMRA	Interrupt Mask A
$E88015	IMRB	Interrupt Mask B
$E88017	VR	Vector Register
$E88019	TACR	Timer A Control Register
$E8801B	TBCR	Timer B Control Register
$E8801D	TCDCR	Timer C/D Control Register
$E8801F	TADR	Timer A Data Register
$E88021	TBDR	Timer B Data Register
$E88023	TCDR	Timer C Data Register
$E88025	TDDR	Timer D Data Register
$E88027	SCR	Sync Character Register
$E88029	UCR	USART Control Register
$E8802B	RSR	Receiver Status Register
$E8802D	TSR	Transmitter Status Register
$E8802F	UDR	USART Data Register

GPIP Register The GPIP register allows reading the status of each bit from GPIP 0 to GPIP 7, and writing output data. In the X68000, all of GPIP 0–GPIP 7 are used as inputs, so this register is read-only.

Next, we explain the signals connected to each GPIP bit. GPIP 7 is connected to the CRTC's H-SYNC (horizontal sync) signal, indicating the CRTC horizontal sync period when it is '1'.

● Figure ····· 3 GPIP, AER, DDR Bit Placement ($E88001, $E88003, $E88005)

|H-SYNC CIRQ V-DISP FMIRQ POW SW EXPON ALARM | | (GPIP7) (GPIP6) (GPIP5) (GPIP4) (GPIP3) (GPIP2) (GPIP1) (GPIP0) | |------------------------|-----------------|--------------|--------------|-----------------|------------|-----------|

- Always '1'

|------------------------|-----------------|--------------|--------------|-----------------|------------|-----------|

- RTC (Clock) ALARM signal

```
1: ALARM is 'H'
0: ALARM is 'L'
```

- EXPON signal state

```
1: EXPON is 'H' (normal)
0: EXPON is 'L'
```

- State of power switch on front panel

```
1: Power switch OFF
0: Power switch ON (normal)
```

- FM sound source IC interrupt request signal

```
1: No interrupt request
0: Interrupt requested
```

- CRTC V-DISP signal state

```
1: V-DISP signal is 'H' (vertical display period)
0: V-DISP signal is 'L' ( / syncronization period)
```

- CRTC cluster interrupt request signal state

```
1: No interrupt request
0: Interrupt requested
```

- CRTC α-D-SYNC signal state

```
1: H-SYNC signal is 'H' (horizontal period)
0: H-SYNC signal is 'L'
```

- GPIP Register: Outputs signals as they are regardless of signal state
- AER Register: Sets the direction of the input/output (I/O) and generates an interrupt under transition 0->1 of the signal

```
0: Generates an interrupt on 0 -> 1 transition
1: Generates an interrupt on 1 -> 0 transition
```

- DDR Register: Sets input/output states for signals as input or output

```
0: Input
1: Output
```

- GPIP 6 is assigned the cluster interrupt request signal from the CRTC. '0' indicates that a cluster interrupt request from the CRTC has occurred. Refer to the description of the CRTC for details on cluster interrupts.
- GPIP 5 is not used in this bit. External fixed to high level, so GPIP 5 is always '1' and read out.
- GPIP 4 is assigned the V-DISP (vertical display period) signal from the CRTC. '1' indicates the vertical display period. A '0' indicates the vertical blanking period.

Note: The text is technical and refers to specific hardware components and signal states associated with them.

GPIP 3 is an interrupt request signal from the FM sound IC. This indicates that an interrupt request has occurred from the FM sound IC.

GPIP 2 and 0 indicate that the power of the X68000 has been turned ON. The X68000's main power switch is the usual ON/OFF push-button switch as well as the expansion slot remote power ON signal and the main unit's remote terminal (both of these signals are treated as the same here). When the RTC (real-time clock: clock) alarm signal goes high, the power is turned on.

In other words, X68000 can determine if the power is ON through software. The expansion slot or main unit remote terminal (both are treated as the same here) when the main power switch is ON, GPIP 2, remote slot or remote terminal goes high (power is ON state) GPIP 1 and the real-time clock alarm signal goes high, GPIP 0 goes high and the interrupt is generated with a value of '0'.

4.2 AER (Active Edge Register)

GPIP can generate interrupts when the values change from '0' to '1', '1' to '0', or stay the same. AER allows each bit's change to be specified. The interrupt is generated when the bit changes from '0' to '1' or from '1' to '0'. The bits of GPIP 3 and GPIP 4 in AER, in particular, are combined with the timer's control signals edge settings. We'll explain this point later.

4.3 DDR (Data Direction Register)

The DDR is a register that specifies whether each bit of the GPIP is used as an input or output. '1' for output, '0' for input.

On the X68000, all GPIP bits are used as input ports, so DDR is set to all '0'.

Page 82

5 Interruption Control

MFP's interruption control-related registers are shown in Figures 4, 5, and 6. The MFP has 16 types of interrupt factors, allocated to two 8-bit registers as shown. The priority order of interruptions is fixed, with GPIP 7 (H-SYNC) having the highest priority, followed by GPIP 6 (CIRQ), Timer A, etc., and GPIP 0 (ALARM) having the lowest priority.

Diagram 4: IERA, IPRA, ISRA, IMRA ($E88007, $E8800B, $E8800F, $E88013) | bit 7 | bit 0 | |-------------|-------| | H-SYNC | Timer B |

- GPIP 7 (H-SYNC)
- GPIP 6 (CIRQ)

Key for progressive stages:

- Reception Buffer Full
- Reception Error
- Transmission Buffer Empty
- Transmission Error

Timers:

- Timer A
- Timer B

Types of MFP Interrupt Factors:

- MPSC Transmission Buffer Empty (Transmission Data Writing Request) Interruption
- MPSC Transmission Error Interruption

- MPSC Reception Buffer Full (Reception Data Reading Request) Interruption
- MPSC Reception Error Interruption

Timer A:

- Timer A End Interruption Request

CRTC:

- CRTC Interrupt (Raster Interrupt Request)
- CRTC H-SYNC (Horizontal Period Signal) Interruption

Legend:

- IERA: Controls the permission/prohibition of interruption occurrence

```
- 1 : Permission for interruption occurrence
- 0 : Prohibition
```

- IPRA: Shows/enables if the interrupt request is pending (preservation)

```
- 1 : The interrupt request is pending
- 0 : Not pending
```

- ISRA: Shows the processing state of the interrupt request (in service)

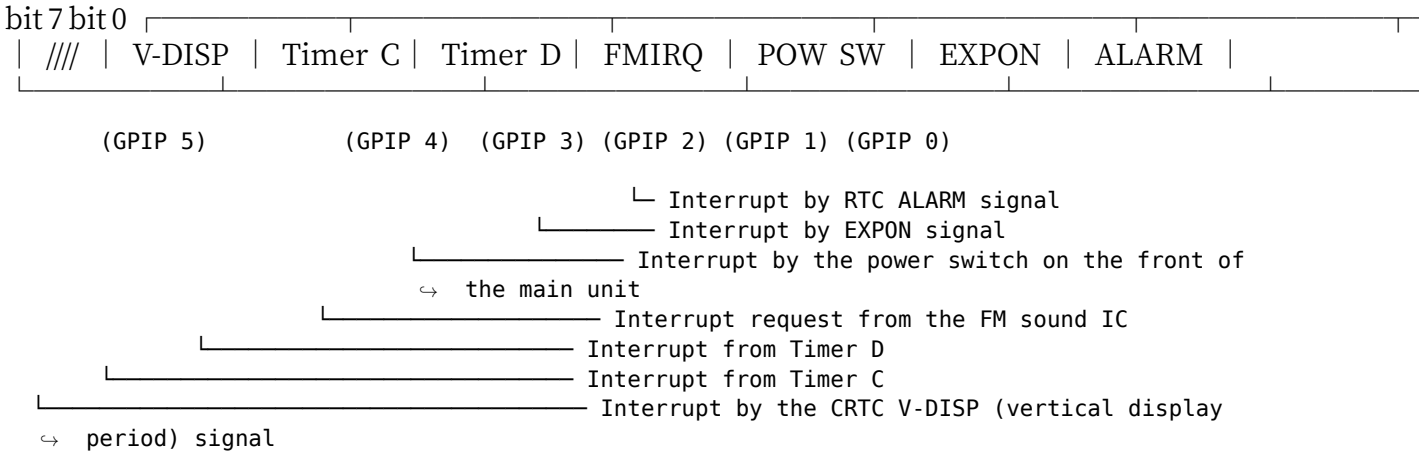
```
- 1 : The interrupt request is being processed
- 0 : Not being processed
```

- IMRA: Controls the interruption mask

```
- 1 : Masks the interrupt request (interruption not triggered)
- 0 : Does not mask (interruption triggered)
```

Page 83

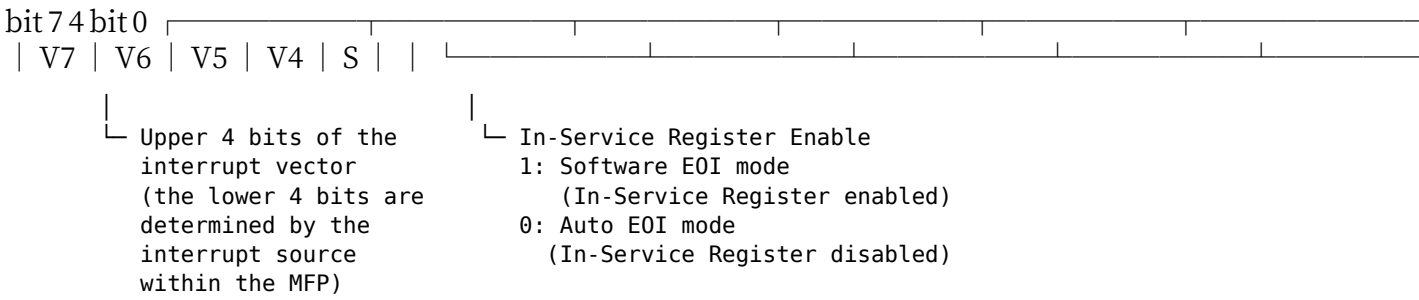
● Fig. 5 IERB, IPRB, ISRB, IMRB ($E88009, $E8800D, $E88011, $E88015)




```
· IERB: Controls enable/disable of interrupt generation
          1: Interrupt generation enabled
          0: //          //          disabled

· IPRB: Indicates that an interrupt request is pending
          1: The interrupt request is pending
          0: //          //          is not pending

· ISRB: Indicates that an interrupt request is in service (being processed)
          1: The interrupt request is in service
          0: //          //          is not in service

· IMRB: Controls the interrupt mask
```

```
1: The interrupt request is not masked (interrupt generation allowed)
0: //          //          masked    (    //          not allowed)
```

● Fig. 6 VR (Vector Register) $E88017



5-1 IERA/IERB (Interrupt Enable Registers A/B)

IERA/IERB are registers that control the permission/prohibition of interrupt occurrences. When set to '1', interrupt occurrences are permitted, and when set to '0', they are prohibited.

5-2 IPRA/IPRB (Interrupt Pending Registers A/B)

IPRA/IPRB are registers that indicate that an interrupt request is pending. IPRA/IPRB are registers set to '1' when an interrupt request is transmitted to the CPU by the MFP (Multi-Function Peripheral) (i.e., a bit in the interrupt vector is written to the CPU) and returns to '0' when the CPU acknowledges the request. In other words, '1' indicates that an interrupt request is pending and has not yet been acknowledged by the CPU.

By setting IPRA/IPRB, IERA/IERB permit or prohibit interrupt occurrences. When the MFP writes '0' to IPRA/IPRB, the CPU will not acknowledge the interrupt, which means the occurrence of the interrupt is prohibited.

5-3 ISRA/ISRB (In-Service Registers A/B)

ISRA/ISRB are registers that indicate the corresponding interrupt is being serviced (processed). When set to '1', they indicate that the interrupt is being processed to the CPU by the MFP (Multi-Function Peripheral) and that the corresponding bit is written to the ISRA/ISRB register. When the bit is set to '0', it indicates that the corresponding interrupt has finished processing and the bit in the ISRA/ISRB register is written as '0'.

MFP can be programmed with firmware to execute automatic End Of Interrupt (EOI). In this state, when the signal is passed to the CPU, it means that the service ends. Even if the corresponding interrupt bit is promptly initialized, the ISRA/ISRB registers indicate that the interrupt service has ended.

ISRA/ISRB being set to '0' indicates that the IPRA/IPRB are '1', and the MFP requests the clearing of the corresponding interrupt bits. In automatic EOI mode, setting this bit clears the request, allowing the change to cascade interrupt requests.

You can configure whether the MFP runs in automatic EOI mode or in vector register mode. For details, refer to the explanation of the vector registers.

Page 85

5.4 IMRA/IMRB (Interrupt Mask Registers A/B)

These registers control the interrupt mask. When the bit is '1', the interrupt can occur. The IERA/IERB registers are quite similar. The difference between the two is whether to prevent interrupts from occurring ('0' setting). When a new interrupt occurs while it is being processed, the MFP's response is delayed. When the IERA/IERB is set to '0', this

interrupt request is ignored. Even when the IMRA/IMRB bit is set to '1', if the IERA/IERB bit is not set to '1', the MFP will not accept the interrupt request, setting the corresponding IPRA/IPRB bit to '1'. When the IMRA/IMRB bit is set to '1', the CPU acknowledges the interrupt.

If the interrupt request source is suppressed by the IERA/IERB, it will be suppressed, making it easy to understand if thought of as merely suppressing the interrupt request from the MFP.

5.5 Vector Registers

When the MFP raises an interrupt request to the CPU, it sends out a signal to set the vector number. The output 8-bit vector number is set from the upper 4 bits to the lower 4 bits of the register. The lower 4 bits of the vector denote the priority level of the MFP's interrupt, set between '1111' indicating the highest priority and lower numbers continuing accordingly, with the lowest priority being GPIP 0 as '0000'.

The S-bit of the vector register determines whether the EOI (End of Interrupt) is in software EOI mode or automatic EOI mode.

When this bit is set to '1', it becomes software EOI mode. After receiving an interrupt and upon completion of EOI processing (which sets the corresponding bit in the ISRA/ISRB register to '0'), it indicates that the interrupt service is ongoing.

Setting the S-bit to '0' puts it in automatic EOI mode, clearing the ISR upon acceptance by the CPU, thus ISRA/ISRB no longer holds each bit.

6 Timer

MFP has four timers ranging from Timer A to Timer D. Among these, Timers C and D can only function in Delay Mode, which divides a simple input frequency by 1/N. However, Timers A and B use dedicated input terminals (TAI/TBI), enabling interval measurements (pulse width measurement mode) and the counting of the number of changes in the input waveform (event count mode).

6-1 Timer Operation Modes

The outline of the operation modes for MFP timers is shown in the diagram on page 88. The diagram shows an 8-bit counter that can be read and written by the CPU. By accessing this counter, the timer's operation status can be checked, and interval measurements and event counting can be performed in a short time. Let's explain the names of these operation modes.

6-1-1 Delay Mode

Delay Mode is used for purposes where it receives any input frequency and generates a divided-period signal. When the counter is programmed in Delay Mode, the MFP divides the 8-bit counter by the value written by the CPU. The term ”prescaler” refers to the fixed ratio division of the input frequency. MFP allows selection from the following prescaler ratios: 1/4, 1/10, 1/16, 1/50, 1/64, 1/100, and 1/200. For example, on the X68000, the user typically uses a 4 MHz clock as the input to the prescaler. Suppose the prescaler divides it by 1/100; the resulting frequency will be 40 kHz (4 MHz/100 = 40 kHz). Each time the clock pulses, the 8-bit counter decreases its value by 1. When the value becomes $01, the clock

pulse creates a divided signal, further triggering the output of the TAO/TBO/TCO/TDO terminal. This 8-bit counter then reloads the initial value set in the Timer Latch Register and starts counting again. Thus, the fixed division by the prescaler and the loaded value in the Timer Latch Register determine the final divided frequency.

● Diagram 7 - Various Operation Modes of the MFP Timer

Timer Mode

● Prescaler XTAL (4 MHz)

● 8-bit Counter

● Divided by 2

TAO/TBO

TCO/TDO

TAI/TBI

AER* or GPIP4/GPIP3 *Input possible

Delay Mode

● Prescaler XTAL (4 MHz)

● 8-bit Counter

● Divided by 2

TAO/TBO

Count Start/Stop

TAI/TBI

AER* or GPIP4/GPIP3 *Input possible

Pulse Width Measurement Mode

● 8-bit Counter

TAI/TBI

AER* or GPIP4/GPIP3 *Input possible

Event Count Mode

Prescaler: Divider ÷4, ÷10, ÷16, ÷50, ÷64, ÷100, ÷200 + selectable

Moreover, For example, choose 1/100 for the prescaler and set the Timer Data Register to 400, then the 8-bit Counter output is 4 MHz / (400 x 100) = 100 Hz, causing the timer to interrupt every 10 ms. The timer output terminals will flip at this interval, so the actual frequency will be 50 Hz, which is half of this.

There are control inputs for TAI and TBI on Timer A and Timer B, but they are not used in this mode.

Page 88

6-12 Pulse Width Measurement Mode

Pulse width measurement mode is a mode that measures the pulse width (the duration of the signal with or without H level) of the input signal by having the TA1/TB1 input trigger the timer to run. In this mode, Timer A and Timer B can both be used. On the X68000,

the TBI terminal is fixed to L level at all times, so in practice, only Timer A can be used in this mode.

In pulse width measurement mode, setting the timer start/stop to TA1 and TB1 input will start/stop the timer, respectively. Moreover, the CPU can be interrupted upon the completion of measurement. Depending on the H or L level at the time of measurement completion, it will either start counting from AER or assign an interrupt. The interrupt promotion pin either becomes Timer A's GPIP4 or Timer B's GPIP3. This means that the AER completion assign function of Timer A corresponds to GPIP4 and that of Timer B corresponds to GPIP3. In this way, by making the timer's count completion interrupt take over the GPIP4 or GPIP3 assign functions, the feature serves to support the pulse width measurement feature with Timer A.

AER is established to allow the CPU to interrupt when the TA1/TB1 input becomes H-level or L-level without starting the timer. Normally used with GPIP input, assigning promotes an interrupt on the changing from 'L' to 'H'. In pulse width measurement mode, an interrupt will occur when '1' becomes '0' or when 'H' changes to 'L', so pay attention to these settings.

Furthermore, during pulse width measurement mode, the basic timer's start/stop control will be controlled by an external signal, which is equivalent to the delete mode. Thus, when the count register holds $01, it increments the timer count and generates an interrupt.

When the measurement is complete, re-conducting the pulse width measurement will rewrite the values written into the timer compare register by the CPU. At this time, confirm that the control signals (TA1/TBI) are active (AER is neither ‘H’ level nor ‘0’ level). Active control signals may cause improper interrupt conduct due to incomplete count loading.

6-1-3 Event Count Mode

This mode is available exclusively for Timer A and Timer B. Event Count Mode is a mode that operates an 8-bit counter by using the inputs TAI and TBI as clocks (naturally, prescalers are not used). Counting in any direction of input changes is done similarly to pulse width measurement mode and is performed on GPIP 4/GPIP 3 of AER. After the data of the counter is reset by a software reset (SO), the count pulse occurs, and it causes Timer A/Timer B interrupt requests to the CPU and recalibrates the value of the Timer A/Timer B counter registers. In X 68000, TAI and V-DISP signals are also valid, and in Human 68 K, Timer A is used in event count mode.

6-2 Registers Related to Timers

The registers for controlling timers are divided into timer control registers, for setting the operational modes of the timers, and timer counter registers for reading/writing the 8-bit counter values. In this way, under certain conditions where Timer C and Timer D act only in delay mode, the control registers will overlap with a single register.

6-2-1 Timer A/Timer B Control Register

The Timer A and Timer B control registers are shown in the configuration diagram. The lower 4 bits indicate and specify the operational mode of the corresponding timer. When these 4 bits are '0000', the timer stops functioning; when '1000', it enters event count mode. If the lower 3 bits are not '000', then it is in delay mode if bit 3 is '0', and pulse width measurement mode if '1'. In delay mode and pulse width measurement mode, you can select the prescaler division by the lower 3 bits.

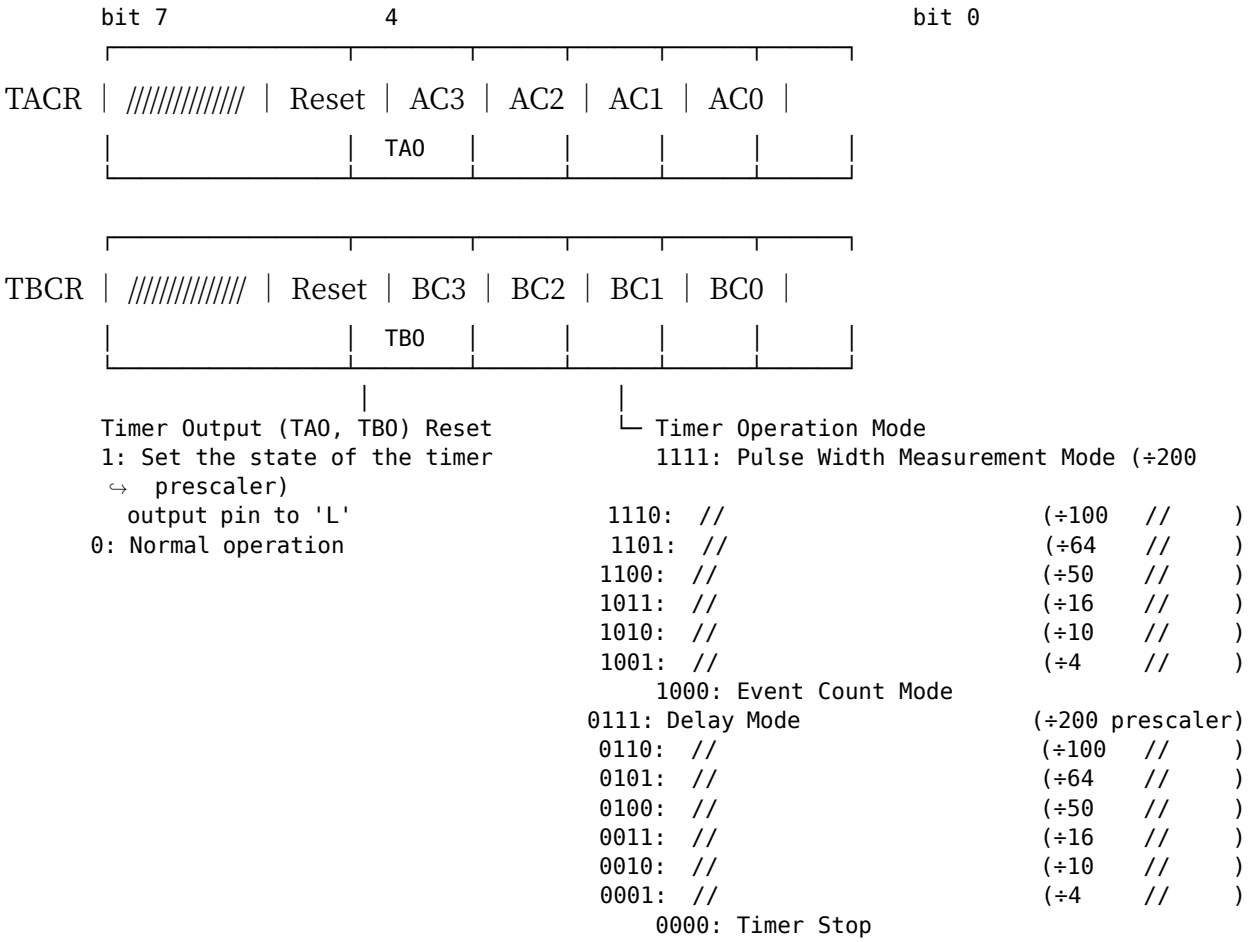
bit 4 is for forced clearing of the Timer output terminals TAO/TBO, so these bits must be

set to '1' to forcefully clear the current output state of the Timer output terminal to '0'. This functionality to clear the output state for the following 8 clock cycles after being written by the CPU is useful.

Page 90

MFP

● Fig. 8 TACR, TBCR ($E88019, $E8801B)



When a count-up pulse arrives from the counter, the output is inverted just

as in normal operation. You can think of this as being provided for cases where you want to start operation from the state in which the timer output is 'L'.

● 6.2 Timer C & D Control Register

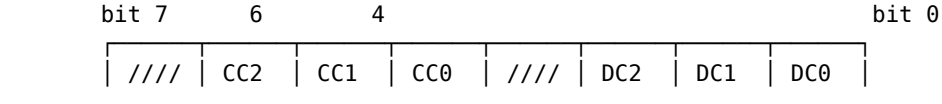
The bit layout of the register that controls Timer C and Timer D is shown in

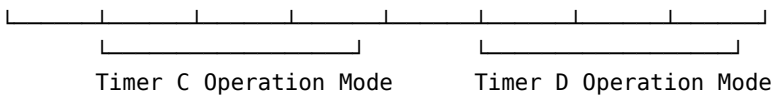
Fig. 9 on page 92. The operation mode of Timer D is selected with bits 0 to 2, and the control of Timer C is performed with bits 4 to 6.

When all 3 of these bits are '0', timer operation is stopped. For any other

value, Timer C and Timer D operate in delay mode, and the prescaler division ratio is selected by the 3 bits. This setting is the same as the case where the most significant bit of the mode setting in the Timer A/Timer B Control Register is always '0'.

● Fig. 9 TCDCR (Timer C & D Control Register)






```
111: Delay Mode    (÷200 prescaler)
110:  //           (÷100   //      )
101:  //           (÷64    //      )
100:  //           (÷50    //      )
011:  //           (÷16    //      )
010:  //           (÷10    //      )
001:  //           (÷4     //      )
000: Timer Stop
```

● 6.2.3 Timer Data Register

```
One timer data register is provided for each timer, used to read/write the
```

timer value. The timer decrements each time a count pulse arrives, and when it reaches $01, the value set in the timer data register is automatically reloaded on the next pulse.

● 7 USART (Serial Port)

```
The USART (Universal Synchronous/Asynchronous Receiver/Transmitter) built
```

into the MFP is a general-purpose serial interface that supports both full-duplex synchronous and asynchronous communication. On the X68000 it is defined to be connected to the keyboard, so it is set to match the keyboard's transmission mode (asynchronous, 2400 bps, 1 start bit, 8 data bits, no parity, 1 stop bit). Furthermore, on the X68000 the USART transmit clock is obtained from Timer B, and since it cannot be synchronized to external data, no clock mode other than 1/16 can be selected.

In this way, the X68000 has little to no room for mode selection, but it would be good to have knowledge of its functions to deal with any situation. Let's explain the functionalities of USART in MFP.

1 SCR (SYNC Character Register) In synchronous transmission mode, the USART waits until the data written in SCR is received. When sending data, the USART sends the data written in SCR automatically when the antenna line is free. Setting anything other than valid SYNC characters in SCR will send nothing. Writing 0 bits (the data length - 1) is not enough. Note that asynchronous transmission requires writing a parity-bitted data to SCR.

In the X68000, the USART is used in non-synchronous mode, making SCR irrelevant and disregarded.

2 UCR (USART Control Register) This register decides the mode of operation for USART. The positioning of UCR bits is shown in Figure 10 on page 94.

1 CLK The communication speed is determined by either the same frequency as the input clock or 1/16 the input clock frequency. Setting '1' results in 1/16 and setting '0' matches the input clock frequency.

In the case of 1/16, the USART will automatically find and sample the start bit of the transmitted data, extracting it from the middle of the data bit. This mode, used for communication with modems and computers, also handles keyboard communication on the X68000 in this mode. The input clock is 2400 bps X 16 = 38400 Hz.

If the transmit/receive clock frequency and input clock frequency are the same, the USART samples data without conditions at each clock tick.

● Fig. 10 UCR (USART Control Register) $E88029

bit 7 bit 0 | CLK | WL1 | WL0 | ST1 | ST0 | PE | E/O | |

```
                                  1: Even parity
                                  0: Odd parity

                              1: Parity enabled
                              0: Parity disabled

              (Sync mode) (Start bit length) (Stop bit length)
11            Asynchronous        1             2
10*           //                  1             1.5
01            //                  1             1
00            Synchronous         0             0
```

Data length setting: 11: 5 bit 10: 6 bit 01: 7 bit 00: 8 bit

```
*: Settable only when CLK bit is '1'
```

1: Transmit/receive rate equals 1/16 of the input clock frequency 0: // equals the same

loading the data, so if the data and clock are not completely synchronized the data will become corrupted. For this reason, when this mode is selected the clock must be connected together with the data, and an external circuit is needed to generate a clock synchronized to the received data. On the X68000, the clock is connected to timer B of the MFP, so this mode cannot be selected.

7·2·2 WL

Sets the data length per character. '00' gives 8 bits, '01' gives 7 bits, '10' gives 6 bits, and '11' gives 5 bits. On the X68000 the keyboard data length is 8 bits, so normally '00' should be set.

7·2·3 ST1, ST0

Selects the start bit and stop bit lengths and the synchronous/asynchronous mode. Setting '00' selects synchronous mode, making both the start bit and stop bit lengths 0. In cases other than '00',

MFP

is in asynchronous mode. Among these, the setting value ”10”, in other words start bit 1, stop bit 1.5 bit mode is only configurable when CLK is 1 or (1/16 mode). For the X68000 keyboard, since both the start bit and the stop bit are 1 bit, this bit is to be set to '01'.

1-4 PE

Select whether to enable or disable parity. Setting ”1” enables parity. On receiving, a parity check will be performed. On transmission, parity will be added to the data. However, please note that for SYNC characters of 8 bits or less, even if PE is ”1”, parity bit will not be added (data will always be prefixed).

1-5 E/O

Select whether parity is even or odd. '1' is for even parity, '0' is for odd parity.

1-3 RSR (Receiver Status Register)

RSR is a register that reads receiver status and controls enable/disable of the receiver. Bit arrangement of RSR is shown in Figure 11 on page 96.

1-3-1 BF

BF (Buffer Full) bit indicates whether the receive buffer contains data. When there is data in the receive buffer, it becomes '1', and UDR (USART Data Register) reads the data from the buffer, making it '0'.

● Fig. 11 RSR (Receiver Status Register) $E8801B

bit 7 bit 0 | BF | OE | PE | FE | F/S or B | M/CIP | SS | RE |

```
                                        1: Receiver Enable
                                        0: Receiver Disable

                                  1: Also takes in characters that match the SCR register
                                   ↪  contents
                                0: Does not take in characters that match the SCR register
                                   ↪  contents

                          · Synchronous mode:
                          1: The word that entered the receive buffer matches the SCR
                           ↪  register contents
                          0: The word that entered the receive buffer does not match the SCR
                           ↪  register contents
                          · Asynchronous mode:
                          1: Start bit detected
                          0: Stop bit detected

                    · Synchronous mode:
                    Writing '0' enters word search mode (data matching the SCR register
                     ↪  contents becomes '1' when received)
                    · Asynchronous mode:
                    1: Break detected (all data with no stop bit is '00')
                    0: Not in break state

              1: Framing error occurred (stop bit not found)
              0: Normal operation

        1: Parity error occurred
        0: Normal operation

  1: Overrun error occurred
  0: Normal operation
```

1: Data is in the receive buffer 0: Receive buffer is empty

7·3·2 OE

OE (Overrun Error) occurs when data that has entered the receive buffer has not been fetched by the CPU and the next data arrives. The newly arrived data is discarded.

OE bit is set to '1' after an overrun error occurs and data in the receive buffer is read out, and it is reset to '0' after reading the RSR register.

7•3•3 PE

PE (Parity Error) occurs when the parity calculated from the received data does not match the received parity. When an error occurs it becomes '1', and when error-free data is received it becomes '0'.

7•3•4 FE

FE (Framing Error) is only valid when in asynchronous mode. After receiving a non-$00 data, if the stop bit is not found, a framing error occurs, and the FE bit is set to '1'. It is reset to '0' when normal data transmission can be received.

7•3•5 F/S or B

F/S or B (Found/Search or Break) bits change their function depending on whether they are in synchronous or asynchronous mode.

In synchronous mode, it enters word search mode by writing '0', and waits until data that matches the content of the SCR register is received. If data identical to the SYNC character is received, the CPU recognizes '1' and issues a receive error interrupt.

In asynchronous mode, when checking for a break condition in which the data line remains '0', it issues a '1' status bit. The break condition can be defined as a state where the data line remains '0' with no stop bit found, or as $00 data in non-asynchronous mode (when no stop bit is found at the edge of non-$00 data).

F/S or B bits are reset to '0' when non-$00 data is received and the RSR is read out.

Note: ”$00” refers to a specific hexadecimal data value.

7-36 M/CIP

The M/CIP (Match/Character in Progress) bit changes functionality depending on whether it is in synchronous mode or asynchronous mode. In synchronous mode, it becomes '1' when a character matching the SYNC character enters the reception buffer and '0' when a non-matching character enters the reception buffer. In asynchronous mode, it becomes '1' when a start bit is detected and '0' when a stop bit is detected.

7-37 SS

The SS (Synchronous Stop) bit determines whether to receive the SYNC character. If the SS bit is set to '0', then data that matches the SYNC character will not enter the reception buffer.

7-38 RE

The RE (Receiver Enable) bit controls the enable/disable of reception operations. Setting the RE bit to '0' stops the reception operation, and all status bits in the RSR are set to '0'. Setting the RE bit to '1' enables the reception operation, but only when the reception clock is not stopped.

7-4 TSR (Transmitter Status Register)

Figure 12 shows the TSR bit allocation. TSR sets the communication state and transmission operation mode registers.

Diagram 12 (Transmitter Status Register)

bit 7	bit 0
BE	UE

bit 7: BE

1: Transmission buffer is empty 0: Transmission buffer is not empty

bit 6: UE

1: Under-run error occurred (data to be transmitted was not written) 0: Normal operation

bit 5: AT

1: Transmission is disabled 0: Transmission is enabled

bit 4: END

1: Automatic loopback clears error and resumes transmission 0: Loopback

bit 3: B

1: Transmission data register is full 0: Transmission data register is not full

bit 2: H

1: High 0: Low

bit 1: L

Loopback mode 11: Loopback mode 10: TE=”0” and S0 terminal=High 01: // = Low 00: // = High-impedance

bit 0: TE

1: Transmission is enabled 0: Transmission is disabled

bit 7 - BE

This bit indicates that the transmission buffer is empty. When the transmission buffer is empty, the BE bit is set to 1. When data is written to the UDR (USART Data Register), the BE bit is cleared to 0.

bit 6 - UE

The UE bit indicates that an under-run error occurred when the data was not written to the transmission buffer. This error occurs when the transmission is finished, and no data has been written to the buffer. The TE bit automatically clears the error and resumes transmission.

or when the TSR register is read, the UE bit is cleared.

7·4·3 AT

When the AT (Auto-Turnaround) bit is '1', the receiver is automatically enabled at the moment transmission of the last data ends. When transmission ends, this bit automatically becomes '0'.

7·4·4 END

If the transmitter is disabled (TE set to '0') while data is being transmitted, the END (End of transmission) bit becomes '1' at the moment data transmission ends. When the transmitter is enabled, the END bit returns to '0'.

7·4·5 B

The B (Break) bit is valid only in asynchronous mode. In asynchronous mode, setting the B bit to '1' puts the transmit data line into the break state after the data currently being transmitted has finished transmitting. Setting the B bit to '0' cancels the break state and returns to normal operation. While this bit is '1', the BE bit will never become '1'.

7·4·6 H, L

The H, L (High/Low) bits determine the state of the transmit data line when the transmitter is disabled. At '00' it is high impedance, at '01' it is the 'L' level, and at '10' it is the 'H' level. '11' is a little special: it enters a kind of self-diagnostic mode called loopback mode. In this mode, the receive data line and receive clock line are connected internally in the MFP to the transmit data line and transmit clock line, and a loopback test is performed in which the transmitted data is received as is. Normally it is advisable to set these bits to '10'.

TE

TE (Transmit Enable) bit controls the permission/prohibition of transmission operation. When the TE bit is set to '1,' the transmission operation becomes enabled, allowing data to be sent.

UDR (USART Data Register)

UDR is a register that performs data transfer. Data written here is transmitted using the transmission line, and received data is received by the CPU through this register.

Initial Setting of MFP

The list of setting values for each MFP register is shown on page 102, Figure 13. The bits that are '1' or '0' are fixed at those settings, P indicates bits that can be changed, or special-purpose bits, and indicates bits that are either always '1' or '0' depending on the condition.

The system setting data is the setting value read out after starting the Human 68K. Since the timer register changes, the maximum value at the time of continuous reading for 100,000 times is recorded in the table.

Figure...13 MFP Setting Values

Address	bit 7	bit 0	System Setting Values	Register Name
$E88001	X X X X X X X X	--	GPIP Data Register	
$E88003	0 0 X P X 1 X X	$06	Active Edge Register	
$E88005	0 0 0 0 0 0 0 0	$00	Data Direction Register	
$E88007	P P P 1 1 P P 0	$18	Interrupt Enable Register A	
$E88009	P 0 P 0 1 0 0 0	$3E	Interrupt Enable Register B	
$E8800B	X X X X X X X X	(no value)	Interrupt Pending Register A	
$E8800D	X X X X X X X X	(no value)	Interrupt Pending Register B	
$E8800F	X X X X X X X X	(no value)	Interrupt Service Register A	
$E88011	X X X X X X X X	(no value)	Interrupt Service Register B	
$E88013	P P P 1 1 P P 0	$18	Interrupt Mask Register A	
$E88015	P 0 P 0 1 0 0 0	$3E	Interrupt Mask Register B	
$E88017	P P P P P X X X	$40	Vector Register	
$E88019	0 0 0 1 0 0 0 0	$08	Timer A Control Register	
$E8801B	0 0 0 0 0 0 0 1	$01	Timer B Control Register	
$E8801D	D P P P P P P P	$77	Timer C Control Register	
$E8801F	P P P P P P P P	$01	Timer A Data Register	
$E88021	0 0 0 0 1 1 0 1	$0D	Timer B Data Register	
$E88023	P P P P P P P P	$C8	Timer C Data Register	
$E88025	P P P P P P P P	$14	Timer D Data Register	
$E88027	0 0 0 0 0 0 0 0	$00	SYNC+ Character Register	
$E88029	1 0 0 0 1 0 0 X	$88	USART Control Register	
$E8802B	X X X X X X 0 P	$01	Receiver Status Register	
$E8802D	X X P X P 1 0 P	$81	Transmitter Status Register	
$E8802F	X X X X X X X X	(no value)	USART Data Register	

- X...To be rewritten to user/desirable data as needed
- P...Can be set according to necessity

● Numerical Calculation ● Processor

A numerical calculation processor is an LSI that performs floating-point arithmetic, and can significantly improve processing speed for applications with many real-number calculations such as ray tracing. Here, we will explain the specific usage methods of the numerical calculation processor.

● 1 Overview

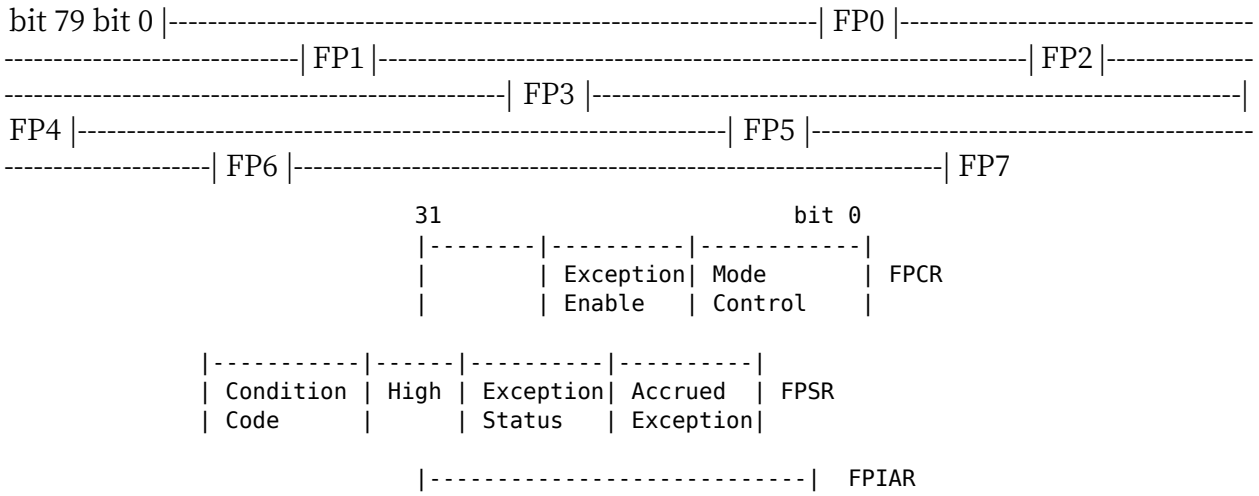
A numerical calculation processor handles numerical computations, particularly improving the floating-point arithmetic with which CPUs generally struggle, through a high-speed dedicated LSI. In the X68000, a numerical calculation processor from the 68000 family, the MC68881, is equipped to accomplish this goal. The physical form factor is a card that installs in an XVI dedicated expansion slot (CZ-6BP1), same as with previous models, and while it appears different, it is uniform in function from a software perspective.

The 68881 was initially designed to be used in conjunction with the 68020 as a coprocessor, but it is now configured to work with other supported CPUs like the 68000. On the X68000, without the 68881, one would have to handle the instructions via software, which is inefficient, but the presence of the 68881 automates this process. For the X68000, operating without automatic handling of these functions requires significant effort. Fortunately, the 68881 has been integrated to relieve these burdens. This section aims to explain in detail how to leverage the capabilities of the 68881, methods for handling operations, and the differences in specific commands for the numerical calculation processor.

● 2 Internal Registers of the 68881

A list of the internal registers of the 68881 is shown in Fig. 1. Access to these registers is performed strictly through arithmetic instructions, data transfer instructions, and the like; note that they are not directly mapped to any address as seen from the CPU.

● Fig. 1 Internal Registers of the 68881



FP0 ~ FP7: Floating-point Data Registers FPCR: Control Register FPSR: Status Register FPIAR: Instruction Address Register

Numeric Arithmetic Processor

2·1 FPn

The eight 80-bit registers FP0 through FP7 are floating-point data registers. The arithmetic processing of the 68881 is performed using these registers. All eight registers are completely equivalent; no single register is treated as special. You can think of them as corresponding to the CPU's data registers (D0~D7).

2·2 FPCR, FPSR, FPIAR

FPCR controls the enable/disable of exceptions generated by the 68881 and specifies the rounding processing of operation results. The function of changing the rounding precision to single or double precision via the PREC bit of the FPCR (Fig. 2) is provided strictly

to maintain compatibility with other computers that perform operations in extended precision; the operation speed also drops compared to extended precision, so normally there is no need to specify anything other than extended precision.

FPSR stores status useful for analyzing the situation and post-processing when an operation error or overflow occurs, as well as the quotient data of division instructions.

The condition code byte of FPSR is set at the end of an arithmetic instruction; it becomes '1' when each condition shown in Fig. 3 on page 107 is satisfied.

The quotient byte is set when a modulo (FMOD) instruction or an IEEE remainder (FREM) instruction is executed.

The exception status byte is used to indicate exception states such as errors or overflows that occurred in the most recently performed floating-point operation or data transfer.

The accrued exception byte contains the five types of exception bits specified by the IEEE. These bits are generated from the exception status byte; however, whereas the exception status byte is set/reset on each operation, the accrued exception byte accumulates the conditions that occurred by ORing them together. Because of this, by clearing the accrued exception byte before a series of operations and simply checking afterward whether it remains in that state, you can determine whether the entire series of operations was performed without any problems.

FPIAR holds the address of the last floating-point instruction executed. This register is directly tied to the 68020 and is also used to resume operation processing that was interrupted by an interrupt after the interrupt processing has finished. As in the case of the X68000, when the 68881 is connected as an I/O device this register has little meaning, but it can still be accessed.

The details of FPCR and FPSR are shown in Fig. 2 and Fig. 3 respectively, for your reference.

● Fig. 2 FPCR (Control Register)

bit 31 16 15 8 7 bit 0 | | Exception Enable | Mode Control |

bit 15 bit 8 | BSUN | SNAN | OPERR | OVFL | UNFL | DZ | INEX 2 | INEX 1 |

```
Unordered branch/set ------- BSUN
Signaling NaN ------------- SNAN
Operand error ------------- OPERR
Overflow ----------------- OVFL
Underflow ---------------- UNFL
Division by zero --------- DZ
Inexact operation -------- INEX 2
Inexact decimal input ---- INEX 1
```

bit 7 4 3 bit 0 | PREC | RND | '0' | (always set to '0')

(Rounding Precision) (Unused) ---------- 11 Double precision --- 10 Single precision --- 01 Extended precision - 00

(Rounding Mode) Round toward plus infinity ----- 11 Round toward minus infinity ---- 10 Round toward zero ---------- 01 Round to nearest ---------- 00

Processing

(Floating Point Processor) ■ Figure 3 FPSR (Status Register) bit 31 24 23 16 15 8 7 0

Condition Code Sign Exception Status Accrued Exception

bit 31 28 27 24 ‘0’ N Z I NAN

Negative Zero Infinite Not a Number (NAN)

bit 23 22 16 S Sign

bit 15 8 7 0

BSUN SNAN OPERR OVFL UNFL DZ INEX2 INEX1

Unordered Signaling NAN Operand Error Overflow Underflow Division by Zero Inexact

bit 7 3 2 0 '0' N

Invalid Operation Overflow Underflow Division by Zero Inexact

3

Data format that the 68881 can handle

The data formats that the 68881 can handle with external interactions are shown in Figure 4.

The 68881's internal calculations are done at extended precision, and the floating-point data register data is stored internally at the specified precision via the FPCR. The specified precision is maintained internally.

◎ Figure 4: Data formats that 68881 can handle

Byte Integer (B)

Word Integer (W)

Longword Integer (L)

Single-precision floating point (S)

Double-precision floating point (D)

Extended-precision floating point (X)

Packed-decimal (P)

± Maximum or NAN (Not A Number) when used. Usually '0'.

Sign of exponent

Sign of mantissa

3-1 Decimal Data Format

The format for decimal data such as bytes, words, long words, etc. is entirely the same data that can be handled by the 68000 CPU, so it goes without saying that it should be familiar.

The format for various decimal points, particularly single-precision, double-precision, and extended-precision formats, all conform to IEEE standards. However, in the case of extended precision, it's 80 bits with an extra 16 bits compared to 64 bits, which is typical. For the 68881, this makes the size of the data in memory different.

To express decimal numbers in base-10, as described in 7.2.10, two's complement notation is used. IEEE also uses this two's complement notation for decimal values. For base-10 integers, they range from 1 to 9. For base-2 integers, they only go up to 1. For fractional parts, if 1 is treated as a decimal point, it is expressed as 1.fx.

In the case of single-precision and double-precision formats, ignoring the integer part simplifies the data and results in a more convenient format. In extended-precision format, the upper part of a 64-bit number is essentially treated as the integer part, with the lower part referred to as the fractional part.

The distinction between integer and fractional parts does exist, but at any given value of the data, the value scales appropriately. For example, in a single-precision format, the specified fractional part might be an integer value like 127 (0x7F). This value is incremented appropriately. If the fractional part is 2^-2, the specified data will be 0x80 or specified as 0xC0 by 2^-3.

The expression of decimal formats, presented in summary on page 110, ought to be referred to for further understanding. Typical floating-point notation shows non-normalized values less frequently. Anti-gravity denotes specific low-value representations. If the fractional part is zero, the entire data is zero, i.e., representing it with zero, indicating its default initialization.

In summary, both the regular and extended decimal formats should be treated such that the integer part initializes similarly to typical integer formats, as annotated.

Figure 5. Summary of Floating Point Formats

`se | f` Single Precision / Double Precision 1: Negative number 0: Positive number

`seu | f` Extended Precision

Not Used Bits

Field Bit Length	Single Precision	Double Precision	Extended Precision
`s`	1	1	1
`e`	8	11	15
`f`	23	52	63
Sum	32	64	96

Expression of Normalized Numbers and Denormalized Numbers:

- Normalized number expression: `(-1)^s * 1.F * 2^(E-127)`, `(-1)^s * 1.F * 2^(E-1023)`, `(-1)^s * 1.F * 2^(E-16383)`
- Denormalized number expression: `(-1)^s * 0.F * 2^(-126)`, `(-1)^s * 0.F * 2^(-1022)`, `(-1)^s * 0.F * 2^(-16382)`

Value Range	Single Precision	Double Precision	Extended Precision
Maximum	3.4 × 10^38	1.8 × 10^308	6 × 10^4933
Minimum	1.2 × 10^-38	2.2 × 10^-308	2 × 10^-4933
Denormalized Minimum	1.4 × 10^-45	4.9 × 10^-324	9 × 10^-4952

Please be careful when expressing numerical values using fractions.

3.2 Special Floating-Point Data

When performing floating-point arithmetic, special cases such as storage underflows and mathematical overflows may occur even without problems. In such cases, the 68881 may have limited expression capabilities, resulting in undefined or invalid results. These are divided into invalid and valid results, as previously explained in section 3.6.

The final NAN, Not A Number, is returned when performing mathematically meaningless operations such as ∞ + -∞ or undefined operations, indicating that the 68881 was unable to compute the result numerically.

3.3 68881 Internal Data Format

The precision during intermediate calculations within the 68881 is in the format shown in Figure 7. To prevent precision loss from repeated calculations, the mantissa part extends to 64 bits, making the total bit length 67 bits.

(Page 110)

Diagram 6: Special Real Number Data Formats

Minimum and Maximum Values Arbitrary Bit Pattern

Normalized Number (normal)

All bits except 0

Denormalized Number (Value close to underflow)

Zero

Maximum Value

Infinity

All bits except 0

NAN

Digits Sign 1: Negative 0: Positive (Notes: During expanded compatibility format, the maximum bit of the fraction (integer part) can be either '1' or '0')

Diagram 7: Format During Computation in 68881

Digits (17 bits)

Overflow Bit Decimal Part (63 bits)

Integer Bit Least Significant Bit of the Decimal Part Sticky Bit Rounded Bit Guard Bit

To simplify the detection of overflow and underflow during execution of arithmetic instructions, the number of digits reserved is 17 bits.

4 Interface With 68881

Refer to page 112, Diagram 8 for the register list needed for communication between 68881 and X68000.

indicates this. These registers are defined by the CIR (Co-Processor Interface Register) specification that the 68020 uses to perform coprocessing and to communicate with various coprocessors such as the math coprocessor and the memory management unit (some unnecessary registers are omitted).

When the 68881 is directly connected to the 68020, the 68020 CPU automatically handles the exchanges with these registers, so the program does not need to be aware of the existence of the registers. However, in the X68000 the 68881 is connected as an I/O device, so the CPU must instead control these registers in software. This inevitably reduces operation performance somewhat compared to a direct connection, but in practical use it rarely becomes a problem. Furthermore, because the 68881 is treated as an I/O device,

it is also possible to control multiple 68881s simultaneously. On Sharp's genuine math coprocessor board CZ-6BP1, two types of addresses can be selected via pin settings. The register addresses are $E9E000–$E9E01F in the standard setting; by changing the pin settings they become $E9E080–$E9E09F (of the two, the floating-point arithmetic driver used with Human68K supports only the standard setting).

● Fig. 8 CIR (Co-Processor Interface Register)

bit 31 16 15 bit 0 +$00 | Response CIR (R) | Control CIR (W) | +$02 +$04 | Save CIR (R) | Restore CIR (R/W) | +$06 +$08 | (Operation Word CIR) | Command CIR (W) | +$0A +$0C | | Condition CIR (W) | +$0E +$10 | Operand CIR (W) | +$14 | Register Select CIR (R) | Instruction Address CIR (W) | +$18 | (Operand Address CIR) | +$1C

Base Address:

```
$E9E000 (Standard)
$E9E080 (For 2nd board)
```

(R) : Read only (W) : Write only (R/W) : Read/Write possible

4.1 Response CIR

The 68881's Response CIR is used to indicate its own operation status or a service request by the host CPU. The Response CIR can be read from the host CPU. The host CPU checks this register value (called Preemptive) to ensure synchronized operation with the 68881. Details of the Response CIR contents will be explained later.

4.2 Control CIR

In accordance with the 68020 coprocessor interface, the Control CIR is used by the host CPU to instruct the coprocessor for execution report commands like FSAVE and FRESTORE. The 68881 receives and accepts all write commands to this register as abort commands. When written to this register, the 68881 suspends its current processing, clears any pending interrupts and data, and transitions to an idle state. If, for example, an exception occurs within the 68881, the host CPU writes to this register to return from the abnormal state.

4.3 Save CIR

The Save CIR is a register intended to read the internal status of the 68881, which is not accessible by software. In multitasking OS environments, where multiple tasks might utilize the 68881, things can get confusing when switching tasks if the 68881 does not execute the command of the task at the correct state. To avoid such situations, the current state is saved to memory, then restored sequentially. This necessity leads to the FSAVE and FRESTORE instructions. The Save CIR is set up for reading the status of 68881's current processing operation. The host CPU reads the data and retrieves the necessary information for required operations.

4-4 Restore CIR

This is a register for executing the RESTORE instruction. The host CPU reads the data (state frame) initially sent to this register when it reads from CIR. When data is written to this register, the 68881 checks the format of the given state frame and begins the restore operation. The host CPU then writes the remaining data to the 68881 and resumes the previously halted operation. If the format is incorrect, the host CPU writes to the control CIR and puts the 68881 into idle mode.

4-5 Operation Word CIR

The 68881 does not use this register. Any write to this register is ignored.

4-6 Command CIR

The host CPU uses this to write instructions to the 68881. This starts the write to all registers with various operation and data transfer instructions. Details of the commands will be explained later.

4-7 Condition CIR

When connected directly to the 68020, this register is used for processing commands like conditional branch instructions based on conditions (e.g., equal, greater than, less than) during operations. On the X68000, since the 68881 is connected via I/O devices, this CIR is used for condition checks (like equal, greater, less, etc.).

4-8 Operand CIR

This is used for data transfer between the host CPU and the 68881. Passing and receiving floating-point data are performed through this register.

4.9 Register Selection CIR

When executing a multiple floating-point data register transfer command (”MOVEM command”), it is used to pass the register mask from the host CPU. By counting the number of data read by the host CPU, it is possible to know the number of registers to be transferred.

4.10 Command Address CIR

It is used when the PC hit is set by the response CIR, and the host CPU obtains the value of the PC (Program Counter). When the 68881 is executing a command, an interrupt may be generated. In this case, the current PC value is passed to the 68881. When used as an I/O device like the X 68000, it would probably not be used in such a circumstances. You can ignore the write request here.

4.11 Operand Address CIR

68881 does not use this CIR. All accesses are ignored.

5 Response Primitives

The general format of response primitives is shown in Figure 9 on page 116. The CA bit indicates that the 68881 is executing some service request. The PC hit indicates that the host CPU is required to fetch the value of the PC (Program Counter) according to the 68881's requirement. This is ignored when used as an I/O device like the X 68000. The 68881 itself also considers such usage assumptions and ignores these requests. The DR indicates the data transfer direction between the 68881 and the host CPU. When it is '0,' it indicates to the host.

Figure 9 68881 Response Primitive Format

bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CA	PC	DR	Function	Parameter												

Function Code:

- 00100: Null Primitive
- 10XXX: Effective Address Evaluation/Data Transfer Primitive
- 01100: Single Main Processor Transfer Primitive
- 00001: Multiple Main Processor Transfer Primitive
- 11100: Command Exception Acquisition Primitive
- 11101: Command Exception Acquisition Primitive

Data Transfer Direction (for Overlapped Data):

- From Host CPU to 68881
- 1: From 68881 to Host CPU

When the function bits are ”00100” (Null Primitive) or ”11100” (Command Exception Acquisition Primitive), set the parameter to '0'.

(The program counter (PC) receives a request (with '1' indicating that there is a request) if the CPU demands a specific service; in such cases, it is marked with '1').

5.1 Null Primitive

A detailed explanation of the Null Primitive is shown in Figure 10. The Null Primitive informs the status of 68881 and indicates synchronization with the host CPU. Combine different Null Primitives and check the corresponding values depicted in the figure for reference. Note that 68881 will not return values other than these combinations.

5.2 Effective Address Evaluation/Data Transfer Primitive

The Effective Address Evaluation/Data Transfer Primitive is used to request the transmission of values of floating-point numbers or control registers from 68881 to the host CPU.

● Fig. 10 Contents of the Null Primitive

bit 15 14 13 12 11 10 9 8 7 2 1 bit 0 | CA | PC | DR | '0' | '0' | '0' | IA | '0' | '0' '0' ... '0' | PF | TF |

```
|    |                        |                             condition-judgment flag
|    |                        68881 internal status         0: true
|    interrupt processing OK   0: instruction executing    1: false
|                              1: idle
```

Value	CA	PC	IA	PF	TF	Contents
$0800	0	0	0	0	0	Response to a write to the Condition CIR (condition = true)
$0801	0	0	0	0	1	(” = false)
$0802	0	0	0	1	0	Indicates that the 68881 is idle
$0900	0	1	0	0	0	Indicates that the 68881 is executing internal processing
$4900	1	1	0	0	0	Request to re-read the response register
$C900	1	1	0	0	0	Same as $8900 except it requests the program counter value

● Fig. 11 Contents of the Effective Address Evaluation / Data Transfer Primitive

bit 15 14 13 12 11 10 7 bit 0 | '1' | PC | DR | '1' | '0' | valid | length |

Value	Corresponding Operation	PC	DR	valid	length	Data to be transferred
$9501/$D501	Floating-point arithmetic instr	X	0	101	$01	byte
$9502/$D502	FMOVE XX, FPm	X	0	101	$02	word
$9504/$D504		X	0	110	$04	longword / single-precision real
$9508/$D508	(OP class: 010)	X	0	110	$08	double-precision real
$960C/$D60C	”	X	0	110	$0C	extended-precision real / packed BCD
---	---	---	---	---	---	---
$B101	FMOVE FPm, XX	0	1	001	$01	byte
$B102		0	1	001	$02	word

Value	Corresponding Operation	PC	DR	valid	length	Data to be transferred
$B104		0	1	001	$04	longword / single-precision real
$B208	(OP class: 011)	0	1	010	$08	double-precision real
$B20C	”	0	1	010	$0C	extended-precision real / packed BCD
----------	----------	----	----	----------	--------	----------
$9704	FMOVE XX, FPcr	0	1	111	$04	control register to be transferred = 4 bytes
$9504	FMOVEM XX, FPcr-list	0	1	101	$04	” 4 ”
$9608	(OP class: 100)	0	1	110	$08	” 8 ”
$960C	”	0	1	110	$0C	” 12 ”
----------	----------	----	----	----------	--------	----------
$B304	FMOVE FPcr, XX	0	1	011	$04	control register to be transferred = 4 bytes
$B104	FMOVEM FPcr-list, XX	0	1	001	$04	” 4 ”
$B208	(OP class: 101)	0	1	010	$08	” 8 ”
$B20C	”	0	1	010	$0C	” 12 ”

The details of this primitive, the correspondence between the primitive values and the 68881 operations, and the correspondence of the data types to be transferred are shown in Fig. 11. This primitive is returned only once at the time of instruction execution; thereafter it changes to the null primitive.

5.3 Single Main Processor Register Transfer Primitive

The format of the primitive is shown in Fig. 12.

This primitive is requested by the 68881 to the main processor when, in a floating-point data register transfer instruction that transfers a register list, a mode is selected that specifies the register list with a data register. When the host CPU is the 68020 and the 68881 is directly connected, the contents of the requested data register are automatically transferred by the CPU; but in the case of the X68000, the written data is merely used next, so the register-number bits hold no real meaning.

● Fig. 12 Contents of the Single Main Processor Register Transfer Primitive

bit 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 bit 0 | '1' | '0' | '0' | '0' | '1' | '1' | '0' | '0' | '0' | '0' | '0' | '0' '0' | register # |

Value	Register #	Register to Transfer
$8C00	000	D0
$8C01	001	D1
$8C02	010	D2
$8C03	011	D3
$8C04	100	D4
$8C05	101	D5
$8C06	110	D6
$8C07	111	D7

5.4 Multiple Coprocessor Register Transfer Primitive

The format of the multiple coprocessor register transfer primitive is shown in Fig. 13. This primitive is used when the 68881 requests the transfer of multiple floating-point data registers to/from an external destination.

The length field indicates the number of bytes of each register to be transferred; in the case of the 68881 it is always $0C.

Figure 13 Contents of multiple processor register transfer primitives

Command before exception acquisition primitive is used to detect abnormalities from 68881, to abort the current operation, and to request initiation of exception processing to

the host CPU. Command mid-execution exception acquisition primitive is a primitive for notifying the transmission destination register of the data when an exception during the command execution is generated.

The respective format of these primitives is shown in Fig. 14 and Fig. 15. The '1' bit of PC relative bit indicates that an exception condition has occurred. The vector number generated by 68881, as well as the handling, is shown for reference.

Figure 14 Contents of command mid-exception acquisition primitive

Value	PC	Vector Number	Contents
$5C0B	1	$0B	Decentralized Emulator
$5C30	0	$30	Division/Condition Branch/Set for Undefined Commands
$1C31	0	$31	Inexact Result
$1C32	0	$32	Zero Division by Exact Division
$1C33	0	$33	Underflow
$1C34	0	$34	Operand Error
$1C35	0	$35	Overflow
$1C36	0	$36	Signaling NAN

![Diagram]..... 15 Contents of Exception Processing Primitive

bit 15	8	7			0
0	0	0	1	1 1 1	0 1

Value	Vector Number	Content
$10D0	$0D	Coprocessor Protocol Violation
$10D1	$31	Inaccurate Result
$10D2	$32	Division by Zero
$10D3	$33	Overflow
$10D4	$34	Underflow
$10D5	$35	Operand Error
$10D6	$36	Signaling NAN

6 68881 and Host CPU Communication

Although there are many types of response primitives, you might think that the communication between the 68881 and the host CPU is confusing, but in reality, the primitive types that correspond to commands are defined, so there aren't that many response types to consider.

6.1 68881 Internal Register Operations / Data Transfer Commands

Here we show the procedure for the 68881 to perform floating-point operations like FADD.X FP 0, FP1 and internal data transfers as examples.

First, the CPU writes these commands into the command CIR. Of course, the responding CIR value is also shown.

The 68881 sets $0900 as the response CIR for the host CPU in case of the $0900 to $4900 internal execution primitives. With this primitive's PC hit reset, the 68881 requests the host CPU to rewrite the program counter (though it might be ignored by a user familiar with 68000, which could lead to an error).

(Note: Writing does not mean error here).

(Page 120)

Communication Between Host CPU and 68881 (Part 1)

Communication during Floating-Point Register Operation/Data Transfer Command (OP Class: 000)

Host CPU Operation | 68881 Operation | Access Timing (Cycle) | Response CIR Value

- Command Write | Idle | Command CIR | $0802
- Response Read | Calculation | Response CIR | $0900 / $4900
- PC Write-back (Abbreviation) | Rounding | Address Command CIR | $0900
- • | Idle | - | $0802

The CPU reads the response CIR and begins the 68881 operation. Upon starting, it performs calculations and rounding, storing the specified calculation result in the data register.

Once the command execution is completed, the 68881 sets the response CIR to $0802 (supervisory active: idle state) and waits for a request from the host CPU.

6:2 Execution Between Registers and External Data/ Data Transfer Command from External to Register

The execution between the floating-point data register and external data (FADD.S #32, FP0, etc.), and the data transfer procedures such as transferring external data to the floating-point data register (FMOVE.S #11, FP2, etc.) are illustrated in the figure on page 122.

First, the host CPU writes the computational command code or data transfer command code to the command CIR. In responding to these commands, the 68881 accepts command evaluation/ data transfer as a supervisory primitive indicated by the response CIR, thus requesting data transfer to the host CPU.

Next, the host CPU transfers the requested data from the operand to the CIR via 68881. After the transfer is complete, the 68881 sets the response CIR value to $0900 (supervisory primitive: internal processing execution state). The data exchange, computations, rounding processing, etc. are carried out, and the result, as specified by the command, is stored in the floating-point data register.

Page: 121

1-17 Host CPU and 68881 Communication (Part 2)

Data transfer commands from registers to external data/external resources between registers (OP class 0x010)

Host CPU's Operation	68881's Operation	Accessed Register	Value of Response CIR
Write command	-------	[Idle]	----------
Respond readout	----------	Command CIR	S0802
Write PC (pointer write) (data)	-------	Respond CIR	Address evaluation/data transf
Readout of response	Fetch address CIR	S8900	
Execute (fetch/write)	Command address CIR/operand CIR	S0900	
Write result	Respond CIR	----------	
Readout of response	-------	[Idle]	S0802

6.3 Data Transfer from Registers to External

The internal motions of the 68881 and the procedure for reading out data from output registers are shown in diagram 18. When specifying the data format and packed data with a progression of up to 10 digits, rounding (if the data is below the numerical cutoff) will be necessary. This data format is called the K factor. Static K factor and Dynamic K factor are used when data is transferred and rounded repetitively. The pattern used for transmission from the registers to the outside differs when the Dynamic K factor is applied and otherwise.

Firstly, in ordinary transmission, the command is written into the command CIR, which returns either S8900 (address evaluation or data transfer primitive) or S0900, and issues another readout to the respond register; the process then begins processing the command format. Once the data transfer and rounding are complete, the retrieved CIR changes to the address and data transfer primitive execution register.

The host CPU will wait for this response and then read the data from the operand CIR.

Figure 18: Host CPU and 68881 Communication (Part 3)

Data Transfer from Registers to External (OP Class 0.011) without Dynamic K Factor

Host CPU Operation	68881 Operation	Register Accessed	Response CIR Value
Write Command	(Idle)	Command CIR	$0802
Read Response		Response CIR	$8900 / SC 900
Write PC (double)	Conversion	Command Address CIR	
Read Response		Response CIR	Physical Address Calculation / Data Transfer Primitives
Read Register		Operand CIR	$8900
Read Response		Response CIR	$0802
(Idle)			

Data Transfer from Registers to External (OP Class 0.011) with Dynamic K Factor

Host CPU Operation	68881 Operation	Register Accessed	Response CIR Value
Write Command	(Idle)	Command CIR	$0802
Read Response		Response CIR	Single-Main Processor Register Transfer Primitive
Write PC (double)	Conversion	Command Address CIR	$8900
Register Transfer		Operand CIR	$8900
Read Response		Response CIR	Physical Address Calculation / Data Transfer Primitives
Read Register		Operand CIR	$8900
Read Response		Response CIR	$0802
(Idle)			

When the read-out is finished, the host CPU re-reads the response CIR. This response CIR is S8082 (Null Privilege Limit Idle State: Idle State).

When a Dynamic K factor is specified, the initial response privilege limit is single main memory privilege limit 68881 is transferred to the CIR. The host CPU takes this response and refers to the parameter CIR and transfers the K factor value 68881.

When 68881 receives the K factor value, it sets the response CIR to S8900 (Null Privilege: Internal Processing Execution) and starts the internal data exchange.

After the burst is completed, the operation proceeds in the same manner as normal, evaluating the execution address/data transport privilege limit. After a second delay, the data is sent, and finally, upon receiving the final response CIR, the null privilege limit is read and the operation ends.

6★4 Control Register Transfer Command

One or more commands of FPCR, FPSR, FPIAR are transferred to registers 19 and below. In essence, low-complexity floating-point data transfer does not require extremely rapid movement and can be completed with ease in circumstances that do not require picky exchange of data. When a command is written, an execution address evaluation/data transport privilege limit response CIR is returned. The host CPU uses this to transfer the data. If there are multiple registers specified for the control register transfer, data transfer will be repeated.

After the transfer is complete, the response CIR is re-read. The value for this is S8082 (Null Privilege Limit: Idle State).

6★5 Transfer of Floating Point Decimal Data Registers

Floating-point decimal data register transfers are shown in figure 20 on page 126.

Floating-point decimal data transfers involve transferring instructions and registers directly (static transfer list) or they can be used as parameters (dynamic transfer list). Transfer can be done in two ways.

For static transfer lists, the initial response CIR is sent. If multiple registers are to be written, the CIR is assessed as circuit 1. Host CPU checks the register selection CIR, and by counting the number of '1' bits, determines the number of registers to be transferred. In this way, the host CPU determines the registers.

Page 124

Figure: 19 Communication between Host CPU and 68881 (4 of 4)

Control register transfer (read) command (OP class: 100)

Host CPU Operation	68881 Operation	Register Accessed	Value of Response CIR
Send Command Write	(Idle)	Command CIR	$0802
Read Response		Response CIR	Effective Address Evaluation/Data Transfer Primitive $8900
Transfer Register		Operand CIR	
Read Response		Response CIR	$0802

Control register transfer (write) command (OP class: 101)

Host CPU Operation	68881 Operation	Register Accessed	Value of Response CIR
Send Command Write	(Idle)	Command CIR	$0802
Read Response		Response CIR	Effective Address Evaluation/Data Transfer Primitive $8900
Data Transfer		Operand CIR	
Read Response		Response CIR	$0802

The CPU completes the data transfer through the Operand CIR and finally reads out the Response CIR.

When a dynamic register list is used, as the initial response, a unit main processor session register transfer primitive is returned, and the host CPU requests the transfer of the register list. The host CPU then sends the register list to 68881. Subsequent operations are the same as in the case of a static register list.

Figure 20 Host CPU and 68881 Communication (5)

Transmission of Floating-Point Data Registers (Dynamic Register List) Host CPU Operation | 68881 Operation | Accessed Registers | Response CIR Value

Command Write | Command CIR | $0802 Response Read | Response CIR Register Write | Operand CIR | $8900 Response Read | Response CIR Register Mask Read | Register Select CIR | $8900 Register Transfer | Operand CIR Response Read | Response CIR | $0802

Transmission of Floating-Point Data Registers (Static Register List) Host CPU Operation | 68881 Operation | Accessed Registers | Response CIR Value

Command Write | Command CIR | $0802 Response Read | Response CIR Register Mask Read | Register Select CIR | $8900 Register Transfer | Operand CIR Response Read | Response CIR | $0802

6-6 Conditional Instruction Execution

Conditional instructions refer to instructions like condition branches (Bcc). The 68881 is directly connected to the 68020 and...

(Note: The document reference numbers and codes such as $0802 and $8900 are kept as they are special values and codes specific to the document's context.)

Figure 21 Host CPU and 68881 communication (Part 6)

Conditional Instruction Processing

Host CPU Operation	68881 Operation	Register being Accessed	Value of Response CIR
Write condition	(Idle)	Condition CIR	$0800/$0801
Read Response	(Idle)	Response CIR	

In the case where a status is returned from 68881, it is analyzed and so forth from 68020. However, in cases such as with X 68000, processing of this kind is not performed by the CPU. There is no use of status fetch instructions. The communication procedure of this command is shown in Figure 21. Please note that the command CIR is used in the initial access. The arriving response CIR is a priority at the time of release of the specified condition. It becomes S0800 if released and becomes S0801 if not released.

Section 6.7 FSAVE/FRESTORE Instruction Processing Operations

The commands for saving/restoring the internal status of the 68881 are as follows. The processing procedure of these commands is shown on page 128, Figure 22.

In the case of the FSAVE command, operations start from reading the save CIR, and the transfer operation starts. There are four types of returned values at this time.

Code	State	Data Transferred
$0018	NULL State	(No data to be transferred)
$0118	Calm Idle	
$XX18	Idle State	Data Transfer of 24 ($18) Bytes
$XXB4	Busy State	Data Transfer of 180 ($B4) Bytes

XX shows the version of 68881. If Calm Idle is returned, the host CPU waits for another read of the response CIR, waiting for a status other than Calm Idle to return. When a status other than Calm Idle returns, the host CPU must do so for each format.

(Page 127)

Note: ”$” indicates a hexadecimal value.

Figure 22: Communication between Host CPU and 68881 (Part 7)

FSAVE Command Processing Operation | Operation of Host CPU | Operation of 68881 | Registers Accessed | Value of Response CIR | |----------|----------|----------|----------| | Save --> Reads Save CIR | | Save CIR | $0802 | | | (Idle) | | | | Transfer Overhead (Data) | | Overhead CIR | |

FRESTORE Command Processing Operation | Operation of Host CPU | Operation of 68881 | Registers Accessed | Value of Response CIR | |----------|----------|----------|----------| | Write to Restore CIR | | Restore CIR | $8900 | | | (Idle) | | | | Transfer Overhead (Data) | | Overhead CIR | |

Beginning of deferred processing execution

Necessary data read, storing the internal status of 68881. The FRESTORE command does the reverse operation. Restore CIR in Restore starts transferring received state information to Save CIR read and new state. Subsequently, restore CIR rereads with NULL, IDLE, BUSY one of states is confirmed, the saved internal state of 68881 has been written.

Math Coprocessor

6.8 Exception Handling Operations

An exception handling operation is the operation performed when the 68881 detects some abnormality during instruction execution. Exception operations include the pre-instruction exception handling operation, the BSUN exception operation, the F-line emulator exception operation (the operation when the 68881 receives an instruction it cannot execute), the FSAVE format exception operation (the operation when an FSAVE instruction is attempted during execution of an FSAVE instruction), and the FRESTORE format exception operation (when written to the restore CIR).

● Fig. 23 Communication between the Host CPU and the 68881 (No. 8)

Pre-Instruction Exception Handling Operation

Host CPU Action	68881 Action	Accessed Register	Response CIR Action
	(idle)		$0802
Write command		Command CIR	Pre-instruction exception acquisition primitive
Read response		Response CIR	
Write exception		Control CIR	$0802

```
acknowledge          |              |                |
                     | (idle)       |                |
```

In-Instruction Exception Handling Operation

Host CPU Action	68881 Action	Accessed Register	Response CIR Action
	(idle)		$0802
Write command		Command CIR	$C900
Read response		Response CIR	$8900
Write PC (optional)	conversion	Instruction Address CIR	Effective address evaluation / data transfer primitive
Read response		Response CIR	$0900
Transfer operand (data)		Operand CIR	
Read response		Response CIR	In-instruction exception acquisition primitive
Write exception		Control CIR	

```
acknowledge          |              |                | $0802
                     | (idle)       |                |
```

Figure 24 Communication between host CPU and 68881 (Example No. 9)

BSUN Exception Operation

Host CPU Operation	68881 Operation	Accessed Register	Response CIR Value
(Idle)			S0802
Write Condition Code		Condition CIR	
Read Response		Response CIR	Command Pre-fetch
Write PC or other (or address)		Command Address CIR R	
Write other example control		Control CIR R	S0802

Figure 25 Communication between host CPU and 68881 (Example No. 10)

F Line Emulator Exception Operation

Host CPU Operation	68881 Operation	Accessed Register	Response CIR Value
(Idle)			S0802
Write Command		Command CIR	
Read Response		Response CIR	Command Pre-fetch
Write PC or other (or address)		Command Address CIR R	
Write other example control		Control CIR R	S0802

When the written data format is strange, etc., there is an operation. Processing procedures are shown in Figures 23, 24, 25, and 26.

In any case, after the host CPU receives an exception notification from 68881, it writes to Control CIR and resets 68881 to an idle state.

Numeric Processing Processor

● Figure 26 Communication between the Host CPU and the 68881 (Part 11)

FSAVE Format Exception Processing Operation

Host CPU operation	68881 operation	Register accessed	Response CIR value
			(Processing previous FSAVE/FRESTORE instruction)
Read Save CIR	($0218)	Save CIR	
Write Abort		Control CIR	$0802
	(Idle)		

FRESTORE Format Exception Processing Operation

Host CPU operation	68881 operation	Register accessed	Response CIR value
Write Restore CIR		Restore CIR	
Read Restore CIR	($02XX)	Restore CIR	
Write Abort		Control CIR	$0802
	(Idle)		

● 7 68881 Instruction Format

Most 68881 instructions are issued by using the command CIR. Here we describe the instructions that can be used in this way.

● 7.1 General Instructions (OP Class 000/010)

68881 instruction formats can be broadly classified by their upper 3 bits, and these 3 bits are called the OP class. The arithmetic instructions normally used by the 68881, and data transfers from external sources to the 68881,

All OP classes are classified as 000 and 010. OP class 000 is a basic register, and 010 is a register-to-register transfer or a transfer instruction to an external register in between calculations of floating-point data.

The formats of these instructions are shown in Figure 27.

The R/M bit indicates whether it is a register or data given externally by the MOVEC command. When set to '0', it is a register, and when set to '1', it is external data. The meaning of the source field changes depending on this R/M bit. When R/M is '0', the source field is treated as a register number, and when R/M is '1', the type of data it indicates is used. When the R/M bit is '1' and the source field is '111', it becomes the FMOVECR command (explained next). The meaning of the function field changes.

The destination register field indicates the result of the floating-point calculation operation to which it is stored. The calculation field is used to specify the operation command. Please refer to the diagram for the relationship between the value of the function field and the calculation.

2 FMOVECR (Move from Constant Rom) Command

68881 has ROM constants like Pi (3.14159...) (~2.71828...) and stores commonly used constants for numeric calculations. FMOVECR command reads this and transfers it to the floating-point register. The format of the FMOVECR command is shown in Figure 28. The upper 9 bits have already appeared in general OP commands when the R/M field and the source field are set to '1'. The lower 9 bits are offset values indicating the ROM constant inside 68881.

In the table note, the offset value of the constants not shown but are used internally by the 68881 is guaranteed not to change without notice.

3 Transfer from Floating-Point Register to External Transfer

The format of commands like the transfer of floating-point registers to external (FMOVE FP 0, XX) are shown in the diagram in 135 pages Figure 29.

Title: Numeric Calculation Processor Command Format for 68881 (OP Class: 000/010)

OP Class

bit	15 14 13 12	10 9 ... 7	6 ...	0
'0'	R/M'0'	Source	Destination Register	Function

Floating Point Commands

R/M	Source	Source Operand	Abbreviation
000	FP0		
001	FP1		
010	FP2		
011	FP3		
100	FP4		
101	FP5		
110	FP6		
111			

Abbreviations: | '0' | Integer | - | | '1' | Long Word Integer | L | | 000 | Single Precision | S | | 001 | Extended Precision | X | | 010 | Packed Decimal Real | P | | 011 | Word Integer | W | | 100 | Short Real | D | | 101 | Byte Integer | B |

Function Commands: | Function | Command | | |----------|---------|-----------------| | $00 | FMOVE to Fp | | Data Transfer | | $01 | FINT | | | | $02 | FSINH | SINH(x) (Hyperbolic Sin) | | | | (*When the operand is small, it might be treated as 0) | | $03 | FINTRZ | | | | $04 | FSQRT | | (√) Square Root | | $06 | FLOGNP1 | Log(x+1) | | $08 | FETOXM1 | e^x-1 | | $09 | FTANH | TANH (Double Tangent) | | $0A | FATAN | TAN^-1 (Arc Tangent) | | $0C | FASINH | SIN^-1 (Arc Sine) | | $0D | FATANH | TANH^-1 (Arc Tanh) | | $0E | FSIN | SIN | | $0F | FTAN | TAN | | $10 | FETOX | e^x | | $11 | FTWOTOX | 2^x | | $12 | FTENTOX | 10^x | | $14 | FLOGN | Log e | | $15 | FLOG10 | Log 10 | | $16 | FLOG2 | Log 2 | | $18 | FABS | Absolute Value | | $19 | FCOSH | COSH (Hyperbolic Cosine) | | $1A | FNEG | | Negate | | $1C | FACOS | COS^-1 (Arc Cosine) | | $1D | FCOS | COS | | $1E | FGETEXP | | Get Exponent | | $1F | FGETMAN | | Get Mantissa | | $20 | FDIV | Division | | $21 | FMOD | | Modulo Division | | $22 | FADD | | Addition | | $23 | FMUL | | Multiplication | | $24 | FSGDIV | Division of Single Precision | | $25 | FREM | | Remainder (IEEE Standard) | | $26 | FSCALE | | FPxINT (x) | | $27 | FSGMUL | Multiplication of Single Precision | | $28 | FSUB | | Subtraction | | $37 | FSINCOS | | Calculate SIN and COS simultaneously | | | | (*When calculated simultaneously, COS goes to 0) | | $38 | FCMP | | Comparison | | $3A | FTST | | Operand Test | | $7F | | | (Unused) |

-Page 133

●Fig. : 28 FMOVECR (Transfer of Floating-point data)

bit 15 10 9 7 6 bit 0 0 1 0 1 1 ROM Offset

ROM Offset Value	Value Stored
$00	π
$0B	Log_10 2
$0C	e
$0D	Log_2 e
$0E	Log_10 e
$0F	0.0
$30	Log_2 2
$31	Log_e 10
$32	10^0
$33	10^1
$34	10^2
$35	10^4
$36	10^8
$37	10^16
$38	10^32
$39	10^64
$3A	10^128
$3B	10^256
$3C	10^512
$3D	10^1024
$3E	10^2048
$3F	10^4096

Bits 7 to 9 specify the number of the floating-point data register of the transmission source. Bits 10 to 12 specify the format of the external appearance in which the data will be output. 68881 outputs floating-point register as specified by general communication standards, and the data undergoes proper conversion before output.

```
The lower 7 bits are used for specifying the format. Selecting 'Packed BCD' (binary-coded
↪ decimal) for this 10-decimal format is very beneficial for decimal fractional
↪ representation and other numerical analysis computations. As previously described, K7
↪ factors refer to static K factors to specify the number of digits for command and
↪ execution. Similarly, dynamic K factors refer to calling separate data for the same
↪ purpose.
```

When the data format is specified as Packed BCD like '011', K factor fields operate as normal. When different formats are specified, all K factor fields must be set to '0'.

```
With '011' as the data format, proper output is executed. For example, if the source data is
↪ π (3.14159265...) and K7 factor is '3', the truncation to three decimal places results
↪ in 3.142 being returned. In contrast, if the K factor is set as '0', the data format
↪ returns 3.14159265.
```

Figure 29 FMOVE FPn.XX (Transfer of floating point small number data from register to outside)

[Diagram]

- Bits 15 to 12: 011 - Designation format
- bit 11: Source register
- Bits 10 to 8: K Factor

Designation Format Table:

- Data Format:

```
- 000: Long Word
- 001: Double Word
- 010: Byte integer
- 011: Pack decimal 10 digits
- 100: Floating K Factor
- 101: Word integer
- 110: Byte integer
- 111: Pack decimal 10 digits (dynamic K Factor)
- Code: L, S, X, P, W, D, B, P
```

K Factor Table (When Designation Format is not ”011” or ”111”):

- K Factor:

```
- -64 to 0: Determines decimals under point
- +1 to +17: Determines virtual decimal position
- +18 to +63: Error, set FPSR OPER bit to 1 in case of operation
```

Designated Register Number

- Register # 0000

Main Processor Data Register

- Note when Designation Format is not ”011”:
  - K Factor does not matter in other cases.

When K Factor is +3, the virtual decimal position determined as 3 digits means that 3.14E + 3 would be 3140.0. When the format '111' is dynamic K Factor, the K Factor field points to a register number of the main processor (D0-D7). This is valid only when 68881 communicates with 68020. For 68000 cases, ignore it (Refer to Communication Procedures for details). When using dynamic K Factor, please read the communication procedure for 68881 use. In static K Factor, the range of K Factor data is not directly meaningful. For detailed procedures, refer to the communication procedure explained earlier.

7-4 Transfer of Control Registers

The instruction that transfers the various registers FPCR, FPSR, and FPIAR held in the 68881 is shown in format in Figure 30. Bits 12, 11, and 10 correspond to FPCR, FPSR, and FPIAR, respectively, and are the registers to be transferred if the bit is '1'. In terms of nomenclature, FMOVE FPcr is responsible for transferring a single control register, and FMOVEM FPcr is responsible for transferring multiple control registers. The format of the instruction is the same for either, with the only difference being whether bits 12, 11, and 10 have a '1' in any one of them, displaying the number of bits is 1 or more.

Figure 30 Transfer of Control Registers (FMOVE FPcr / FMOVEM FPcr) bit 15 | 13 | 12 | 10 | 9 | 4 | 3 | 0 | bit 1 | dr | Register Selection | 0 0 0 | 0 0 | 0 0 0 | 0

1: Transfers FPIAR 0: Does not transfer

1: Transfers FPSR 0: Does not transfer

1: Transfers FPCR 0: Does not transfer

Transfer Direction 1: From 68881 to external 0: From external to 68881

7-5 Transfer of Multiple Floating-Point Data Registers

The instruction that corresponds to the 68000 MOVEM command is the transfer of multiple floating-point data registers (FMOVEM). The format of the instruction is shown in Figure 31. Whether to transfer or not transfer any of the eight floating-point data registers is specified in a register selection field (static or dynamic register list). Furthermore, each bit of the register list corresponds to a specific register. There are two ways to specify the transfer of multiple data registers: either with a pre-decrement or a post-increment mode.

Page 136

Numeric Processing Processor

● Figure 31 Transfer of Multiple Floating-Point Data Registers (MOVEM FPn)

bit 15 13 12 11 10 9 8 7 bit 0

```
 1  1   dr  Mode   0  0  0     Register selection

                                    Determines whether or not to transfer
                                    each floating-point data register
```

Mode	Register selection	Transfer mode
	bit 7 6 5 4 3 2 1 bit 0	
00	FP7 FP6 FP5 FP4 FP3 FP2 FP1 FP0	Static register list, -(An)
10	FP0 FP1 FP2 FP3 FP4 FP5 FP6 FP7	// (An)+
01	r r r r 0 0 0 0	Dynamic register list, -(An)
11	0 r r r r 0 0 0	// (An)+

rrr: The register number of the data register that holds the register-selection data.

```
The bit arrangement is the same as for the static register list; it changes
according to the lower bits of the mode bits.
```

Transfer direction 1: From 68881 to external 0: From external to 68881

Addressing modes such as post-increment ((A0)+, etc.) or pre-decrement ((-A0), etc.) are generally used, but with these two the order in which data is transferred is reversed (data saved in the order FP0, FP1, FP2 using pre-decrement is retrieved in the order FP2, FP1, FP0 using post-increment). The 68881 is designed so that data input/output is possible in either order.

Which of the total of four transfer modes obtained by combining these is used is selected by the mode field, and the direction of data transfer is determined by the dr bit.

● 7.6 Format of Conditional Instructions

Conditional instructions are a general term for conditional branch instructions and the like, but in the case of the X68000 the 68881 is connected as an I/O device, so these instructions are simply instructions that are given a condition and check whether the arithmetic result up to that point matches it or not.

The format of these instructions is shown in Figure 32 on page 138.

When a condition is given in the lower 6 bits (condition field), the 68881 then obtains it as the arithmetic result

Comparing the set flags, whether the given conditions are met or not will be returned by the response CIR.

●Figure. 32 Conditional instructions

Condition Code	Mnemonic	Definition	BSUN bit	Formula
$00	F	False	NAN' Z' N	False
$01	EQ	Equal	Z	E (NAN' N)
$02	OGT	Ordered Greater Than	Z' (NAN' Z)	
$03	OGE	Ordered Greater or Equal	Z	Z' (NAN' Z)
$04	OLT	Ordered Less Than	Z' NAN' Z	
$05	OLE	Ordered Less or Equal		
$06	OGL	Ordered Greater or Less Than		
$07	OR	Ordered	NAN	
$08	UN	Unordered	NAN	
$09	UEQ	Unordered or Equal	NAN' NAN	
$0A	UGT	Unordered or Greater Than	NAN' Z'	
$0B	UGE	Unordered or Greater or Equal	NAN' Z' N	
$0C	ULT	Unordered or Less Than	NAN' Z'	
$0D	ULE	Unordered or Less or Equal	Z' (NAN' Z)	
$0E	NE	Not Equal	Z	Z'
$0F	T	True	True	
$10	SF	Signaling False	NAN' Z' NAN' Z N	
$11	SEQ	Signaling Equal	NAN' Z' N	True
$12	GT	Greater Than		NAN' Z'
$13	GE	Greater or Equal		Z (NAN' N)
$14	LT	Less Than		NAN NAN
$15	LE	Less or Equal		Z' NAN(Z' Z)N
$16	GL	Greater or Less Than		NAN NAN'
$17	GLE	Greater or Less or Equal		NAN
$18	NGLE	Not Greater or Less or Equal		NAN NAN' Z
$19	NGL	Greater or Less		NAN' Z
$1A	NLE	Not Less or Equal		NAN' Z' NAN
$1B	NLT	Not Less Than		NAN' Z' N
$1C	NEQ	Not Equal		NAN' Z' Z
$1D	NGE	Not Greater or Equal		NAN
$1E	NGT	Not Greater Than		Z' Z
$1F	SNE	Signaling Not Equal		NAN' Z' Z' N
$1F	ST	Signaling True		True

Notes: In cases where the value of the condition field is 10 or more, if a meaningless operation such as a division by zero or an overflow occurs inside the 68881, the BSUN bit of the FPSR register is set.

Digital Signal Processor

8 Sample Programs

As an example of how to use the 68881, we have prepared three sample programs: reading data from ROM, single precision calculation (SIN (1.0)), and double precision calculation (3.1415 + 2.7182). Please refer to them.

List 1: Reading Data from ROM

```
/*
    • Reading circumference data inside 68881 */
/*
    • In the case of XC,
    • please insert one line with #define volatile */
include ”stdio.h”
union DAT {
    unsigned char cdat;
    unsigned short sdat;
    unsigned int idat;
    float fdat;
    double ddat;
} dat;
struct CIR {
    unsigned short response;
    unsigned short control;
    unsigned short save;
    unsigned short restore;
    unsigned short operation_word; /* Not used */
    unsigned short command;
    unsigned short reserver1;
    unsigned short condition;
    unsigned int operand;
    unsigned short register_select;
};
    unsigned short   reserve2;
    unsigned int     instruction_address;
    unsigned int     operand_address;          /* Not used */
};
volatile struct CIR *cir = (struct CIR *)0xe9e000;
void main(); void wait_copro();
void main() {
    SUPER(0);
    cir->command = 0x5c00;       /* FMOVECR #0, FP0 */
    wait_copro(0x0802);
    cir->command = 0x6400;       /* FMOVE.S FP0, xxx */
    wait_copro(0xb104);
```

```
    dat.idat = cir->operand;
    wait_copro(0x0802);
    printf("PAI = %f\n", dat.fdat);
```

}

void wait_copro(response) unsigned short response; {

```
    unsigned int     i, ack;
    for (i=0; i<0x20; i++) {
        ack = cir->response;
        printf("%04x\n", ack);
        if ((ack & 0xbfff) == response)
            break;
    }
    printf("**********\n");
```

}

/*---- Execution result ----

```
 * 0900
 * 0802
 * **********
 * 8900
 * b104
 * **********
```

140

- 0802

- PAI = 3.141593 */

●List 2 Simple Operation (SIN(1.0))

/*

- Calculation of sin(1.0) */

/* For XC,

- please add
- #define volatile
- in one line */

#include ”stdio.h”

union DAT {

```
    unsigned char cdat;
    unsigned short sdat;
    unsigned int idat;
    float fdat;
    double ddat;
```

} dat;

struct CIR {

```
    unsigned short response;
    unsigned short control;
    unsigned short save;
    unsigned short restore;
    unsigned short operation_word;    /* Not used */
    unsigned short command;
```

```
    unsigned short reserve1;
    unsigned short condition;
    unsigned int operand;
    unsigned short register_select;
    unsigned short reserve2;
    unsigned int instruction_address;
    unsigned int operand_address;     /* Not used */
```

};

/* --- Execution Result ---- *

- 9504
-

-
- 0900
- 0802
-

---

*/

- 8900
- b104

- 0802

- SIN(1.0) = 0.841471 */

●List...3 Binary Operation (3.1415 + 2.7182)

/*

- Calculation of 3.1415 + 2.7182 */

/* In the case of XC,

- please include the line
- #define volatile */ #include ”stdio.h”

union DAT {

```
    unsigned char cdat;
    unsigned short sdat;
    unsigned int idat;
    float fdat;
    double ddat;
```

} dat;

struct CIR {

```
    unsigned short response;
    unsigned short control;
    unsigned short save;
    unsigned short restore;
    unsigned short operation_word;     /* Not used */
    unsigned short command;
    unsigned short reserve1;
```

```
    unsigned short condition;
    unsigned int operand;
    unsigned short register_select;
    unsigned short reserve2;
```

};

Is there anything else you would like to know?

unsigned int instruction_address; unsigned int operand_address; /* Not used */ };

volatile struct CIR *cir = (struct CIR *)0xe9e000;

void main(); void wait_copro();

void main() {

```
    SUPER(0);
    dat.fdat = 3.1415;
    cir->command = 0x4400;    /* FMOVE #3.1415, FP0 */
    wait_copro(0x9504);
    cir->operand = dat.idat;
    wait_copro(0x0802);

    dat.fdat = 2.7182;
    cir->command = 0x4422;    /* FADD.S #2.7183, FP0 */
    wait_copro(0x9504);
    cir->operand = dat.idat;
    wait_copro(0x0802);

    cir->command = 0x6400;    /* FMOVE.S FP0, xxx */
    wait_copro(0xb104);
    dat.idat = cir->operand;
    wait_copro(0x0802);
    printf("3.1415 + 2.7182 = %f\n", dat.fdat);
```

}

void wait_copro(response) unsigned short response; {

```
    unsigned int i, ack;
    for (i=0; i<0x20; i++) {
        ack = cir->response;
        printf("%04x\n", ack);
        if ((ack & 0xbfff) == response)
            break;
    }
```

}

Summary of comments:

- `/* FMOVE #3.1415, FP0 */`: Float move 3.1415 to FP0 (floating point register 0).
- `/* FADD.S #2.7183, FP0 */`: Float add single 2.7183 to FP0.
- `/* FMOVE.S FP0, xxx */`: Floating move single from FP0 to some address.

(Blank page in the original.)

RTC

The RTC is an LSI that keeps the current date and time. In the X68000, in addition to keeping the clock running, it is used to realize timer operation that automatically starts at a specified time. Here, we explain how to access the RTC and other related matters.

1 RTC Peripheral Block Diagram

The X68000 uses an IC called RP5C15 as an RTC (real-time clock) for maintaining the date, time, and executing alarm (timer) operations. In the X68000, this IC is used not only for clock management but also for realizing timer operation that automatically turns on the main power supply at a specified time, and for controlling the TIMER LED on the front panel.

The peripheral block diagram for RTC in the X68000 is shown in Figure 1 on page 148. A crystal oscillator of 32.768 kHz is typically used as the basic clock source for clock ICs, and from this reference oscillator, the clock inside the IC is generated.

The RP5C15 has two output signal pins called ALARM and CLKOUT. The ALARM output is a level signal that matches the preset date, day, hour, minute, and second.

When using it for timer functions in the X68000, this output is connected to the GPIO0 pin of the MFP (Multi-Function Peripherals), and the data from this pin is used.

The output from CLKOUT is programmable by software to allow selection between six different frequencies of level and high/low pulse outputs. In the X68000, it can be selected from ...

Note: The text cuts off at the end.

Fig. 1 — RTC Peripheral Block Diagram

The TIMER-LED is used for flashing operation.

The power supply to the RP5C15 is designed so that when the main switch on the front panel is OFF, it is powered by the VCC2 power supply and battery. When VCC2 is supplied, the power to the RP5C15 is provided, and at the same time, the battery is charged through a resistor. When VCC2 is cut off (for instance, when the main switch on the front panel is turned off or during a power failure), the clock continues to operate from this battery.

2. RTC Registers

The list of RTC holding registers is shown in Fig. 2. The RTC usually accesses in 8-bit (byte) units, but the RTC holds only 4 bits of data, with the effective bits being the lower 4 bits.

The RTC registers consist of two bank structures, and to specify which bank to access, you set the MODE register (address: $E8A01B). Among the RTC registers,

RTC Registers

Address	Bank 0	Bank 1
$E8A001	1-second	CLKOUT selection register
$E8A003	10-second	Adjust
$E8A005	1-minute	Alarm 1-minute
$E8A007	10-minute	Alarm 10-minute
$E8A009	1-hour	Alarm 1-hour
$E8A00B	10-hour	Alarm 10-hour
$E8A00D	Day of the week	Alarm day of the week
$E8A00F	1-day	Alarm 1-day
$E8A011	10-day	Alarm 10-day
$E8A013	1-month	(unused)
$E8A015	10-month	12/24-hour select
$E8A017	1-year	Leap-year counter
$E8A019	10-year	(unused)
$E8A01B	MODE register	(both banks)
$E8A01D	TEST register	(both banks)

Address	Bank 0	Bank 1
$E8A01F	RESET controller	(both banks)

The MODE, TEST, and RESET-controller registers are not bank-specific; they can be accessed at $E8A01B, $E8A01D, and $E8A01F respectively.

The year/month/day registers are all accessed in BCD: the ones digit and the tens digit live in separate registers. These registers do not validate input, so take care not to set impossible values (e.g. a value greater than 9 in a ones digit, or $\geq$ $0A).

select" and "leap-year counter" onto one row and placed the MODE register at the wrong address ($E8A00F $\rightarrow$ $E8A01B).]

1 CLKOUT Select Register

We show the bit configuration of the CLKOUT select register in the figure. The CLKOUT register determines what type of signal to output to the CLKOUT terminal. The significant bits of the CLKOUT register are the lower three bits, which allow you to select one of the eight types of outputs shown in the figure. In the X68000, the CLKOUT terminal is used to light up the TIMER-LED, and thus the device outputs to the CLKOUT terminal when it is at high level. Therefore, if you set this register to "111," the device will turn off (consider "high impedance" at high level) the signal, and if you set it to "101" it will blink with a timing that lights up at one-second intervals.

When set to '101', the edge (rising edge from L to H) matches the timing of the second counter increment, and when set to '110', the rising edge matches the timing when the minute counter in increments.

Figure...3 Bit configuration of the CLKOUT select register BANK 1, $E8A001

7	6	5	4	3	2	1	0
Bank	$E8A001	CLKOUT					

CLKOUT terminal output waveform selection

- 000: High impedance
- 001: 16.384kHz
- 010: 1.024kHz
- 011: 128Hz
- 100: 16Hz
- 101: 1Hz (*1)
- 110: 8Hz (*2)
- 111: Fixed to "L"

*1: Synchronizes with the rise edge of CLKOUT and the second counter. *2: Synchronizes with the rise edge of CLKOUT and the minute counter.

2 Adjustment Register

We show the bit configuration of the adjustment register in Figure 4. The adjustment register is used to adjust the second counter (1-second counter) by 10 seconds, or simply

to clear it to 0. Unlike a mere clear, when you adjust it when the value of the second counter is greater than 30, the minute counter is incremented.

RTC

● Figure 4 Adjust Register BANK 1, $E8A003

bit 7 4 3 bit 0 |----|----|----|----|----|----|----|----| Adjust

```
Second-counter adjust
1: Adjust ON
0:   //   OFF

* If you adjust when the second counter is 0–29,
  the second count simply becomes 0.
  If you adjust when the second counter is 30–59,
  the minute counter is advanced and then the
  second counter is set to 0.
```

is set. For example, adjusting at 10:29:29 gives 10:29:00, and adjusting at 10:29:30 gives 10:30:00.

In the adjust register only the least significant bit is valid; setting this bit to '1' performs the adjust operation.

● 2.3 12/24 Hour Selector

The 12/24 hour selector specifies whether the clock counts in 12-hour mode or 24-hour mode. The bit arrangement of this register is shown in Figure 5. When set to 12-hour mode, bit 1 of the 10-hour register becomes the AM/PM bit. '0' indicates AM and '1' indicates PM. The X68000 counts in 24-hour mode.

● Figure 5 12/24 Hour Selector BANK 1, $E8A015

bit 7 4 3 1 bit 0 |----|----|----|----|----|----|----|----| 12/24

```
12-hour clock / 24-hour clock selection
1: 24-hour clock
0: 12-hour clock

* In 12-hour mode, bit 1 of the 10-hour counter
  indicates AM/PM.
  (0: AM, 1: PM)
```


2.4 Leap Year Counter

The bit placement is shown in diagram 6. The leap year counter is a register that indicates how many years have passed since the last leap year. When '00' is set, it means that it is a leap year, and February 29th will be part of the count. The leap year counter is automatically incremented every year, so no exceptional case (where 100 years are not fully divisible by 400) will occur. Even if the counter keeps its value unchanged, calculations are automatically performed based on the leap year rules.

An example of overflow handling is the year 2100, which is more than 100 years from now (2100 AD is not a leap year because it is not divisible by 400, while the year 2000 AD is a leap year because it is divisible by 400). This system can write the remainder when divided by 4 for the number of years from the last leap year.

```
Leap Year Counter BANK 1, $E8A017
```

```
bit 7 →  | -- unused -- |7 |6 |5 |4 |3 |2 |1 |  Leap  |0 | ← bit 0

Number of years since last leap year
      00: This year is a leap year
      01: 1 year after last leap year
      10: 2 years after last leap year
      11: 3 years after last leap year
```

- From the year 2099 onward, always write 00 to this register.

2.5 Mode Register

The MODE register is a register that allows/prohibits calculations and alarm operations, and selects the register bank. The bit placement is as shown in diagram 7. The bit determines whether to access the RP5C15 register bank on the sink side. Once written, the value remains the same until the next write operation. Changes to this register are typically not made during normal operations of Human 68K, so it is okay to perform debugging and other write-in operations.

RTC

● Figure 7 MODE Register $E8A01B

bit 7 4 3 2 1 bit 0 |----|----|----|----| Timer EN Alarm EN BANK 1/0

```
Register bank selection
1: BANK 1
0: BANK 0

Alarm operation enable/disable
1: Alarm operation enabled
0:   //   disabled

Timer operation enable/disable
1: Timer operation enabled (normal operation)
0:   //   disabled (counting below seconds stops)
```

bit 2 is the enable/disable control for the alarm operation (in the X68000, used for the timer operation that powers on at a specified time); writing '1' enables the alarm operation.

bit 3 is the enable/disable control for the timer operation (count operation); writing '0' disables the counting below the second unit, and writing '1' restores normal operation. Even if the timer operation is disabled, counting within the second unit continues operating, and a single count-up is held internally, so as long as the period is within one second, leaving this bit at '1' will not cause the clock to drift.

● 2.6 Test Register

The bit arrangement of the test register is shown in Figure 8 on page 154. These are used by the chip maker of the RP5C15 (Ricoh) for shipping inspection (apparently making the clock run faster than normal). Normally, do not set any data other than 0.

● 2.7 RESET Controller

The bit arrangement of the RESET controller is shown in Figure 9 on page 154. The RESET controller is a register that performs initialization of the alarm, resetting of the counter below the second unit, selection of the output pulse from the ALARM output terminal, and so on.

The ALARM output terminal of the RP5C15 outputs a 1 Hz pulse, a 16 Hz pulse (both with a 50 percent duty cycle), and the OR condition of the three factors of a match between the

internally set time and the current time.

Figure 8: Test Register $E8A01D

|----|----|----| bit | 7 | 4 | 3 | | bit 0 | |----|----|----|----------| | | TEST 3 | TEST 2 | TEST 1 | TEST 0 |

This is used for chip maker (reflow) exit inspection. Normally, set all to '0'.

Figure 9: RESET Controller $E8A01F

|----|----|----| bit | 7 | 4 | 3 | | bit 0 | |----|----|----|----------| | | 1Hz ON | 16Hz ON | Timer RESET | Alarm RESET |

1: Reset the alarm 0: Release

This section resets/releases the counter under a second unit (core part not read by software). Clearing occurs when bit '1' is set; normal operation resumes when '0' is set.

Alarm terminal 16Hz pulse output control 1: Output OFF 0: Output ON

Alarm terminal 1Hz pulse output control 1: Output OFF 0: Output ON

Bits 4 and 3 indicate whether to output a 1Hz or 16Hz pulse from the ALARM output terminal. ‘0’ turns the output ON, and ‘1’ turns it OFF for both. Practically, it's better to have both set to ON, given that the pulses are sine waves of 1Hz and 16Hz frequency.

For the X68000, the ALARM output is only for signal operation, so these bits should usually be set to ‘1’ (OFF).

The timer reset part is the core section's counter. The counter is reset when this bit is set to '1' and resumes normal operation at '0'.

Alarm reset resets the circuits related to matching alarm operation. With this reset bit, RP 5 15’s alarm condition of year, day, hour, and minute matching is cleared. This fixes any discrepancies forcibly to match the conditions above. However, it only matches partial registers and the rest remain inconsistent.

RTC Access

This somewhat easy operation is, for example, considering the action of performing ”every day at 18:00 and 22:00”, saving the effort of setting the day, date, and day of the week every day. By resetting the alarm, the value on the alarm counter register side is not sent forcibly and a consistent state is maintained, allowing the setting operation to be saved. In this example, you can change only the register of the alarm time each time.

3 RTC Access

Since RTC timekeeping operates independently of the CPU, and the CPU can't access the registers all at once, you need to take care to access it.

3.1 Reading Time

When reading time, what you need to be careful about is the possibility of a discrepancy due to the CPU reading registers sequentially. For example, in the case of reading between 19:59:59 and 20:00:00, if reading happens sequentially, the read timing might result in a read of 20:00:59 instead of 19:59:59. Here is the method to avoid this:

1) Stop timekeeping before reading (using the TIME EN bit of the MODE register), and resume after reading.
2) Read the time data twice, and read again if they don't match.

3) Read data in sync with a 1 Hz signal (use CLKOUT or ALARM as a 1 Hz output and read into GPIP).

On the X68000, since CLKOUT and Alarm pins are used for LED lighting and timer functions, practically usable methods are 1) and 2). From a software viewpoint, since timekeeping stops momentarily, method 2) is considered safer. Also, note that the timing for RTC internal time change is the rise of CLKOUT (transition from L to H).

However, the alarm output is about 180 degrees out of phase with CLKOUT2, and when the alarm output is active, the RTC timekeeping is updated 96 μs later.

3-2 Writing Clock Data

When writing clock data, if noise or glitches occur while writing, the settings will be affected. The following are some methods to avoid this:

1) Stop the seconds hand in the same way as when reading the clock, and set it.
2) Clear the lower 4 bits of the seconds register reset control register, and write data by synchronizing with the 1 Hz signal.
3) Write data while synchronizing with the 1 Hz signal.

Of these three methods, it is thought that the first method is difficult to use on the X68000 because the counting operation of the clock suspension method continues, and the setting error is around 1 second. Therefore, it is considered that using the second method is suitable.

When writing clock data, be sure to set whether it is in 12-hour mode or 24-hour mode, and set the year counter settings.

3-3 Other Settings

Below is a summary of points related to settings that have not been touched upon so far for your reference.

3-3-1 Year Counter

The RTC year counter operates independently from the century processing, so there is no need to exclude the lower 2 digits of the year when setting. In Human 68K, it is set to treat the values as years since 1980.

RTC

3.3.2 Day Counter

The Day Counter is a 7-stage counter that increments the value from 0 to 6 every day. It is up to the user to determine whether the values correspond to the days of the week. In the Human 68K system, Sunday corresponds to 0.

3.3.3 Alarm Function

Please follow the steps below to set the alarm:

1) Make the alarm disable (Set the MODE register bit to 0)
2) Reset the alarm (Set the RESET controller register bit to 1)
3) 100 μs delay
4) Set the alarm register

Also, when using the alarm function, ensure that the main power switch on the back of the device is not turned off.

•4 Sample Program

Here is a sample that performs simple clock reading. Please refer to it. This program uses the double read method.

List 1: Clock Reading

```
/*
 * RTC Reading Sample
 */
```

/* For XC

```
 * Please insert one line of
 * #define volatile
 */
```

#define TRUE 1 #define FALSE 0

char *dayofweek[7] = {”SUN”,”MON”,”TUE”,”WED”,”THU”,”FRI”,”SAT”};

volatile unsigned char *rtc_base = (unsigned char *)0xe8a001; volatile unsigned char *rtc_mode = (unsigned char *)0xe8a01b;

unsigned char c_time[2][7];

void main(); int cmp_time(); void read_time(); void print_time(); void bank();

void main() {

```
    unsigned int bnk;
    SUPER(0);
    bank(0);
    bnk = 0;
    read_time(c_time[bnk ^ 1]);
    while(!cmp_time(c_time[0], c_time[1]))
        read_time(c_time[bnk ^ 1]);
    print_time(c_time[0]);
```

} int cmp_time(src, dst) unsigned char *src, *dst; {

```
    unsigned int i;
    for (i=0; i<7; i++)
        if (*src++ != *dst++)
            return(FALSE);
    return(TRUE);
```

}

void read_time(buf) unsigned char *buf; {

```
    volatile unsigned char *rtc;
    unsigned int i;
    rtc = rtc_base;
    for (i=0; i<3; i++, rtc += 4) {
         *buf++ = (*rtc & 0xf) + (*(rtc+2) & 0xf)*10;
    }
    *buf++ = *rtc & 0xf;
    rtc += 2;
    for (i=0; i<3; i++, rtc += 4)
        *buf++ = (*rtc & 0xf) + (*(rtc+2) & 0xf)*10;
```

}

void print_time(buf) unsigned char buf[]; {

```
    unsigned int i;
    printf("[YY/MM/DD HH:MM:SS] <%04d/%02d/%02d %02d:%02d:%02d>\n",
```

```
        1980+buf[6],buf[5],buf[4],buf[2],buf[1],buf[0]);
    printf("[Day Of Week] <%s>\n",dayofweek[buf[3]]);
}
```

void bank(bnk) unsigned int bnk; {

```
    if (bnk)
        *(rtc_mode) |= 1;
    else  *(rtc_mode) &= ~1;
}
```

This code snippet seems to be dealing with reading, formatting, and banking the time from a Real-Time Clock (RTC).

(Blank page in the original.)

Screen Control

The screen control system, built around a group of LSIs uniquely developed by Sharp, is arguably the most distinctive feature of the X68000. This section explains the various display modes and screen control mechanisms of the X68000.

1 Screen Configuration of the X68000

The X68000 has a screen display mechanism so powerful that it is not seen in other personal computers. In business-use personal computers, most screen display amounts to no more than text and very simple graph display, and the hardware is matched to this, typically providing only a kanji display screen and a graphics screen capable of handling about 16 colors. In contrast, the X68000 is designed to respond flexibly to a wide variety of ”display” demands — from real-time action games where drawing speed is critical, to high-resolution images of tens of thousands of colors or more typified by ray tracing, and window systems supporting various character fonts — all while minimizing the load on the CPU as much as possible.

The screen configuration of the X68000 is shown in Figure 1 on page 162.

The X68000 has four types of independent screens: graphics screens (1 to 4 planes), text screens (4 planes), sprites (up to 128 on the screen, up to 32 on the same horizontal line), and BG screens (background screens, up to 2 planes). These are combined, and after compositing with video images (superimpose) is performed, they are displayed on the CRT as a single screen. These

■Figure 1 Screen Configuration of X68000

Sprites 16 x 16 dots (12 colors) (256 patterns) (256 patterns x 2)

BG Screen (512 x 512 x 1) (256 x 256 x 2)

Text Screen (1024 x 1024 x 4)

Graphic Screen 256 x 256 1 byte (512 x 512) 2 byte (1024 x 1024) (65536 colors 1/2/4 screens)

Video Image

To CRT

Each screen has different specifications and can be used separately or in combination according to the purpose, allowing for diverse expressions. Here, we will briefly summarize the structure and characteristics of the graphics, text, BG, and sprite screens. The structures of each screen are shown in Figure 2 for your reference.

Page 162

Screen Control

● Figure 2 Structural Overview of the Graphic, Text, BG, and Sprite Screens

1 word (16 bits)

Graphic screen

bit 15 |---------------------------------------| bit 0 Color code 65536-color mode bit 15 |---------------------------------------| bit 0 Color code 256-color mode bit 15 |---------------------------------------| bit 0 Color code 16-color mode

Text screen

bit 15 |--| bit 0 bit 15 |---------------------------------------| bit 0

BG screen

BG data area RAM Pattern number

PCG RAM Pattern data (out) Pattern data (◎) Selection

PCG RAM Pattern data (out) BG screen

```
64 divisions
64 divisions
```

Sprite screen

Sprite scroll register X coordinate Y coordinate Pattern number

PCG RAM Pattern data (◎) Selection

Sprite screen

0.1 Graphic Screen

The graphics screen is suitable for use when dealing with many colors, such as in painting software or title rendering. Catalogs and other large-scale displays often utilize the 65536 color simultaneous display feature on this screen. In this mode, you can use not only the 65536 color mode but also the 256 color and 16 color modes. In this screen mode, one dot on the screen corresponds to one word (16 bits). When drawing, it becomes easier to explain by considering the alignment of bits in horizontal and vertical directions on the screen, which is sometimes called ”plane-type”. When writing data to the specified dot position, the color of that dot changes based on the written color code, read by counting the increasing number of colors. Unlike the memory operation for text display, which involves more memory operations, the graphic display is advantageous for drawing shapes with numerous dots at the same time; it is not suited for text display, which primarily involves writing one dot at a time in positions akin to half-points or quarter-points.

0.2 Text Screen

The text screen has characteristics almost opposite to those of the graphics screen. Because it is named the text screen, it may be thought to allow only text display, but it is still a type of graphics screen on the X68000. The X68000's text screen allows placing and clearing dots at any position, making it useful for such purposes.

What sets the text screen apart from the earlier explained graphics screen is that the text screen's bit alignment is in horizontal rows (horizontal) instead of vertical. Each word of data corresponds to 16 dots in horizontal direction. To write 16 dots on a graphics screen, it would typically require 16 memory write operations for a monochrome display, which can be avoided by writing once to the corresponding bit on the text screen. This allows for convenient display of pre-prepared patterns like in ”text spaces”.

The X68000 can possess a blank text screen containing planes corresponding to each bit of RGB colors. This enables up to 16 color (or selectable from 65536 colors) displays.

● 3 BG Screen

The BG (background) screen is a screen with strong color development, similar to the sprite that will be explained next. In game screens, it is common to use it not only for drawing characters but also for drawing backgrounds such as cities and terrain. On this screen, it is possible to display graphics as with the previous text screens. In games, it is often used to effectively display screens with a limited number of patterns.

The BG screen is divided into 64 parts and divided into 32 x 32 tiles. It is displayed on the screen by registering the memory addresses corresponding to each tile in the Map Data Table and writing the pattern numbers (0–255) in the BG space. Unlike characters like sprites, the same pattern cannot be displayed independently in dot units, creating boundaries between scrolling and moving elements.

The X68000 can hold 2 BG screens. Also, part of the BG screen can be displayed in dot units independently from the sprite screen. This ensures smooth background scrolling.

● 4 Sprites

Sprites are displayed with defined patterns at any desired location in dot units. On the X68000, a single sprite definition consists of 16 x16 dots, and up to 256 different patterns, up to 128 sprites can be displayed simultaneously on the screen.

On the other hand, sprites are mainly used in action games and other instances where characters move rapidly, unlike the text screen and graphics screen where smooth scrolling is important. This greatly reduces the burden on the CPU by handling the collision detection and overlapping management of multiple characters in hardware, rather than software.

A game such as ”Gradius,” which came bundled with the original X68000 model, is a good example of fully utilizing this sprite function.

2 Configuration and Address Layout of Each Screen

The X68000's screen display circuit is quite elaborately built: while simultaneously handling four types of screens with different characteristics — graphic, text, BG, and sprite — it also supports superimposing with TV and screen capture. For this reason, trying to grasp all the registers and so on at once tends to make them hard to understand. Therefore, here we will first explain the structure of each of the X68000's screens and the address layout of the display memory, and we will explain functions such as screen ON/OFF and priority control in the following sections and onward.

2.1 Configuration of the Graphic Screen

2.0.1 Screen Modes of the Graphic Screen

A list of the graphics screen modes that the X68000 is designed to support is shown in Table 1 on page 167.

The X68000 has many display modes, but if we focus on the number of dots, they consist of combinations of 2 types of actual screens and 4 types of display screens. In the table, the entries marked with a double circle indicate that the screen mode is supported by BASIC, the XC library, IOCS calls, and so on. The screen modes marked with an asterisk (*) indicate screen modes that are not supported (or at least not publicly documented) by the system software, but that are described as existing in the Programmer's Manual and similar documentation that accompanies XC and the like.

Also, in the table the terms ”high-resolution mode” and ”standard-resolution mode” are used; these simply indicate that the horizontal scan frequency is 31 kHz and 15 kHz respectively. On the X68000, 31 kHz mode is normally used, so calling the 15 kHz mode ”standard resolution” is a little

Table 1: List of Graphical Screen Modes for X68000

Actual Screen

Display Screen	High-Resolution Mode (Horizontal Frequency 31kHz)	Standard Resolution Mode (Horizontal Frequency
1024 × 1024	768 × 512	X
	512 × 512	512 × 512
	512 × 512	X (Interlaced)
		512 × 512
	256 × 256	○ (Two Screens)

Note:

- X: Not available
- ○: Available
- (Graphics of X-BASIC V, Supported by IOS, CRTC settings required)

Strangely enough, Sharp’s manual type describes this mode, so I decided to follow the trend.

The horizontal frequency of TV broadcasts is 15 kHz, so when displaying superimposed graphics, the 15 kHz mode is used. In the 15 kHz mode, the only screen modes available are those with a horizontal dot number that does not reach 512, such as the screen mode with 256 dots.

COLUMN: Interlace and Two Screens Reading

Interlace is a method designed considering low transfer rate of screen data while reducing flickering during TV broadcasts. To display motion clearly enough not to be visible to the human eye, the screen must be refreshed at a speed greater than 1/24 second per frame. In TV broadcasts, the screen is usually refreshed at 30 frames per second, but even at this speed, motion can be flickering if more refined control is not applied. Therefore, instead of sending the whole frame at once, the TV broadcast sends odd-numbered and even-numbered lines alternately (each of these frames is called a ”field”) to suppress flickering by refreshing the screen at 60 frames per second. This display method is called the interlace method.

The X68000's CRT interface supports the interlace method, and the 512 × 512 dot display in 15 kHz mode uses the interlace method. The X68000's CRT vertical frequency is fixed at 60 Hz (where the TV's vertical frequency is adjusted internally), so the number of vertical dots in the screen mode is 512 at 31 kHz mode, 512 at 15 kHz mode, and typically 256. The CRT recognizes the difference between odd and even frames for precise screen control.

COLUMN

Overscan

Similar to interlace, overscan is related to how TV broadcasts are displayed. Overscan is a means of displaying the screen image slightly larger than the CRT. When displaying the broadcasting image on the CRT, the entire screen is shown. However, on a computer display, the displayed image can extend beyond the edges of the CRT, leading to the phenomenon known as overscan. Thus, on many computers, the entire screen image is adjusted to fit within the CRT's visible area.

For the X68000, in 31 kHz mode, such an adjustment (called underscan) is made, while in 15 kHz mode, overscan is considered, hence actual display through overscan is done. When overscan happens, the edges of the X68000's screen may extend beyond the CRT boundaries. If the X68000 does not use underscan, then part of the screen might get cut off on a TV set. Therefore, during 15 kHz mode, the display functions use underscan so that the entire CRT display is visible.

2.1.2 Graphic VRAM Address Allocation

For an actual screen size of 1024 x 1024 dots, the graphic VRAM address allocation for a screen size of 512 x 512 dots is shown in Figure 3 and Figure 4. The graphic VRAM address allocation varies depending on the screen mode, but in any case, one dot on the screen corresponds to one word (16 bits). Specifically, the dot one position to the right of a given dot is the second word, and so forth until the fourth word.

For a screen size of 1024 x 1024 dots, the VRAM area uses 1 page for 2MB bytes. If the screen size is 512 x 512 dots, it uses four pages of 512KB bytes each. In other words, page 0 is $C00000 to $C7FFFF, page 1 is $C80000 to $CFFFFF, page 2 is $D00000 to $D7FFFF, and page 3 is $D80000 to $DFFFFF.

When writing codes to memory, data is stored incrementally according to the word. When in 65536 color mode, the upper 8 bits remain as they are. In 256 color mode, the lower 8 bits are valid. In 16 color mode, the lower 4 bits are valid and the upper 4 bits are ignored.

Figure 3. Graphic VRAM Address Allocation (Actual Screen Size of 1024 x 1024 Dots)

Figure 4 Configuration of Graphic VRAM Address (Display Area 512 x 512 Dots)

16-color (4-page) mode Page 0

Page 1

Page 2

Page 3

Page 0

(The annotation under each block of memory addresses likely remains unchanged since it usually denotes hexadecimal addresses in English as well.)

bit 15 | bit 0

Page 0

Page 1

Page 2

Page 3

Not used

256-color (2-page) mode Page 0

Page 1

bit 15 | bit 0

Page 0

Page 1

Not used

65536-color (1-page) mode Page 0

bit 15 | bit 0

Page 0

Not used

COLUMN

Page and Plane

Pages and planes are often similar concepts, but in this book, a page is a unit that can specify display colors or scroll position combinations independently, as in a text screen. When defined separately, each screen is called a plane.

A graphics screen can have one to four planes, depending on the screen mode setting. Each plane displays independently of the other planes on the same page (such as color specification, scroll, priority, ON/OFF control, etc.), so we call it a page.

On the other hand, a text screen is composed of four planes. Each plane corresponds to one bit in the color number mode. Using these four bits (4 planes) allows the specification of 16 (4 bits) colors. Also, features like scroll position and ON/OFF controls operate continuously across all four planes. Thus, in a text screen, these are referred to as planes.

2-2 Text Screen Configuration

2-2-1 Text Screen Modes

The text screen mode differs from the graphics screen in that it is purely planar. The display size corresponds to the graphics screen size, but it is usually 1024x1024 dots in text screen modes. Each graphical plane has four planes.

2-2-2 Text VRAM Address Arrangement

Each plane on the text screen corresponds to a word compiling into 16 dots horizontally. There are four planes in total, each called T0 plane, T1 plane, T2 plane, and T3 plane. The address ranges are T0 plane: E00000~E1FFFF, T1 plane: E20000~E3FFFF, T2 plane: E40000~E5FFFF, and T3 plane: E60000~E7FFFF (Figure 5).

● Figure 5 Address Layout of the Text Screen

$E60000 T3 plane

$E40000 T2 plane

$E20000 T1 plane

$E00000 $E00002 $E00080 T0 plane

$E1FFFE $E3FFFE $E5FFFE $E7FFFE

1024 dots

1024 dots

bit 15 bit 0 (left) (right)

$E00000 T0 plane

$E1FFFE $E20000 T1 plane

$E3FFFE $E40000 T2 plane

$E5FFFE $E60000 T3 plane

$E7FFFE

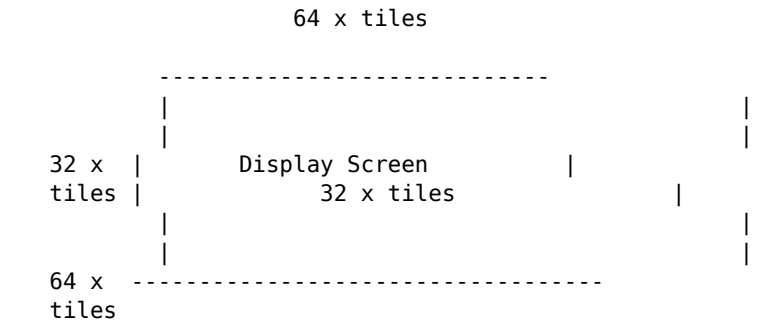
The color code of the text screen is not written as data directly the way it is on the graphics screen; instead, it is selected from among 16 colors by the data corresponding to the same position in each of the planes T0 to T3. To find the color code of a given dot, a read of all four planes (4 reads) is required, which is somewhat troublesome. However, since there is a function for writing to multiple planes simultaneously, increasing the number of colors used does not have much effect on the drawing speed.

◆3 Configuration of BG Screen

◆3-1 BG Screen Mode

When the display screen is 512 x 512 dots, one page is used; when it is 256 x 256 dots, two pages of BG screen are used. Figure 6 shows the relationship between the actual size of the BG screen and the display screen. The actual screen area of the BG screen is twice the size of the display screen. The size of the pattern used on the BG screen is 16 x 16 dots when the display screen is 512 x 512 dots, and 8 x 8 dots when it is 256 x 256 dots. Therefore, the number of tiles on the BG screen corresponds to the screen mode. In other words, when the actual screen area is 64 x 64 tiles, the display screen is 32 x 32 tiles (changing the display screen size to the screen mode register (address $EB0810) which is HRES bit (bits 0, 1).

Figure 6: The actual screen area and the display screen



Actual Screen

|------------------------------| 32 x tiles
|------------------------------| Actual screen

|------------------------------| 64 tiles |------------------------------|

Screen Mode Register HRES Bit Display Screen Size Actual Screen Size Pattern Size (1 tile)

0 0	256 x 256 dots	512 x 512 dots	8 x 8 dots
0 1	512 x 512 dots	1024 x 1024 dots	16 x 16 dots

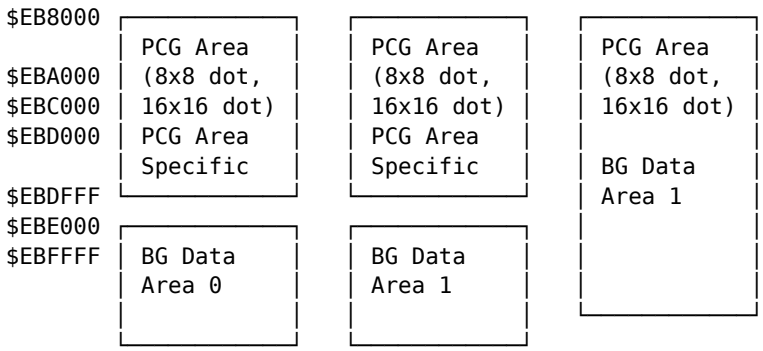
2-3-2 BG Screen Memory Address Allocation

The address allocation for the BG screen memory area is shown in diagram 7. The RAM for the BG screen is divided into areas for registering patterns used for display (PCG area) and for determining which of the 64x64 (total: 4096) divided patterns to use and where to display them (BG data area). Of these, the PCG area is shared with the sprite.

The RAM for the BG screen, 16 KB in the first half ($EB8000-$EBBFFF), is exclusively used for the PCG area. When the pattern of the PCG uses 1 bit per dot registration, 1 pattern size is 16x16 dots (screen mode is 512x512 dots) with 1 pattern equaling 128 bytes, or 8x8 dots (screen mode is 256x256 dots) with 1 pattern equaling 32 bytes. When the pattern is 16x16 dots, 128 patterns can be assigned in this area, or when it's 8x8 dots, then 512 patterns. However, the BG data area can only use 256 patterns.

The RAM for the BG screen is divided into 16 KB in the first half and 8 KB in the second half ($EBC000-$EBFDFF, $EBE000-$EBFFFF). When using the BG screen in 2 sides, both sides are allocated to the BG data area. When not using both sides of the BG screen, the first half 8 KB may also be allocated to the PCG area.

Diagram 7: PCG Area and BG Data Area Address Allocation



3 possible configurations to choose from

Screen limitations

Therefore, if the BG screen is only used on one side, up to 192 (i.e., 128+64) 16 x 16 dot patterns can be registered. If the BG screen is not used at all (using all as sprite patterns), up to 256 patterns can be registered.

2-3 PCG Area Configuration

The data structure for the PCG area is shown in the diagram on page 176. The fundamental unit of PCG data is the 8 x 8 dot image (a single pattern of 32 bytes).

As shown in the screen mode of 256 x 256 dots and the BG screen mode, when displaying an 8 x 8 dot pattern, this corresponding pattern is used as it is. When displaying a 16 x 16 dot pattern, the PCG data is combined for each pattern number, and each pattern number is taken as a unit. In other words, the pattern numbers 0, 1, 2, and 3 are combined in the order of 4, 5, 6, and 7 for the 16 x 16 dots.

Each PCG's registered data and patterns are shown enlarged on the right side of the image to show the correspondence. PCG areas can be accessed with long words (32 bits), so by arranging data for exactly 8 dots in a row, the upper 4 bits correspond to the leftmost 4 dots, and the lower 4 bits correspond to the rightmost 4 dots, making it easier to handle.

PCG data is expressed as 4 bits per dot, which means each dot's color code is represented by 4 lower bits. The upper 4 bits are in the BG data area or sprite scroll registers, and

it's possible to specify BG patterns and sprites. By combining these, 256 colors can be represented, allowing 16 colors per pattern.

Page 175

Figure 8: Structure of the PCG Area

For the PCG Area

PCG Area | Address | Pattern Number | |------------|-----------------| | $EB8000 | 0 | | $EB8040 | 1 | | $EB8060 | 2 | | $EB8080 | 3 | | $EB80A0 | 4 | | $EB80C0 | 5 | | $EB80E0 | 6 |

| $EB9F80 | 252 | | $EB9FA0 | 253 | | $EB9FC0 | 254 | | $EB9FE0 | 255 |

| $EBA000 | 64 | | $EBA020 | | | $EBA040 | | | $EBA060 | |

For BG Data Area

| $EBDFF80 | 190 | | $EBDFA0 | 191 | | SEBDFCO | | | SEBDFEO | |

| $EBFF80 | 255 | | $EBFFA0 | | | SEBFFCO | | | SEBFFEO | |

PCG Area | bit 31 16 15 bit 0 | | + $00 P7 P6 P5 P4 P3 P2 P1 P0 | | + $04 P15 P14 P13 P12 P11 P10 P9 P8 | | + $08 P23 P22 P21 P20 P19 P18 P17 P16 | | + $0C P31 P30 P29 P28 P27 P26 P25 P24 | | + $10 P39 P38 P37 P36 P35 P34 P33 P32 | | + $14 P47 P46 P45 P44 P43 P42 P41 P40 | | + $18 P55 P54 P53 P52 P51 P50 P49 P48 | | + $1C P63 P62 P61 P60 P59 P58 P57 P56 |

8 Dot

DOT Pattern Example

16 x 16 Dot Pattern (Pattern Number 0)
16 Dots 16 x 16 Dots

Notes

- The diagram illustrates how 8x8 dot patterns are stored in memory.
- Pattern numbering is used to access the PCG (Programmable Character Generator) area data.
- The 16x16 dot pattern expands into 4 subpatterns numbered as 0, 1, 2, and 3.

This diagram appears to be describing how dot patterns are encoded and stored at specific addresses within memory for a Programmable Character Generator in a computing or graphical system.

Screen Control

2.3.4 Structure of the BG Data Area

The structure of the BG data area is shown in Figure 9. One word is allocated per block of BG. The lower 8 bits (bits 0-7) indicate the number of the pattern registered in the PCG area, and the 4 bits, bits 8-11 (COLOR), indicate the upper 4 bits of the color code (the lower 4 bits are specified per dot in the PCG area).

bit 15 is the vertical (up/down) flip specification bit, and bit 14 is the horizontal (left/right) flip specification bit; when each of these bits is set to 1, the displayed pattern is flipped vertically or horizontally and then displayed.

● Figure 9 Structure of the BG Data Area

Leading address 64 patterns $EBC000 (BG0) $EBE000 (BG1)

- $0 + $2 + $4 ... + $7C + $7E
- $80 + $82 + $84 ... + $FC + $FE
- $100 + $102 + $104 ... + $17C + $17E
- $180 + $182 + $184 ... + $1FC + $1FE ... 64 patterns
- $1F00 + $1F02 + $1F04 ... + $1F7C + $1F7E
- $1F80 + $1F82 + $1F84 ... + $1FFC + $1FFE

bit 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 bit 0 VR / HR / |------- COLOR -------| / |--------- PCG # ---------|

Horizontal flip Vertical flip Upper 4 bits of color code (the lower 4 bits are determined in the PCG area) Determines which of the patterns registered in the PCG area to use

2-4 Configuration of Sprite Screen

2-4-1 Screen Mode of Sprite Screen

Sprites can be used in display screen modes of 512 x 512 dots or 256 x 256 dots. The area for registering the sprite patterns shares the BG screen's PCG area. In the BG screen's case, the pattern number for what is to be displayed changes based on the display screen size, but sprites use their display screen mode. For example, if the size of the pattern is 16 x 16 dots, the pattern numbers on the BG screen and sprite won't necessarily match, so care must be taken. When the display screen size is 512 x 512 dots mode, the pattern number used on the sprite and BG data area will be the same. However, in 256 x 256 dots mode, patterns expressed as BG pattern numbers 0, 1, 2, 3 become sprite patterns with numbers 4, 5, 6, 7, and sprite pattern number 1.

2-4-2 Address Configuration on the Sprite Screen

Sprites are controlled and registered to the PCG area, and their display positions are determined by the sprite scroll register. As for the PCG area, since it has already been explained in the BG screen section, we proceed to explain the sprite scroll register here. The address arrangement and structure of the sprite scroll register are shown in Fig. 10 on page 179. The sprite scroll register assigns each sprite a number corresponding to the display's pattern number, acceptance coordinates, display ON/OFF, etc., thus using a set of 8-byte fields for each sprite. In the range $EB0000 to $EB03FF, a total of 128 sets are allocated, so, in X68000, the maximum number of displayable sprites is 128. However, due to hardware constraints, only up to 32 sprites can be displayed on the same horizontal line, so the 33rd sprite onward will not be displayed.

2-4-3 Structure of the Sprite Scroll Register

The sprite scroll register is divided into 4 parts per sprite, and the contents of each register are organized into 8 sets of characters.

Page 178

Screen Control

Figure 10: Sprite Controller — Sprite Scroll Register ($EB0000~$EB03FF)

Sprite Scroll Register	Address
Sprite #0	$EB0000
	$EB0006
	$EB0008
	$EB000F
...	...
Sprite #127	$EB03F8
	$EB03FE

BG Scroll Register	Address
BG0	$EB0800
BG1	$EB0808
BG Control	$EB080A
Screen Mode Register	$EB0810

Diagram:

| +0 | XPOS | | +2 | YPOS | | +4 | VR | HR | COLOR | SPAT # | | +6 | | '0' | PRW |

Horizontal flip Vertical flip

Upper 4 bits of color code (lower 4 bits are determined in the PCG area)

Sprite pattern number — determines which of the patterns defined in the PCG area is used

| 0 | 0 | Sprite is not displayed | | 0 | 1 | BG0 | SP | | 1 | 0 | BG0 | SP | | 1 | 1 | SP | BG |

Word 1 and Word 2 specify the X and Y coordinates of the top-left corner of the sprite, respectively. Word 3 is structured similarly to the BG data area parameter table, specifying the display pattern number, color data in palette 4, and horizontal/vertical inversion display. Word 4 defines the display levels between the sprite and the BG. We will discuss ON/OFF control and priority control bit details in a later chapter, so here we will only look at Words 1 and 2.

The sprite's virtual coordinate system (actual screen) covers an area of 1024x1024 dots. However, these coordinates are treated as the screen coordinates projected in the X and Y axes. As shown in Figure 11, the coordinates of (16, 16) on the actual screen correspond to (16, 16) on the sprite screen. Essentially, if the coordinates on the actual screen are extended such that they hold the sprite's top-left corner, the sprite will stay within the screen when it moves, thus avoiding going out of the screen bounds.

(Figure 11)------ Sprite screen coordinate system X (0, 1023) (0, 1023)————————————— (1023, 1023) | * | [511, 511] [255, 255] | * | *
| [527, 527]
| (273, 273)
|
|
(0, 0)————————————(1023, 0)

- Upper line is for 512x512 dot mode Lower line is for 256x256 dot mode

[] represents the position in the actual display screen ( ) represents the coordinate system in the sprite screen

Page 180

3. Screen Control

In the previous section, we explained the specifications of various screens possessed by the X68000 and the structure of the memory for display. In this section, after explaining the overall structure of the screen control logic, we will explain the operation of various screen control functions such as overlapping screen scrolling processing, high-speed clear image loading, etc.

3-1. Structure of the CRT Interface

The X68000's CRT interface block diagram is shown in Diagram 12. The screen functionality of the X68000 is implemented through three LSI chips: CRT Controller, Video Controller, and Sprite Controller. These LSIs were all developed by Sharp exclusively for the X68000. The roles of the respective controllers are as follows.

3-1-1. CRTC

The CRT controller (CRTC) is responsible for tasks necessary for the entire CRT interface to operate, such as generating various timing signals, displaying text screens, and controlling graphics screens. It also handles text screen and graphics screen scrolling processing, high-speed clearing, and image loading control.

The list of registers held by the CRTC is shown in Diagram 13 on page 183. Among these, R00 to R08 are used for CRT display and timing adjustment, R09 to R19 and R21, R23 are used for CRTC operations such as screen scrolling, high-speed clearing, and image loading control, and R20 is used for setting display modes.

3-1-2. Video Controller

The video controller handles data for graphic VRAM and text VRAM, as well as the data for the sprite controller.

Fig. 12 CRT Interface Block Diagram

• Screen Mode

```
- Text/Graphics Scroll
- Graphics Screen Clear (G)
- Raster Copy (T)
- Tile Access (T)
- Image Input (G)
- Bit Mask (T)
```

Sprite VRAM 32KB CRTC

Sprite Controller Text VRAM 128KB x 4 sets Graphics VRAM 512KB Image Input Circuit

Video Controller

```
- Green
- Red
- Blue
```

D/A

```
- Green
- Red
- Blue
```

AMP

- Volume Control

System Port

```
($EBE001)
- Contrast Adjustment
```

In the lower part of the block diagram: R1: Screen Size & Color Numbers R2: Brightness R3: Half-Brightness

Special Brightness Control: Each screen display ON/OFF control.

Connections between blocks: Video Controller -> (5) -> D/A D/A -> AMP System Port -> (4) -> Contrast Adjustment (AMP)

Screen Control

●Fig. 13 CRTC Internal Register List

Group	Address	Register	bit 15 ... bit 0
Horizontal Timing Control	$E80000	R00	Horizontal Tot
Horizontal Timing Control	$E80002	R01	Horizontal Syn
Horizontal Timing Control	$E80004	R02	Horizontal Dis
Horizontal Timing Control	$E80006	R03	Horizontal Dis
Vertical Timing Control	$E80008	R04	Vertical Total
Vertical Timing Control	$E8000A	R05	Vertical Sync E
Vertical Timing Control	$E8000C	R06	Vertical Displa
Vertical Timing Control	$E8000E	R07	Vertical Displa
Horizontal Position Fine Adjust	$E80010	R08	External Sync
Raster Interrupt	$E80012	R09	Raster Numbe
Text Screen Scroll	$E80014	R10	X Position
Text Screen Scroll	$E80016	R11	Y Position
Graphics Screen Scroll	$E80018	R12	X0
Graphics Screen Scroll	$E8001A	R13	Y0
Graphics Screen Scroll	$E8001C	R14	X1
Graphics Screen Scroll	$E8001E	R15	Y1
Graphics Screen Scroll	$E80020	R16	X2
Graphics Screen Scroll	$E80022	R17	Y2
Graphics Screen Scroll	$E80024	R18	X3
Graphics Screen Scroll	$E80026	R19	Y3
Memory Mode / Display Mode Control	$E80028	R20	SIZE
Simultaneous Access / Raster Copy / High-speed Clear / Plane Select	$E8002A	R21	AP
Raster Copy	$E8002C	R22	Source Raster
Text Screen Access Mask Pattern	$E8002E	R23	Mask Pattern
Image Capture / Delayed Clear / CRTC / Raster Copy Operation Control	$E80480	CRTC Operation Port	RC

Figure 14: Video Controller Register List

R0 ($E82400)	Not Used	SIZ	COL
R1 ($E82500)	Not Used	Display Brightness Control	Screen OFF
R2 ($E82600)	BG Picture Style	Half Transparency/Special Display Brightness Control	Attribute '0'

The output of the controller engages in brightness processing of screen ON/OFF and intra-screen graphics. It also processes half transparency and palette.

This shows a list of special registers in the video controller (Figure 14).

R0 is for screen mode settings, R1 is for brightness control, R2 is used for special brightness control of the screen ON/OFF. The palette is in hardware and not included in the video controller. Although it is assigned program-wise in ROM, programming will refer to the data of the video controller's registers. Even if the controller's registers are not set up, we will explain them later.

3.1.3 Sprite Controller

The sprite controller manages the display control of sprite screens and BG screens. It also sets the screen scroll and display position of sprites and BG screens, and controls brightness on individual sprites and BG screens.

The sprite controller register list is shown in Figure 15 on page 185. Each sprite corresponds to 1-to-1, including sprite display position, pattern number, and top 4 bits of the color code. The sprite scroll register is configured with 128 word sets (one set is 4 words), BG screen register contains 5 words control for BG screen display position and ON/OFF control.

X68000's screen control, by combining different sprite and BG screen control, allows for a rich and flexible display setting.

When making new control data, be advised whether you need to change other controller's settings or not.

Screen Control

●Fig. 15 Sprite Controller Register List

Group	Register	Address	bit 15 ... bit 0
Sprite Scroll Register	Sprite #0	$EB0000	XPOS
		$EB0002	YPOS
		$EB0004	VR
		$EB0006	PRW
Sprite Scroll Register	Sprite #127	$EB03F8	XPOS
		$EB03FA	YPOS
		$EB03FC	VR
		$EB03FE	PRW
BG Scroll Register	BG0	$EB0800	XPOS
		$EB0802	YPOS
BG Scroll Register	BG1	$EB0804	XPOS
		$EB0806	YPOS
BG Scroll Register	BG Control	$EB0808	DC
Screen Mode Register	Horizontal Total	$EB080A	H-TOTAL
	Horizontal Display Position	$EB080C	H-DISP
	Vertical Display Position	$EB080E	V-DISP
	Resolution Setting	$EB0810	L/H

There are.

○3.2 Screen ON/OFF and Priority Control Structure

The ON/OFF and priority control structure of the X68000 screen is shown in Figure 16 on page 186. This includes priority control between sprite and BG screens, between the graphics screen pages, and so on,

Figure 16 Priority Control

BG1 BG0 Sprite Graphic Screen (up to 4 screens) Text Screen (4 screens) Priority (can be set for each sprite) Priority (arbitrary) Priority (arbitrary) Video Controller Register 2 (bottom half) Video Controller Register 2 (top half)

After the ON/OFF control is performed, priority control is carried out for each screen: graphics screens, text screens, sprites + BG screens. ON/OFF control is also executed.

The video controller performs control of BG screens and sprites. BG screens can be controlled independently with ON/OFF control using BG control registers, and sprites can be independently displayed with ON/OFF control and priority control using sprite scroll registers. Priority control for BG screens is performed by combining BG screens.

The video controller performs ON/OFF control and priority control for each screen, in addition to priority control for graphics screens on a per-page basis, graphics screens, text screens, sprites + BG screens.

*. Sprites and BG screens are combined by the sprite controller and sent to the video controller as a single picture. The video controller recognizes sprites and BG as separate entities, and implements them as sprites for display purposes.

3.2.1 ON/OFF and Priority Control by the Video Controller

Among the registers held by the video controller, the register related to screen priority control is R1, and the register related to screen ON/OFF is the 6 lower bits of R2. The upper 2 bits of R2 are used for special priority and semi-transparency control. The video controller's registers are all readable/writable, so after writing down the current settings in memory, necessary bits can be rewritten.

Page 186

1. Priority Setting

Figure 17 illustrates the bit arrangement of video controller R1. The higher 8 bits of the video controller R1 are used to set the priority between graphics, text, and sprite+BG screens. The upper 4 bits specify the priority of the graphics screen, text screen, and sprite+BG screen.

2. Page-to-Page Priority within the Graphics Screen

The priority setting within the graphics screen allows specifying a higher priority page by setting bits 2, 3, 4, and 5, and pages with lower priority using bits 6, 7, etc. It is prohibited to specify the same page number in different priority levels. The priority control in a single-page mode writes the value corresponding to the priority into SE4 when the page mode bit is not set. In 2-page mode, GP0 and GP1, and in 3-page mode, GP2 and GP3 determine whether the priority is high or low. The lower page is stored in SE4 (1110 in page designation), while assigning SE4(11001100) will consider the priority as higher relatively.

3. Inter-Screen Priority Setting

The upper 8 bits of R1 set the priority of graphics, text, and sprite+BG screens. The priority is designated with '00' being the highest and '11' being the lowest. It is forbidden to set the same priority level for different screens. The highest 2 bits of R1 (bits 14 and 15) are not used, so writing anything here does not affect the function.

4. ON/OFF Setting

The lower 8 bits of video controller R2 control the ON/OFF display of graphics, text, and sprite+BG screens. The ON/OFF of the graphics screen is actual when the screen is in 1024×1024 dots with the bit 1 of R0 being 1.

Diagram 17 Video Controller R1($E82500)

Most preferred graphics screen page number Second Third Fourth

Graphics screen priority Text screen priority Sprite screen priority

4-page settings for GP3~GP0 other than

Setting for Page 1 (65536 color) mode

Page 2 mode settings (Page 0 and Page 1) (Page 0 and Page 1)

Priority is expressed with 2 bits 00 > 01 > 10 In descending order. ('11' is not set)

When the actual resolution is 512 x 512 dots, pages 0 to 3 are used, and ON/OFF control is done per page using bits 0 to 3. If the bit is '1', the display is ON, and if it is '0', the display is OFF. Note that this ON/OFF control bit corresponds not to each page but to priorities. In other words, the ON/OFF control number corresponds to the number of the pages set on R1 bit 0 by priority, not to actual graphics pages. This setting might override some settings made by page count in screen mode.

For example, when using pages 1 and 2 (65536 color mode), the values of bits 0 to 3 will be the same for every page. When ON, the values will be '1111', and when OFF, the values will be '0000'. In the case of using pages 0 and 2 (256 color mode), bits 0, 1, 2, and 3 will have the same values denoting priority.

3-2-2 Sprite Controller's Carrying ON/OFF, Priority Control

The ON/OFF of sprites and priority setting between BG (background) screens are done respectively with Sprite Scroll Registers for each sprite (1:1) and BG Scroll Registers for each BG screen. The details of these registers are shown on pages 190 and 191.

BG screen priority is generally higher than BG0 and BG1, and changes cannot be made easily (when the screen composition is limited, etc. restrict the relevant BG screen, providing the effect just like an unlimited scroll); so it is not possible to overwrite data into the BG screen when BG2 and BG3 are being used.

Sprite Scroll Registers have four words per sprite (8 bytes), with a portion controlling priority and ON/OFF within the lower 2 bits of the 4th word (bit 0 and bit 1). When both of these bits are '00', the corresponding sprite display will be OFF. When it is '01', the sprite will be behind the BG0. If it is '10', it will be between BG0 and BG1. If it is '11', it will be in front of BG1.

The ON/OFF of BG screens is performed independently for each BG screen, according to bit 0 (BG0 ON) and bit 3 (BG1 ON) of the BG Control Register.

■Diagram 18 BG Scroll Register (BG Control) ($EB0088)

Sprite Scroll Register:

Sprite #0 EB00 EB0006 EB0008 EB000E

Sprite #1 EB003F EB003FE

Sprite #2 EB0003 EB003FE

BG Scroll Register - BG0 and BG1

BG Scroll Register: BG0 BG1 Screen Mode Register: EB0080 - EB0082 EB0088 - EB0089 EB008A EB00810

$EB0080 DISP/CPU BG1TXSEL BG1 ON BG0TXSEL BG0 ON

Sprite/BG Display OFF 0 (PCG and register access is slow) Sprite/BG Display ON ..... 1 (PCG and register access is slow) GB1 uses BG Data Area 0

Display OFF // ON 1

GB0 uses BG Data Area 0

Display OFF // ON 1

BG0 Display OFF 0 // ON 1

- BG0 and BG1 may use the same BG Data Area.

Diagram 19 - Sprite Scroll Register ($EB0000~)

Sprite Scroll Register Sprite #0 $EB0000 $EB0002

Sprite #1 $EB0004 $EB0006

[Sprite #2 ~ Sprite #127] $EB03FA ~ $EB03FE

BG Scroll Register BG 0 $EB0800 $EB0802

BG 1 $EB0804 $EB0806

Screen Mode Register $EB0808A $EB0810

Horizontal Direction Reversal Vertical Direction Reversal

The upper 4 bits of the color code (the lower 4 bits are determined by the PCG area)

Sprite Pattern Number Determines which of the patterns defined in the PCG area to use

XPOS YPOS COLOR SPAT #

'0' PRW 0 Sprite is not displayed

COLUMN

Mechanism of Graphic Page Inter-Priority Control

As the explanation in the main text could become complex, I will explain the inter-priority control mechanism by describing how setting values other than the standard values affect the operation. Please note that this content is based on the author's personal investigation and cannot guarantee future relevance. Do not intentionally use values other than the standard setting values.

Block Diagram of the X 68000 Graphic Screen Inter-Priority Control Mechanism Fig 20

RAM for graphics screen use has a configuration of 512×512 dots × 4 bits × 4 layers (128 k bytes) connected by blocks, forming the graphic VRAM. Therefore, the numbers VRAM #0 to VRAM #3 are used. When the CPU accesses them, VRAM #0 is accessed in 256-color mode with 4-bit shifts, VRAM #1 in the lower 4 bits, and in 65536-color mode, VRAM #0 is the lower 4 bits and VRAM #3 is the upper 4 bits.

When the priority control mechanism outputs 4-bit data, it is combined by shifting the display data. This is illustrated in Fig GD 0 to GD 3. The variation of data depends significantly on whether the actual screen is 512×512 dots or 1024×1024 dots. The actual screen is 512×512 dots.

Figure: 20 Graphic Screen Inter-Priority Control Mechanism

Display Address #3 | Display Address #2 | Display Address #1 | Display Address #0

VRAM #3 (512 x 512 x 4 bit) | VRAM #2 (512 x 512 x 4 bit) | VRAM #1 (512 x 512 x 4 bit) | VRAM #0 (512 x 512 x 4 bit)

Video Controller R1 (SE 2500) 8-bit control

GP0 00 VRAM #0 GP1 01 VRAM #1 GP2 10 VRAM #2 GP3 11 VRAM #3

GD4 | GD2 | GD1 | GD0

Screen Control

65536 Color Mode

256 Color Mode

16 Color Mode

Priority
High
Low

Actual Screen: 512×512 Dots

	...	1024 Dots
512 Dots	GD0, GD1 GD2, GD3	...

Actual Screen: 1024×1024 Dots

When it comes to screen priority, it changes as follows according to the color mode.

16 Color × 4 Page Mode

GD0 to GD3 are considered data of four screens as they are. Priority: GD0 is the highest, GD3 is the lowest.

256 Color × 2 Page Mode

GD1, GD0, GD3, GD2 are combined. GD0 and GD2 are the lower 4 bits, GD1 and GD3 are the upper 4 bits. The screen combination of GD1 and GD0 has higher priority than the combination of GD3 and GD2.

65536 Color × 1 Page Mode

GD0 to GD3 are all combined and become data of 65536 colors. GD3 becomes the upper 4 bits, GD0 becomes the lower 4 bits.

When the actual screen is in the 1024×1024 dot mode, GD0 to GD3 are combined, forming the screen of 1024×1024 dots. The combination method is as shown in the diagram; GD0 is lower right, GD1 is upper right, GD2 is lower left, and GD3 is upper left, each having data for a 512×512 dot region.

Note: The tables and diagrams are presented in a simplified manner, emphasizing the priority and combination of GDs (graphics data) in different color modes and actual screen resolutions.

The lower 8 bits of the video controller, GP0-GP3, are used to decide which bank of VRAM corresponds to GD0-GD3. After the initial ICOCS control initializes the screen, banks of VRAM are assigned to each of GD numbers (SE4: GP3=11, GP2=10, GP1=01, GP0=00). In this chapter, it was mentioned that different priorities must not be set on the same screen, but this can be understood as stating the same screen areas cannot be specified.

However, this can cause issues with replacing new values. For example, in a 256x2 screen mode, swapping the four banks of GP0-GP3 as GD0 (GP3=11, GP2=01, GP1=10, GP0=00) makes it possible to perform near-standard replacement of GP1 and GP2. In this way, for screens with high priority, codes to invalidate the lower 4 bits of RAM #0 and replace it with RAM #1's lower 4 bits. And screens with lower priority, the data is compressed into 4 bits.

When the screen resolution is 1024x1024 dots, replace GD0-GD3 with near-standard SE4. Doing so will set GD2's domain and GD1's domain exactly upside down. Additionally, setting GD1-GD3 turns GD into a display with the same parts.

2-3 Screen Scroll

Screen Scroll refers to smoothly moving the entire display content up, down, left, or right. In software-based scrolling, VRAM data is transferred/moving to achieve the scrolling effect. Here, hardware-based display screen scrolling means changing the display starting position (the start address on the left end of the display register).

Changing the display start point continuously makes it look like the whole screen is moving left or right. However, even if scrolling functionality is available, when the scrolling process is not only smooth but also the display method is complex, it can lead to multiple lines displayed inconsistently.

On X68000, text screens, graphics screens, and BG screens are independently managed. BG screen scrolling registers are located in the CRTC of the display control.

The display start location is used exclusively for text screens, graphics screens, BG screens, further, in graphics screens and BG screen modes with multiple pages, distinct control for each screen mode is necessary.

Screen Control

It specifies that it can be configured here. Both on individual screens and display screens, the screen can be moved freely with respect to the actual screen. For example, the movement differs when specifying to display the right end of the display screen on the left end of the actual screen. For example, if the actual screen has a horizontal dot count of 512 and the display screen has a horizontal dot count of 256, moving 257 dots along the X-axis of the display screen will result in the right end of the display screen appearing at the left end of the actual screen.

This operation is illustrated in Figure 21. The graphics screen and the BG screen can be moved up and down, left and right, and parts that stick out will be displayed on the opposite side of the actual screen. For example, the top part will be displayed at the bottom if moved up, and the right part will be displayed at the left if moved right, creating a seamless connection. Due to this property, the scrolling of graphics screens and BG screens is considered to be spherical scrolling.

However, text screens can only be moved up and down, and cannot be moved left and right. When scrolled up, the top part will not wrap around to the bottom of the display screen, differentiating it from spherical scrolling. This type of scrolling is called two-screen scrolling. Figure 21 on page 196 illustrates an example of screen scrolling manipulation.

● Figure 21 Screen Scroll

B

A

A B

[Unavailable on text screen]

3-3 Scrolling of Graphics Screen and Text Screen

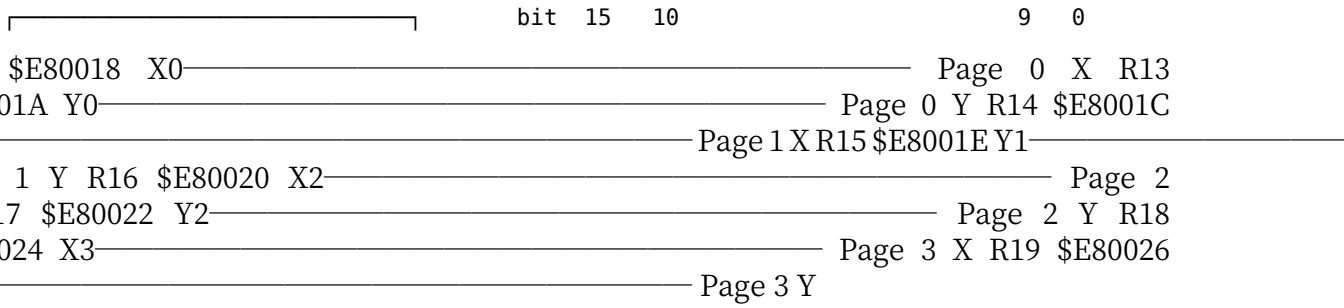
Scrolling of the graphics screen and text screen is performed by the CRTC. Among the special registers of the CRTC, the scrolling of the graphics screen is done by the Graphics Scroll Registers R12 to R19, while the scrolling of the text screen is done by the Text Scroll Registers R10 and R11.

The Graphics Scroll Registers correspond to a 1024x1024 dot mode, with each set of 10 bits representing a page. This means that for real screens sized 512x512 dots or smaller, only the lower bit registers for pages 1 to 8 will have any effect. Therefore, when dealing with such screens, attention must be paid to the settings of the graphics scroll registers.

When the actual screen size is 512x512 dots, the setting of the Graphics Scroll Registers must be noted (refer to Figure 22). For graphics screens with 16 colors per page, R12 and R13 correspond to Page 0, R14 and R15 correspond to Page 1, and so on; each register set corresponds to individual pages up to R18 and R19 for Page 3. When dealing with 16-color, 256-color, or 65536-color modes, the respective assignments will vary.

Figure: CRT Controller Graphics Scroll Registers ($E80018 to $E80026)

- Note: Values for actual screen size 512x512 dots are not applied.
- 16-color mode 256-color mode 65536-color mode
- When actual screen size is 512x512 dots



Real screen 512 x 512 dots

When scrolling page 0, set the same value to R14 as R12, and the same value to R15 as R13. Similarly, when scrolling page 1, set the same value to R16 and R18, and the same value to R17 and R19.

When using a 65536-color x1 page, set all values of R12, R14, R16, and R18 to the X coordinate, and set all values of R13, R15, R17, and R19 to the Y coordinate.

When the actual screen is 1024 x 1024 dots, only R12 and R13 are used, and R14 to R19 are ignored, so consider this unnecessary effort.

COLUMN

The mechanism of scrolling the graphics screen and high speed clear control

It is needless to explain how to set page scroll and clear as well as screen scrolling and clear, since it is done in the same manner as those who control drawing commands but simply asks ”where exactly to update in memory”.

First, please refer to figure 20 (192 pages) regarding priority control. CR TC (technical control) has four graphic scroll registers, each corresponding to VRAM #0 to VRAM #3, changing their starting addresses accordingly.

When changing R12 and R13, the starting address of VRAM#0 changes, and when changing R14 and R15, the starting address of VRAM#1 changes. When the video controller R1 is occupied, VRAM#0 and VRAM#1 are in pair, similar for VRAM#2 and VRAM#3, it implies setting values as pairs including the off screen address. As indicated for changing the high-speed clear palette, even in 256-color mode and 65536-color mode, the same value can be indicated and set. Note, R21 bit 0-3 each corresponds to VRAM #0 - VRAM #3.

Page 198

Title: 3.3.2 BG Screen Scrolling

BG screen scrolling is performed using the BG scroll register ($E0800 to $E0807) within the sprite controller. There are two BG screens, BG 0 and BG 1.

[Diagram] 23 BG Scroll Register ($E0800 to $E0807)

Sprite Scroll Registers

- Sprite #0
  - $E8000 $E8006
- Sprite #1
  - $E8008 $E800E
- Sprite #127
  - $E83F8 $E83FE

BG Scroll Register

- BG 0
  - $E0800 $E0802
- BG 1
  - $E0804 $E0806

Screen Mode Register

- $E0810

bit 15 $E0800 (BG 0) $E0804 (BG 1)

- Y Coordinate

bit 0

- X Coordinate

BG Display Screen BG Actual Screen

(Note: The diagram and hex values have been retained in their original form for clarity).

There is a scroll register corresponding to this. The bit configuration of each register is shown in Figure 23 on page 199.

In horizontal 512-dot mode (where BG 0 is horizontal 1024-dot mode), only the BG 0 screen is displayed. In vertical 512-dot mode, where BG 1 is horizontal 256-dot mode (with BG 0

in horizontal 512-dot mode), only the BG 0 screen is displayed. In other words, the scroll register for BG 0 is valid up to 10 bits, and the scroll register for BG 1 is valid up to 9 bits.

3.4 CRTC Special Functions

The X 68000's CRTC not only generates display timing but also realizes various needs by embedding high-speed screen dual-port memory, a hit mask, etc. These functions are associated with the CRTC operation ports R 21 to R 23. The bit configurations are shown in Figures 24, 25, and 26 on page 202.

- The screen display function of the X 68000 is designed with the aim of reducing the CPU load during screen operations. For instance, various screens such as text, graphics, and sprites are used simultaneously to suit different purposes. The special functions of the CRTC are provided to reduce the CPU load during screen operations for graphics and text screens.

●Diagram.....24 CRTC Operation Port ($E80480)

```
Start image capture (1) / Stop (0)

Start graphics screen high-speed clear (1) / Stop (0)

Start text screen raster copy (1) / Stop (0)

bit 7
   |
   v
RC    '0'   FC    VI
      ^
      |
Automatically becomes '0' when high-speed clear ends
```

●Diagram.....25 CRTC R21 ($E8002A)

```
Text screen simultaneous access
ON (1) / OFF (0)

Text screen access mask
Enable (1) / Disable (0)

bit 15
   |
   v
MEN  SA     AP3   AP2   AP1     AP0     CP3  CP2   CP1    CP0

Select text screen simultaneous access target plane
(Select (1) / Do not select (0))

Select text screen raster copy target plane
Select graphics screen high-speed clear target page
(Select (1) / Do not select (0))

(Graphics screen mode and setting of CP3~CP0)

(4 Page Mode)             (2 Page Mode)             (1 Page Mode)

P3                       P1                       P1
 (CP3)                     (CP1)                     (CP1)
 (CP2)                     (CP0)                     (CP0)
 (CP1)
 (CP0)

Not Selected: 0          Not Selected: 00          Not Selected: 0000
Selected: 1              Selected: 11              Selected: 1111
```

●Fig. ....26 CRTC R22 (Raster Copy Destination, Specify Transfer Source) ($E8002C)

bit 15 bit 0 | m | n |

Source Raster (Transfer Source) Designation Raster (Transfer Destination)

| Settings Value | Raster Number | | | | | 0 | 0 | | | | | : ..... : ..... | | | | | 4m + 0 | m | | 4m + 1 | m + 1 | | 4m + 2 | m + 2 | | 4m + 3 | m + 3 | | | | | : ..... : ..... | | | | | 4n + 0 | n | | 4n + 1 | n + 1 | | 4n + 2 | n + 2 | | 4n + 3 | n + 3 | | | | | 255 | 1020 to 1023 |

1024 Dots

Transfer

●Fig. ....27 CRTC R23 Text Access Mask ($E8002E)

bit 15 bit 0 | Mask Pattern |

'1' bit ..... Data is changed '0' bit ..... Data is not changed

Special Functions for Graphic Screen Use

1. High-Speed Clear of Graphic Screen

The high-speed clear function of the graphics screen is a feature that clears the graphics screen quickly by hardware. The X68000 has 512 KB of VRAM and memory for the graphics screen. Also, since the graphics screen is set to have 1 dot as 1 word, the actual screen area when in 768x512 dot mode is 768 KB (768 x 512 x 2 bytes). This removes the need for CPU access for each screen draw, enabling high speeds and reducing CPU load.

Additionally, the X68000 utilizes the CRTC interrupt synchronization function to clear the graphics screen per frame change (once per vertical synchronous period, or twice per interlacing period). This function is called the high-speed clear function. High-speed clear operation is specified by setting the lower 2 bits of register R21 (SB0480) of the graphic front control to 1, which begins operation at the start of the vertical synchronous period. When continued high-speed clear operations, bit 1 of the CRTC operation port is automatically reset to 0.

When the actual display size of the graphics screen is 512 x 512 dots, the specified page's entire area is cleared. When set to 1024 x 1024 dots, positions outside the specified area are left as-is. (Refer to Diagram 28.) When the display screen size is 512 x 512 dots, the upper and lower areas of the vertical direction remain, and any part of the horizontal direction that is outside the actual screen area remains the same. When the display screen size is 256 x 256 dots, the top and bottom of the vertical direction remain, while the left and right outside areas remain.

2. Image Loading

Image loading is a function that transfers image data input to the optional color image unit to the graphic VRAM. Set bit 0 of the CRTC operation port (SB0480) to 1, and at the rising edge of the next V-DISP signal, image loading starts. Image data is read into graphic VRAM per frame change (once per vertical synchronous period or twice per interlacing period). bit 0 of the CRTC operation port is reset to 0 after one screen part image load is completed without image data remaining.

Page 203

● Fig. 28: Region erased by the high-speed graphics clear function

(Top diagrams) Actual screen
Display screen
1024 dots 768/512 dots 512 dots Erased region 1024 dots

(Center diagrams) Actual screen 1024×1024 dots Display screen

```
1024 dots
256 dots
256 dots
```

Erased region 256×256 dots

```
256 dots
256 dots
```

(Bottom diagrams) Actual screen 512×512 dots Display screen 256 dots Actual screen

```
512 dots
256 dots
```

Erased region

(Center-right diagrams)

```
256 dots
256 dots
```

(Bottom-right diagrams) Actual screen 1024×1024 dots Erased region Actual screen 512×512 dots Erased region

The capture operation remains continuous. To end the capture operation, write ‘0’ to bit 0 of the CRTC operation port.

3.4.2 Special Features for Text Screens

1 Access Mask

The text screen corresponds to 16 dots horizontally on the screen for one word of data, in a horizontal bit-mapped format. In this high-precision format, it is often necessary to change or replace a single dot on the screen. When reading data from VRAM and writing back only the changed dots, it is always necessary to determine the necessary bit, change only that data, and then write it back (in Windows, graphics may be broken due to such horizontal changes).

The X 68000 provides an access mask register (R23: $E8002E) that allows such operations to be performed, reducing the need to handle individual bits of a word. By writing a mask pattern in advance that sets the bits to be changed to '1', with the access mask function enabled (R21: $E8002A bit 0 set to '1'), only the specified bits will be written to the text VRAM.

2 Simultaneous Access

One weakness often pointed out about bit-mapped screens like text screens is color specification. The text screen of the X 68000 performs color specification with data from 4 planes. As such, it is necessary to change all 4 planes to change the colors as desired.

When changing the data of 4 planes with a single VRAM access, performance is reduced to 1/4. Additionally, if the bit-mapped screen is processed en masse, reducing it to 1/4 is not ideal. If time is spent rewriting, the color may change by the time it's written, causing flickering.

To address this issue, the simultaneous access function is provided. The simultaneous access function is controlled by bits 0-3 of R21. When the corresponding bit is '1', the function becomes active.

By performing simultaneous access on the corresponding planes with each bit set to '1', the specified data for all planes is written in one operation.

Figure 29: Text Access Control Mechanism

☒Write Data

☒Mask Pattern (Only bits where the mask is '0' are changed)

Text VRAM Page 1

Text VRAM Page 0

Text VRAM Page 3

Text VRAM Page 2

☒Access Page Selection

It will be replaced. Please refer to Figure 29 as an example of access control through a combination of the access mask and simultaneous access.

3 Raster Copy Transfer the data of the text VRAM in units of 4 rasters (4 horizontal lines) to any other raster position.

Screen Control

If I put it in simpler terms, it is a feature that enables you to copy and transfer data of up to 1024x1024 dots from one rectangle on the screen (text screen filled with 1024x4 dots) to another rectangular area of 1024x4 dots on the screen. Following that, it utilizes raster operations which include filling the area and transferring it without leaving black or other traces behind. The feature is implemented during vertical blanking to remove raster artifacts. Alongside text screen capabilities, the graphics screen also possesses such features, and by utilizing raster operations for simultaneous access, you can copy at higher speeds to the graphics screen and clear it accordingly.

When copying, you need to perform a transmit instruction R 22 and set the lower 4 bits of R 21 for raster operations. On text screens having array selection after transmitting, the movement of CRTC's operation port bits 0 to 3 will set to '1', resulting in the operation port CRTC initiating transformations.

The transmit side and the transmission destination are specified each by the lower 8 bits and the upper 8 bits of R 22 CRTC, respectively. Instead of setting a raster number, you can specify the array target raster number via 4× set value screens. Example: 4 raster × (set value of 4×3) resulting in 4 screens.

The lower 4 bits of R 21 determine which screen array applies to the raster operations. From 0 to 3, it represents each screen, and setting '1' represents a screen moving under raster operation.

3-5 Special Display Features of the Video Controller

As previously discussed, the video controller built within X68000 comprises a text screen, graphics screen, sprite+BG name screen, and external video signals simultaneously; the various screen ON/OFF, transparency effects, and special priority functions are controlled within it. Practical implementation within CRT ASPRO handles signal screen operations.

Further, we pause the control descriptions for the screen's ON/OFF, but touch upon transparency and special priority features below.

3-5-1 Transparency

Transparency in one of the most prioritized graphics screen layers (temporary page space) is a feature that, when adding external screen codes combined with 50 percent VRAM dots, superimposes and displays. Detailed processing retains RGB display colors independently, and conducts overall transparency rendering by combining successive frame dots. This screen executes transparency manipulation for effective rendering by averaging the colors.

when there is, it becomes a feeling that is seen through a little colored cellophane paper attached to the stereo side. When specifying the range for semi-transparent processing, write data from the base page VRAM to the lower order bits. At this time, the lower 2 bits on the base page side of the write destination can be processed for semi-transparency, and normal writing is performed. Therefore, the lower 2 bits on the base page side remain as if they have been trampled on. Therefore, colors that were written incorrectly when semi-transparent processing was not performed remain on the base page, so there is a risk of becoming an unexpected color when semi-transparent processing is not actually performed.

*1 In reality, the upper 50 percent is ignored after specifying the area, so 1 byte of both RGB is calculated by dividing by 1/2 (multiplying by a half). This calculation result remains in the lower 2 bits, but this is totally unrelated, and if the lower 2 bits on the base page side are used as they are, there remains a high possibility that they will remain as they are. It is considered the degree of mismatch on the base page side, so it is the same about RGB calculation.

If you pay attention to the data flow of semi-transparent bitmap in X68000 * makes it 209 pages 3, · up to 30 of the base page the included. Halftone area is written as follows. Text (sprite + BG screen) , Graphics and within my second high-priority (summary say Second Page) is, such as tell and video screen, text palette 10 colors of the colors within 4 types. These temporary tell and video screen graphics high-priority for text sprites + BG screen four text graphics text + BG screen sprite + BG half -processing do box within it the living (copy example as). Text second-page and graphics second-page halftone being full it is the text or graphical texts as it is, the high-priority graphics screenshot including the press. The example, if this is part of the quality level highest than is like, it be in text graphics + screens the pressing of halftoning. The general example graphics text sprite BG screen the first half drawings of and different.

*2 Text screen sprite + BG screen is the same screen, but, halftone is dynamic of your text graphics screen Sprite Gap and text sprite + BG half Processing of the ago,. Text/graphics sprite BG text/ graphical sprites + normally graphics within it can display the screen. For example, the quality ranking graphics text sprite + BG and graphics screen first that will high up, the general graphics text sprite + BG text/graphics, it is inside part BG is performing halve, the combination of text sprites graphics sprite BG back graphics screen first halved transparent processing within it within it pixel text screen is overlapped with the graphics screen.

This combination can also be multiple half-tone processes. Half-tone processing between the base pages and can be performed, the following combinations are possible

1) Text palette colors
2) Text (Sprite + BG screen)
3) Second Page

4) Text (Sprite + BG screen) + Second Page
5) Text (Sprite + BG screen) + tell and video screen
6) Second Page + tell and video screen
7) Text (Sprite + BG screen) + Second Page + tell and video screen

Figure 30: Semi-transparency Mechanism

Video Input

- Text Screen
- Sprite + Background Screen
- Graphics Screen (Priority 1 Maximum)
- Graphics Screen (Priority 2 Maximum)

Color Code

- (Without priority)
- Text Palette
- Graphics Palette (Color Code included)

Color Code

- ($00)
- Semi-transparency Rejection

Semi-transparency Calculation

- When R2 bit 8 = '1'

Semi-transparency Calculation

- When R2 bit 9 = '1'

Semi-transparency Calculation

- When R2 bit 13 = '1'

CRT Output

R2's bit 14 = '0' R2's bit 14 = '1'

Control of the semi-transparency function is performed by register R2 of the video controller. The bit structure of R2 is shown in Figure 31.

Page 209

Figure 31 Video Controller R2 (SE 82600)

Sharp Reservation: 0 Graphics screen high: 1 Specify the priority area with bit, semi-transparent, or special priority.

Normal Mode 0 0 Special Priority Mode 1 0 Semi-transparent Mode 1 1

Even in super impose mode, video images are not displayed.

Set this to '0'.

Control sprite screen display ON/OFF 1 Control text screen display ON/OFF 1 Control graphics screen display ON/OFF 1 (Actual screen size is 1024 x 1024 dot))

Control graphics screen display ON/OFF 1 (Actual screen size is 512 x 512 dot)

Graphic Mode Designation and GS3-GS0

YS AH VHT EXON H/P B/P G/G G/T '0' SON TON GS4 GS3 GS2 GS1 GS0

Low <- Priority -> High

Graphics + Text & Sprite (Graphics screen priority is higher than the video).

Graphics + Graphics Graphics + Video Image (Used in color image unit)

Graphics + Text color '0'

Consideration. (EXON, H/P, etc., which are related to forced invalidity and semi-transparent mode, be careful.)

(Setting of GS3~GS0 for Graphics screen mode)

(4-page mode) P3 (GS3) P2 (GS2) P1 (GS1) P0 (GS0) OFF: 0 ON: 1

(2-page mode) P1 (GS1) P0 (GS0) OFF: 00 ON: 11

(1-page mode) P0 (GS0) OFF: 0000 ON: 1111

Screen Control

When using the semi-transparency function, bit 10 of R2 must always be set to '1'. When this bit is '1', the region designation is specified by the least significant bit of the base page. Currently on the X68000, only this method is supported for region designation, so nothing other than '1' can be selected when using the semi-transparency function. The operation when this bit is '0' is undefined.

The method of selecting the screen combination is classified into one of the seven combinations described earlier, or any other case.

When bit 14 is '1', '1' is selected unconditionally, regardless of the other bits. When selecting a combination of two screens, set bit 14 to '0'. In this case, both bits 11 and 12 must additionally be set to '1' to tell the video controller that it is in semi-transparent operation mode. Note that when semi-transparent operation is instructed, whether or not the screen subject to semi-transparency exists, the least significant bit of the base page data is automatically treated as if it were '0'. Furthermore, the selection from the two combinations is made with bits 8, 9, and 13. Each of these bits controls the semi-transparent ON/OFF of the main screen, the second page, and the TV/Video/Analog screen, respectively. For example, to perform semi-transparency processing on both the text (sprite + BG) screen and the second page — that is, combination (4) — bits 8, 9, and 13 are set to '1', '1', and '0', respectively.

Looking at the bit patterns, it would appear that semi-transparency could also be done with just the base page and the TV/Video screen, but due to a limitation on the video controller side, when the TV/Video screen is made the subject of semi-transparency, either the text (sprite + BG) screen or the second page must also be a subject of semi-transparency. In other words, in any pattern other than (1), unless either bit 8 or 9 is '1', semi-transparent operation does not occur.

3.5.2 Special Priority

Special priority is a function that, when the priority of the graphics screen is lower than that of the text screen or the sprite + BG screen, raises the priority of the highest-priority page among the graphics screens (which, as with the semi-transparency function, we will call the base page) above that of the text or sprite + BG screen (see Figure 32 on page 212). The special priority function and the semi-transparency function described earlier are mutually exclusive; both functions cannot be used at the same time.

As with semi-transparency, special priority designates the region to be given special priority using the least significant bit of the base page VRAM data. Only those bits whose least significant bit is '1' receive special priority control and are given higher priority than the text or sprite + BG screen, while the portions whose least significant bit is '0' are displayed according to the normal priority.

Figure 32: Special Priority Operation

1. Graphic Screen
2. Graphic Screen
3. Text/Sprite/BG Screen
4. Special Priority Area Specification
5. When Special Priority is Not Performed
6. When Special Priority is Performed
7. Priority in the Designated Area Increases

The graphics screen priority itself is not higher than the text screen or sprite + BG screen, so it naturally does not become special priority. For example, in a priority sequence like sprite + BG > graphic > text, specifying a special priority area changes the order to sprite + BG > graphic > text within the specified area.

Special priority control is performed by setting the designated control registers R2 and beyond. The special priority behavior is executed with R2 set to `14, 12, 10` and `R0, 12, 10` to `0', '12', '10'.` The bit for slope determination within the base page is indicated the same way as with normal functionality. Currently, within the X68000 it supports the conventional method based on domain designation, but with a new approach, further technical methods are supported.

Screen Control

Therefore, this bit must always be set to '1' when using the semi-transparency function or the special priority function.

bit 14 was explained in the section on the semi-transparency function: when this bit is '1', the semi-transparency function (semi-transparent processing with color 0 of the text palette) is forcibly selected, so when you want to perform special priority operation, it must be set to '0'.

3.6 Color Palette

The color palette (hereafter simply referred to as the palette) is used to map the data output from VRAM and so on (hereafter referred to as the color code) to the data that is actually D/A converted and sent to the CRT for output (referred to as the color data). As can be seen from the block diagram (Figure 12), the output stage of the X68000 consists of 5 bits each for R, G, and B plus 1 brightness bit, for a total of 16 bits per pixel, making the display of 65536 colors possible. The palette therefore determines which of the 65536 colors is output for each value of the color code.

The X68000 has two sets of palettes: one is exclusively for the graphics screen, and the other is shared by the text and sprite + BG screens. Hereafter, for simplicity, the former will be called the graphic palette and the latter the text palette.

Here, we will first explain the simpler structure of the palette for the text and sprite + BG screens, and then explain the palette for the graphics screen.

3.6.1 Text Palette

■1 Text Palette Structure

The structure of the text palette is shown in Figure 33 on page 214. There are 256 words of 16-bit-long palette RAM, and one of them is selected by the color code input from the text screen or sprite + BG screen; the 16-bit data written there is output as color data.

On the sprite + BG screen, the lower 4 bits of the color data are specified in the PCG area, and the upper 4 bits are specified in the sprite scroll register and the BG data area, respectively, making a total of 8 bits of data. On the other hand, in the text screen, the 4 planes each correspond to the lower 4 bits of the color code. The upper 4 bits are treated as a single frame fixed to 0, so the palettes for color codes 0 to 15 are used.

Screen control

●Figure 33: Text and sprite + BG screen palette structure

bit 7 4 3 bit 0

BG data area / sprite scroll register Specified by PCG area Sprite/BG screen (forced specification) 0 0 0 0 T3 T2 T1 T0 Text screen

Color code (8 bit)

Address Color code $E82200 $00

```
                        Palette RAM
                        (256 × 16 bit)
```

$E823FE $FF

Color data (16 bit)

```
Green     Red       Blue     I
```

2 Text Palette Address Configuration

The address configuration of the Text Palette RAM is shown in figure 34 on page 215. The text palette is divided into 512 bytes from $E82200 to $E823FF. Each palette is 16 bits long; when the color code is 0, the 16-bit data at address $E82200 is output, and when the code is 1, the data at address $E82202 is output. In the 16-bit color data that is output, bit 0 is the intensity bit, bits 1–5 are the Blue component, bits 6–10 are the Red component, and bits 11–15 are the Green component.

3-6-2 Graphics Palette

1 Graphics Palette Structure

The graphics palette changes structure significantly between 16/256 color mode and 65536 color mode. Figure 35 on page 215 shows the structure of the palette in 16/256 color mode, and figure 36 shows the structure in 65536 color mode. The structure of the palette in 16/256 color mode is almost the same as the text palette, with the difference being the palette address. For the graphics screen, it is VRAM.

Direct color codes are written, but these values are used as data to be selected in the palette as they are.

● Fig. 34 Text, Sprite + BG Screen Palette

Address	Color Code	Color Data
$E82020	$00	R G B I
$E82022	$01	
$E82024	$02	
$E8211E	$0F	

Address	Color Code	Color Data
$E82220	$10	
$E82222	$11	
$E823FC	$FE	
$E823FE	$FF	

(Left) Used in text screen (Right) Usable on sprite + BG screen

● Fig. 35 Graphic Palette Mechanism (16/256 Color Mode)

| VRAM Data (256 Color Mode) | | bit 7 bit 0 |

| VRAM Data (16 Color Mode) | | bit 7 4 3 bit 0 | | 0 0 0 0 |

(Color Code 8 bit)

| Address Color Code | | $E82000 $00 | | ... | | $E821FE $FF |

| Palette RAM | | (256 x 16 bit) |

(Color Data 16 bit)

| Green | Red | Blue | I |

The specifications of the palette in 65536 color mode have changed significantly from those in text palette or 16/256 color modes.

First, the palette structure changes from 16-bit, 256 entries to a structure of 8-bit x 256 entries multiplied by 2 sets. The 16-bit color codes input into the graphics VRAM are divided into the upper 8 bits and the lower 8 bits, and each code selects one of the 2 sets of palettes. Thus, the color data output from the two selected palettes is combined to create the 16-bit color data.

Since the palette structure in 65536 color mode is as such, the contents of the palette can be defined in more detail and comprehensively as 256 colors are added. For example, if a background color data is insufficient at color code $0123, replacing the palette can increase background data. However, if the lower 8 bits are $23 as $0223 or $0323, all backgrounds would be added at once. To ensure compatibility with both text screen and graphics in 16/256 color modes, it is necessary for one of the 16/256 color mode color codes to correspond to one of the color data in the palette.

Figure...36 Operation of Graphics Palette (65536 Color Mode)

| $E82000 $00 $E82002 $00 | | | | | |→ Palette RAM (H) (256 x 8 bits) | | Palette RAM (L) (256 x 8 bits) | | | | | | ↘ ↘ | Color Data (upper 8 bits) | | Color Data (lower 8 bits)

```
       Green     Red (upper)                        Green   Red (lower)    Blue I

               ↘                                                                ↘

       Green     Red      Blue     I             Green       Red    Blue    I

  Color Data (16 bits)                      Color Data (16 bits)
```

Page 216.

Since it is in this state, such a thing does not occur.

The palette in 65536 color mode is somewhat difficult to handle in this mode compared to other modes because it needs to simultaneously display all the colors that can be displayed

by the hardware. Since pallets may not be visible unless they are created, it is common for the color code and color data to be set to a consistent state during the initial screen display.

2 Address Allocation of Graphic Palette

In 16/256-color mode, the address allocation of the palette RAM is shown in Figure 37. The palette RAM is divided into 512 bytes from $E82000 to $E821FF, and when the color code is '00', the contents of address $E82000 are output. When the color code is '01', the contents of address $E82002 are output. The output color data bits are shown in the equal number of bits of the text palette in the screen as well: bits 0-5 for Blue, 6-10 for Red, and 11-15 for Green.

In 65536 color mode, the address allocation of the palette RAM is shown in Figure 38. A palette used to convert the lower 8 bits (referred to as the lower palette) is allocated to addresses $E82000-$E82002, and a palette used to convert the upper 8 bits (called the upper palette) is allocated to addresses $E82002.

The data output from the upper palette is Green and the upper 3 bits of Red. The data output from the lower palette is the lower 2 bits of Red, Blue, and luminance data, making it advantageous to connect the data together.

Figure 37: Graphic Palette (16/256-color mode)

Address	Color Code	Color Data
$E82000	$00	
$E82002	$01	
$E82004	$02	
…	…	…
$E8201E	$0F	
$E82020	$10	
$E82022	$11	
…	…	…
$E821FC	$FE	
$E821FF	$FF	

16-color mode | 256-color mode

[Diagram]: 38 Graphic Palette (65536 Color Mode)

Upper Palette

Address Color Code Data (Upper)
Code G R

$E82002 $00

```
$01
$02
```

$E82006 $03 $E82004 …
$E821FE $EE $FF

Lower Palette

Address Color Code Data (Lower) Code R B I

$E82000 $00

```
                    $01
                    $02
```

$E82004 $03 $E821FC … $E821FF $FE $FF

bit 15

$E82000 Red Blue I Red Blue I

Color Code (Lower) $00 Use Color Code (Lower) $01 Use

The arrangement of 16-bit data bits in graphics pallets in 16/256 color mode is the same as that of graphic palettes and text palettes in 16/256 color mode.

In the palette RAM arrangement, when the color code is an even number, data of even numbers and data of odd numbers are combined and accessed as 1 word (16-bit). For example, address $E82000, 16-bit data is arranged with the lower 8 bits of the even number color code $00 color data (correctly lower 8 bits of data), and the lower 8 bits of the even number color code $01's color data is allocated.

4 CGROM (Character Generator ROM)

CGROM contains English letter and Kanji character patterns (hereinafter referred to as fonts).

Screen control

This refers to the ROM. The X68000 employs both a text screen and a bitmap method, and since it can display characters of any shape, a variety of character patterns are provided in the CGROM. Figure 39 shows an overview of these pre-installed character patterns.

Within the CGROM there are alphanumeric (half-width, 1/4-width) fonts in four types — 8×8 dots, 8×16 dots, 12×12 dots, and 12×24 dots — and kanji (full-width) fonts in two types — 16×16 dots and 24×24 dots — for a total of six kinds of fonts. The half-width 8×16 dot characters and full-width 16×16 dot characters are the ones supported by the Human 68K IOCS calls and the like, but other fonts can also be displayed under SX-WINDOW. In the figure, the ”character letter face” entries represent the actual character size. If alphanumeric characters were arranged at their full size, they would be hard to read when placed closely together. For this reason, the actual character patterns are made smaller than the area defined as the font size, leaving the edge dots blank, so that the text remains easy to read even when packed tightly together.

The address configuration of the CGROM is as shown in figure 40. Full-width 16×16 dot fonts are allocated from $F00000 to $F388BF; 8×8 dot fonts from $F3A000 to $F3A7FF; 8×16 dot fonts from $F3A800 to $F3B7FF; 12×12 dot half-width fonts from $F3B800 to $F3CFFF; and 12×24 dot half-width fonts from $F3D000 to $F3FFFF. Furthermore, full-width 24×24 dot fonts are stored from $F40000 to $FBF3AF. The gap between the end of the 16×16 dot and 8×8 dot font data areas, and the gap from the end of the 24×24 dot font area to $FBFFFF, the final address of the CGROM area, is simply empty space.

Next, let me explain how each of these fonts is stored within the CGROM.

●Figure 39: Character fonts held in ROM

Character type	Font size	Character letter face	Character code
Alphanumeric			
- 1/4-width characters	8 × 8 / 12 × 12	6 × 7 / 10 × 10	$00〜$FF
- Half-width characters	8 × 16 / 12 × 24	7 × 13 / 10 × 18	
Kanji / non-kanji			
- Full-width characters	16 × 16 / 24 × 24	15 × 16 / 24 × 24	Non-kanji: JIS code upper $21–$28; Level 1: upper $30

Number of characters:

- Alphanumeric: 256
- Level 1: 3008
- Level 2: 3478

(Horizontal × Vertical)

• Figure 40 CGROM Address Allocation

$F00000 16 × 16 dot font (Non-Kanji 752 characters)

$F05E00 16 × 16 dot font (First-level Kanji 3008 characters)

$F1D600 16 × 16 dot font (Second-level Kanji 3478 characters)

$F388C0

$F3A000 8 × 8 dot font (256 characters)

$F3A800 8 × 16 dot font (256 characters)

$F3B800 12 × 12 dot font (256 characters)

$FD0000

$F40000 24 × 24 dot font (Non-Kanji 752 characters)

$F4D380 24 × 24 dot font (First-level Kanji 3008 characters)

$F82180 24 × 24 dot font (Second-level Kanji 3478 characters)

$FBF380

$FBFFFF

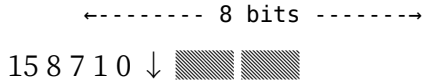
Note: The values with ”$” preceding them are hexadecimal address spaces. The numbers in parentheses represent character counts.

4.1 8×8 Dot Font

The structure of the 8×8 dot font data is shown in Figure 41. The font data starts at address $F3A000, and uses an 8-byte memory area for each character.

The character font is expressed in 8 dots horizontally by 1 byte, and the pattern for 8×8 dots is expressed in 8 bytes. Horizontally, the rightmost bit is bit 0, and the leftmost bit is bit 7.

Figure 41 - 8×8 Dot Font



$F3A000 → P0 P0 $F3A002 P2 P1 $F3A004 P4 P5 $F3A006 P6 P7

4.2 8×16 Dot Font

The structure of the 8×16 dot font data is shown in Figure 42. The start address is $F3A800, and 16 bytes are used per character, other than that it is the same as the 8×8 dot font.

Note: Diagrams of the font structures have been transcribed as closely as possible with text.

●Figure 42: 8×16 dot font

8 dots bit 15 8 7 bit 0 P0 $F3A800 P 0 P 1 P1 $F3A802 P 2 P 3 P2 $F3A804 P 4 P 5 P3 $F3A806 P 6 P 7 P4 $F3A808 P 8 P 9 Data for P5 16 dots $F3A80A P 10 P 11 one character P6 $F3A80C P 12 P 13 P7 $F3A80E P 14 P 15 P8 P9 $F3B7FC ... ~ P15 $F3B7FE

4-3 12×12 dot font

The 12×12 dot font data is stored as shown in figure 43 on page 223. The start address is $F3B800, using 2 bytes per horizontal line and 24 bytes per character.

For a CPU that accesses memory in 8-bit units, the number 12 is somewhat awkward. In the CGROM, 2 bytes (16 bits) of space are used per line, and the pattern is registered in the upper 12 bits of this. The remaining lower 4 bits are all read out as 0.

When read out in units of one word (16 bits), the right end of the pattern is bit 4 and the left end is bit 15.

-43 12×12 Dot Font

1 character worth of data

Screen Control

4 12×24 Dot Font

The 12×24 dot font data is as organized in Figure 44. The starting address is $F3D000, with each horizontal line using 2 bytes, and 48 bytes used per character.

The horizontal data structure applies 2 bytes (16 bits) per line, just like in the 12×12 dot font, and the upper 12 bits are reserved for data registration. The lower 4 bits remain the same as in the 12×12 dot font and are filled with '0' (zero).

●Figure 44: 12×24 dot font

12 dots bit 15 8 7 4 3 bit 0 P0 P1 $F3D000 P 0 P 1 '0' P2 P3 $F3D002 P 2 P 3 '0' P4 P5 $F3D004 P 4 P 5 '0' P6 P7 $F3D006 P 6 P 7 '0' P8 P9 $F3D008 P 8 P 9 '0' P10 P11 $F3D00A P 10 P 11 '0' P12 P13 $F3D00C P 12 P 13 '0' P14 P15 $F3D00E P 14 P 15 '0' P16 P17 $F3D010 P 16 P 17 '0' P18 P19 $F3D012 P 18 P 19 '0' P20 P21 $F3D014 P 20 P 21 '0' P22 P23 $F3D016 P 22 P 23 '0' Data for P24 P25 $F3D018 P 24 P 25 '0' one character P26 P27 $F3D01A P 26 P 27 '0' P28 P29 $F3D01C P 28 P 29 '0' P30 P31 $F3D01E P 30 P 31 '0' P32 P33 $F3D020 P 32 P 33 '0' P34 P35 $F3D022 P 34 P 35 '0' P36 P37 $F3D024 P 36 P 37 '0' P38 P39 $F3D026 P 38 P 39 '0' P40 P41 $F3D028 P 40 P 41 '0' P42 P43 $F3D02A P 42 P 43 '0' P44 P45 $F3D02C P 44 P 45 '0' P46 P47 $F3D02E P 46 P 47 '0' (24 dots) ~

```
$F3FFFC                                '0'
$F3FFFE                                '0'
```

The lower 4 bits are always '0'

4-5 16×16 dot font

The 16×16 dot font data is stored as shown in figure 45 on page 225. The start address is $F00000, using 2 bytes per horizontal line and 32 bytes per character.

The 16 horizontal dots of the character font are stored directly as one word of data. In the bit arrangement, the right end is bit 0 and the left end is bit 15.

”■...45 16x16 Dot Font 16 Dots

◇6 24x24 Dot Font

24x24 dot font data is stored as shown in Figure 46. The start address is $F40000, and 1 line of 24 dots horizontally is 3 bytes, using 72 bytes per character. Although the data for one line is 24 bits, the CPU's memory access is basically in bytes (8 bits), words (16 bits),

or long words (32 bits). Therefore, when accessing the 24-bit part, the access is repeated three times in units of bytes, divided into 16 bits + 8 bits twice, or read in 32-bit units and then processed into 8-bit units, etc. (Be cautious when programming since address errors will occur if access is made in odd-numbered bytes, word units, or long word units.)”

$F388BC $F388BE

COLUMN

Practice of CGROM Pattern Arrangement

In this text, I will supplement how the actual arrangement of patterns has been made, as it has not been mentioned so far. Please note that the arrangement of this font is something the author personally researched, and there is no guarantee that it will remain unchanged in the future.

From the values published for the letter face and the actual contents of the CGROM, the approximate allocation area of the various fonts is considered. Specifically, the exact areas and how the patterns are arranged are examined for extrament, in the diagrams 47-49. Even if the patterns are actually arranged in the area at the top, the letter face area of the lower half and below are usually not shown. Checking closely, while almost all uppercase letters, numbers, etc. are allocated to the letter face area of the alphabet, it seems that a significant portion like some letters and numbers are extracted to the letter face area.

Additionally, the letter face areas of the 16x16 dot font, 24x24 dot font renders as 15x16, 24x24 respectively, but the major alphabets of letters and numbers are written within the dot line as shown in the figure as 15x13 and 24x24.

Figure...47 Pattern Allocation Area (1)

8x8 Dot Font 6x7 Dot

8x16 Dot Font 7x13 Dot

12x12 Dot Font 10x10 Dot

Upper: Letter Face Area Lower: Actual Pattern Allocation Area

(At the bottom right of the page) Page 227

It seems to be placed in the area of 20×19 dots. Considering the case of dense placement, Kanji

Fig. 48 Button Use Area (2)

12×24 Dot Font 10×18 Dots

16×16 Dot Font 15×16 Dots (15×13 Dots) (within is Alphabet uppercase letterface)

I think it may be in order to maintain the balance of the characters.

●Figure 49 Pattern Usage Area (3) 24x24 Dot Font

24x24 dots (20x19 dots)

(The inside showcases an alphabet letter, kana letter, or another script letter face)

(No unused area)

Page 229

5: Screen Mode Control

The screen display of the X 68000 is handled by the CRT controller, video controller, and sprite controller, and involves the setting of screen modes through these controllers. Here, registers related to screen mode settings are organized.

5-1: CRTC

Among the registers in the CRTC, those related to screen mode settings are R00 to R08, and R20. Among these, R00 to R07 are registers that adjust the basic timing of the CRT interface, and R20 is a register that switches color modes and VRAM banks.

5-1-1: Timing Control Registers

The allocation of R00 to R08 is as shown in Figure 50.

- Diagram 50: CRTC (R00~R08)

bit 15 - 8	bit 7	bit 0
R00	$E80000	
R01	$E80002	
R02	$E80004	
R03	$E80006	
R04	$E80008	
R05	$E8000A	
R06	$E8000C	
R07	$E8000E	
R08	$E80010	

- Horizontal Total '1'
- Horizontal Sync End Position
- Display Start Position
- Display End Position
- Vertical Total
- Vertical Sync End Position
- Vertical Display Start Position
- Vertical Display End Position
- External Image Horizontal Adjust

- Note: Cycle values are odd numbers (with the lower bit set to '1').

Page 230

Screen Control

In this register, adjustments are made for the timing of display periods and synchronization signals given to the CRTC. The standard timing of the CRT interface for X68000 and the setting values for each video mode are shown on page 51. The relationship between each timing name and signal form is shown on page 232. The method of calculating the setting values of this register is detailed in the diagram on page 233, but make sure to change the values of R0 to always be even numbers (set bit 1 to ‘1’).

■•2▌ CRTC R20

The location of R20 in the CRTC is shown in the diagram on page 54. The upper byte of this register sets the vertical sync mode, and the lower byte sets the horizontal sync frequency (light pen mode), standard sync mode, and the horizontal and vertical period values. For this, set the values of the upper byte bits 7, 8, 9, 10 to the same values as the lower byte bits 7, 8, 9, 10 of the pattern controller R.

■Diagram....51 Basic Timing of CRT Interface & CRTC Standard Set Values

■Timing High Resolution Standard Resolution Sync Frequency Horizontal: 31.5 kHz 15.98 kHz Vertical: 55.46 Hz 61.46 Hz Data Display Period (1)

```
Horizontal: 22.09 µs          52.69 µs
Vertical: 16.25 ms             15.019 ms
```

Sync Period (2)

```
Horizontal: 31.75 µs           62.58 µs
Vertical: 18.03 ms             16.270 ms
```

Sync Pulse Width (3)

```
Horizontal: 3.45 µs             3.30 µs
Vertical: 0.191 ms             0.187 ms
```

Back Porch (4)

```
Horizontal: 4.14 µs             4.94 µs
Vertical: 1.111 ms             0.876 ms
```

Front Porch (5)

```
Horizontal: 2.07 µs             1.65 µs
Vertical: 0.476 ms             0.187 ms
```

Page 231

Tables: Registers: | No. | Address | Screen Mode (High Resolution) | Screen Mode (Standard Resolution) | |-----|---------|-------------------------------|-----------------------------------| | | 768 x 512 | 512 x 512 | 512 x 256 | 256 x 256 | 512 x 512 | 512 x 256 | 256 x 256 | | R00 | $E80000 | $89 (137) | $5B (91) | $5B (91) | $2D (45) | $4B (75) | $4B (75) | $25 (37) | | R01 | $E80002 | $0E (14) | $09 (9) | $09 (9) | $04 (4) | $03 (3) | $03 (3) | $01 (1) | | R02 | $E80004 | $1C (28) | $11 (17) | $11 (17) | $06 (6) | $05 (5) | $05 (5) | $00 (0) | | R03 | $E80006 | $7C (124) | $51 (81) | $51 (81) | $26 (38) | $45 (69) | $45 (69) | $20 (32) | | R04 | $E80008 | $237 (567) | $237 (567) | $237 (567) | $237 (567) | $103 (259) | $103 (259) | $103 (259) | | R05 | $E8000A | $05 (5) | $05 (5) | $05 (5) | $05 (5) | $02 (2) | $02 (2) | $02 (2) | | R06 | $E8000C | $28 (40) | $28 (40) | $28 (40) | $28 (40) | $10 (16) | $10 (16) | $10 (16) | | R07 | $E8000E | $228 (552) | $228 (552) | $228 (552) | $228 (552) | $100 (256) | $100 (256) | $100 (256) | | R08 | $E80010 | $1B (27) | $1B (27) | $1B (27) | $1B (27) | $2C (44) | $2C (44) | $24 (36) |

(Numbers in parentheses are decimal values)

Diagram: Fig: 52 CRT Signal

- Display Signal
- Sync Signal

Display Signal: 0.7 V p-p (75 Ω termination)/accuracy
Sync Signal: TTL level positive polarity

Page 232

Screen control

●Figure 53: Method of calculating the CRTC R00–R07 setting values

[R00] = (Horizontal sync period) × (Horizontal display dot count) / (Data display period) × 8 − 1

[R01] = (Horizontal sync pulse width) × (Horizontal display dot count) / (Data display period) × 8 − 1

[R02] = {(Horizontal sync pulse width) + (Horizontal back porch)} × (Horizontal display dot count) / (Data display period) × 8 − 5

[R03] = {(Horizontal sync pulse width) + (Horizontal front porch)} × (Horizontal display dot count) / (Data display period) × 8 − 5

[R04] = (Vertical sync period) / (Horizontal sync period) − 1

[R05] = (Vertical sync pulse width) / (Horizontal sync period) − 1

[R06] = {(Vertical sync pulse width) + (Vertical back porch)} / (Horizontal sync period) − 1

[R07] = {(Vertical sync period) − (Vertical front porch)} / (Horizontal sync period) − 1

●Figure 54: CRTC R20 ($E80028)

bit 15 bit 0 SIZ COL HF VD HD

Actual screen 512 × 512 dot mode 0 Actual screen 1024 × 1024 dot mode 1

```
16 color mode      0 0
256 color mode     0 1
```

Undefined 1 0 65536 color mode 1 1

HF — Horizontal frequency: 0 = 15.98 kHz 1 = 31.5 kHz

VD — Vertical: 0 0 = 256 dots 0 1 = 512 dots 1 0 = undefined 1 1 = undefined

HD — Horizontal: 0 0 = 256 dots 0 1 = 512 dots 1 0 = undefined 1 1 = 768 dots

Set the lower 3 bits of the video controller's R0 (Address: $E82400) to the same values.

5·2 Video Controller

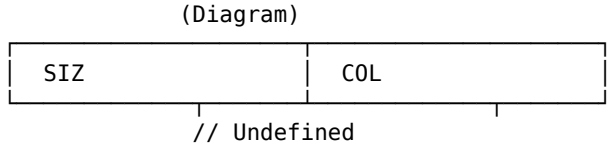
Among the registers of the video controller, it is R0 that relates to the screen mode. The bit layout of the register is shown in Figure 55. The lower bit of R0 executes the selection of the actual screen size and color mode of the screen. This setting value is the same as bits 8, 9, and 10 of CRTC's R20.

■ Figure: 55 Video Controller R0 ($E82400)

```
16 Color Mode 0
256 Color Mode 1
```

Undefined 0 65536 Color Mode 1

Actual Screen 512 x 512 Dot Mode 0 Actual Screen 1024 x 1024 Dot Mode 1



CRTC’s R20 (Address: $E80028) The same as bits 8-10

5·3 Sprite Controller

In the sprite controller, the setting for the screen mode is executed by the screen mode register according to the screen mode. The screen mode register consists of four 16-bit registers, and the bit layout is as shown in Figure 56. Among these, the standard values of the setting of H-TOTAL ($EB080A Address), H-DISP ($EB080B Address), V-DISP (valid in $EB080E) are as shown in Figure 57 on page 236. The calculation formulas for each setting value are as follows:

1 H-TOTAL

When in low-resolution 256 x 256 dot mode, the value is the same as CRTC's R00, otherwise:

Figure 56
Sprite Controller Screen Mode Register ($EB080A-$EB080F)

Sprite #0	$EB0000	$EB2006	-
Sprite #1	$EB0008	---------	---
Sprite Control Register	$EB000E	---------	---
Sprite #127	$EB03F8	---------	---

BG Control Register	BGO	BG1
BG Scroll Register	$EB0800	$EB0800
Screen Mode Register	$EB0808	$EB0810

$EB080A	bit 15	---	---	---	H-TOTAL*	'1'	bit 0	

$EB080C	---	---	---	---	---	H-DISP	---	---	---	---

$EB080E	---	---	---	---	---	V-DISP	---	---	---	

$EB0810	---	L/H freq	V	Res	H	Res
---	Horizontal synchronization frequency 15.98 kHz: 0	---	31.5kHz	:1		
---	Vertical 256 line mode: 0	---	/ 512	---	---	Unde
---	Horizontal 256 dot mode: 0	---	/ 512	---	---	Unde
----------	---	---	---	---	---	

- The lowest bit of H-TOTAL (bit 0) must always be '1'.
  Set $FF for H-TOTAL. The value set for this register, like for $R00, be sure to always set the lowest bit (least significant bit) to '1'.

●Figure 57: Sprite controller — Standard register setting values for screen modes

Screen mode register		High-resolution mode			Standard-resolution mode	
Name	Address	512×512	512×256	256×256	512×512	512×256
H-TOTAL	$EB080A	$FF (255)	$FF (255)	$FF (255)	$FF (255)	$FF (255)
H-DISP	$EB080C	$15 (21)	$15 (21)	$0A (10)	$09 (9)	$09 (9)
V-DISP	$EB080E	$28 (40)	$28 (40)	$28 (40)	$10 (16)	$10 (16)
	$EB0810	$15 (21)	$11 (17)	$10 (16)	$05 (5)	$01 (1)
BG screen count		1	1	2	1	1

Values in ( ) are decimal.

2 H-DISP Set the value obtained by adding 4 to the setting value of the CRTC register R02.

3 V-DISP Set the same value as the CRTC register R06. Also, set the $EB0810 register to the same value as the lower 8 bits (bits 0–7) of the CRTC register R20.

5-4 Precautions for Settings

When making settings to the CRTC and so on, there are several points that require attention, so they are supplemented here.

5-4-1 Precautions When Setting the CRTC

When setting the CRTC registers R00 to R07 and R20, perform the settings in the following order.

Screen Control

- When changing from a high display mode to a low display mode R20 → R01 → R02 → R03 → R04 → R05 → R06 → R07 → R00
- When changing from a low display mode to a high display mode R00 → R01 → R02 → R03 → R04 → R05 → R06 → R07 → R20

The sequence for screen mode is judged by bits 4, 1, and 0 of R20, and if arranged in descending order, the priority is as follows:

```
768 × 512 dots (High-resolution mode)
512 × 512/512 × 256 (High-resolution mode)
512 × 512/512 × 256 (Standard resolution mode)
256 × 256 (Standard resolution mode)
```

5·4·2 Setting CRTC When Capturing Images

When capturing images, the value of R08 in CRTC is changed as follows.

```
· 512 × 512/512× 256 dot mode … $9A
· 256 × 256 dot mode … $EB
```

5·4·3 Precautions When Setting Sprite Screen Mode Registers

When setting the value of the H-TOTAL register (located at $EB08A) in the screen mode register of the sprite controller (to 256 × 256 dots in standard resolution mode), perform the H-DISP register setting after 130μs or more.

**5·4·4 Precautions for Sprite RAM Access

After turning on the power, when accessing the sprite VRAM (PCG area, BG data area), perform the setting after setting bit 0 of the BG control register (located at $EB0808) to ”1”.

●6 Sample Program

We have created some samples that perform operations on the CRTC and video controller, so please refer to them.

6-1 Text Screen Scroll (C1.C)

Scroll the text screen up, down, left, and right. It's designed to move like a planetary surface scroll. Also, please check the behavior when it overflows.

● List 1 Text Screen Scroll

/*

- Text screen scroll sample
- (Please check the behavior when it overflows horizontally)
-
- volatile is not supported by XC,

- so please uncomment the next line to disable volatile
- #define volatile */

#include ”doslib.h” #define GP_VDISP 0x10 volatile short *crtc_rl0; volatile short *crtc_rl1; volatile char *gpip;

void set_xpos(int xpos); void set_ypos(int xpos); void wait_vdisp(void);

main() {

```
int i, j;
(short *) crtc_rl0 = 0xe80014;
(short *) crtc_rl1 = 0xe80016;
```

6-2 4-Way Scrolling on the Graphic Screen (C2.C)

In 512x512x16 color x 4 plane mode, each screen scrolls independently.

6.3 Text Screen Scroll Using the Raster Copy Function (C3.C)

We have implemented vertical scrolling of the text screen using the raster copy function. Use the cursor keys to move up and down, and the screen moves accordingly. To end the session, press CTRL+C.

◇List……3 Text Screen Scroll Using the Raster Copy Function /*

```
 * Raster Copy Function Sample (Text Screen Scroll)
 *
 * Use the cursor keys to move the screen up and down, and end with Ctrl+C.
 *
 * Since XC does not support volatile,
 * please disable volatile by including the following line.
 * #define volatile
 */
```

#include ”basic0.h” #include ”doslib.h”

volatile short *crtc_r21; volatile short *crtc_r22; volatile short *crtc_mode;

Screen Control

volatile char *gpip;

char raster_scroll(void); void raster_copy(int src, int dst); void wait_h_sync(void); void start_raster_copy(void); void stop_raster_copy(void);

void main() {

```
int i;
short   r21dat, r22dat;
gpip      = (char *)0xe88001;
crtc_r21  = (short *)0xe8002a;
crtc_r22  = (short *)0xe8002c;
crtc_mode = (short *)0xe80480;
C_CUROFF();
SUPER(0);
r21dat = *crtc_r21;
r22dat = *crtc_r22;

while(raster_scroll() != 0x3)
    ;
wait_h_sync();
stop_raster_copy();
*crtc_r21 = r21dat;
```

```
    *crtc_r22 = r22dat;
    C_CURON();
    exit(0);
```

}

char raster_scroll() {

```
    char    keybuf[8];

    b_inkey0(keybuf);
    if (keybuf[0] == 0x1b) {
        b_inkey0(keybuf);
        switch(keybuf[0]) {
            case    0x55:   roll_up();
                    break;
            case    0x4a:   roll_down();
```



```
break:
    default: break;
```

} } return(keybuf[0]); } roll_up() {

```
    int i;
    raster_copy(0, 128);
    for (i=0; i<123; i++)
        raster_copy(i+1, i);
    raster_copy(128, 123);
```

} roll_down() {

```
    int i;
    raster_copy(123, 128);
    for (i=123; i>0; i--)
        raster_copy(i-1, i);
    raster_copy(128, 0);
```

} void raster_copy(src, dst) int src, dst; {

```
    dst &= 0xff;
    src &= 0xff;
    wait_h_sync();
    stop_raster_copy();
    *crtc_r21 = 0x3;
    *crtc_r22 = (src << 8) | dst;
    start_raster_copy();
```

} void start_raster_copy() {

This code includes function definitions for `roll_up()`, `roll_down()`, `raster_copy()`, and `start_raster_copy()`.

6-4 High-Speed Clearing of Graphic Screens (C4.C)

We tried using the high-speed clearing feature for graphics screens in 256x256 dot mode. It may look like only part of the actual screen is being cleared as you move the cursor around, but please confirm that the actual screen is being cleared.

List ..... 4 High-Speed Clearing of Graphic Screens

/* Sample for high-speed clearing feature

```
 *
 *  This is high-speed clearing in 256x256 dot mode.
 *  You can confirm that the cursor moves and clears the screen.
 *  You can confirm which areas are being cleared.
```

```
 *
 *  Since volatile is not supported by XC,
 *  please invalidate volatile by including the following line
 *  #define volatile
 */
```

#include ”basic0.h” short *vram;

volatile short *crtc_r21; volatile short *crtc_mode; volatile char *gpip; volatile short *crtc_r12; volatile short *crtc_r13;

short r21dat; int pos_x, pos_y;

void init(void); void h_clr(void); void wait_v_sync(void); char g_move(void);

main() {

```
    screen(0,0,1,1);
    vram = (short *)0xc00000;
    gpip      = (char *)0xe88001;
    crtc_r21  = (short *)0xe8002a;
    crtc_mode = (short *)0xe80480;
    crtc_r12  = (short *)0xe80018;
    crtc_r13  = (short *)0xe8001a;
    pos_x = pos_y = 0;
    printf("High Speed Clear Test\n");
    SUPER(0);
    r21dat = *crtc_r21;
    init();
    h_clr();
    while(g_move() != 3)
        ;
    *crtc_r21 = r21dat;
    exit(0);
```

} void init() {

```
    int i;
    short   col;
    for (i=0; i<1024*1024; i++)
        *vram++ = col++;
```


6.5 4-Plane Independent Scroll in 65536-Color Mode (C5.C)

By independently controlling the scroll registers of the graphics screen, you can see that in 65536-color mode the 4 planes can be scrolled independently. This is not a usage guaranteed to work by the manufacturer, so please treat it strictly as reference only.

● List….5 4-Plane Independent Scroll in 65536-Color Mode

/*

```
 *  4-Plane Independent Scroll in 65536-Color Mode (Reference)
 *
 * Since volatile is not supported by XC,
 * insert the following line to disable volatile.
 * #define   volatile
 */
```

#include ”basic0.h” #include ”graph.h”

void main(); void init_screen(); void move_screen(); void delay();

volatile unsigned short *x0 = (unsigned short *)0xe80018; volatile unsigned short *y0 = (unsigned short *)0xe8001a; volatile unsigned short *x1 = (unsigned short *)0xe8001c; volatile unsigned short *y1 = (unsigned short *)0xe8001e; volatile unsigned short *x2 = (unsigned short *)0xe80020; volatile unsigned short *y2 = (unsigned short *)0xe80022; volatile unsigned short *x3 = (unsigned short *)0xe80024; volatile unsigned short *y3 = (unsigned short *)0xe80026;

void main(argc, argv) int argc; char *argv[]; { init_screen();

Screen Control

```
    SUPER(0);
    move_screen();
```

}

void init_screen() {

```
    unsigned int    i, j, col;
    screen( 1, 3, 1, 1);
    window(0, 0, 511, 511);
    for (i=0; i<256; i++) {
        col = i*256;
        for (j=0; j<256; j++)
            pset(128+j, 128+i, col+j);
    }
```

}

void move_screen() {

```
    unsigned int    i;
    for (i=0; i<128; delay(), i++) {
        *x0 = 511-i;
        *y0 = 511-i;
        *x1 = 511-i;
        *y1 = i;
        *x2 = i;
        *y2 = 511-i;
        *x3 = i;
        *y3 = i;
    }
```

}

void delay() {

```
    unsigned int    i;
    for (i=0; i<5000; i++)
        ;
```

}

6. 768x512 Dot Mode Display with 65536 Colors (V1.C)

Normally, the 768x512 dot mode only displays 16 colors on the graphics screen, but by bypassing the CRTC in the video controller, it is possible to display 65536 colors in the 512x512 dot area of the 768x512 dot screen. Since the dot density is high and the vertical dot spacing is almost the same, for example, in the 512x512 dot mode, if 100 horizontal dots are displayed, the vertical and horizontal intervals should appear as squares of 100 dots per side. This mode is not guaranteed to operate by the manufacturer, so please note that this is for reference only.

■List….6….768x512 Dot Mode Display with 65536 Colors

/*

```
 * 768x512 Dot Mode Display with 65536 Colors (for reference)
 *
 * Due to lack of support for 'volatile' in XC,
 * please exclude the following line using 'volatile'
 # define volatile
 */
```

#include ”stdio.h” #include ”doslib.h” main() {

```
   short *vram, *crtc r20, *vcr1, *palette;
   int i, h, s, v;
   vram = (short *)0xc00000;  /* G-Vram Start Address */
   crtc r20 = (short *)0xe80028;  /* CRTC R20 */
   vcr1 = (short *)0xe82400;  /* Video Controller R1 */
   palette = (short *)0xe82000;  /* Palette Register */
   SUPER(0) ;
   screen(2,0,1,1);
   *crtcr20= 0x3016;
   *vcr1= 3;
   for (i=0x0001; i<=0x10000; i+=0x0202) {
       *palette++ = i;
       *palette++ = i;
   }

   for (h= 0; h < 192; h++)
```

This includes the code snippet within the text.

6.7 Blending Between Graphics Screen 2 and Text Screen with Transparency (V2.C)

The blending between graphics screen 2 and the text screen involves semi-transparency. Although it is not supported by X-BASIC, this semi-transparency functionality produces quite interesting effects, so it might be worthwhile to touch it up a bit.

List 7: Blending Between Graphics Screen 2 and Text Screen with Semi-Transparency

/*

```
 *   Semi-Transparency Sample Code for Blending Graphics Screen 2 and Text Screen
 *
 *   Because `volatile` is not supported in XC,
 *   please disable `volatile` by uncommenting the line below.
 *   #define volatile
 */
```

#include ”basic0.h” #include ”graph.h”

#define GREEN 0x0f800 #define RED 0x07c0 #define BLUE 0x003e #define INTENS 0x0001

void init_palette();

unsigned short *gpal = (unsigned short *)0xe82000; volatile unsigned short *video_r1 = (unsigned short *)0xe82500; volatile unsigned short *video_r2 = (unsigned short *)0xe82600;

main() {

```
    screen(1, 2, 1, 1);
    locate(20, 10);
```

}

6: 8 BG Screen Settings & Scroll (S1.C)

We have performed PCG registration, BG screen settings, scrolling, and other operations.

● List ..... 8 BG screen settings and scrolls

/* *

- BG screen settings & scroll sample
-

*/

- Since `volatile` is not supported in XC,
- please insert the following line to disable `volatile`
- #define volatile */

#include ”basic0.h”

volatile unsigned short *bgscrlx0= (unsigned short *)0x00e0b0800; volatile unsigned short *bgscrly0= (unsigned short *)0x00e0b0802; volatile unsigned short *bgctrl = (unsigned short *)0x00e0b0808; volatile unsigned short *bgtext = (unsigned short *)0x00e0b0c000; volatile unsigned short *pcg = (unsigned short *)0x00e0b8000; volatile unsigned short *spscrl = (unsigned short *)0x00e0b0006; volatile unsigned short *videor3 = (unsigned short *)0x00e0b2600; volatile unsigned short *videor2 = (unsigned short *)0x00e0b2500; volatile unsigned short *palette = (unsigned short *)0x00e082220;

void main() {

```
    unsigned int i, j;
    screen(1,3,1,1);
    SUPER(0);
    *bgscrlx0 = 0;
    *bgscrly0 = 0;
    *videor3 = 0x40;
    *videor2 = (*videor2 & 0xff) | 0x1200;
    for (i=0; i<0x10; i++)
       *palette++ = ((i & 1)?0x3e:0) | ((i & 2)?0x7c:0) | ((i & 4)?0xf800:0) | ((i & 8)?1:0);
    for (i=0; i<0x80; spscrl += 4, i++)
        *spscrl = 0;
    for (i=0; i<0x10; i++)
        *pcg++ = 0x1111;
    for (i=0; i<0x10; i++)
        *pcg++ = 0x2222;
    for (i=0; i<0x10; i++)
        *pcg++ = 0x4444;
    for (i=0; i<0x10; i++)
        *pcg++ = 0x8888;
    for (j=0; j<4; j++)
        for (i=0; i<0x10; i++)
            *pcg++ = 0;
```

}

COLUMN Period When CPU Can Access

Sprite Scroll Register It is accessible during the entire display period.

The access period is guaranteed to be available once every character clock (QD). Therefore, a wait time of up to 1.8 QD (580 ns or approximately 1440 ns) may occur.

- QD period = 320 ns (high resolution); 800 ns (standard resolution)

- If you set the DISP/CPU bit (Display Cut) of the Scroll Control Register; D09 of $EB0808 to '0', you can forbid temporary access to the Sprite Scroll Register, allowing access during the entire period without high-speed access. However, during that period, all background and screen displays are cut off. Therefore, first, turn the DISP/CPU bit to '0' (Display Cut) and then start accessing the Sprite Scroll Register in succession.

1/2 QD (160 ns or 1/2 QD 400 ns)

CPU Access Timing

Note: Columns might be slightly off due to formatting.

Screen Control

Timing for Reading Registers for Display

The period for reading the registers for display is as shown below (the actual time may vary slightly).

Diagram B: Timing for Reading Registers

3-4 QD period (1-3.2 µs) H-DISP |<---->| |<---->| 1-4 M/QD 1.5ms (22 Hs - 55.7µs) Display Enabled 2.1-2.7-3.4MHz V-DISP Register Reading Timings: (22.42 ne☒ 1181.5sf42.2) 2.42-1181 1- 3 | 3- 4 QD | Register Reading timings 1.66ms at 122 µs upd 2irer

2:51 V-DISP / 3:63.5 3.5ms after second QD Display Disabled (63.5ms after 1:122 µs | V Active Period). Register Reading Timings

Therefore, when writing the sprite scroll registers during the V-sync period, it is necessary to write before the second line if V-DISP is enabled.

Also, if the sprite scroll register contents are rewritten, they will be reflected 3 lines after a 2-line delay.

- In the case of high-resolution mode or super interlace mode, some H period timings may stop the QD pulse. In such a case, there may be a delay in the CPU access even during the period when the CPU can access it (about 60 ms at the maximum).

Background Scroll Register and Screen Mode Register

These registers can be accessed during any period outside the display period. Fundamentally, there are two registers: the internal register for the CPU and the internal/external registers. The internal register for the CPU is accessible within one wait. The contents written to the CPU-use register will be transferred to the internal/external registers once per horizontal scan. Thus, registers become effective after writing to the CPU-use register.

Note: The transfer diagram can be seen on page 256

- Address of background scroll registers: $EB0808 DISP/CPU Clear(D09)
- Addresses of screen mode registers: $EB080A H-TOTAL Bit/D07-00

CPU and internal transfer timing closures are generally about 256 pages are the same.

For instance, if you want to write the background scroll register over a single scan line, you must write to the horizontal scan period.

Diagram C: Timing of Register Transmission

When the transmission period coincides with the CPU's write cycle, the CPU will experience a wait state. During the read cycle, no wait state is inserted.

Access to background scroll registers and screen mode registers is not affected by the superimpose timings.

PCG and Text

It is possible to access all periods including the display period.

During the CPU period, it is ensured by once every 2 character clocks (QD) in time. Therefore, a minimum wait of about 2.8 QD (900 ns or 2240 ns) is required.

Since the registers are DISP/CPU bits (background control registers; E800H to 00 DH) set by the CPU, all PCG and text displays are refreshed every frame to achieve high-speed access. However, during that time, sprites and background displays for the entire screen are halted. Therefore, once the horizontal sync period begins, after setting the DISP/CPU bit to 0 (display), PCG and text access starts and the display resumes.

Diagram D: CPU Access Timing

The periods when PCG and text are read or written are as shown in the diagram.

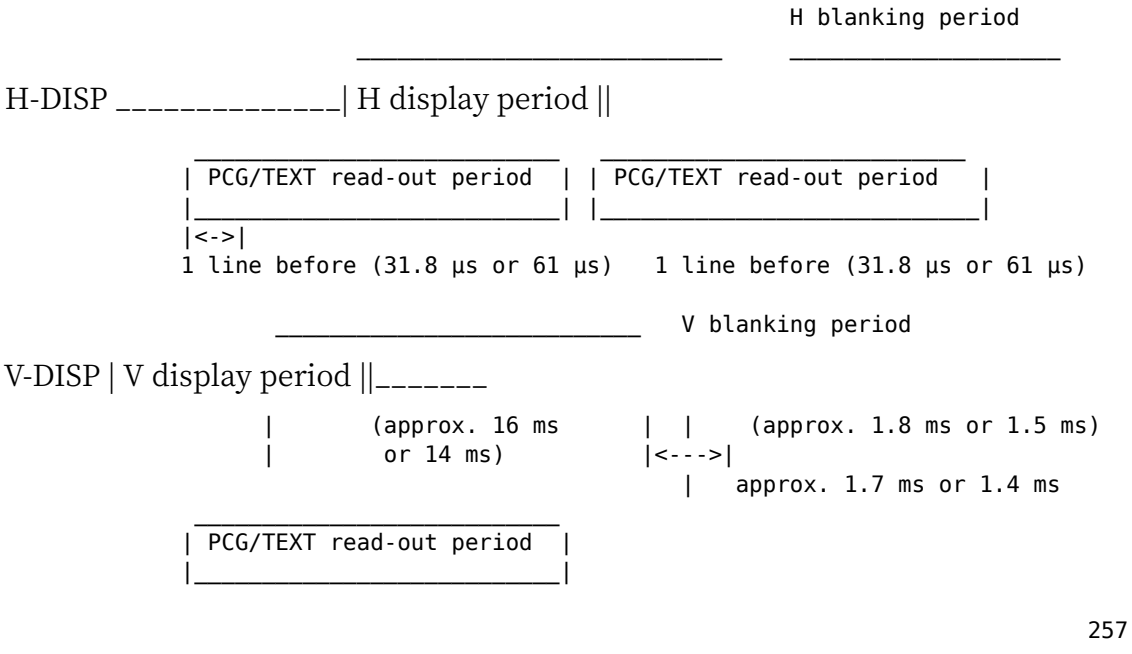
Therefore, when rewriting PCG or text during the V sync period, it should be completed before one line of V-DISP. Also, during the H sync period, the rewriting period is generally short (depends on the display mode).

*In the case of standard mode or superimpose mode, there can be a QD within some parts of the H sync period. Therefore, if the CPU access period conflicts with it, the wait can be extended (about a maximum of 60 μs).

Page 256

Screen Control

● Fig....E Register Read-out Timing



257

(Blank page in the original.)

Sound Mechanism

The sound mechanism of the X68000, which is equipped with both FM sound source and sampling (ADPCM) sound source as standard, broadly supports everything from sound effects to voice output. Here, we will explain the sound mechanism of the X68000.

1. Configuration of the X68000's Sound

The block diagram of the X68000's sound system is shown in Figure 1 on page 260. The X68000 uses ADPCM sound source, which performs compression and replay of audio, and FM sound source, which is responsible for producing musical tones. These are controlled by two types of sound LSI.

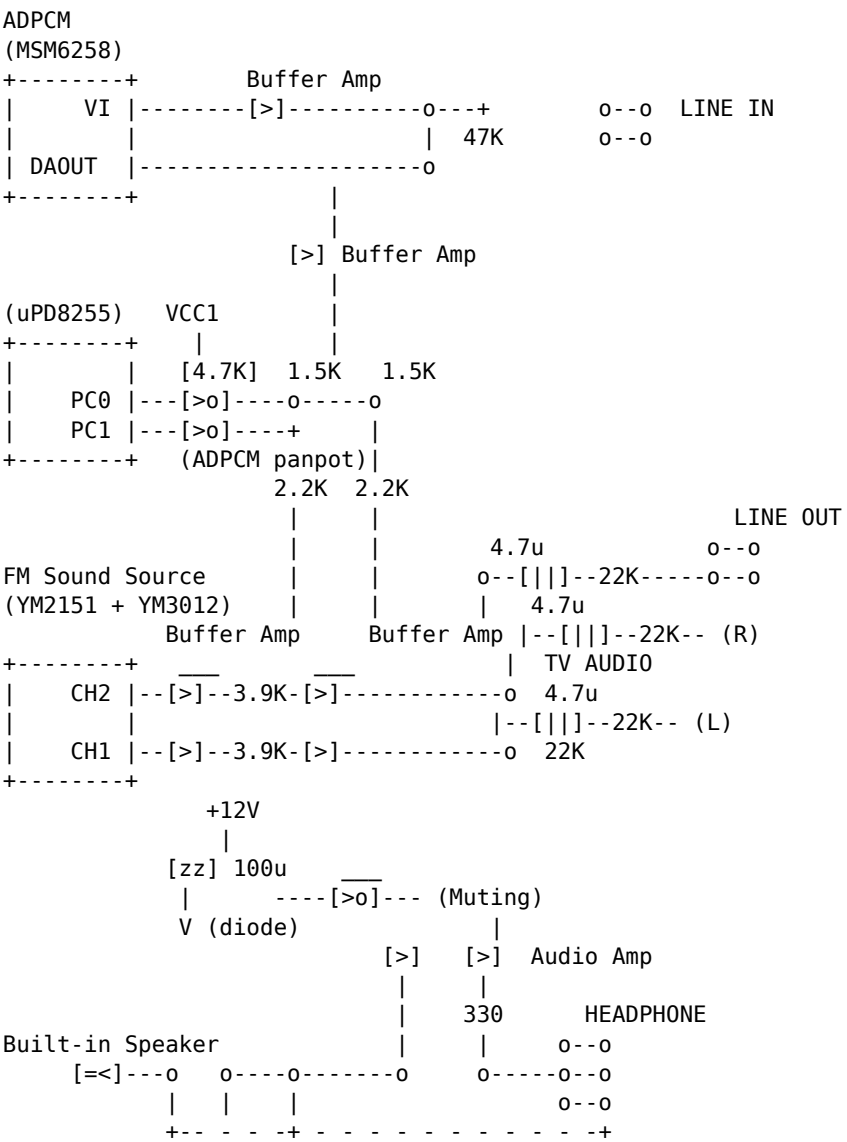
The ADPCM output is sent to the buffer amp, split into left and right, and then mixed with the FM sound IC output. After being split, the part connected to and controlled by the μPD8255 outputs ADPCM to the left, right, or center (or does not output at all).

To the left of PC0 and to the right of PC1 correspond channels. If the output is '1' (High Level), the corresponding channel will output sound; if it is '0' (Low Level), the channel will be OFF thus no sound will be emitted.

When the microcomputer is reset, the 8255 I/O pins will all be cleared. Therefore, ADPCM output will be OFF for both channels. Mixing of left and right ADPCM will also be turned OFF. This system is similar to how audio amplifiers process sound, making it highly versatile for musical composition and output.

Page 259

● Fig....1 Sound System Block Diagram



Sound Device

Example circuit (a circuit that prevents a loud ”pop” sound when the device is turned on).

#2 FM Sound Source

The X68000 uses Yamaha's YM2151 as the FM sound source LSI. Yamaha manufactures several FM sound source LSIs, each with a different suffix. The YM2151 is also known as OPM (Fm Operator Type-m). This name is widely used, and many people may be familiar with it. In this book, we will refer to the FM sound source LSI as OPM.

2-1 Internal Block of OPM

The internal block diagram of OPM is shown in Figure 2 on page 262. OPM has eight sound output circuits, and each output can be mixed to produce the final signal. Each channel can be assigned numbers from 1 to 8, and generally, each sound corresponds to one instrument. For example, in a three-voice system, sounds for channels 2, 3, and 7 can be assigned.

Each channel consists of four blocks called slots, which are labeled M1 (Modulator 1), M2 (Modulator 2), C1 (Carrier 1), and C2 (Carrier 2). These slots can be combined to create complex sounds by connecting the outputs directly or through modulation.

In addition, OPM controls the level fluctuation (attack, decay, sustain, and release) from key-on to key-off with an Envelope Generator (EG). It can also apply periodic changes like tremolo or vibrato to the amplitude using an LFO (Low Frequency Oscillator). Many other features, such as starting all slots simultaneously for composite sounds or switching between 32 slots, are also built into the OPM.

Page 261

Diagram 2: Internal Block Diagram of OPM

Channel 1 Slot 1 Slot 9 Slot 17 Slot 25 M1 M2 C1 C2 OP

Channel 2 Slot 2 Slot 10 Slot 18 Slot 26 M1 M2 C1 C2 OP

Channel 3 Slot 3 Slot 11 Slot 19 Slot 27 M1 M2 C1 C2 OP

Channel 4 Slot 4 Slot 12 Slot 20 Slot 28 M1 M2 C1 C2 OP

Channel 5 Slot 5 Slot 13 Slot 21 Slot 29 M1 M2 C1 C2 OP

Channel 6 Slot 6 Slot 14 Slot 22 Slot 30 M1 M2 C1 C2 OP

Channel 7 Slot 7 Slot 15 Slot 23 Slot 31 M1 M2 C1 C2 OP

Channel 8 Slot 8 Slot 16 Slot 24 Slot 32 M1 M2 C1 C2 OP

PG: Phase Generator SIN Table EG: Envelope Generator Each slot's structure OPM (YM2151) ACC DAC (YM3012) CH2 CH1 LFO Timer

M1: Modulator 1 M2: 2 C1: Carrier 1 C2: //2 OP: FM Operator LFO: Low Frequency Oscillator ACC: Accumulator PG: Phase Generator EG: Envelope Generator

Noise . . . . .

Basic Structure of Slot

PG: Phase Generator (Sin is controlled by inputting ϕ at time of t)

Phase Modulation

By combining oscillator + envelope generator, you can make the sound have a complex trajectory within the sound envelope.

As for effects, there is basic ADSR envelope generation.

Controllers

By using LFOs (Low Frequency Oscillators), many types of modulation effects can be added. For instance, Chorus, Flanger and Phasers can be created.

Modulation Algorithms

By chaining multiple LFOs and EGs (Envelope Generators), complex sounds of any kind can be formed. The key point is how to chain the LFOs and EGs efficiently.

Digital Envelope Generator

Unlike analog ones, these can offer precise windows of modulation and more complex shapes can be formed.

Output

Combination --> (Algorithm) --> Output.

Decoding Algorithms

Digital Signal Processing can decode the sound. The effects added can multiply the immersive experience of the sound module.

Please note: Using envelope parameters correctly is key to creating a good sound synthesis output.

Figure 3 Basic Structure of Slot

Input from other slots (up to 2 maximum) -->

SIN waveform table

--> Output

PG (Phase Generator)

EG (Envelope Generator)

Other inputs -->

LFO

Parameters ( ) indicate that they are set per note.

Key: KC: Key Code KF: Key Fraction MUL: Phase Multiply DT1: Detune 1 DT2: Detune 2 PMS: Phase Modulation Sensitivity AR: Attack Rate D1R: 1st Decay Rate D2R: 2nd Decay Rate RR: Release Rate KS: Key Scaling D1L: 1st Decay Level TL: Total Level AMS: Amplitude Modulation Sensitivity AMS-EN: AMS Enable PMD: Phase Modulation Depth AMD: Amplitude Modulation Depth W: Waveform LFRQ: Low Frequency

*1: Feedback exists only in Slot M1.

Sound System

then, using the EG output beta, the output Aout can be

```
Aout = beta SIN(omega t + alpha SIN(psi t))
```

expressed as above. The output waveform of the EG is determined by parameters in the OPM registers such as AR, D1R, D2R, RR, KS, D1L, and TL, and furthermore the ON/OFF of output-level fluctuation by the LFO and the degree of that fluctuation are determined by AMS-EN and AMS. Among the PG and EG parameters in Fig. 3, those enclosed in parentheses ( ) indicate parameters that are set per channel, while those not enclosed indicate parameters that are set per slot. The block labeled LFO, shown with a dotted line in Fig. 3, is a signal generator for making the PG and EG outputs waver at a low frequency. The OPM has only one of these inside the chip, and this output is shared and used by all slots. The LFO has two outputs, one for the PG and one for the EG, and their respective output levels are specified by the parameters PMD and AMD. The waveform type and frequency of the LFO output are determined by the parameters W and LFRQ, respectively. Only the M1 slot has a feedback circuit that uses its own output as its own input signal, and the amount of this feedback is specified by the FL parameter.

2.3 Basic Structure of the Other Parts

```
The basic structure of the parts of the OPM other than the slots is shown in Fig. 4 on
```

page 266. The operator has the parameter CON, which specifies the combination of slots, and the parameter LR, which specifies whether the channel's sound is output from the left, right, or center. (In the OPM block diagram published by the manufacturer, LR is input at the accumulator section, but since it is easier to understand intuitively if you think of it as acting on the operator, here it is treated as being input to the operator.) There is only one noise generator inside the OPM. The ON/OFF of the noise output is specified by NE, and the tone quality by NFRQ. When the noise output is turned ON, the output of the SIN-waveform table of slot 32 is replaced by the output of the noise generator. (Since this is difficult to show in the figure, it is written as switching with the slot output.)

Fig. 4 Basic structure of the parts other than the slot

OP (operator)

slot -> M1 slot -> M2 slot -> C1 slot -> C2 --o *1

```
                  -> R ch output
                  -> L ch output
```

NG (noise generator)

NE NFRQ

CON LR

*1: The NG output is used by switching it with slot 32

Timer A -> All-slots simultaneous key ON Timer B -> Interrupt to the CPU

CLKA1 CLKB CSM F-RESET CLKA2 LOAD IRQEN

NE : Noise Enable NFRQ : Noise FReQuency

CON : CONnection LR : Left channel Enable / Right channel Enable

2.4 OPM Address Layout

The OPM port addresses are shown in Fig. 5.

After setting the register number at address $E90001, the CPU accesses these registers using the data port at address $E90003. The OPM has many internal registers, but the registers related to sound creation are all write-only; on a read, regardless of the register number setting, the status register is always read out.

Fig. 5 OPM Port Addresses

Address	Data (D7 D6 D5 D4 D3 D2 D1 D0)	Contents
$E90001		Register number setting port
$E90003		Data Read/Write port

2.5 OPM Read Register

When a read is performed from the OPM, the contents of the status register are always read out. The bit layout of this register is shown in Fig. 6. bit 7 is the OPM write BUSY flag. It indicates that the OPM is in a state where it cannot accept the next access from the CPU. When writing to the OPM, you must first confirm that this bit is ”0” before doing so.

bit 0 and bit 1 are bits that indicate which of the two timers inside the OPM has overflowed. The OPM timers can issue an interrupt to the CPU after a preset time has elapsed, or apply a key-on to all slots simultaneously. This bit is mainly used so that, when used in a way that issues interrupts to the CPU, the CPU can determine which timer caused the interrupt.

Fig. 6 OPM Status Register

bit 7	6	5	4	3	2	1	bit 0
B						1ST(A)	1ST(B)

Write BUSY flag 1: Data write in progress (the CPU must not write the next data) 0: Normal operation (” the CPU may write)

1: Timer A overflow has occurred 0: ” A has not overflowed

1: Timer B overflow has occurred 0: ” B has not overflowed

2.6 OPM Write Registers

The layout of the OPM write registers is shown in Fig. 7 and Fig. 8. Register numbers $00 to $1F are registers for noise, timer, LFO, etc., of which there is only one inside the OPM; $20 to $3F are specified on a per-channel basis, and $40 to $FF are specified on a per-slot basis. Next, we will explain these registers in address order.

Fig. 7 OPM Register List (Part 1)

Register No.	bit 7	bit 6	bit 5	bit 4	bit 3
$00					
$01			'0'		LFO Reset
$02 - $07					
$08		KON (C2)(C1)(M2)(M1)			(CH No.)
$09 - $0E					
$0F	NE		NFRQ		fnoise (Hz) = 4000 / (32 x NFRQ)
$10			CLKA (upper)		TA (ms) = 64 x (1024 - CLKA) / 40
$11			CLKA (lower)		
$12			CLKB		TB (ms) = 1024 x (256 - CLKB) / 4
$13					
$14	CSM		F-RESET (B)(A)	IRQEN (B)(A)	LOAD (B)(A)
$15 - $17					
$18			LFRQ		LFO frequency setting
$19	F	PMD / AMD		F = 1: PMD, F = 0: AMD	
$1A					

Register No.	bit 7	bit 6	bit 5	bit 4	bit 3
$1B	CT1	CT2			
$1C - $1F					

Sound Mechanism

Fig. 8 OPM Register List (Part 2)

Register No.	bit 7	bit 6	bit 5	bit 4	bit 3
$20 - $27	R-ch EN	L-ch EN	FL		CON
$28 - $2F		KC (OCT)		(NOTE)	
$30 - $37		KF			
$38 - $3F			PMS		AMS
$40 - $5F		DT1		MUL	
$60 - $7F			TL		TL: Total Level (output level setting)
$80 - $9F	KS		AR		KS: Key Scaling (selects EG's rate de
$A0 - $BF	AMS EN			D1R	
$C0 - $DF	DT2		D2R		DT2: Detune 2 (large frequency char
$E0 - $FF	D1L	RR		D1L: 1st Decay Level, RR: Release Rate	Set per slot

2.6.1 Test Register

The bit layout of the test register is shown in Fig. 9.

Fig. 9 Test Register

bit 7	6	5	4	3	2	1	bit 0
$01	TEST		LFO RESET	TEST			

These bits are all for test purposes, so do not write anything other than '0'.

1: Resets the LFO 0: Starts the LFO output

This register's main purpose is for use during the manufacturer's shipping inspection. The only bit that is made public is bit 1; please make sure to use '0' for all other bits. Setting bit 1 to '1' resets the LFO, and returning it to '0' starts the LFO. This is effective when you want to synchronize and run the LFO with other tones.

2.6.2 KON Register

This is a register that performs key on / key off control. The bit layout is shown in Fig. 10. Since it controls the ON/OFF of a tone, in terms of a keyboard instrument, pressing a key is key on, and releasing the finger from the key is key off.

The lower 3 bits of the KON register specify the channel you want to control, and bits 3 to 6 determine whether to key on or key off each slot of that channel. When the corresponding bit is '1' it is key on, and when '0' it is key off.

Since only one channel at a time can be turned on with the KON register, to key on multiple channels

Fig. 10 KON

bit 7	6	5	4	3	2	1	bit 0
+$08	C2	C1	M2	M1		CH No.	

Channel number selection 111: channel 8 110: ” 7 101: ” 6 100: ” 5 011: ” 4 010: ” 3 001: ” 2 000: ” 1

1: M1 output ON 0: ” OFF

1: M2 output ON 0: ” OFF

1: C1 output ON 0: ” OFF

1: C2 output ON 0: ” OFF

Sound Mechanism

When you do so, the phases of the output waveforms of each channel will, naturally, not match. When you want to start them with their phases reliably aligned, use the key-on function provided by Timer A.

2.6.3 Noise Generator Control Register

This is a register that performs ON/OFF control of the OPM's internal noise generator and control of the noise frequency. The bit layout is shown in Fig. 11. The lower 5 bits set the noise frequency, and bit 7 performs ON/OFF control of the noise generator. When bit 7 is '1', the noise generator is enabled, and slot 32 is replaced by noise. The envelope generator for slot 32 is used as is, but the envelope curve becomes one whose attack is exponential and everything else changes linearly.

Fig. 11 Noise Generator Control

bit 7	6	5	4	3	2	1	bit 0
$0F	NE			NFRQ			

Noise enable 1: Noise output ON 0: ” OFF

Noise frequency setting Noise frequency Fnoise = 4000 / (32 x NFRQ) (kHz) Noise period Tnoise = 2^-1 / (Fnoise x 10^3) (Hz)

2.6.4 Timer A Setting Register

Of the two timers the OPM has, this is the register that sets the time for Timer A. The bit layout is as shown in Fig. 12 on page 272. The data length is 10 bits, so the upper 8 bits are assigned to register number $10 and the lower 2 bits to $11. If the value set in this register is taken as CLKA, an overflow occurs after 64 x (1024 - CLKA) / 4000 (ms). It can issue an interrupt to the CPU, or apply a key-on to all slots simultaneously.

Fig. 12 Timer A Setting

bit 7	6	5	4	3	2	1	bit 0
$10		CLKA (upper)					
$11						CLKA (lower)	

bit 9							bit 0
CLKA (upper)				CLKA (lower)			

The two are combined and handled as 10-bit data

Timer time TA = 64 x (1024 - CLKA) / 4000 (ms)

2.6.5 Timer B Setting Register

This is a register that sets the time for Timer B. The bit layout is as shown in Fig. 13. Since Timer B has a bit length of only 8 bits, the count value can be set with register number $12 alone. If the value set in this register is taken as CLKB, an interrupt can be issued to the CPU after 1024 x (256 - CLKB) / 4000 (ms).

Fig. 13 Timer B Setting

bit 7	6	5	4	3	2	1	bit 0
$12			CLKB				

Timer time TB = 1024 x (256 - CLKB) / 4000 (ms)

2.6.6 Timer Control Register

The bit layout of the timer control register is shown in Fig. 14. This register specifies things such as ON/OFF control of the timer operation and whether to generate an interrupt.

Bits 0 and 1 control the ON/OFF of the timer operation; bit 0 corresponds to Timer A and bit 1 to Timer B. '1' starts the timer and '0' stops it.

Timing Control

bit 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 $14 | CSM | F-RESET | IRQEN | LOAD | | (B) | (A) | (B) | (A) | (B) | (A) | (B) | (A)

- 1: Start Timer A operation
- 0: Stop Timer A operation
- 1: Start Timer B operation
- 0: Stop Timer B operation
- 1: Enable Timer A interrupt (Allowed)
- 0: Disable Timer A interrupt (Prohibited)
- 1: Enable Timer B interrupt
- 0: Disable Timer B interrupt
- 1: Timer A Overflow Flag Reset
- 0: Normal Operation
- 1: Timer B Overflow Flag Reset
- 0: Normal Operation
- 1: All slots are turned ON when Timer A overflows
- 0: Normal Operation

Bits 2, 3 determine whether the CPU is interrupted when a timer overflows. bit 2 is for Timer A and bit 3 is for Timer B. If you set this bit to '1', the CPU is allowed to interrupt when an overflow occurs, and the 1ST bit of the status register is set to '1'.

Bits 4, 5 are control bits to clear the 1ST bit of the status register, bit 4 is used for Timer A, and bit 5 is used for Timer B. If this bit is set to '1', the corresponding 1ST bit will be cleared.

bit 7 specifies whether to use the slot-on function by Timer A. When set to '1', all slots are automatically turned on when Timer A overflow occurs.

2.6.7 LFO Frequency Setting Register

The bit position is shown in Figure 15 on page 274. This determines the frequency for applying vibrato, etc. The relationship between the set value and the number of cycles is shown in Table 17.

Figure 1-5: LFO Frequency Setting ![$18 LFRQ] (Setting the frequency of LFO output.)

Table 1: Relationship between LFRQ Settings and LFO Output Frequencies | DATA (HEX) | FREQ (Hz) | DATA (HEX) | FREQ (Hz) | DATA (HEX) | FREQ (Hz) | DATA (HEX) | FREQ (Hz) | DATA (HEX) | FREQ (Hz) | |------------|----------|------------|----------|------------|----------|------------|----------|------------|----------| | FF | 59.1278 | BF | 3.6955 | 7F | 0.2310 | 3F | 0.0144 | | FE | 57.2205 | BE | 3.6763 | 7E | 0.2235 | 3E | 0.0140 | | FD | 55.3131 | BD | 3.4571 | 7D | 0.2161 | 3D | 0.0135 | | FC | 53.4058 | BC | 3.3379 | 7C | 0.2066 | 3C | 0.0130 | | FB | 51.4984 | BB | 3.2187 | 7B | 0.2012 | 3B | 0.0126 | | FA | 49.5911 | BA | 3.0994 | 7A | 0.1937 | 3A | 0.0121 | | F9 | 47.6837 | B9 | 2.9802 | 79 | 0.1863 | 39 | 0.0116 | | F8 | 45.7764 | B8 | 2.8610 | 78 | 0.1788 | 38 | 0.0112 | | F7 | 43.8690 | B7 | 2.7418 | 77 | 0.1714 | 37 | 0.0107 | | F6 | 41.9617 | B6 | 2.6226 | 76 | 0.1639 | 36 | 0.0102 | | F5 | 40.0543 | B5 | 2.5034 | 75 | 0.1565 | 35 | 0.0098 | | F4 | 38.1470 | B4 | 2.3842 | 74 | 0.1490 | 34 | 0.0093 | | F3 | 36.2396 | B3 | 2.2650 | 73 | 0.1416 | 33 | 0.0088 | | F2 | 34.3323 | B2 | 2.1458 | 72 | 0.1341 | 32 | 0.0084 | | F1 | 32.4249 | B1 | 2.0265 | 71 | 0.1267 | 31 | 0.0079 | | F0 | 30.5176 | B0 | 1.9073 | 70 | 0.1192 | 30 | 0.0075 | | EF | 29.1595 | AF | 1.8941 | 6F | 0.1155 | 2F | 0.0072 | | EE | 28.6610 | AE | 1.7881 | 6E | 0.1118 | 2E | 0.0070 | | ED | 27.5062 | AD | 1.7285 | 6D | 0.1082 | 2D | 0.0068 | | EC | 25.7092 | AC | 1.6689 | 6C | 0.1043 | 2C | 0.0065 | | EB | 24.3529 | AB | 1.6093 | 6B | 0.1006 | 2B | 0.0063 | | EA | 23.4553 | AA | 1.5497 | 6A | 0.0959 | 2A | 0.0061 | | E9 | 22.8819 | A9 | 1.4901 | 69 | 0.0911 | 29 | 0.0058 | | E8 | 22.4938 | A8 | 1.4305 | 68 | 0.0884 | 28 | 0.005

2-6-8 PMD/AMD Setting Register

This register sets the output levels of the LFO for PG, EG, and BW. The layout of the bits of this register is shown in Figure 16. Whether the value written in bit 7 is PMD or AMD is determined. If bit 7 is '1', PMD, if it is '0', AMD is selected. The larger this value, the larger the output level, and the deeper the modulation will be.

Figure 16: PMD/AMD Setting

19 bit 7 bit 6 bit 5 bit 4 bit 3 bit 2 bit 1 bit 0 PMD/AMD ■F LFO output level setting

The lower 7 bits decide the PMD or AMD value. 1: PMD 0: AMD

2-6-9 General-purpose output/LFO output waveform control register

The bit layout is shown in Figure 17 on page 276. OPM has versatile output terminals that can be used for various purposes, but on the X68000, it is used as a forced READY signal to bring the FDC (Floppy Disk Controller) READY terminal to the READY state, and as an ADPCM clock switching signal.

The lower 2 bits of the register determine the output waveform of the LFO from the 4 types shown in the figure.

●Figure 17: Amplified Output/LFO Output Waveform Control

$1B bit 7 6 5 4 3 2 1 0 CT 1 CT 2 W

Selection of LFO output waveform (Refer to the figure below)

N	Waveform supplied to PG (Phase Generator)	Waveform supplied to EG (Envelope Generator)
0	(+)	(+)

N	Waveform supplied to PG (Phase Generator)	Waveform supplied to EG (Envelope Generator)
	(-)	0 (-)

| 1 | (+) | (+) | | | (-) | 0 (-) |

| 2 | (+) | (+) | | | (-) | 0 (-) |

| 3 | (+) | (+) | | | (Noise) | 0 (Noise) |

*1: (+) becomes higher and (-) becomes lower *2: (+) becomes higher, frequency becomes higher

FDC READY terminal control 1: Forced to be READY state 0: Normal operation

ADPCM clock switching 1: 4 MHz 0: 8 MHz

If you select '3' for W by setting it to '11', the LFO output will become a noise waveform, allowing EG and PG to fluctuate randomly. Unlike the noise generator that is independent noise sound, LFO noise output fluctuates the frequency and output level randomly, making it different.

2•6•10 Channel Configuration, Output Control Register

This register is used to make the slot connections, M1 slot feedback amount, and output selection to left and right channels. The bit arrangement is as shown in figure 18. bit 6 controls the ON/OFF of the outputs to the left and right channels, so writing '1' will turn ON output to the respective channel and writing '0' will turn OFF output to the respective channel. When both sides are ON, it feels like they are being written from both sides towards the center.

Bits 3-5 are used for selecting the feedback amount loop through the M1 slot. So, the higher the slot level becomes, the greater the feedback amount, and when the output level decreases, less feedback is applied. The value of bits 0-2 decides the slot connection method, as shown in figure 19 and figure 20, there are 8 types of settings, and which one to use is determined.

M1 decides the amount of feedback within 7

```
Feedback amount X α
```

111: Feedback amount 4 α
110: Feedback amount 2 α
101: Feedback amount α + α
100: Feedback amount α
011: Feedback amount 1/4 α
010: Feedback amount 1/3 α
001: Feedback amount 1/6 α
000: Feedback OFF

1: Output to the left channel is ON
0: Output to the left channel is OFF

1: Output to the right channel is ON
0: Output to the right channel is OFF

Fig. 19 Types of connections (Part 1)

P1 - P4: Output signals of the PG (phase generator)

```
CON = 0
  P1 -> (+) -> M1 -> (+) -> C1 -> (+) -> M2 -> (+) -> C2 -> OUT
        ^feedback     ^P3       ^P2       ^P4
  (M1 output is fed back to its own input)

CON = 1
  P3 -> C1 ----------+
  P1 -> (+) -> M1 -> (+) -> M2 -> (+) -> C2 -> OUT
        ^feedback     ^P2       ^P4
  (M1 output is fed back to its own input; C1 and M1 sum into M2)

CON = 2
  P1 -> (+) -> M1 --------+
        ^feedback         |
  P3 -> C1 -> (+) -> M2 -> (+) -> C2 -> OUT
              ^P2         ^P4
  (M1 output is fed back to its own input; M1 and M2 sum before C2)

CON = 3
  P2 -> M2 --------------+
  P1 -> (+) -> M1 -> (+) -> C1 -> (+) -> C2 -> OUT
        ^feedback     ^P3       ^P4
  (M1 output is fed back to its own input; M2 and C1 sum before C2)
```

Top left: ”Diagram 20: Types of Connections (Part 2)”

Top right: ”Sound Equipment”

Inside diagrams: ”CON = 4” ”CON = 5” ”CON = 6” ”CON = 7”

”OUT”

References for parts: ”P1” ”P2” ”P3” ”P4”

2.6.11 KC (Key Code) Register

The KC register provides basic frequency values necessary for the channel's operation, selectable in terms of musical notes. The bit configuration is shown in figure 21, allowing you to set the octave and the specific note within the octave. The OPM is clocked at 3.579545 MHz, making it more accurate when assigning the table. However, the X 68000 is clocked at 4 MHz, resulting in slightly higher-pitched (about 192.27 cents) sounds. Therefore, according to the OPM manual, a sound played as A on OPM might appear as B on X 68000. Since this pitch difference can be as large as 200 cents, those who find it bothersome should make adjustments using the KF register.

▼図……21 KC (Key Code) (Channel-specific setting)

OCT	NOTE Specification
$E: D# (L)	(11)
$D: C# (L)	(10)
$C: C (F)	(9)
$B: B (8)	(8)
9: A# (7)	(7)
8: A (6)	(6)
7: G# (5)	(5)
6: G (4)	(4)
5: F# (3)	(3)
4: F (ファ)	(2)
1: E (ミ)	(1)
0: D# (0)	(n)

Octave Designation The output frequency fout can be derived by applying OCT and NOTE values through the following equation;

[fout = 440 \times 2 \left(OCT - 4 + \frac{n}{12} \right) \times 4 \times 3.579545]

Given that the standard note A (OCT=4, note ‘ラ’) is 440 Hz.

[fout = 440 \times 2 \left(OCT - 4 + \frac{n-6}{12} \right)]

Therefore, it is important to adjust this using the KF register to match.

2.6.12 KF (Key Fraction) Register

The bit configuration of the register is shown in figure 22. Only the upper 6 bits are utilized, adding fine-tuning of the frequency set by KC with a unit correction of approximately 1.6 (100/64) cents. The relationship between KC and KF is shown in Figure 23 for reference.

Sound system

Fig. 22 KF (key fraction) (set per channel)

$30 bit 7 6 5 4 3 2 1 bit 0 | +-------------------------------+-------+ $37 | KF |///////| +-------------------------------+-------+

Performs fine adjustment of pitch within one note (100 cents) in steps of about 1.6 cents (= 100/64).

Fig. 23 Pitch specification by KC, KF

```
        +-----+---+---+---+---+---+---+---+
        | D6-D4| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
  KC    +-----+---+---+---+---+---+---+---+
        | Oct | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
        +-----+---+---+---+---+---+---+---+

        +-----+---+---+---+---+---+---+---+---+---+----+----+----+----+
        | D3-D0| 0 | 1 | 2 | 4 | 5 | 6 | 8 | 9 |$A |$C |$D |$E |
        +-----+---+---+---+---+---+---+---+---+---+----+----+----+----+
        | Note| D#| E | F | F#| G | G#| A | A#| B | C | C#| D |
        +-----+---+---+---+---+---+---+---+---+---+----+----+----+----+

        +-----+---+~~~+----+~~~+----+~~~+----+~~~+----+---+
        | D7-D2| 0 |   | 16 |   | 32 |   | 48 |   | 63 | 0 |
  KF    +-----+---+~~~+----+~~~+----+~~~+----+~~~+--------+---+
        | Cent| 0 |   | 25 |   | 50 |   | 75 |   |63/64 x 63| 0 |
        +-----+---+~~~+----+~~~+----+~~~+----+~~~+----------+---+
```

2.6.13 PMS/AMS setting registers

Whereas PMD/AMD set the LFO output level, PMS/AMS set the degree of the LFO effect on a per-channel basis. The bit layout of the registers is as shown in Fig. 24 on page 282. Bits 4-6 set the effect on the PG, and bits 0, 1 set the degree of effect on the EG. The numeric values written in the figure indicate the value when the LFO output is at its maximum level.

![Figure] 2-24 PMS/AMS (Setting per channel)

bit 7 6 5 4 3 2 1 0

```
$38 — PMS — — — —
$3F — AMS — — —
```

LFO frequency modulation depth setting *

```
111: ±700 cents
110: ±400
101: ±100
100: ±50
011: ±20
010: ±10
001: ±5
000: 0
```

LFO output level modulation depth setting *

```
11: 95.625dB
10: 47.8125dB
01: 23.90625dB
00: 0dB
```

- Values are when LFO output level is maximum

2-6-14 DT1/MUL Setting Register

Illustrated in Figure 25, the setting for MUL, DT1 and DT2 (ranging from $C0-$DF) creates deviations in the fundamental pitch frequency (provisionally called the fundamental frequency) for each slot, based on the frequency value given by KC and KF. Therefore, the range of MUL is from 1/2 to 15 and is used to make these deviations. The degree of deviation in DT2 changes depending on the amplitude of the fundamental frequency level, and slots with a DT1 value of 0-3 generate multiples of the fundamental frequency (at least the fundamental frequency divided by 2). When DT1 equals DT2, no setup is generated. Changes in the DT1 frequency deviation value depend on the KC value. Refer to Table 2 for further details.

2-6-15 TL (Total Level) Setting

TL controls the output level of the slot, and the decrease amount defined by this register determines the actual EG output. The layout of the register is shown in Figure 26 on page 284. Only the upper 6 bits of the TL register are valid, and the decrease amount is 0.75xTL(dB). The higher the value of TL bit 6, the greater the output signal level, and a value of $F (15) results in the smallest decrease.

Diagram 25 DT1/MUL (Set for each slot)

- Setting the frequency given by KC, KF further: (fine adjustment)
- Setting to a multiple of the frequency given by KC, KF below:

```
$F  :  15
$E  :  14
$D  :  13
$C  :  12
$B  :  11
$A  :  10
9    :   9
8    :   8
7    :   7
6    :   6
5    :   5
4    :   4
3    :   3
2    :   2
1    :   1
0    :  0.5
```

Table 2 How to Shift DT1 Setting Value and Frequency

OCT	NOTE	Frequency Shift (cents)		Frequency Shift (Hz)	
		DT1=0	DT1=1	DT1=2	DT1=3
0	0	0.000	0.000	5.025	10.036
0	1	0.000	0.000	4.228	8.445
0	2	0.000	0.000	3.559	7.110
0	3	0.000	0.000	3.993	5.980
0	4	0.000	2.515	5.025	5.025
1	0	0.000	0.000	2.115	4.228
1	1	0.000	1.778	3.555	6.338
1	2	0.000	1.496	2.990	5.435
1	3	0.000	1.428	2.990	5.435
2	0	0.000	1.258	2.515	4.025
2	1	0.000	1.057	3.170	4.225
2	2	0.000	0.889	2.667	3.555
2	3	0.000	0.748	2.242	3.735
3	0	0.000	1.258	2.515	3.143
3	1	0.000	1.057	2.114	3.170
3	2	0.000	0.889	1.778	2.667
3	3	0.000	0.748	1.869	

```
6  0   0.000   0.393   0.865   1.258   0.000   0.267   0.587   0.853
6  1   0.000   0.397   0.793   1.123   0.000   0.320   0.640   0.907
6  2   0.000   0.334   0.723   1.056   0.000   0.320   0.693   1.013
6  3   0.000   0.327   0.654   0.935   0.000   0.373   0.747   1.067
7  0   0.000   0.315   0.629   0.865   0.000   0.427   0.853   1.173
7  1   0.000   0.315   0.629   0.865   0.000   0.427   0.853   1.173
7  2   0.000   0.315   0.629   0.865   0.000   0.427   0.853   1.173
7  3   0.000   0.315   0.629   0.865   0.000   0.427   0.853   1.173
```

Fig. 26 TL (total level) (set per slot)

$60 bit 7 6 5 4 3 2 1 bit 0 | +-------+----------------------------+ $7F |///////| TL | +-------+----------------------------+

Sets the output level of each slot in units of 0.75 dB. If the maximum output level is Lmax, the output level L is expressed as

```
L = Lmax x 10^(-(0.75/20) x TL)
```

2.6.16 KS/AR setting registers

The bit layout is shown in Fig. 27. In the case of a musical instrument, the higher the pitch becomes, the shorter the attack and the lingering (release) time of the sound tend to become. KS exists to obtain this effect; corrections based on KS are applied to AR, D1R, D2R, RR, and so on, which are described later. This correction amount depends on KC (key code) and is given by (KC%4) % (2^(3-KS)) (% denotes integer division, "^" denotes exponentiation).

AR is the attack rate, which determines the rise of the sound immediately after key-on. The larger the value, the sharper the rise.

For specific numeric values, two cases are shown in Table 3 and Table 4 on page 289 and following: the time from minimum volume (0%: -96 dB) to maximum (100%: 0 dB), and the time from 10% to 90%. Please refer to them.

Note that the RATE value in this table uses, in the case of RR, the value calculated as RR x 4 + 2 + KS; in other cases, where XR is the value set in the register, the value calculated as XR x 2 + KS is used, and if the calculated value becomes 63 or greater, the value for RR = 63 is applied. The leftmost column of the table shows the calculated RR value; the two

numeric values to its right show the value when RR is split into the upper 4 bits and the lower bits.

KS/AR

Sound Mechanism

- Attack rate (Time till volume reaches 0dB after KEY ON)
- Setting values for KC (key code) by attack, first decay, second decay, and release time
- (However, rounding down any remainders below decimal point during division)

AMS-EN/D1R Setting Register

”The bit arrangement is shown on the right. AMS-EN determines whether to apply modulation by the LFO to the EG. Writing '1' indicates that modulation will be applied, while '0' means normal operation.

*For D1R, it decides the time to transition from first decay to second decay. This process is scaled similarly to AR (Attack Rate). Please refer to the comments for AR as they apply similarly here.”

Diagram of AMS-EN/D1R Setting Register

LFO output level modulation selection: 1: Modulation applied 0: No modulation

Setting time for first decay

2-618 DT2/D2R Setting Register

The bit arrangement of this register is as shown in Figure 29. DT2, like DT1/MUL, gives a large change in frequency numbers decided by KC and KF, which means a change of several hundred cents.

D2R sets the time from the moment you transition from the first decay rate to the second decay rate until the sound completely dies out. This time is also scaled by KS.

Figure...29 DT2/D2R (set for each slot)

bit 7 6 5 4 3 2 1 0 $C0 D2R $DF DT2

```
Setting values to shift the frequency
given by KC and KF further
        Second decay time setting
```

11: +950 cents (×1.73 times) 10: +781 cents (×1.57 times) 01: +600 cents (×1.41 times) 00: +0 cents (×1.00 times)

2-619 D1L/RR

The bit arrangement is shown in Figure 30. The upper 4 bits are D1L and the lower 4 bits are RR. D1L determines the level when transitioning from the first decay rate to the second decay rate, with values from 0 to $E indicating this level as -3×D1L (dB), and $F indicating -3×D1L-48 (dB).

RR sets the release rate time from when you release the key until the sound dies out. Like AR, D1R, and D2R, RR is also scaled by KS.

EG output level and register value relationship is shown in Figure 31 for reference. Note that scaling by KS is not considered in this figure.

Sound system

Fig. 30 D1L/RR (set per slot)

$E0 bit 7 6 5 4 3 2 1 bit 0 | +------------------+----------------+ $FF | D1L | RR |

```
        +------------------+----------------+
                                  |
                                  Release time setting
```

Determines the level at which the transition from the first decay to the second decay occurs. If this level is taken as L_D1, then

```
L_D1 = -3 x D1L      (dB):  when 0 <= D1L <= $E
L_D1 = -3 x D1L - 48 (dB):  when D1L = $F
```

Fig. 31 Relationship between the envelope generator output waveform and the

```
        setting values

   (Key ON)                                  (Key OFF)
   ___                                       ___________
      |                                     |
  0dB |         /\                          |
      |        /  \___
   TL |       /       \____
  D1L |      /             \_________
      |     /                        \____
 -96dB|___/                              \____|\___
           <-AR-><-D1R->          <-RR->
                      <------- D2R ------->
```

2.7 Relationship between setting values and OPM operation

Unfortunately, documentation showing numerically how the OPM setting values affect the actual audio output waveform has not been published (and according to the manufacturer there are no plans to release it). Indeed, from the standpoint of sound design, parameter adjustment is ultimately done while listening with one's ears, not while looking at the output waveform. However, on a machine like the X68000 that is also equipped with ADPCM, one can also conceive of usages such as outputting from the ADPCM using the parameters for the FM sound source.

Therefore, I am now going to explain the basic concept of connecting operators in series and the method of calculating each rate of the envelope generator (EG) respectively.

The following formulas are as close to the original as possible (it is a near approximation obtained by watching the waveform while performing synchronized calculations) and are not necessarily the values that OPM (Operator Pulse Modulation) itself has incorporated into its specifications, so please be aware of this.

○ 2-1 Output Waveform When Operators are Connected in Series

The output level of the SIN waveform table when two operators are connected in series is given by;

$A = SIN(\omega t + \alpha SIN(\psi t))$

where ωt and ψt are frequencies, which can be easily obtained, but α cannot be determined absolutely. Therefore, the actual waveform cannot always be obtained with these formulas. Experimental results, each slot output is determined by TL (Total Level) value (such as when AR*0, D1R, and D2R are at their maximum values),

$\alpha = 10^{\wedge}(-(0.75/20 \times TL + \varepsilon/2))$

where ε here indicates the base of natural logarithms (=2.71828...).

Thus, it was found that the calculation result of αSIN (ψt) sometimes closely approximates the angle at which the sum of ωt results.

We do not yet know M1's feedback amount, but TL can be assumed to equal 1 (radian) when 0 dB, based on observation. For example, FL (Feedback Level) = 3 is equivalent to 0.7853 (3.14159/4), and a waveform close to this can be obtained by feeding back the calculated value.

○ 2-2 Relationship between EG Rate and Time

EG values follow almost logarithmic functions. The attack time from 0 to 100% is calculated by using the 4 upper bits and the 2 lower bits of RATE (second item in Table 2, third column). Suppose the 4 upper bits are RATEH and the 2 lower bits are RATEL, then t can be calculated as:

t = (10^4.202682) / (2^RATEH) × (1 / (1 + 0.25×RATEL)) × 3.58 / 4.00

Sound system

is expressed. Other times can be calculated simply by multiplying this value by a coefficient. Note, however, that because of the limit of the OPM's time resolution, when t becomes small the actual value deviates from the value calculated by this formula, so please be careful.

Table 3 Each rate setting value and time for the EG (0 - 100%)

Attack AR x 2 + KS	Time (ms)	First decay / second decay / release XR x 2 + KS RR x 4 + 2 + KS	Time (ms)
0 : 0	infinity	0 : 0 0	infinity
1 : 0	infinity	1 : 0 1	infinity
2 : 0	infinity	2 : 0 2	infinity
3 : 0	infinity	3 : 0 3	infinity
4 : 1 0	7136.33	4 : 1 0	98637.69
5 : 1 1	5709.06	5 : 1 1	78910.15
6 : 1 2	4757.55	6 : 1 2	65758.46
7 : 1 3	4077.90	7 : 1 3	56364.40
8 : 2 0	3568.16	8 : 2 0	49318.84
9 : 2 1	2854.53	9 : 2 1	39455.07
10 : 2 2	2378.78	10 : 2 2	32879.23
11 : 2 3	2038.95	11 : 2 3	28182.20
12 : 3 0	1784.08	12 : 3 0	24659.42
13 : 3 1	1427.27	13 : 3 1	19727.54
14 : 3 2	1189.38	14 : 3 2	16439.61
15 : 3 3	1019.48	15 : 3 3	14091.09
16 : 4 0	892.04	16 : 4 0	12329.71
17 : 4 1	713.63	17 : 4 1	9863.77
18 : 4 2	594.69	18 : 4 2	8219.81
19 : 4 3	509.74	19 : 4 3	7045.55
20 : 5 0	446.02	20 : 5 0	6164.86
21 : 5 1	356.82	21 : 5 1	4931.89
22 : 5 2	297.35	22 : 5 2	4109.90
23 : 5 3	254.87	23 : 5 3	3522.77
24 : 6 0	223.01	24 : 6 0	3082.42
25 : 6 1	178.41	25 : 6 1	2465.94
26 : 6 2	148.68	26 : 6 2	2054.95
27 : 6 3	127.43	27 : 6 3	1761.38
28 : 7 0	111.51	28 : 7 0	1541.22
29 : 7 1	89.20	29 : 7 1	1232.97
30 : 7 2	74.34	30 : 7 2	1027.48
31 : 7 3	63.72	31 : 7 3	880.59
32 : 8 0	55.75	32 : 8 0	770.60
33 : 8 1	44.60	33 : 8 1	616.48
34 : 8 2	37.16	34 : 8 2	513.74
35 : 8 3	31.86	35 : 8 3	440.35
36 : 9 0	27.88	36 : 9 0	385.31

```
37 :  9  1     22.30       | 37 :  9  1       308.25
38 :  9  2     18.58       | 38 :  9  2       256.86
39 :  9  3     15.93       | 39 :  9  3       220.17
40 : 10  0     13.94       | 40 : 10  0       192.65
41 : 10  1     11.15       | 41 : 10  1       154.12
42 : 10  2      9.29       | 42 : 10  2       128.43
43 : 10  3      7.97       | 43 : 10  3       110.09
44 : 11  0      6.97       | 44 : 11  0        96.33
45 : 11  1      5.58       | 45 : 11  1        77.06
46 : 11  2      4.65       | 46 : 11  2        64.22
47 : 11  3      3.98       | 47 : 11  3        55.04
48 : 12  0      3.48       | 48 : 12  0        48.18
49 : 12  1      2.78       | 49 : 12  1        38.53
50 : 12  2      2.33       | 50 : 12  2        32.11
51 : 12  3      1.99       | 51 : 12  3        27.52
52 : 13  0      1.74       | 52 : 13  0        24.08
53 : 13  1      1.53       | 53 : 13  1        19.27
54 : 13  2      1.16       | 54 : 13  2        16.06
55 : 13  3      1.09       | 55 : 13  3        13.77
```

Table 14-4 EG various rate settings and time (10-90%)

Attack Time (ms) First Decay/Second Decay/Release

AR X2 + KS XR X2 + KS RR X4 + 2 KS Time (ms) Time (ms)

```
0: 0    ∞Max                                  0: 0      ∞Max
1: 0    ∞Max                                  1: 0      ∞Max
2: 0    ∞Max                                  2: 0      ∞Max
3: 0    ∞Max                                  3: 0      ∞Max
4: 1    4008.07                                4: 1      19942.60
5: 1    3206.45                                5: 1      15954.08
6: 1    2276.35                                6: 1      12395.07
7: 1    1701.92                                7: 1      11335.78
8: 2    1301.24                                8: 2      9971.30
9: 2    1003.63                                9: 2      7977.05
```

10: 2 755.12 10: 2 6647.53 11: 2 571.85 11: 2 5697.88 12: 3 436.15 12: 3 4988.65 13: 3 332.56 13: 3 3985.52 14: 3 247.01 14: 3 3223.77 15: 3 177.02 15: 3 2482.35 16: 4 131.01 16: 4 1949.26 17: 4 100.81 17: 4 1661.88 18: 4 76.34 18: 4 1248.47 19: 4 58.29 19: 4 910.99 20: 5 250.50 20: 5 1246.41 21: 5 200.41 21: 5 997.13 22: 5 167.01 22: 5 830.94 23: 5 143.15 23: 5 712.23 24: 6 120.26 24: 6 623.21 25: 6 100.20 25: 6 498.57 26: 6 83.50 26: 6 411.47 27: 6 71.57 27: 6 356.12 28: 7 62.62 28: 7 311.60 29: 7 50.10 29: 7 249.28 30: 7 41.75 30: 7 207.74 31: 7 35.78 31: 7 178.04 32: 8 31.32 32: 8 155.80 33: 8 25.05 33: 8 124.64 34: 8 20.87 34: 8 103.86 35: 8 17.89 35: 8 89.03 36: 9 15.65 36: 9 77.30 37: 9 12.52 37: 9 52.82 38: 9 10.44 38: 9 62.52 39: 9 8.93 39: 9 44.59

```
40: 10   7.83                                 40: 10     38.95
41: 10   6.47                                 41: 10     31.16
42: 10   5.22                                 42: 10     25.96
43: 10   4.48                                 43: 10     22.26
44: 11   3.91                                 44: 11     19.48
45: 11   3.13                                 45: 11     15.58
46: 11   2.61                                 46: 11     12.98
47: 11   2.26                                 47: 11     11.12
48: 12   1.96                                 48: 12     9.74
49: 12   1.57                                 49: 12     7.79
50: 12   1.31                                 50: 12     6.49
51: 12   1.12                                 51: 12     5.57
52: 13   0.98
```

ADPCM

3.1 Overview of ADPCM

OPM (FM sound source) generates waveforms by calculation with SIN curves and noise, whereas ADPCM captures and reproduces natural sounds. The input waveform is sampled periodically and the voltage at that point is converted directly to digital data. This method, known as PCM, is widely adopted in CDs. With PCM, there is no significant problem if the sampling frequency is high, but the temporal resolution of the waveform becomes a big problem. For example, in the case of a CD, 1 second of data is divided into 16 bits each sampled at a frequency of 44.1 kHz, resulting in 88.2 kB of data per second.

To reduce this amount of data, a method is used where the difference from the previous voltage is recorded as data. This method, called DPCM (Differential PCM), measures the change from the previous data and digitizes the difference. ADPCM (Adaptive Differential PCM) further improves upon DPCM, adjusting the volume to accommodate large voltage changes. When the previous change is large, the width of the change is also large; when small, the width is likewise small.

The X68000 uses an LSI called MSM 6258V for both converting the input waveform to ADPCM data and reconverting ADPCM data back to the input waveform. This LSI is monaural and for X68000, a mixing process is added between the right, left, and central output signals.

Next, the features of the MSM 6258V are summarized.

Page 291

3.1 ADPCM Data

The MSM 6258V creates data as either 3-bit or 4-bit ADPCM data, consolidating data sampled twice into a 1-byte data for CPU transfer. While it can be switched to 3-bit ADPCM or 4-bit ADPCM via an LSI pin, the X68000 is fixed to 4-bit ADPCM mode.

3.1.2 Sampling Frequency

The sampling frequencies can be selected from 1/512, 1/768, or 1/1024 of the LSI clock frequency. Since the X68000 allows switching between 4 MHz and 8 MHz as the clock, there are five frequency options available: 3.9 kHz, 5.2 kHz, 7.8 kHz, 10.4 kHz, and 15.6 kHz (for 4 MHz it's 1/512 and for 8 MHz it's either 1/1024 or 7.8 kHz, thus one frequency less). The memory usage needed for 1 second is half the sampling frequency (for 15.6 kHz, it's 7.8 KB).

3.1.3 A/D, D/A Converter

The A/D converter (which converts input voltage to digital data) and the D/A converter (which converts digital data to output voltage) integrated into the MSM 6258V have an accuracy of 10 bits each. The MSM 6258V converts 8-bit input data to 10-bit PCM data, then converts the 2-bit ADPCM data to 10-bit PCM data for output as sound signals.

3.2 ADPCM-Related Registers

An overview of the registers related to ADPCM control is shown in Figure 32. Registers holding the statuses manipulated by the MSM 6258V cannot be read. As the X68000 can

only monitor the stage signal of the sampling frequency, ADPCM output switching is captured on PPI(8255) ports. ADPCM's basic clock selection and OPM's versatile output terminal CT2 are used to perform switching.

Page 292

Figure 32 Registers Related to ADPCM Operation

ADPCM (MSM6258V)

Address	READ/WRITE	bit 7	6	5	4	3	2	1	bit 0	Notes
$E92001	R	PLAY/ REC	'1'	'0'	'0'	REC ST	PLAY ST	ST/	ADPCM Status	
$E92003	W					Data	'0'	Data	Data Output	

PPI (8255) Port Control Word Register

Address	READ/WRITE	bit 7	6	5	4	3	2	1	bit 0	Note
$EA9005	R/W	IOA6	IOA5	PC4	Sampling	PC1	PAN	ADPCM Sample/Channel Control		
$EA9007	W	'0'	Bit	Sel	Data					Con

OPM (YM2151) Register No. = S1B ($E90001 writes $1B and then accesses it)

Address	READ/WRITE	bit 6	5	4	3	2	1	bit 0	Notes
$E90003	W	CT2	CT1			W			ADPCM Base Clock Switching

Additionally, since it is more convenient to perform ADPCM data transfer with DMAC, one of the channels of the DMAC on X68000 is allocated for ADPCM use. Please refer to the DMA section for how to set up DMAC.

3-2-1 ADPCM Status Register

The bit arrangement of the ADPCM status register is shown in Figure 33 on page 294. bit 7 indicates whether it is in a stop state (no sound data output) or in a sound data output state. Bits '1' and '0' indicate recording/stop state and playback state, respectively.

3-2-2 ADPCM Command Register

Controls ADPCM operations/start/stop control. The bit arrangement is as shown in Figure 34 on page 294. bit 2 starts recording, bit 1 controls the start of playback, while bit 0 instructs stopping of recording/playback. Once playback stops, if not stopped by the command register, it will repeatedly output the data given after ADPCM ended, so caution is advised.

33 ADPCM Status Register ($E92001) bit 7 6 5 4 3 2 1 bit 0 PLAY / REC '1' '0'

1: Recording/Standby 0: Playback

34 ADPCM Command Register ($E92001) bit 7 6 5 4 3 2 1 bit 0 '0' REC ST PLAY ST SP

```
1: ADPCM Playback Start                1: ADPCM Recording Start
0: Do Not                             0: Do Not
```

Be sure to write the stop command. When the stop command is not written, the output level of the ADPCM is maintained in the last state, and when the next playback start command is received it may return to 1/2 VDD (the neutral level of maximum amplitude), and a ”pop” noise may occur.

3.2.3 ADPCM Data Register

This is the register for inputting ADPCM data. The bit arrangement is as shown in Figure 35. The ADPCM data is transmitted in two-sample units, each divided into upper 4 bits and lower 4 bits as shown in the figure. The data created by the ADPCM consists of 4-bit data with the upper bit first and the lower bit last. Each of these 4-bit data is represented by the upper bit set as the highest value.

(Note: Figures and bit patterns are represented as closely as possible to the original).

Sound Function

Fig. 35 ADPCM Data Register ($E92003)

bit 7 6 5 4 3 2 1 0 Data n+1 Data n n+1th ADPCM Data n-th ADPCM Data FLAG ABS - DATA Polarity 1: Negative 0: Positive

3.2.4 PPI(8255) Port C

Using spare bits of PPI used for the joystick, they are used for selecting the sampling rate and controlling ADPCM sampling. The layout of this port is shown in FIG. 36 on page 296.

Bits 2 and 1 select one of the ADPCM sampling rates, which is divided by the master clock by 512, 768, or 1024 times. When both Bits 2 and 1 are set to '11', it is unused. When actually set to '01', the result is the same as when setting to '01'.

bit 1 and bit 0 are panning control bits, where bit 0 controls the left channel, and bit 1 controls the right channel. When set to 0, the output is off; when set to 1, the output is on. If both are on, sound can be heard from the center channel.

It should be noted that on the X68000, even though Port C of 8255 is used as an output port, due to the 8255 characteristics, it can read the output port, and the output data can be read as it is. Therefore, it is possible to change only the necessary bits while reading the currently set output data and keeping it intact.

Figure 36 8255 Port C ($E9A005)

bit 7	6	5	4	3	2	1	bit 0
IOA 6	IOA 5	PCS	PC4	Sampling Rate	PCM PAN		

ADPCM Output Control

- 11: Both left and right OFF
- 10: Left ON only
- 01: Right ON only
- 00: Both left and right ON

ADPCM Sampling Rate Switch

- 11: (Not used)
- 10: Basic clock ÷37 (7.8K / 4MHz, 15.6K / 8MHz)
- 01: 75⌉ (5.2K / 10.4K /)
- 00: 10⌉ (3.9K / 7.8K /)

Joystick #1 Control

- 1: Operation prohibited
- 0: Normal operation

Joystick #2 Control

- 1: Operation prohibited
- 0: Normal operation

Joystick #1 Trigger A

- 1: When the A button is pressed
- 0: Normal operation

Joystick #1 Trigger B

- 1: When the B button is pressed
- 0: Normal operation

3-2-5 PPI (8255) Control Word Register

PPI has a special function to control any bit of the port for bit set/reset operations. These operations are performed using the PPI control word register. Bit set/reset commands are shown in Figure 37. When bit 7 is 0, the 8255 recognizes the bit set/reset command, and the specified Port C bits from bit 1 to bit 3 will be changed to the data provided.

Of course, since Port C can be read/write, there's no issue in reading, performing operations like AND or OR on the data, and then rewriting it, but operation is possible directly as needed by setting and resetting bits.

Page 296

Fig. ..... 37 8255 Control Word Register ($E9A007)

bit 7 6 5 4 3 2 1 0 '0' Bit Sel Data

Specify the data to be set in the bit selected by Bit Sel 1: Set to '1' 0: '0'

Specify the port and bit number to operate on 111: Set bit 7 data 110: // 6 101: // 5 100: // 4 011: // 3 010: // 2 001: // 1 000: // 0

If you only have to manipulate one bit at a time, it is more convenient to use the bit set/reset feature, so this function has been explained.

3•2•6 OPM Register 1B

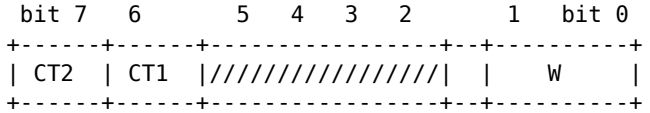
In the X68000, the clock switch signal of the ADPCM is output using the versatile output terminal CT2 of the OPM (FM sound source LSI). This bit is bit 7 of the OPM register 1B. The bit layout of the register is as shown in the figure on page 298. bit 7 is the clock selection bit, '0' for 8 MHz, '1' for 4 MHz. This register is write-only and cannot be read back, so if you are using it for system purposes, be sure to store its value in another bit.

3•3 Sample Program

We have created a sample program to operate the ADPCM for your reference. In this example, the transition signal of S1P is switched. The options specified here are: start option (bit 1), PPI busy control bit (bit 0, 1) and three values specified by the PPI subcircuit selector (bit 2, 3), three values indicated at ADPCM base clock selection (OPM...

Page 297

Fig. 38 OPM register (register No. = $1B)



```
    |      |                              |
    |      |                              Selection of the oscillation
    |      |                              waveform of the LFO inside the OPM
    |      |
    |      FDD READY terminal control
    |        1: force busy state
    |        0: normal operation
    |
    ADPCM clock switching
      1: 4 MHz
      0: 8 MHz
```

is the value to set in register $1B.

In this sample, the bit set/reset function of the PPI control is used, so please refer to that as well.

List 1 ADPCM operation (playback of continuous data from $1F)

```
/*
 * ADPCM operation test
 *
 * Because volatile is not supported under XC,
 * include the following line to disable volatile
 * #define volatile
 */

#include <doslib.h>

struct DMAREG {
    unsigned char   csr;
    unsigned char   cer;
    unsigned short  spare1;
    unsigned char   dcr;
    unsigned char   ocr;
    unsigned char   scr;
    unsigned char   ccr;
    unsigned short  spare2;
    unsigned short  mtc;

/* 8255 Control Word Register */

/* OPM Register Number Designation Register */

/* OPM Data Register */

/* ADPCM Command Register */

/* ADPCM Status Register */

/* ADPCM Data Register */

void adpcm_sample(); void adpcm_clksel(); void adpcm_stop(); void adpcm_start(); void dma_setup(); void dma_start(); void wait_complete(); void clear_flag();

void main(argc,argv)
    int argc;
    char *argv[];

{
    unsigned int    i, pan, sample, clk;
    if (argc >= 2)
        pan = atoi(argv[1]);
    else    pan = 0;
```

```
    if (argc >= 3)
        sample = atoi(argv[2]);
    else    sample = 0;
    if (argc >= 4)
        clk = atoi(argv[3]);
    else    clk = 0;
    SUPER(0);
    dma =       (struct DMAREG *)0xe840c0;
    ppi_cwr =   (unsigned char *)0xe9a007;
    opm_regno = (unsigned char *)0xe90001;
    opm_data =  (unsigned char *)0xe90003;
    adpcm_command = (unsigned char *)0xe92001;
    adpcm_status =  (unsigned char *)0xe92001;
    adpcm_data =    (unsigned char *)0xe92003;

    adpcm_stop();
    create_adpcmdata(pcmbuf,BUFSIZE);
    adpcm_outsel(pan);  /* Panpot Control */
    adpcm_sample(sample);    /* Sampling rate */
    adpcm_clksel(clk);  /* ADPCM Clock */
    clear_flag();
    dma_setup();
    dma_start();
    adpcm_start();
```

}

This code snippet is self-explanatory in its context and contains function declarations, a main function, and several hardware related operations.

Sound system

```
    wait_complete();
    adpcm_stop();
    clear_flag();
```

}

void create_adpcmdata(buf, length)

```
    unsigned char   *buf;
    unsigned int    length;
```

{

```
    while(length--)
        *buf++ = 0x1f;
```

}

void adpcm_outsel(sel) unsigned int sel; {

```
    *ppi_cwr = (0 << 1) | ((sel >> 1) & 1); /* Left  */
    *ppi_cwr = (1 << 1) | (sel & 1);        /* Right */
```

} void adpcm_sample(rate) unsigned int rate; {

```
    *ppi_cwr = (2 << 1) | ((rate >> 1) & 1);
    *ppi_cwr = (3 << 1) | (rate & 1);
```

}

void adpcm_clksel(sel) unsigned int sel; {

```
    *opm_regno = 0x1b;
    *opm_data  = (sel & 1) << 7;
```

}

void adpcm_stop() { *adpcm_command = 0x1; }

void adpcm_start() { *adpcm_command = 0x2; }

ADPCM Data

Investigations have been made into how the specific algorithm for converting to ADPCM data is structured, but it appears that it is proprietary knowledge (know-how) belonging to the manufacturer (OKI) and has not been disclosed.

For this reason, it is unclear how the X68000 ADPCM data is composed.

Please refer to the ADPCM audio algorithm column embedded in Yamaha's Y8950 (MSX-AUDIO) or LSI and YM2608 (OPNA) when you are investigating ADPCM.

COLUMN

Algorithm of ADPCM (Procedure for ADPCM Voice Analysis)

```
1 A/D conversion ... Convert audio to 8-bit PCM data for each sampling rate.
2 8→16 ..... Convert the obtained PCM data to 16-bit data by multiplying it by 256; convert
↪  it to Xn.
3 Calculation of dn ... Compare this Xn with the predicted value xn and find the difference,
↪  dn.
4 Determination of ADPCM data

        ........ When dn is positive, set the MSB (L4) of the ADPCM data to '0', and when
        ↪  negative, set it to '1'.
        Absolute value of the difference: From the relationship between | dn | and
        ↪  quantization
        width Δn, determine the remaining 3 bits (L3, L2, L1) of the ADPCM data.
        The symbol of ADPCM data is as shown in the table.

         **Table - ADPCM Data and Quantization Width (f)**
```

dn ≥ 0 dn < 0
L4 L3 L2 L1 f Condition (f = 1/n < 1/λn)

```
0 0 0 0 1 57/64          1/4 ≤ |n <1/2
0 0 0 1 1 57/64          1/2 ≤ |n <3/4
0 0 1 0 1 57/64          3/4 ≤ |n <1
0 0 1 1 1 77/64          1 ≤ |n <5/4
0 1 0 0 1 102/64         5/4 ≤ |n <3/2
0 1 0 1 1 128/64         3/2 ≤ |n <7/4
0 1 1 0 1 153/64         7/4 ≤ |n <5
```

With the above operation, the conversion to ADPCM data from the audio data is replaced.

5 Updating the predicted value and quantization width ..... Once ADPCM data is obtained, update the prediction value for the next step, xn+1, and the quantization width. Update Δn: Δn+1.

Xn+1 = {1-2×L4}×(L3+L2/2+L1/4+1/8)×Δn+xn Δn+1=f(L3, L2, L1)×Δn : Δnmin=127, Δnmax=24576

- Initial setting: Predicted value x1=0 Quantization width Δ1=127

Below, repeat operations 1 - 5 for each sampling time to perform voice analysis.

(Blank page in the original.)

SCC

SCC is a serial communication LSI that supports synchronous communication, asynchronous communication, and data modulation. Here, we explain how the SCC is used in the X68000, and how to set various registers in the SCC.

1. Overview of SCC

The X68000 uses the Zilog 8530 SCC (Serial Communication Controller; hereinafter simply referred to as SCC), a member of the Z8000 series LSI, to support the RS-232C interface and mouse. The block diagram centered around the SCC is shown in Figure 1 on page 306.

The SCC has two serial boards: channel A and channel B. On the X68000, channel B's RTS and RxD are used for the mouse, while channel A is used for RS-232C communication. The SCC features several communication lines, and channels that do not support RS-232C use CTS and DCD if not used; the DTR switches the clock line used for CD de-lining. Thus, the X68000 provides a standard framework not only for asynchronous communication but also for synchronized communication such as Monosync and Bisync, and SDLC. The SCC LSI has separate clock functions to separate the transmission bit clock and data, with robust support for synchronized communication.

●Figure 1 SCC Peripheral Block Diagram

X68000 hardware allows using the communication modes shown below. However, in the case of the X68000, DMA cannot be used for data transmission with the SCC, so the transfer speed depends on the CPU's response speed. Therefore, the actually usable transfer speed is considered to be significantly lower than the values indicated here.

Channel A

- Asynchronous Mode

```
- Character Length: 5/6/7/8 bits
- Stop Bit Length: 1/1.5/2 bits
- Parity: Even/Odd/None
- Clock: x1, x16, x32, x64 (x1 requires external synchronization)
- Error Detection:
  - Parity Error
  - Overrun Error
  - Framing Error
```

- Synchronous Mode

```
- Byte-oriented Synchronous Mode: (Monosync, Bisync)
- Character Length Selection: Can be either internal or external
- Synchronous Character Count: 1/2 characters
- Synchronous Character Length: 6/8 bits
- Synchronous Character Control: Insertion/Removal can be automatic
- CRC: Can be automatically generated/checked
```

- SDLC Mode

```
- Automatic generation/detection of abort sequence
- Automatic insertion/removal of SDLC control characters
- Automatic insertion of flag between messages
- Automatic detection of address field
- Termination processing of information field
- CRC automatic generation/check
- Automatic online/offline loop by detection of EOP in SDLC loop mode
```

- Data Transfer Rate

```
- Asynchronous Mode: 38.4 Kbps (in x16 mode)
- Monosync/Bisync: 1.5 Mbps
- FM Encoding Method, DPLL: 375 Kbps
- NRZI Encoding Method, DPLL: 187 Kbps
```

Channel B

Asynchronous communication only

Character length: 8 bits Stop bit: 2 bits Parity: None Baud rate: 4800 bps

1. SCC Data Communication Mode

We have summarized the communication modes supported by SCC and their data formats in Figure 2. The asynchronous mode, which is currently the most commonly used, especially in devices using RS-232 C, indicates that SCC supports data transmission using this data format.

Other formats are all synchronous transmission modes. Synchronous transmission mode always requires a clock signal to establish the timing for data transmission. We illustrate the relationship with the clock and the data in Figure 3. On the sender side, data is synchronized with the rising edge of the clock signal, thereby changing the data.

Figure 2 .... Data Formats Supported by SCC

Async (Asynchronous) (Graphic of asynchronous data format showing start bit, data, parity bit, and stop bit)

Monosync (Graphic of monosynchronous data format showing SYNC, Data, CRC1, CRC2)

Bisync (Graphic of bisynchronous data format showing SYNC SYNC, Data, Data, CRC1, CRC2)

External Sync (Not used for X68000)

(Graphic showing external synchronous data format with SYNC indicating start) (Data, Data, CRC1, CRC2)

SDLC/HDLC X.25 (Graphic showing SDLC/HDLC data format with Flag, Address, Information, CRC1, CRC2, Flag)

Page 308

Figure 3: Synchronous Transmission Operation

By ensuring that the receiver captures the timing data on the rising and falling edges of the clock signal, the data can be accurately transmitted. Before the data, the Sync Sync Flag indicates the start of a series of data, allowing the receiver to align the data and timing. The CRC following the data is a check code used to detect if there are any errors in the received data.

Among these, in External Sync Mode, the timing for reading data uses the LSI or SYNC terminal of the synchronous signal, similar to RS-232C signals. However, in the X 68000, the LSI of the SYNC terminal is not exposed externally. Thus, in the X 68000, this mode cannot be used.

Next, let's take a closer look at these data formats.

0.01 Async (Asynchronous) Mode

Figure 2 shows a typical data format for asynchronous communication mode. A '0', called the start bit, is sent before the data. Following the data, a '0' is sent as a parity bit for check purposes, and a '1' called the post stop bit at the end of the character data. Sometimes there is no parity bit. Additionally, the stop bit can range from 1 bit, 1.5 bits, 2 bits, or any combination of these.

The receiver begins preparing to receive data by noticing that the data line has become '0'. It then sequentially samples the midpoint (intended center) of each bit. The parity bit checks the correctness of the transmitted data. The parity bit is one bit of data to decide whether the number of bits that contain data, including data and parity, is odd or even.

Additionally, the recipient confirms whether the stop bit, which should indicate the end of the data, is set to '1'. If it is '0', it is detected as a framing error.

0:02 Monosync Mode

Monosync mode, regardless of whether it is asynchronous or synchronous communication mode, requires a clock signal to be synchronized with data output/input. While signals other than data, such as start bits or stop bits and parity bits, are necessary in asynchronous mode, the efficiency of data transmission increases because no extra data is appended in synchronous modes. At the beginning of a frame (a series of data rows), a SYNC (synchronization) character is added to the synchronous data, and at the end, the data is appended with CRC-check data (SYNC character data is stored in the WR7 register of the SCC).

In synchronous communication mode, unlike asynchronous mode where a start bit is added to the foreground of characters, it becomes difficult to determine which bit is the start of a character. For this reason, specific data patterns designated for SYNC characters are prepared to recognize the top bit of a frame in order to facilitate detection.

When utilizing Monosync mode on SCC, if starting to read data or collecting starting data is not synchronized, a seize failure occurs and gets handled by re-syncing in the same way as asynchronous. The SCC waits until identical SYNC patterns are found in the bit pattern to re-entry into the sequence of reading held data.

0:03 Bisync Mode

Bisync (Binary Synchronous Communication) is a protocol introduced by IBM, featuring a message format that connects two SYNC characters unlike Monosync. Figure 1 depicts the format of a Bisync message. The last BCC means Block Check Code, which is essentially CRC shown in the figure. SOH and STX refer to delimiter codes assigned to various data. These control codes will be summarized in list form in Figure 5.

In Bisync and Monosync alike, control codes like SYN are specifically treated as special data codes, meaning that if this data enters the text, it's indistinguishable whether it's treated as text continuation or control codes, thus creating confusion in processing. That is to say, this format prevents the transmission of specific binary data (such as audio data, image data, executable files) without additional handling procedures.

■Figure 4 Bisync Message Format

SYN SYN	SOH	Header	STX	TEXT	ETX or	BCC				ETB	

■Figure 5 Example of Transmission Control Characters

Symbol	Value	Name	Meaning
SOH	$01	Start Of Heading	Heading Start
STX	$02	Start of Text	Text Start
ETX	$03	End of Text	Text End
EOT	$04	End Of Transmission	Transmission End
ENQ	$05	ENQuiry	Inquiry (request for a response from the other station)
ACK	$06	ACKnowledge	Positive Acknowledge
DLE	$10	Data Link Escape	Transmission Control Extension
NAK	$15	Negative Acknowledge	Negative Acknowledge
SYN	$16	SYNchronous Idle	Synchro-

nization Signal | | ETB | $17 | End of Transmission Block | Transmission Block End

At IBM, the method adopted to deal with this is to insert DLE ($10) before the control data. This mode is called transparent transmission mode (the normal, non-transparent mode is called normal transmission). In this mode, for example, SYN ($16) is sent as the two-byte data DLE SYN ($1016). When you want to send the data $10, it is handled by converting it into the two-byte data DLE $10 ($1010). The DLE codes inserted in transparent mode are not included in the CRC calculation.

The SCC's Bisync mode does not support transparent mode, so when transparent mode is used the CPU must perform the insertion/removal of DLE codes and the CRC calculation.

●1.04 External Sync (External Synchronization) Mode

In External Sync mode, the recognition of the start position of data is not done through special data like Monosync or Bisync, but is controlled by the hardware (the SYNC terminal). Already...

As mentioned earlier, since the X 68000 does not use the SCC or SYNC terminal, transmission in this mode cannot be carried out.

0·05 SDLC Mode

In contrast to Monosync and Bisync, which transmit data in byte units, SDLC is a mode that considers transmission in bit units, allowing transmission of information with a minimal number of bits.

Figure 16 shows the SDLC message format. In Bisync or similar, special data such as DLE or SYN are treated as special data. In SDLC, where transmission is basically in bit units, such method cannot be taken, 7 bits of continuous data '1111110' are used as the flag data to indicate the head of a frame, and other than that, if '0' is not inserted after 5 consecutive '1's.

On the receiving side, if '0' is found after 5 consecutive '1's, the '0' is deleted. If '1' continues 6 times, it is considered as flag data thus avoiding the mistake of mistaking data for flag and reading it.

The address field indicates the number of the recipient specified by the sender. SDLC envisions not only the transmission of data between 1-to-1, but also the use of one transmission line by multiple stations in a network environment. In such an environment, it is necessary to show to which station on the transmission network line the data should be sent. In SDLC, each station has an 8-bit number (address), and frames sent by the sender show the number of the frame sent. In the value of the address, $FF is called a global address and is used to send to an unspecified partner.

The control field is data to distinguish the type of subsequent framework or frame, and the data length is fixed.

The information field contains the actual data to be transmitted. There is no specification as to what content this is, and the number of bytes is not fixed.

Following the information field is the FCS (Frame Check Sequence), which is 16-bit CRC data for checking whether the contents of the frame were transmitted correctly.

Diagram: SDLC Message Format

Flag (8 bits) | Address (8 bits) | Control (8 bits) | Information | FCS (16 bits) | Flag

1.0.6 SDLC Loop Mode

The SDLC loop mode is a slightly modified version of normal SDLC. In this mode, a single primary station (controller: primary station) controls several secondary stations (secondary stations: secondary stations). It manages the data flow within the loop. The details of the SDLC loop mode system configuration are shown in Figure 7.

In SDLC loop mode, a message is not transmitted directly from the primary station to all secondary stations at once. Instead, it is relayed from one secondary station to the next within the loop. A secondary station that receives the transmitted message from the primary station will forward the message to the next secondary station after reading it.

Message transmission within the loop relies on a special configuration. If a secondary station does not have data to send, End Of Poll (EOP) data (”11111111”) called flag characters will be sent. Similar to normal data transmission in SDLC, a series of ”1” and ”0” beyond the specified length will be automatically inserted. Therefore, there is no risk that the EOP data will be accidentally transmitted as normal data. If a secondary station receiving the EOP message has a message to send, it will change the last “0” of EOP to “1“ and transmit the message. After transmitting its own message, the station will append EOP to the end of the message. If there is no message to send, the station will pass the EOP as is.

Figure 7: SDLC loop

- Controller

```
- Secondary Station 1
- Secondary Station 2
- Secondary Station 4
- Secondary Station 3
```

1-2 Baud Rate Generator

SCC is called a baud rate generator internally. It is a standard clock generation circuit for transmission, and you can select the desired transmission speed. The baud rate generator has a 16-level register. By setting the value of this register and the ratio of the supplied clock frequency to the baud rate generator, you can adjust the frequency output to the SCC PCLK (X 68000's case is 5 MHz) by PC or PC/(2X(N+2)).

Reducing the actual transmission rate to synchronous communication can lead to data bursts, but you can take timing from that and make data-to-data sampling intervals irregular, making it necessary to consider high transmission speeds (units: bps) at 16x or 32x clock frequencies when deciding the output frequency from the baud rate generator.

For instance, without proper synchronous timing, data timing might not come correctly. Therefore, SCC uses a high clock rate for data sampling bitwise during asynchronous communication to read data from the data line almost at the midpoint of the data bit. SCC allows you to choose the ratio of the clock frequency to the transmission rate from 16, 32, or 64 when in asynchronous mode. Please refer to the diagram in Fig. 6 for the relationship during asynchronous transmission speed and baud register set value at 16 mode. With X 68000, because the clock turns 5 MHz, an extremely small deviation from the theoretical and actual transmission speeds generally appears. Because asynchronous mode synchronizes every character, a deviation of 2 percent or more won't cause any problem.

1-3 Data Encoding

SCC can choose from four types of data encoding methods. We will use Fig. 7 to illustrate the differences in the respective encoding methods on the data line.

For NRZ, '1' stays output as '1', and '0' stays output as '0', which is generally used widely.

For NRZI, the output level inverts when data is '0' and stays the same for '1', while in FM...

Figure 8: Baud Rate Generator Setting Values (Reference)

Public Transmission Speed (bps)	Baud Rate Generator Setting Value	Actual Transmission Speed (bps)	Ratio to Public
38400	2 ($0002)	39062.5	1.017
19200	6 ($0006)	19531.3	1.017
9600	14 ($000E)	9765.6	1.017
4800	31 ($001F)	4734.8	0.986
2400	63 ($003F)	2403.8	1.002
1200	128 ($0080)	1201.9	1.002
600	258 ($0102)	601.0	1.002
300	519 ($0207)	299.9	1.000
150	1040 ($0410)	150.0	1.000
75	2081 ($0821)	75.0	1.000

*Note: These values are for the 16-clock mode.

Figure 9: Symbolization Modes Supported by SCC

'1' '1' '1' '0' '0' '1' '1' '0'
NRZ
NRZI
FM1
FM0
MANCHESTER*

*Note: MANCHESTER can be used for FM and DPLL, and the receiver can interpret it as NRZ mode.

- '1' = High
- '0' = Low
- '1' = Non-inverted
- '0' = Inverted
- '1' = No change in the middle of the bit
- '0' = Change in the middle of the bit

When the data is '0', the signal is inverted in the middle of the bit boundary, and when it is '1', the signal is inverted in the middle of the output. FM0 and FM1 use this method of inverting in the middle of each bit. When receiving non-NRZ encoded data, the data extracted from the demodulated signal is sequentially passed to the DPLL circuit.

Also, as a special mode, SCC supports NRZI mode (using bits 5 and 6 of WR10), and checking characters can be received when using FM mode (using bits 5, 6, and 7 of WR14). (Transmission is prohibited).

When transmitting non-NRZ encoded data, DPLL generates a clock synchronized with the incoming signal and can separate data and clock, so there is no need to add special circuits for clock separation or for transmitting signals even if the transmission speed changes.

0:4 DPLL SCC internally has DPLL (Digital Phase Locked Loop) circuit built-in, and can separate data and clock from the NRZI or FM encoded data without adding special external circuitry. DPLL's basic clock is 16 times the transmission rate (unit: bps) when extracting NRZI or FM encoded data.

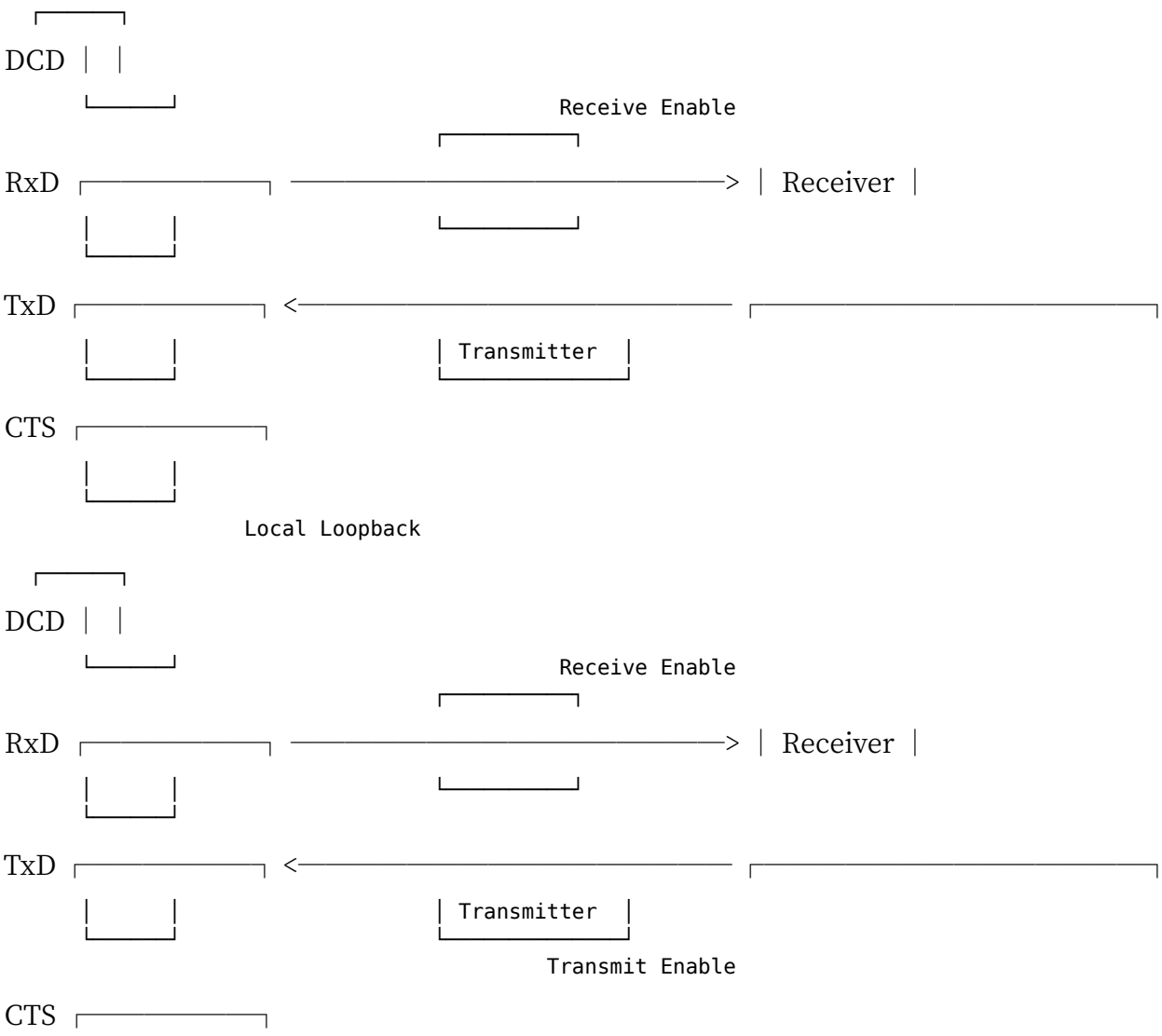
DPLL continuously adjusts its sampling clock according to the phase changes of the received signal to synchronize data transmission speed without external adjustments. Therefore, data can be extracted accurately even with slight mismatches in transmission/reception speed.

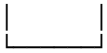
0:5 Local Loopback and Auto-Echo Functions Local loopback is a function where transmitted data is received by itself as it is, and auto-echo is a function where the received data is automatically sent back as it is. The following figure shows an overview of each function. These modes can be used in any mode, whether asynchronous, SYNC, or SDLC.

In local loopback, the transmitter is connected to the receiver internally, and even if the receiver's DCD terminal is programmed on the start mode, it will not function as the control signal for the receiver's operation.

SCC

●Figure 10 Local Loopback and Auto Echo





Auto Echo

In auto echo, the data received on RxD is output directly to TxD. Since the output of the transmitter is disconnected, the CTS terminal does not function as an operational control signal for the transmitter, even in auto mode.

●1.6 Interrupts

The SCC has four kinds of interrupt generation factors for each channel, and each factor can generate a different interrupt vector. The Human 68K uses a mode in which the interrupt level changes from 1 to 3 according to the interrupt factor, and the generated vector numbers are between $50 and $5E.

It means:

As per the document, there are four types of interrupts per channel.

Special Conditions An overrun can occur if there is a parity error in the data received by the receiver.

Receive Character Ready This interrupt triggers when valid data arrives in the receive buffer. To fetch data from the buffer upon receiving an interrupt, the buffer must be read.

Transmit Buffer Empty This interrupt signals that the transmit buffer is empty and ready for the next data. Upon receiving this interrupt, data must be written to the buffer.

E/S (External Status) Change Change in RS-232 control line functions like breaking signals happening inside the SCC mentioned as status (such as communication under layers) and hardware failures that might not correspond to other interrupts are all classified as this interrupt. This interrupt, when triggered, makes it possible to know the factor of discontentment by referring to RR.

0.7 SCC Registers

In X68000, the SCC port addresses are shown in Figure 11 and the register list is shown in Figure 12. SCC has two dedicated channels and each is assigned a separate port address. Out of these, the command port ($E98001, $E98005) is for access to internal registers of SCC, and the data port ($E98003, $E98007) is for inputting/outputting actual data.

Internally, SCC has 16 read-write registers as shown in Figure 12.

Figure 11: SCC Port Addresses

Address Description $E98001 Channel A Command Port $E98003 Channel A Data Port $E98005 Channel B Command Port $E98007 Channel B Data Port

SCC

■Figure 12 SCC Register List

Register	Register Function
WR 0	CRC initialization, SCC initialization command, selection of register to access
WR 1	Transmit/Receive interrupt settings
WR 2	Interrupt vector settings
WR 3	Receive operation parameter settings
WR 4	Settings for parameters related to transmit/receive operation
WR 5	Transmit operation parameter settings
WR 6	Synchronous Character / SDLC Address settings
WR 7	// SDLC Flag settings
WR 8	Transmit Buffer (same as $E98007 (Channel A), $E98003 (Channel B))

Register	Register Function
WR 9	Interrupt generation control to the CPU, SCC reset
WR 10	Various controls for Transmitter / Receiver
WR 11	Clock Mode Control
WR 12	┐
WR 13	┘ Baud Rate Generator (R13: upper, R12: lower)
WR 14	DPLL operation mode settings, etc.
WR 15	External / Status interrupt generation enable / disable control

| RR 0 | Transmit/Receive buffer and control terminal status | | RR 1 | Special Rx Condition status, reading of fractional code, etc. | | RR 2 | Interrupt vector (Channel A: WR2 setting; Channel B: number of last-generated interrupt vector) | | RR 3 | Reading of pending interrupt factors (exists only on the Channel A side) | | RR 8 | Receive Buffer (same as $E98007 (Channel A), $E98003 (Channel B)) | | RR 10 | Missing Clock in FM mode, operation status in SDLC, etc. | | RR 12 | ┐ | | RR 13 | ┘ Setting value for the Baud Rate Generator (value set to WR12/WR13) | | RR 15 | The value set in WR15 is read out |

There are 9 of them, but to access these registers with just one command port, the SCC uses a somewhat unusual method.

Normally the SCC is set up so that register number 0 (WR0 or RR0) can be accessed. To access a register other than this, you write the register number to WR0, and then write the value you want to set into that register. When the write is finished, WR0 or RR0 becomes accessible again.

With this method, if the number of writes gets out of sync due to a program error or the like, you may end up writing the register number to WR0 again, thinking you are writing to another register, and overwrite the wrong register. When there is such a concern, you can perform a dummy read of the SCC's command port. If you do this read after setting the register number, register number 0 becomes accessible by this read; and even if register number 0 has already been set, only the contents of RR0 are read out, so there is no effect on the operation of the SCC.

Access to register number 8 (WR8 and RR8) is the same as access to the data port. Direct access is possible, so there is no need to go to the trouble of specifying register number 8 and then reading or writing; but the SCC supports this access method as well (since WR8 and RR8 are data registers, it may be more accurate to say there are 15 write registers and 8 read registers).

Next, let's look at the contents of each register in a little more detail.

●1.7 WR0

The bit arrangement of WR0 is shown in Figure 13. The details of each bit are as follows.

■ bit 7,6 (CRC Reset Command)

Used to control the CRC checker/generator. The CRC generator is not initialized by issuing a channel reset command to the SCC, so it must always be initialized with this command.

The Transmit Underrun/EOM (End Of Message) latch command is used for transmit CRC control. When bit 6 of RR0 is '0' and a transmit underrun/EOM occurs (i.e., it is judged that all data has been sent), the SCC transmits the CRC data and sets bit 6 of RR0 to '1'. The command that clears this is the Transmit Underrun/EOM latch command.

○Figure 13 WR 0

SCC

CRC Reset Command Command Code Register Select

SCC Operation Commands Register Number to Access

111: Upper Byte IUS Reset 110: Error Reset 101: Transmit Interrupt Pending Bit Reset 100: Next Command Output Disable 011: Abort Transmit 010: External Data Transfer Enable 001: Upper Byte (RR/WR8-15) Select 000: Null Code (No Operation)

11: Transmit Underrun/EOM Latch Reset 10: Transmit CRC Checker Reset 01: Receive CRC Checker Reset 00: Null Code (No Operation)

2 Bits 5, 4, 3 (Command Code)

The meanings of bits 5, 4, and 3 in WR 0 as command codes to the SCC are as follows:

111 (Upper Byte IUS Reset) Clears the highest priority among the service requests currently being processed, enabling subsequent service request processing. This command must be issued at the end of processing the interrupt factor; otherwise, the next interrupt cannot be processed.

110 (Error Reset) Clears the special Rx condition interrupt cause (requires the upper byte of RR1 to be read). When a special Rx condition interrupt cause is generated and this command is issued, the data in the buffer for the interrupt cause will be erased. In other words, after this command is issued following a special Rx condition interrupt cause, buffer data may be necessary reading.

101 (Transmit Interrupt Pending Bit Reset) Issued when there is a send interrupt (send buffer empty interrupt), and there is no further data to send. This command stops further transmit interrupts but triggers the upper byte IUS reset command. If further interrupts occur again, the command is issued accordingly, leading to next interrupts.

100 (Enable Next Receive Interrupt)

When bits 3 and 4 of WR1 are set to '01' ('interrupt on first character'), if you issue this command during the first receive-processing routine, an interrupt will be generated when the next data is received as well. Even when data has already been placed in the receive buffer, issuing this command will generate a receive interrupt.

011 (Send Abort)

In SDLC mode, this is used to send an Abort (8 to 13 consecutive '1's). When this command is issued, the transmit buffer is automatically emptied, and bit 6 of RR0 (Tx Underrun/EOM) is set to '1'.

010 (Reset External/Status Change Interrupt)

When an E/S (External/Status change) interrupt occurs, issue this command within the interrupt-processing routine. The cause of an E/S interrupt can be determined by checking that the relevant bit of RR0 has become '1'. Issuing this command clears that status to '0'.

001 (Select Upper Register)

The lower 3 bits of WR0 are used to select the SCC's internal registers; to select a register numbered 8 or higher, set bits 5, 4, 3 to '001'. Bits 7 and 6 are normally '00' (null code), so if you simply write a register number to WR0, when that register number is 8 or higher the Command Code bits naturally become '001' (i.e. this command).

000 (Null Code)

This does not affect the operation of the SCC. When selecting a register numbered 0 to 7, these bits naturally become '000'.

3 Bits 2, 1, 0 (Register Select)

These bits specify which of the SCC's internal registers the next read/write operation will access. Values '000' to '111' indicate register numbers 0 to 7. For registers numbered 8 or higher, set the Command Code to '001', and these bits then select registers 8 to 15.

●1.72 WR1

The settings for the WR1 register are shown in Figure 14. WR1 can control interrupt enable/disable, data transfer modes, etc.

●Figure 14 WR1

bit	7	6	5	4	3	2	1	0
Wait/DREQ	Enable	Wait/DREQ	Function	Wait/DREQ on	Rx/Tx	Rx INT	Mode	Parity is Spec. Condition

```
1: External status interrupt enabled
0: External status interrupt disabled

1: Transmit interrupt enabled
0: Transmit interrupt disabled

1: Make a parity error a special Rx condition interrupt
0: Do not
```

Receive interrupt mode

```
11: Interrupt occurs only on a special Rx condition
10: Interrupt occurs on all received characters (special Rx condition interrupt also
↪  enabled)
01: Interrupt occurs only on the first character (special Rx condition interrupt also
↪  enabled)
00: Receive interrupt disabled
```

1: Use the W/REQ signal for receive operation 0: Transmit

1: W/REQ terminal operates as a DMA request signal 0: Operates as a Wait signal

1: W/REQ function enabled 0: Disabled

■1 bit 7, 6, 5 (W/REQ Signal Control)

These bits control the operation of the W/REQ signal held by the SCC. The W/REQ signal indicates that the SCC is ready for data transfer, and as such, can be used as a wait signal that waits until DMA or CPU is ready for access, or as a transfer request signal to the DMAC. On the X68000, this signal line is not connected.

bit 7 selects whether to use the W/REQ signal function. When set to 1, the W/REQ signal becomes valid. The X68000 does not use the W/REQ signal, so this bit is set to 0.

bit 6 selects whether to make the W/REQ terminal function as a wait signal or as a DMA transfer request signal. When set to 1, it operates as a DMA request; when set to 0, it operates as a Wait signal.

bit 5 selects whether the W/REQ signal is used for receive operation or for transmit operation. When set to 1, it operates for receive operation; when set to 0, it operates for transmit operation.

2. Bits 4, 3 (Receive Interrupt Request Mode)

This specifies the conditions under which a receive interrupt request occurs.

11 (Special Rx Condition Time Entry Slot)
This mode generates a receive interrupt request when a special receive condition occurs. In this mode, data remaining in the receive buffer at the time of occurrence is transferred as it is to WR0 with an error set command. Although this mode is convenient when using DMA transfer, since the X68000 and SCC are not connected to the DMA, it is not likely to be used.

10 (End of Receive Character Time Entry Slot)
Normally, the X68000 will use this mode. An interrupt request is generated when data is requested by the CPU upon receipt. When a special Rx condition expands, a special Rx condition interrupt request is generated. The special Rx condition interrupt bit is located in the RR1.

01 (First Character Time Entry Slot)
In this mode, once the receive action has begun, an interrupt is generated with the first received character. When a special Rx condition expands, a special Rx condition interrupt occurs.

00 (Receive Interrupt Inhibited)
In this mode, the occurrence of receive interrupts is suppressed. Interrupt requests to the CPU are inhibited, but it operates in synchronization with interrupt request occurrences and status bit changes, and the CPU can perform receive actions while checking RR0 and RR2.

3. bit 2 (Parity Error to Special Rx Condition Interrupt Mode)

You can select whether or not to generate a special Rx condition interrupt on parity errors. When '1' is set, parity errors cause a special Rx condition interrupt.

SCC

■4 bit 1 (Transmit Interrupt Control)

Controls whether to generate a transmit interrupt. When this bit is '1', the transmit interrupt is enabled. When it is '0', it is disabled.

■5 bit 0 (E/S Interrupt Control)

The E/S (external/status change) interrupt is generated when any of the conditions other than bit 2 and bit 0 of RR0 is met. This bit controls the enable/disable of this E/S interrupt; when '1', E/S interrupt generation is enabled, and when '0', it is disabled.

●1.73 WR2

The bit layout of WR2 is shown in Figure 15. This register sets the interrupt vector that the SCC generates. When bit 0 of WR9 is '1', the SCC generates four interrupt vectors per channel, eight in total, according to the interrupt cause. At this time, bit 4 of WR9 selects whether bits 4 to 6 are varied or bits 1 to 3 are varied.

In Human 68K, WR2 is set to $50 and bits 4 and 0 of WR9 are set to '0' and '1' respectively, using the mode in which vector bits 1 to 3 vary according to the interrupt cause. As a result, the SCC generates one of the eight values $50, $52, $54, $56, $58, $5A, $5C, $5E depending on the interrupt cause.

●Figure 15 WR2

bit 7	6	5	4	3	2	1	bit 0
Vector Number							

Interrupt vector

1-74 WR3

WR3 bit configuration is shown in Figure 16. This register controls operations such as the bit length of the received character. The meaning of each bit is as follows:

1 bit 7, 6 (Length of received character)

Specifies the bit length per character when receiving in asynchronous mode. In SYNC modes (Monosync, Bisync) or SDLC mode, it is also specified in units of 8 bits. If the received character length is less than 8 bits, the upper bits are filled with 1s.

Figure...16 WR3

bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit
Rx bit/char.	Auto Enable	Enter Hunt Mode	RX CRC Enable	Address Search Mode	SYNC Char. Load INH	Rx Ena

1: Receive Operation Enabled 0: // Prohibited

SDLC Mode: 1: During WR6 Write 0: Does not detect characters differing from WR6 0: Normal Operation

SDLC Mode: 1: Compares only the higher 4 bits of the address 0: Normal Operation

SDLC Mode: 1: Does not recognize addresses differing from the written WR6 address. *Does not recognize messages with addresses 0: Normal Operation 1: Performs CRC calculation of received characters 0: //

1: Characters matching sync characters/flags are entered in synchronous mode 0: Synchronization established

1: Auto mode (Reception at ”DCD” and transmission at ”CTS”) 0: Normal mode

Specifies the bit length of received characters: 11: 8 bits/character 10: 6 // 01: 7 // 00: 5 //

bit 5 (Auto-Enable)

When this bit is set to '1', the operation of the transmitter and receiver is controlled by the CTS and DCD signals. In auto mode, if the CTS terminal goes 'Low', transmission will be enabled, and if the DCD terminal goes 'Low', Reception will be enabled. In other words, the CTS signal functions as the transmit enable signal and the DCD signal serves as the receive enable signal. As mentioned, if programmed for auto mode, operations such as the local loopback, remote loopback, and auto echo mode will be deactivated if CTS is used as the control signal.

bit 4 (Enter Hunt Mode)

When this bit is set to '1', the SCC enters a mode that realigns the data and timing of the received data. SCC will wait for synchronization and character recognition data to match with the character length values written in WR 6. If they match, the RR0 bit 4 is set (ISS flag is issued). Hunt mode can also be entered automatically by SCC upon receiving a non-synchronized data character or an abort.

bit 3 (Receive CRC Check Enable)

When this bit is set to '1', the received characters are treated as data for CRC calculation. Received characters are included in CRC calculations. In asynchronous mode, this bit setting is ignored.

bit 2 (Address Search Mode)

This bit is effective only in SDLC mode. When set to '1', SCC will compare the address field in the received SDLC message against the value set in WR 6, and discard mismatched messages. If bit 1 of WR 3 (SYNC Character Load Inhibit) is also set to '1', SCC will compare only the high-order bit of the address field.

bit 1 (SYNC Character Load Inhibit)

In modes other than SDLC mode, SCC will compare the received data with the value set in WR 6 when this bit is set to '1', and discard the data if the comparison fails. Broken data cannot be transmitted.

The CRC calculation for T is not included.

Even in Monosync mode, where the synchronous character length is 6 bits, SCC handles characters in 8-bit units, so caution is necessary. Also, in Bisync mode where the synchronous character length is 12 bits, the settings for this bit are invalid. In SDLC mode, when an address recognition mode is selected, if this bit is set to '1', comparison of the Address Field and WR 6 setting values is done only with the upper 4 bits, and all messages to the station whose upper 4 bits match will be received.

7 bit 0 (Receive Interrupt Enable)

'1' allows receiving operations, and '0' disables them. If a channel reset or hardware reset occurs, SCC will automatically set this bit to '0'.

1•0•5 WR 4

The bit layout is shown in Fig. I7. These registers are for setting each type of parameter for transmitters and receivers.

1 Bits 7, 6 (Clock/Data Transfer Rate Comparison)

Determines the clock and data transfer rate comparison. For example, if X16 mode is selected, data transfer speed is given by clock /16. In asynchronous mode, modes other than X1 should be used.

2 Bits 5, 4 (Sync Mode)

Specifies the method of synchronizing received data. If bits 3 and 2 are '00', i.e., when asynchronous mode is selected, these bit settings are invalid.

11 (External Synchronization Mode)

Mode that synchronizes with the input of the SYNC terminal on the SCC. This mode cannot be used on X 68000, as it does not have a SYNC terminal.

⦿×...17 WR 4 bit 7 6 5 4 3 2 1 0

```
      Clock Mode    SYNC Mode     STOP Bit         Parity   Parity
                          Even/Odd   Enable
                                                                  1:   Parity ON
                                                                  0:   OFF
```

1: Even 0: Odd

11: Stop bits 2 bits 10: // 1.5 // 01: // 1 // 00: // None (sync mode)

11: External sync mode (Not usable in X68000) 10: SDLC mode ('01111110' is a flag) 01: 16-bit sync character (BIsync) 00: 8 (Monosync)

11: ×64 Clock mode 10: ×32 // 01: ×16 // 00: ×1 //

10 (SDLC Mode) Operates in SDLC mode. In this case, set the flag data ('01111110') at WR7, set the address at WR6, and select CRC-CCITT at WR5.

01 (Bisync Mode) Set the sync character continuously at WR6 and WR7. If the sync character is 12 bits or 16 bits, specify it at bit 0 of WR10.

00 (Monosync Mode) Set the sync character at WR7. SCC synchronizes by matching the same character. Choose a sync character length of 6 bits or 8 bits at bit 0 of WR10.

3 Bits 3, 2 (Stop bit length) Specify the stop bit length for asynchronous mode. Set these bits to '00' when using synchronous mode.

4 bit 1, 0 (Parity Selection)

Parity bit selection in asynchronous mode is performed. bit 0 selects whether there is parity or not, and bit 1 selects odd or even parity. If the 8-bit character length is selected with no parity, caution is needed because the parity bit will also be stored in the receive buffer.

1.7.6 WR5

WR5 sets the transmission parameters and controls transmission. The bit arrangement is as shown in the figure below (Figure 18).

Figure 18 WR 5

Bit	7	6	5	4	3	2	1	bit 0
DTR	Tx Bit/char.	Send Break	Tx Enable	SDLC/CRC-16	RTS	Tx CRC Enable		

Description of Each Bit:

1: Force 'L' at DTR output terminal

Character length specification for transmitted character:

- 11: 8 bits / character

- 10: 6 bits / character
- 01: 7 bits / character
- 00: 5 bits / character

1: Enable transmit function 0: Prohibit

1: Perform send character CRC computation 0: Do not perform send character CRC computation

- Use '0' if not using SDLC

1: Select to use CRC-CCITT polynomial 0: Select to use CRC-16 polynomial

- SDLC enabled: Use '0'

1: RTS terminal to 'L' 0: 'H'

1: Send break signal (keep TxD '0') 0: Normal operation

1. bit 7 (DTR Control) SCC operates the state of the DTR signal. When this bit is '1', the SCC's DTR output terminal becomes 'Low' level (ready state), and when it is '0', it becomes 'High' level.
2. bit 6, 5 (Transmission Character Bit Length) Specify the bit length of the transmission character. Data will be transmitted in order from the lower bits.
3. bit 4 (Break Transmission)

```
When this bit is '1', TxD becomes '0'. This feature works in conjunction with the
↪  transmitter enable.
When Monosync or Loop mode is selected and synchronization is confirmed by the receiver,
↪  this bit becomes '0' and the transmitter starts sending synchronous characters or
↪  data. This bit automatically becomes '0' when the SCC channel is reset or hardware
↪  reset.
```

4. bit 3 (Transmission Enable)

```
When this bit is '0', transmission operation stops, and the TxD terminal becomes '1'.
↪  When this bit becomes '0' during the transmission of CRC characters, CRC transmission
↪  continues instead of sending synchronous characters or flags.
This bit becomes '0' when SCC channel reset or hardware reset is performed.
```

5. bit 2 (CRC Polynomial Selection)

```
Selects the CRC polynomial to be used for transmission and reception. When '1', CRC-16
↪  polynomial is used, and when '0', CRC-CCITT polynomial is used. In SDLC mode,
↪  CRC-CCITT polynomial is selected.
The CRC generator and checker can be reset entirely to '1' or '0' by bit 7 of WR10.
```

6 bit 1 (RTS control)

Operates the state of the SCC's RTS signal. When this bit is set to '1', the SCC's RTS output terminal is at 'Low' level (ready state). When set to '0', it is at 'High' level.

7 bit 0 (Enable transmission character CRC)

Specifies whether the CRC calculation for the transmission character is to be performed. When set to '1', CRC calculation for the transmission character is performed, and when a transmission underrun occurs, the CRC data is sent.

1-7 WR6 / WR7

Bit allocation is shown in Fig. 19. In Monosync and Bisync modes, WR6 and WR7 set the synchronous character. In Bisync mode, WR6 sets the lower byte and WR7 sets the

upper byte. In SDLC mode, WR6 sets the station address and WR7 sets the flag character ('01111110').

1-7-8 WR9

WR9 performs interrupt control. Bit allocation is as shown in Figure 20 on page 334. WR9 connects internally so that only one of them is accessed from any channel.

1 Bits 7, 6 (Reset command)

Resets each channel of the SCC. bit 7 corresponds to channel A, and bit 6 corresponds to channel B. When either is set, the corresponding channel is reset. When set to '1', bits 0 and 1 of WR0, and bits 2, 3, and 4 of WR9 will not change, and the hardware reset operation is similar.

SCC

■ WR 6/WR 7 WR6

```
bit         7        6        5        4        3        2        1        0
                                SYNC Char./Address
```

Mode WR6 Value bit 7 6 5 4 3 2 1 0 Monosync 8bits SYNC7 SYNC6 SYNC5 SYNC4 SYNC3 SYNC2 SYNC1 SYNC0 Monosync 6bits SYNC1 SYNC1 SYNC5 SYNC4 SYNC3 SYNC2 SYNC1 SYNC0 Bisync 16bits SYNC7 SYNC6 SYNC5 SYNC4 SYNC3 SYNC2 SYNC1 SYNC0 Bisync 12bits SYNC3 SYNC2 SYNC1 SYNC0 ‘1’ ‘1’ ‘1’ ‘1’
SDLC ADR7 ADR6 ADR5 ADR4 ADR3 ADR2 ADR1 ADR0 SDLC(Address0) ADR7 ADR6 ADR5 ADR4

WR7

```
bit        7        6        5        4        3        2        1        0
                                SYNC Char.
```

Mode WR7 Value bit 7 6 5 4 3 2 1 0 Monosync 8bits SYNC7 SYNC6 SYNC5 SYNC4 SYNC3 SYNC2 SYNC1 SYNC0 Monosync 6bits SYNC1 SYNC1 SYNC5 SYNC4 SYNC3 SYNC2 SYNC1 SYNC0 Bisync 16bits SYNC15 SYNC14 SYNC13 SYNC12 SYNC11 SYNC10 SYNC9 SYNC8 Bisync 12bits SYNC11 SYNC10 SYNC9 SYNC8 ‘1’ ‘1’ ‘1’ ‘1’ SDLC ‘0’ ‘1’ ‘1’ ‘1’ ‘1’ ‘1’ ‘1’ ‘0’

2 bit 4 (Vector Change Mode Selection) SCC has a function to change the vector number generated by the interrupt cause depending on the interrupt cause. At this time, this bit selects whether to change bits 1-6 of the vector or to change bits 3-1. For examples of interrupt causes and vector numbers, refer to Figure 2.

![Figure]...20 WR9

bit 7 | 6 | 5 | 4 | 3 | 2 | 1 | bit 0 Reset Command | '0' | Status High Status Low | MIE | DLC | NV | VIS

1: Vectors change according to the interrupt cause 0: The vector does not change

1: Prohibit lower position chain 0: Allow

1: Allow SCC interrupt generation 0: Prohibit

1: Change the vector bits 4-6 according to the interrupt cause 0: bit 3-1

11: Forced hardware priority 10: Channel reset A 01: Channel reset B 00: No reset

3 bit 3 (Permission/Prohibition of Interrupt Generation)

'0' means that interruptions from the SCC to the CPU are prohibited, so no interrupt requests will occur. This bit will be reset to '0' by hardware.

4 bit 2 (Prohibition of Lower Position Chains)

This functionality is effective when connecting a device like Z8000 Family LSIs in a daisy chain configuration, allowing a single interrupt signal to be handled by multiple LSIs. Since the X68000 uses an SCC independently, this bit is usually not used. After reset, this bit goes to '0'.

5 bit 1 (No Vector)

If this bit is set to '1', the SCC will not output interrupt vectors. When using the SCC in connection with an interrupt controller that issues unique interrupt vectors to the interrupt controller, SCC will be in this mode, and the vectors output by the interrupt controller, as well as the vectors output by SCC...

Do not let any vectors conflict.

6 bit 0 (Vector Include Status)

Select whether to allow vector changes due to interrupt factors. Setting this bit to '1' allows changes in vector numbers due to interrupt factors, and setting it to '0' outputs the vector number written in WR2 regardless of interrupt factors.

7 WR 10

WR 10 is a control register for receive operation. The bit layout of WR 10 is shown in Fig. 21 on page 336.

1 bit 7 (CRC Preset)

Specifies whether to initialize the CRC checker/generator. Setting this bit to '1' initializes everything, and setting it to '0' does not.

2 Bits 6, 5 (Data Encoding)

Select the encoding method for the data to be output by the SCC. Generally, the '1' represents Mark and '0' represents Space, and the NRZ bit is a mode where 'High' corresponds to 'Mark' and 'Low' corresponds to 'Space'. Other modes like NRZI or FM can also be selected to separate clock and data using the internal DPLL of the SCC. When using the DPLL, set bits 7, 6, and 5 of WR 14. However, encoding can still be specified even if not using the DPLL.

3 bit 4 (Active in Loop Mode)

A bit used when operating in SDLC Loop Mode primarily for internal loopback tests or when EOP is transmitted, Transmit Inverts, or it asserts the On Loop bit returning to normal operation. When the flag is being sent, this bit is '1', and when the SCC sends flag or data characters, it sends a contiguous stream of '1's. When SDLC loop optimization transmits or receives any non-flag character, normal procedure for a 1-bit delay mode for (RxD), followed by transmission or not at all. Bit sequences in TxD are returned.

Except when SDLC loop acknowledgment transactions are to be sent, normally this bit is '0'.

WR 10

bit	7	6	5	4	3	2
Function	CRC Preset 1/0	Data Encoding	Go Active on Poll	Mark/Flag Idle	Abort/Flag on Underrun	Loop Mode

Monosync Mode: 1: Simultaneous character 6bit
0: 8bit

Bisync Mode: 1: Receiving synchronous character 12bit
0: 16bit

SDLC Mode, Simultaneous Mode: 1: Loop mode (internally connects TxD and RxD) 0: Normal operation

SDLC Mode: 1: Sends abort and flag on underrun
0: Sends CRC

SDLC Mode: 1: Sets TxD to '1' (mark state) during idle
0: Sends flag (flag idle)

SDLC Loop Mode: 1: Sends flag data during flag transmission.
0: After flag transmission ends, switches to 1-bit delay state.

TxD conversion mode 11: FM (normal '0')
10: FM (/'1')
01: NRZI
00: NRZ

1: CRC generator, CRC check initial value is All '1'
0: All '0'

You must set this bit to '1' before going active.

4 bit 3 (Mark/Flag Idle) This mode is only effective in SDLC mode, controlling the state of TxD during idle. Setting this bit to '0' means SCC sends a flag during idle. Setting it to '1' means once the flag's transmission ends, TxD remains '1' during idle.

5 bit 2 (Abort/Flag on Underrun) This bit is only effective in SDLC mode. It defines SCC's behavior when transmission underrun occurs.

Page 336

SCC

It selects between '0' and '1'. When it is '0', it sends CRC data when a transmission underrun occurs. When it is '1', it sends an abort flag. At the same time, bit 6 of RR0 (transmission underrun/EOM) goes to '1', and the external/status (E/OS) loopback stops. Also, once CRC transmission is complete, TxD is fixed to '1' and the loopback software interrupt routine is invoked. Usually, in SDLC mode, after writing to the front of the data, it writes '1' to the last byte and then sets this bit to '0'. In SDLC loop mode, this bit is ignored.

6 bit 4 (Loop Mode) SCC enters loop mode. To enable the transmitter and receiver, this bit is set last.

In SDLC mode, after bit 4 of WR10 is set to '1', SCC enters loop mode once EOP is received. Thereafter, the bit is reset to '0', and SCC exits loop mode upon the next EOP. In synchronous modes other than SDLC, this bit is used to keep the receiver and transmitter in sync. The receiver receives synchronous characters, and the transmitter is enabled at the boundary of these characters (it is released from TxD break state). This bit is ignored in non-synchronous modes.

7 bit 0 (Sync Character Length) In Monosync and Bisync modes, this bit is used to set the synchronization character length to 6 bits or 12 bits, instead of the usual 8 bits in Monosync or 16 bits in Bisync. This bit's setting is ignored in SDLC mode and non-synchronous modes.

10 WR11 WR11 selects the functions of the transmission timing and reception timing clocks and the SYNC terminal. The layout is shown in figure 22 on page 338.

●Figure 22 WR11

bit 7 6 5 4 3 2 1 bit 0 RTxC XTAL / No XTAL | Receive Clock | Transmit Clock | TRxC Out/In | TRxC Output

Signal source output from the TRxC terminal 11: DPLL output 10: Baud rate generator output 01: Transmit clock 00: Crystal oscillator circuit output

1: TRxC terminal is an output terminal 0: ” input terminal

Transmit clock selection 11: DPLL output 10: Baud rate generator output 01: Same as TRxC terminal 00: Same as RTxC terminal

Receive clock selection 11: DPLL output 10: Baud rate generator output 01: Same as TRxC terminal 00: Same as RTxC terminal

1: Configure a crystal oscillator circuit using the RTxC and SYNC terminals 0: RTxC becomes a clock input terminal (use this setting on the X68000)

■1 bit 7 (RTxC Crystal Present/Absent)

When a crystal resonator is connected between the RTxC terminal and the SYNC terminal, the SCC forms an oscillator circuit and can perform oscillation by itself. When this bit is '1', the SCC performs oscillation using the crystal resonator connected to the SYNC terminal; when set to '0', RTxC becomes a clock input terminal for an external clock. On the X68000, both channel A and channel B are used with '0'.

■2 bit 6, 5 (Receive Clock Selection)

Selects the clock source for receive operation. In the normal asynchronous mode, '10' is used, that is, the output of the baud rate generator. After a hardware reset, the receive clock is in the mode where it is supplied from RTxC.

3 bit 4, 3 (Transmission Clock Selection)

Select the transmission clock. In normal asynchronous mode, ”Z” (i.e., the output of the baud rate generator) is used. After a hardware reset, the mode becomes where the transmission clock is supplied from TRxC.

4 bit 2 (TRxC Input/Output)

Select whether to use the TRxC terminal of the SCC as a clock input terminal or as a clock output terminal. If the terminal is selected for TRxC in the asynchronous or transmission clock mode, the TRxC terminal will be forcibly used as an input terminal irrespective of the setting of this bit. In the X68000, channels A and B can commonly use the TRxC terminals for either input or output. When using it as an output on either side, set the DTR terminal to '1'(Low level). When using the RS-232 connector of channel B's DTR terminal as a control, the DTR terminal of channel A of the TRxC terminal should be set to ‘0’ (High level), otherwise the input and SCC outputs of channel B will collide, so please be careful.

5 bit 1, 0 (TRxC Output Source)

When the TRxC terminal is used as an output terminal, select the clock source output from this terminal. When selecting DPLL output, the DPLL output generating clock used by the receiver is output from the TRxC terminal.

0 • 0 11 WR12/WR13

These are registers for controlling the output frequency of the baud rate generator. It is configured as shown in Figure 23, with WR 12 as the lower 8-bit and WR 13 as the upper

8-bit operating as a 16-bit register. When writing to these registers, be sure to stop the operation of the baud rate generator.

Box 23 WR12, WR13

WR12	WR13
bit 7 6 5 4 3 2 1 bit 0	
TC (lower)	
TC (upper)	

Baud rate generator output frequency = Input clock frequency to the baud rate generator / 2 × (TC + 2)

1•7•12 WR14

WR14 is used for the control of the baud rate generator and DPLL. The bit arrangement is as shown in Fig 24.

1. Bits 7, 6, 5 (DPLL Command)

Selects the operation mode of the DPLL.

111 (NRZI Mode Select) DPLL operates for NRZI signal decoding use. After reset, the DPLL becomes this mode.

110 (FM Mode) DPLL operates for FM signal and Manchester signal decoding use, generating a clock synchronized with the input FM signal.

101 (DPLL Clock Source = RTxC) DPLL uses input from the RTxC terminal as the clock source.

100 (DPLL Clock Source = BRG) DPLL uses the output from baud rate generator as the clock source. When DPLL is operating in NRZI mode, the clock of the baud rate generator needs to be 32 times the transmission rate; when operating in FM mode, a clock of 16 times the baud rate generator needs to be input to the baud rate generator.

011 (DPLL Disable) Stops the operation of the DPLL. The clock like bit (bit 7, 6 of RR10) is cleared,

Page 340

![Diagram] WR14

bit 7	6	5	4	3	2	1	bit 0
DPLL Command	Local Loopback	Auto Echo	DTR/REQ Function	BRG Source	BRG Enable		

- 1: Operate Baud Rate Generator
- 0: Stop

Baud rate generator's clock selection
1: SCC uses PCLK (5MHz for K68000)
0: Same as RTxC

1: Send back communication type/Operate at the same time; transmission side sets DTR to 'L'
0: DTR bit is set to 'L' in WRS when DTR is stopped

- 1: TxD is externally disconnected, internally connected with TxD
  0: Operate normally

- 1: RxD is externally disconnected, internally connected with TxD
  0: Operate normally

DPLL Operation Mode Selection

- 111: Set NRZI Mode (Operate in DPLL NRZI mode)
- 110: Set FM Mode (Operate in FM mode)
- 101: Set Source = RTxC (Clock source for DPLL is RTxC)
- 100: Set Source = BRG (Output from Baud Rate Generator)
- 011: Disable DPLL (DPLL not operate)
- 010: Reset Missing Clock (Clear Missing Clock in RR10 bit 6, 7)
- 001: Enter Search Mode (DPLL enters search mode)
- 000: Null Command (No effect on DPLL operation)

- NRZI = Non-Return to Zero Inverted: 32 data bits for DPLL input, 16 bits for FM mode
- Search mode
- 010 (Clock Fault Reset) Clear the clock fault, and the state of the next clock fault will be detected (clock fault bits are only available in FM mode).
- 001 (Enter Search Mode) After receiving this command, DPLL enters search mode and synchronizes with input data. In FM mode, if no edge of the input signal is detected within a given period, the clock fault becomes '1 time clock fault' and the RR10 7-bit becomes '1'. Furthermore, if no edge of the input signal is detected consecutively two times, it becomes '2 times clock fault', and the RR10 6-bit becomes '1', DPLL enters search mode again.
- 000 (Null Command) Does not affect DPLL operation.

bit 4 (Local Loopback)

Setting it to '1' puts SCC into local loopback mode, allowing the transmitter output to be fed back directly to the receiver input. The R×D pin is not used in this mode. After a reset, this bit is '0'.

bit 3 (Auto Echo)

Setting it to '1' puts SCC in auto-echo mode, where the input to R×D is directly echoed back to the T×D pin. Transmit output is disabled. After a reset, this bit is '0'.

bit 2 (DTR/REQ Function Selector)

This bit decides whether the DTR/REQ pins on the SCC are used as software-controllable DTR signals or as data transfer request signals. When set to '1', these pins act as DTR signals. When set to '0', WR5 bit 5 can determine the function. In this latter mode, the pins go 'Low' when transmission requests are pending, simultaneous mode is used, or CRC data is being sent.

In the X68000, the DTR signal is output to the RS-232C connector, so normally this bit is not used and remains at '0'.

bit 1 (BRG Clock Source)

This bit selects whether the baud rate generator's clock signal is used as input to the RT×C pin or whether the SCC utilizes an external base clock (input via the PCLK pin). Normally, for synchronous mode, an external clock is required, so setting this bit to '1' will typically be used.

For the X68000, the PCLK pin receives a 5 MHz clock signal.

bit 0 (BRG Enable)

This bit enables or disables the BRG (baud rate generator). Setting it to '1' enables the baud rate generator. After a reset, it will be '0'.

WR12, WR13

In the case of configuring in WR12 and WR13, set this bit to '0' to stop the operation of the baud rate generator, and after finishing the setup, return it to '1'.

1 · 7 13 WR15

The bit arrangement of WR15 is shown in Figure 25. WR15 indicates the necessity of the interrupt status for E/S (SCI Status). Regarding the selection of whether to generate interrupts respectively for these, interruption is allowable if set to '1', and is prohibited if set to '0'.

Figure 25 WR15

bit 7	6	5	4	3	2	1	0
Break/Abort IE	Tx Underrun/EOM IE	CTS IE	Sync/Hunt IE	DCD IE	'0'	Zero Count IE	

bit 0:

```
1: Interrupt is generated upon zero count value of the baud rate generator
0: No interrupt generated on zero count
```

bit 1:

```
1: Interrupt is generated upon change of DCD terminal state
0: Interrupt is not generated
```

bit 2 (non-synchronous mode):

```
1: Interrupt is generated upon change of SYNC terminal state
0: Interrupt is not generated
```

bit 2 (synchronous mode/SDLC mode):

```
1: Interrupt is generated upon verification of synchronization or flag
0: Interrupt is not generated
```

bit 3:

```
1: Interrupt is generated upon change of CTS terminal state
0: Interrupt is not generated
```

bit 4:

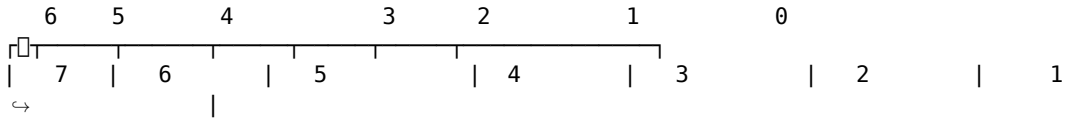
```
1: Interrupt is generated upon sending/receiving address end
0: Interrupt is not generated
```

bit 5:

```
1: Interrupt is generated upon entry into break/abort state
0: Interrupt is not generated
```

1 · 7 14 RR0

The bit arrangement of RR0 is shown in Figure 26 on page 344. RR0 indicates the state for the six data sending/receiving buffer E/S interrupt needs.



```
|      | Tx       | CTS         |SYNC|    DCD  |Tx Buffer|Zero     |Rx Character           |
|       |Underrun/|                   | /Hunt |                |Empty     |Count
↪  |Available                      |
|       | EOM      |                  |              |               |               |
↪                                   |
└──┴──────┴──────┴──────┴──────┴──────┴──────────────┘

       Whether there is a character (data) in the receive buffer:

           1: Character present
           0: Buffer empty

       1: Count value of the baud rate generator is zero

       0: Normal operation (WR15 D3=1 only: 0 is normally set)

            1: Tx buffer is empty
       or there is a character

        Shows the status of the DCD terminal:
           1: DCD terminal at L level (carrier detected)
           0: //         "H"  //         (   //      //  not detected)

        Asynchronous mode:
           1: When SYNC terminal is at L level
           0: //         "H"  //

       Synchronous Mode/SDLC mode:
           1: Sync not established (Hunt Mode)
           0: Sync established

       Shows the status of the CTS terminal:
           1: CTS terminal at L level (transmission possible state)
           0: //         "H"  // (transmission not possible state)

           1: An underrun (sent a character from the buffer that did not have a character
           ↪  on next or after next transmission), or normal operation

       SDLC Mode:
           1: Detected 7 consecutive '1's (Abort Sequence)
           0: Detected end of Abort Sequence or normal operation
```

1 bit 7 (Break / Abort / EOP)

In Asynchronous mode, detecting the break condition on RxD will set this bit to '1'. When the break condition ends, null data (S00) will be read immediately. This data should be discarded upon read.

It is necessary.

In SDLC mode, this bit detects the Abort sequence (seven or more contiguous 1's) and becomes ”1”, and becomes ”0” when the Abort sequence ends. An E/S interrupt occurs when this bit changes from 0 to 1.

2 bit 6 (Transmitter Underrun/EOM)

Becomes ”1” due to modulation, transmitter underrun disable, and abort exit commands, etc., and an E/S interrupt occurs.

This bit is reset by writing a ”Transmitter Underrun/EOM Reset” command to WR0.

3 bit 5 (CTS Line Status)

Indicates the status of the CTS (Clear To Send) terminal. In WR15, monitoring of changes

to CTS by bit 5 is enabled; when monitoring CTS in this condition, transmission continues without causing an E/S interrupt if the CTS status stays stable. If an E/S interrupt is needed when CTS changes state, then an E/S interrupt will occur when bit 5 in WR15 is set to ”1”. If monitoring by CTS is disabled by bit 5, the status of the CTS terminal is read unchanged.

4 bit 4 (SYNC/HUNT)

In asynchronous mode, indicates the status of the SYNC terminal. On the X68000, the SYNC terminal is fixed at the 'High' level and does not act as a status indicator.

In SDLC mode, after loading the Enter Hunt command, (all) receptions become invalid, and the opening flag of the first frame is detected to become ”0”. And, if bit 4 in WR15 is ”1”, an E/S interrupt occurs.

5 bit 3 (DCD Line Status)

Indicates the status of the DCD terminal. In WR15, monitoring of changes to the DCD by bit 3 is enabled; when monitoring the DCD under this condition, transmission continues without causing an E/S interrupt if the DCD status stays stable. If an E/S interrupt is needed when DCD changes states, then an E/S interrupt will occur when bit 3 in WR15 is set to ”1”. If monitoring by DCD is disabled by bit 3, the status of the DCD terminal is read unchanged.

6 bit 2 (Transmission Buffer Empty)

When the transmission buffer is empty, it becomes '1'. In synchronous mode or SDLC mode, this bit remains '0' even if CRC transmission is performed. This bit is set to '1' upon reset.

7 bit 1 (Zero Count)

When bit 15 of WR1 is set to '1' and when the count value of the Baud Rate Generator reaches 0, this bit will be set to '1' and generate an E/S interrupt. This interrupt is generally not used when the Baud Rate Generator is used as a non-synchronous mode or clock signal.

8 bit 0 (Receive Character Available)

When at least one character is present in the receive buffer, it becomes '1' and will be '0' when the receive buffer is empty. The receive buffer becomes empty upon reset.

0•1 7 15 RR1

The bit arrangement of RR1 is shown in Figure 27. The upper 4 bits of this register are assigned as special Rx condition status bits, and the lower 4 bits are allocated as terminal status bits in SDLC mode.

1 bit 7 (End of Frame)

Used only in SDLC mode. When a normal flag is received, a CRC error, or an end marker is detected, it becomes '1' and resets to '0' when an error reset command or the first frame after is received.

2 bit 6 (CRC/Framing Error)

In non-synchronous mode, when a framing error (a stop bit should be present) occurs and it becomes '0', in synchronous mode, this bit indicates the CRC check results. CRC error is shown by becoming '1'.

Page 346

■ Data···27 RR 1

```
    bit 7        6        5       4      3        2      1       0
End-of-   CRC/Framing   Rx   Overrun  Parity      Residue Residue Residue All Sent
Frame         Error     Error   Error   Error         Code 0   Code 1   Code 2

            SDLC mode, final received character
            Indicates the number of valid bits
                111: 1 bit
                000: 2 //
                100: 3 //
                010: 4 //
                110: 5 //
                001: 6 //
                011: 7 //
            Asynchronous mode
                1: Character reception is complete
                0: Character being sent

            1: Parity error detected in received data
            0: Normal operation

            1: Received buffer is overrun
            0: Normal operation

            Asynchronous mode
                1: Framing error detected (stop bit not found)
                0: Normal operation

            Synchronous/SDLC mode
                1: CRC error detected
                0: Normal operation

            SDLC mode
                1: Terminal bit matched the CRC error bit
                0: Normal operation
```

When this condition occurs, '1' is set. This bit is reset to '0' by the error reset command or successful character reception.

3 bit 5 (Overrun Error)

Reception overrun, that is, if the receive buffer is full and a new character is received, this overrun causes the character to be stored at the point where it occurred, and the value is set to '1'. If this condition persists, and the user reset command is not issued, subsequent characters will also cause special RX condition interrupts.

■4 bit 4 (Parity Error)

When parity is enabled, this becomes '1' if an error is detected by the parity check. Once an error is detected, it remains '1' until it is reset by the error reset command. If bit 2 of WR1 is set so that a parity error generates a special Rx condition interrupt, an interrupt is generated on the character that caused the parity error. If the error reset command is not issued when this interrupt occurs, a special Rx condition interrupt will be generated every time a received character comes in thereafter.

■5 bit 3, 2, 1 (Residue Code)

In SDLC mode, data can be transmitted with an arbitrary number of bits, and the SCC fetches it into the receive buffer 8 bits at a time. For this reason, the number of valid bits in the last data is somewhere between 1 and 8 bits. These bits indicate that number of valid bits, and are the residue bits.

■6 bit 0 (All Characters Sent)

In asynchronous mode, this becomes '1' when this character has all been sent. It resembles transmit buffer empty, but the point at which the transmit buffer becomes empty indicates that transmission of the previously written character has started and that space has become available in the buffer, whereas this bit indicates that everything has finished being sent, so be careful not to confuse them.

●1.716 RR2

On the channel A side, the vector number written to WR2 is set as is; on the channel B side, a vector number that has been changed according to the interrupt cause is set. As already described, there is a mode in which bits 4 to 6 change and a mode in which bits 1 to 3 change according to the interrupt cause. Figure 28 shows the correspondence between the vectors and interrupt causes in the mode where bits 1 to 3 change, which is the mode used in Human 68K.

RR 2

Channel A

```
bit 7  6  5  4  3  2  1  bit 0
--------------------------
|          Vector Number          |
--------------------------
```

Value written to WR2

Channel B

```
bit 7  6  5  4  3  2  1  bit 0
--------------------------
|  CPU-set Vector Number  |
--------------------------
               |
```

Indicates reason for interrupt

```
111: Channel A special Rx condition
110: Channel A receive character available
101: Channel A external/status change
100: Channel A Tx buffer empty
011: Channel B special Rx condition
010: Channel B receive character available
001: Channel B external/status change
000: Channel B Tx buffer empty
```

- Regarding bit 3-1 of WR9, depending on the setting of bits 4-6, it operates only when the bit value changes.

RR 3

The bit layout of this register is shown in the diagram on page 350, figure 29. This register indicates the pending (held) interrupt reason within the contents. This register is held only by Channel A, and when Channel B is read, $00 will be read.

RR 10

The bit layout is shown in the diagram on page 350, figure 30. This register accumulates status not contained in other registers.

Page 349

Figure 29 RR 3 (Valid for Channel A only)

Channel A	'0'	'0'	Channel A Rx IP	Channel A Tx IP	Channel A EXT/STAT IP	Channel B Rx IP	Channel B T
bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0

- Channel B External Status Change Interrupt
- Channel B Transmission End Interrupt
- Channel B Reception End Interrupt
- Channel A External Status Change Interrupt
- Channel A Transmission End Interrupt
- Channel A Reception End Interrupt

- When any of these bits are set, it's in the pending state.

Figure 30 RR 10

bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
One Clock Missing	Two Clocks Missing	'0'	Loop Sending	'0'	On Loop		

- In SDLC mode:

```
1: While in Loop Mode, the SCC is actually in the loop.
0: Other states
```

- In Monosync mode:

```
1: During transmission, if the transmit mixer is active.
0: Other states
```

- In SDLC mode:

```
1: While in Loop Mode, the SCC is sending on the loop.
0: Other states
```

- 1: In FM mode, during the second consecutive test, a clock edge was not found during the expected period. 0: Other states
- 1: During FM mode, RxD was '1' during the expected period but no clock edge was detected. 0: Other states

1 – Bits 7, 6 (Clock Failure Detection) In FM mode, if the DPLL does not detect a clock edge during the expected period where there should be an edge in the input waveform, ...

SCC

...if bit 7 is '1' and no edge is found even after two consecutive attempts, bit 6 becomes '1'.

■2 bit 4 (Loop Sending)

In SDLC loop mode, this becomes '1' only during the period when the transmitter is under loop control and the SCC is performing transmit operation.

■3 bit 1 (On Loop)

In SDLC loop mode, this becomes '1' during the period when the SCC is actually on the loop. When set to Monosync loop mode, it becomes '1' while the transmitter is active.

●1.719 RR12/RR13

The bit layout is shown in Figure 31. These registers read out, as is, the values set into the baud rate generator (WR12/WR13).

●Figure 31 RR12, RR13

- The values written to the baud rate generator in WR12, WR13 are read out.

●1.720 RR15

The bit layout is shown in Figure 32 on page 352. This register reads out, as is, the value written into WR15.

◆ ■32 RR 15

```
bit 7 6 5 4 3 2 1 0
┌─────┬─────┬─────┬─────┬─────┬─────┬─────┬─────┐
| Break/ | Tx Underrun | CTS | SYNC/Hunt | DCD | '0' | Zero Count | '0' |
|Abort IE| /EOM IE | IE | IE | IE |   | IE   |   |
└─────┴─────┴─────┴─────┴─────┴─────┴─────┴─────┘

1: Zero count interrupt enabled
0:              //   disabled

1: DCD change interrupt enabled
0:              //   disabled

1: SYNC/Hunt status change interrupt enabled
0:              //   disabled

1: CTS change interrupt enabled
0:              //   disabled

1: Tx underrun / EOM occurrence interrupt enabled
0:              //   disabled

1: Break/Abort status detection interrupt enabled
0:              //   disabled
```

Keyboard/Mouse

The point of direct contact with the user is the keyboard and mouse. The X68000 not only supports text input via the keyboard, but also includes functions for controlling the display and the mouse, providing comprehensive support for the user interface.

1. Overview of the Keyboard/Mouse

A block diagram of the keyboard and mouse interface is shown in Figure 1 on page 354. On the X68000, data input and output with the keyboard is performed via the MFP or USART (serial port), and data communication from the mouse is performed via the SCC on CPU Board B.

Data communication from the keyboard is transmitted at a speed of 2400 bps, data length 8 bits, start bit 1 bit, stop bit 1 bit, no parity. Mouse data communication is transmitted at a speed of 4800 bps, data length 8 bits, stop bit 1 bit, parity 2 bits with parity enabled.

The X68000 allows you to control the power ON/OFF of the dedicated display, TV channel switching, etc. This control is performed by the CPU (Work Icon No. 8: 8C51) in the keyboard, as well as by system software #2. The power for the keyboard is provided continuously even when the main power supply is OFF, and the CPU in the keyboard continues

to operate. Therefore, control such as TV channel switching is possible even when the keyboard is connected to the main unit.

Additionally, although the X68000 has both keyboard and mouse connectors, this system...

(Page 353)

●Figure 1 Keyboard/Mouse System Block Diagram

X68000

Mouse Data rate: 4800bps Data: 8 bit Stop: 2 bit Parity: None

SCC RTSB LS19 7438 Vcc1 Vcc1 RxDB LS367 MSCTRL Mouse Connector MSDATA

Mouse Connector MSCTRL Vcc2 LS374 Q D P0.7 P3.7

MFP SI LS244 KY RxD Vcc2 SO LS244 KY TxD RR 74S08 KY READY

System Port #4 ($E8E007) bit 3

System Port #4 ($E8E007) bit 3 ALS244 Vcc1 interlock Vcc1 LS11 Vcc1 KY RMT

System Port #2 ($E8E003) bit 3

To Display Control Connector

MSDATA MSDATA

Keyboard Connector

Main Body Connection Connector

TxD RxD T0 T1 80C51

Keyboard Data rate: 2400bps Data length: 8 bit Stop bit: 1 bit Parity: None

Keyboard / Mouse

The data line is electrically connected. However, the MSCTRL signal that requests data output to the mouse is controlled by the CPU on the keyboard side, and the MSCTRL signal control on the keyboard side is executed according to commands to the CPU.

2 Keyboard / Mouse Related Ports

We have summarized the I/O ports related to keyboard and mouse control in Table 2. For MFP and SCC, please refer to the description page of each device.

Table 2-2 Keyboard / Mouse Related Ports

Device	Address	Read/Write	bit 7	6	5
$E88027	R/W	SYNC		Synchronization Character Register	
$E88029	R/W	CLK0 WL1	ST1	STO	PE
$E88028	R/W	BF FE	PT	F/S	M
$E88020	R/W	BE UE	AT	END	B
$E8802F	R		USART Data Register 9		
$E80003	R/W	TV CTRL	TV CTRL/NMI		TV CTRL/RESE
$E80007	R/W		System Port #4		
$E98001	R/W		SCC Channel 3 Command Port		
$E98003	R/W		SCC Data Port		

1 System Port #2

The bit layout of System Port #2 (Address: $E80003) is shown in Figure 3 on page 356. bit 0 is related to the control of the 3D scope of the option, and bits 3 and 4 are related to the display. bit 3 acts as a control signal for the display when writing, and as a disc display when reading.

■ Figure 3: System Board #2 (See SEB 003)

bit 7 6 5 4 3 2 1 0 bit 0

```
TV CTRL             3D-L       3D-R

                    3D Scope Control / Status Readout
                    11: Close left/right
                    10: Open only right
                    01: Open only left
                    00: Open left/right
```

WRITE 1: Display control signal becomes '1'. 0: Display control signal becomes same as keyboard control signal (default).

READ 1: Display power is OFF. 0: Display power is ON.

Status of the display power ON/OFF.

The output of bit 3 is OR-ed with the display control signal from the keyboard as a gate. Usually, the display control signal from the keyboard is '0', so changing this bit to '1' will make the display control signal '1', and if it's '0', it will be '0'. This way, you can perform software-based display control regardless of the keyboard's state. If you leave this bit as '1', the display control signal will be forced to '1' and won't respond to the keyboard control signal. This will also prevent remote control signals from being OR-ed with the display control signal, making the remote control non-operational. Usually, this bit should be set to '0'.

2.2 System Board #4

System board #4 (address: $EB007), bit arrangement is shown in Figure 4. bit 3 indicates the possibility of keyboard data output based on the board. Usually, when data is sent from the keyboard, MFP reads the RR (Receiver Ready) signal as '1' (Low level), and the CPU reads the data, changing it to '0' (High level). The keyboard checks its signals. By sending a key code, the CPU won't receive unnecessary data.

● 4 System Port #4 ($E60E007)

bit 7 6 5 4 3 2 1 bit 0 KEY CTRL NMI RESET HRL

Switch Dot Clock (Usually set to '0') 0: // 1: Resets NMI 0: Does not reset

When writing 1: Key data transmission allowed 0: Not allowed

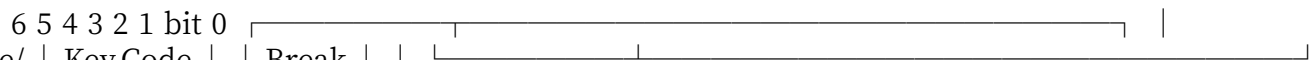
When reading 1: Key Jack (Keyboard Connector) inserted 0: Not inserted

Writing '0' to bit 3 of system port #4 forces this signal to '1' (Low level), disabling keyboard data transmission. The keyboard will send key data and later perform a key scan (checking if the key is pressed or released). In this state, display control from the keyboard will not be performed. While reading, this bit functions as a keyboard extraction status. If the keyboard is connected, it reads '1', if not connected, it reads '0'.

● 3 Input data from the keyboard

The format of the key data sent from the keyboard to the main unit is shown in figure 5 on page 358. The key data is reported when a key is pressed or released. The key code for keys that have changed at the lower 7 bits is reported by setting or clearing bit 7, indicating whether the key is pressed or released. The key layout and key code correspondence for X68000 are as shown in figure 6 on page 358.

●Figure 5 Key Data



1: Key released 0: // pressed

●Figure 6 Key Layout and Key Codes

```
61  62    63  64  65  66  67    68  69  6A  6B  6C        5A  5B  5C    5D  52  53  54

01  02  03  04  05  06  07  08  09  0A  0B  0C  0D  0E  0F      36  5E  37    3F  40  41  42
10  11  12  13  14  15  16  17  18  19  1A  1B  1C          1D                    43  44  45  46
71  1F  20  21  22  23  24  25  26  27  28  29              3B      3D    4B  4C  4D    4E
70  2A  2B  2C  2D  2E  2F  30  31  32  33  34          70      72  73    4F  50  51
        5F  55  56          35          57  58  59  60
```

Key codes are in hexadecimal.

BREAK COPY F1 F2 F3 F4 F5 F6 F7 F8 F9 F10 Kana Romaji Input Code Input CAPS Register HELP

ESC ... BS INS HOME DEL CLR / * - TAB ... ROLL UP ROLL DOWN UNDO 7 8 9 + CTRL ... 4 5 6 = SHIFT ... SHIFT 1 2 3 Hiragana XF1 XF2 XF3 XF4 Full-width 0 . , ENTER OPT1 OPT2

●4 Output Data to the Keyboard

The list of commands sent from the X68000 main unit to the keyboard is shown in Fig. 7. In the X68000, the CPU inside the keyboard generates the display control signals and controls the mouse control signal (MS CTRL), so there are many commands to support those functions.

Keyboard / Mouse

●Figure 7 Control Commands to the Keyboard

| Data | Function | | bit 7 | 6 | 5 | 4 | 3 | 2 | 1 | bit 0 | | | |------|---|---|---|---|---|---|---|------|-------------|---------------------------------| | '0' | '0' | | TV CTRL Code | | Dedicated display (display TV) control | | '0' | '0' | | MS CTRL | | | MSCTRL signal control for the mouse connector on the keyboard | | '0' | '1' | | KEY | | | Keyboard data transmission enable/disable | | '0' | '1' | | X68K/X1 | | | Display control mode selection by keyboard operation | | '0' | '1' | | BRIGHT | | | Brightness selection of the LED on the keyboard | | '1' | '1' | | CTRL EN | | | Display control from the main unit enable/disable | | '1' | '1' | | OPT2 EN | | | OPT2 key display control enable/disable | | '0' | '1' | '1' | '1' | REP. DELAY | | | Time setting from key press until repeat begins | | '0' | '1' | '1' | '1' | REP. TIME | | | Repeat interval setting | | '1' | Full-width | Hiragana | INS | CAPS | Code Input | Kana | Romaji | Kana | ON/OFF control of the LEDs on the keyboard |

It is as follows.

●1 Display Control

On the X68000, you can control the display not only by operating the keyboard, but you can also request the CPU to generate display control signals from the main unit. The list of commands for this is shown in Fig. 8 on page 360.

These commands include items that cannot be done by operating the keyboard, such as power ON/OFF and superimpose at normal contrast. In particular, superimpose at normal contrast makes the TV screen brighter than ordinary superimpose, so it is better to use this mode when superimposing.

●2 Mouse Control Signal Control

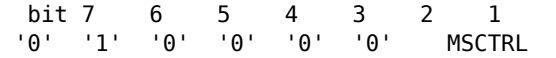
The format of the command is shown in Fig. 9 on page 360. With bit 0 you select the state of the MSCTRL signal of the mouse connector attached to the keyboard. The mouse operates from when MSCTRL goes High...

[Figure 8] Display Control Command List

Control Code	SHIFT Key + Code in Brackets	Name	Function
S00		Vol. up	(Invalid) Volume (up)
S01			// up
S02		Vol. down	// down
S03		Vol. normal	// Normal
S04	CLR	Call	Channel Call
S05	(transmission key-less)	CS down	TV screen (initialization, reset)
S06	0	Mute	High Volume Mute
S07	CH16		(Invalid)
S08	-	BR up	TV/Video Switch (toggle)
S09	-	BR down	TV/External Input Switch (toggle)
S0A	(transmission key-less)	BR ½	Contrast Normal
S0B	-	CH up	Channel Up
S0C	-	CH down	Channel Down
S0D	-		(Invalid)
S0E	(transmission key-less)	Power ON/OFF	Power ON/OFF (toggle)
S0F	CS ½		Superimpose ZON/OFF (toggle), Contrast Down
S10		CH 1	Channel 1
S11	// 2	CH 2	Channel 2
S12	// 3	CH 3	Channel 3
S13	// 4	CH 4	Channel 4
S14	// 5	CH 5	Channel 5
S15	// 6	CH 6	Channel 6
S16	// 7	CH 7	Channel 7
S17	// 8	CH 8	Channel 8
S18	// 9	CH 9	Channel 9
S19	// 10	CH 10	Channel 10
S1A	// 11*	CH 11	Channel 11
S1B	// 12	CH 12	Channel 12
S1C	// 13	CH 13	TV Screen
S1D	// 14	CH 14	Computer Screen
S1E	// 15	CH 15	Superimpose ZON/OFF (toggle), Contrast Normal
S1F	(transmission key-less)		(blank)

- For S1C~S1F, please verify correspondence during X1 Compatibility Mode.

[Figure 9] Mouse Control Signal Diagram



1: Set MSCTRL to 'High' 0: Set to 'Low'

Begin data transmission when it becomes Low.

4-3 Key Data Transmission Permission/Prohibition

If the CPU fails to retrieve key data, the system board will either allow or prohibit the keyboard from sending key data. If the keyboard is unable to send key data, it will no longer be able to control the display via the keyboard, resulting in broken functionality.

To avoid such situations, this command is used to prohibit the keyboard from sending key data and also to disable the key scan operation until the CPU can retrieve the key data properly. By using this command to halt the key data transmission, even if key data cannot be sent to the CPU, it allows the keyboard to normally control the display.

The command format is as shown in Figure 10. Setting the highest significant bit to '0' will prohibit key data transmission, and setting it to '1' will allow for normal operation mode.

● Fig. 10 Key Data Transmission Permission/Prohibition

Bit:	7	6	5	4	3	2	1	0
Key EN	x	x	x	x	x	'1'	'1'	'1'

1: Key Data Transmission Permission
0: Prohibition

4-4 Display Control Key Mode

This section describes the selection between the X1 compatiblity mode for the display control via key operations. The format of the command is as shown in Figure 11.

● Fig. 11 Display Control Key Mode

Bit:	7	6	5	4	3	2	1	0
X68K/X1	'0'	'1'	'0'	'1'	'0'	'0'	x	x

1: X68000 Mode
0: X1 Compatibility Mode

Normal mode (referred to as X68000 mode) and X1 converter mode differ as shown in figure 12. In X68000 mode, the Superimpose input switch toggles each time data is sent, but in X1 converter mode, inputs are used to select between Superimpose, TV, and Computer.

■ Figure 12 TV Control Operations SHIFT Key Pressed Simultaneously with | X68000 Mode | X1 Converter Mode |

- | Superimpose ON/OFF Toggle | Superimpose | 0 | TV/External Input Switch (Toggle) | TV |
- | TV/Computer Switch (Toggle) | Computer |

●5 LED Brightness Selection

You can select the brightness of the LED on the keyboard. The command format is as shown in Figure 13. The lower 2 bits adjust the brightness of the LED in 4 levels.

■ Figure 13 LED Brightness Selection | bit 7|6|5|4|3|2|1|bit 0| | '0' | '1' | '0' | '1' | '0' | '1' | '0' | BRIGHT | | Keyboard LED Brightness | | 11: Bright, | | 10: Somewhat Bright, | | 01: Somewhat Dim, | | 00: Dim |

4.6 Selection of Display Control Enable/Disable from the Main Unit

Select whether or not to accept the display control command requested from the keyboard. The command format follows the structure in Figure 14. It becomes invalid if the lowest bit (bit 0) is set to '0'.

■Figure 14 Selection of Display Control Enable/Disable from the Main Unit bit 7 6 5 4 3 2 1 bit 0 '0' '1' '1' '0' '1' '0' '0' CTRL EN 1: Enable display control from the main unit 0: Disable

4.7 OPT.2 Key Display Control Enable/Disable

The command format is shown in Figure 15. Normally, display control from the keyboard uses the SHIFT key, but the OPT.2 key can be used as an alternative to the SHIFT key. This command selects whether to permit or prohibit using the OPT.2 key for display control.

■Figure 15 OPT.2 Key Display Control bit 7 6 5 4 3 2 1 bit 0 '0' '1' '0' '1' '1' '1' '0' OPT.2 EN 1: Permit display control via OPT.2 key 0: Prohibit

4.8 Setting the Key Repeat Start Time

The command format is as shown in Figure 16. This sets the time until the key repeat starts from the moment the key is pressed and held down. By using the lower 4 bits, the time until the repeat starts can be set in units of 100 ms from 200 ms to 1700 ms. When the keyboard is reset, the initial setting of this time is 500 ms.

Figure 16: Setting the Key Repeat Start Time

![Diagram]

The time until the key repeat starts (computed as):

[\text{Time} = 200 + (\text{REP. DELAY} \times 100) , \text{ms}] (The initial setting is 500 ms at reset)

4.9 Setting the Key Repeat Interval

The command format is as shown in Figure 17. The interval of the key repeat can be set between 30 ms and 1155 ms. When the keyboard is reset, this interval is set to 110 ms.

Figure 17: Key Repeat Interval Command

![Diagram]

The key repeat interval (computed as):

[\text{Interval} = 30 + (\text{REP. TIME}^2 \times 5) , \text{ms}] (The initial setting is 110 ms at reset)

4.10 Keyboard LED Control

You can control the lighting on/off of the LED attached to the key on the keyboard. The command format is shown in Figure 18. Each of the lower 7 bits corresponds to a key, and when the bit is '1', the LED is turned off, and when it is '0', the LED is turned on.

■ Figure 18 Keyboard LED Control

bit 7 5 4 3 2 1 bit 0

```
'1'  KANA (Full) Hiragana INS    CAPS  Code Input Romaji  Katakana
       Each key's LED on/off control
                   1: Off
                   0: On
```

5. Display Control Signal

When the display control signal is not coming from the keyboard or if the keyboard connector is disconnected, it is possible to control the display by controlling bit 2 of the system board and outputting a display control signal. Essentially, this display control performs the job of displaying.

The format of the display control signal is shown in Figure 19 on page 366. Data transmission is done by sending the front and back signals at an interval of 48 ms. Figure 20 shows the detailed contents of the data. The principle is to send the basic altitude data to the screen side so that the data can be correctly received on both sides, and we can confirm if commands are executed correctly to prevent malfunction.

The transmission of the display control signal involves sending single-bit data as 1/0 signals, which are switched at 250 μs intervals as shown in the diagram. If there are multiple signals in series, the intervals between pulses will decide the content of the data.

These methods include those conducted by wireless remote control. For the display inside, these are wireless methods executed similarly.

Page 365

●Figure 19 Display Control Signal

Display Control Signal: '1' (Front Signal), '0' (Back Signal) Waveform of Each Bit:

Data '0': 1ms [blank space] 2ms Front Signal to Back Signal: 250μs blank Data '1': 250μs blank

●Figure 20 Data to Send to the Display

The data sent to the display (sent in order from the leftmost bit)

Signal	Data sent to the display (sent in order from the leftmost bit)
Front Signal	'0' '0' '0' '0' '0' '0' '0' '0' '0' '0' 'Display Control Read'
Back Signal	'0' '0' 'Display Control Read' '1' '1' '1' '1' '1'

The signal received from the) erasure is ORed with the signal sent from the main body as single remote data.

6 Keyboard Special Features

Although not closely related to the main usage of the device, let's introduce some additional features the keyboard has.

6.1 Specifying LED Brightness

When resetting the keyboard (unplugging the keyboard), you can choose the brightness of the LED.

Keyboard / Mouse

...obtained.

- When started up without pressing anything : Bright
- When started up while pressing XF3 : Slightly bright
- When started up while pressing XF4 : Slightly dim
- When started up while pressing XF5 : Dim

●6.2 LED Check

When you reset the keyboard while pressing the three keys F1, F2, and F3 simultaneously, the LED blinks repeatedly. In this state, input from the keyboard and so on cannot be performed at all, so when you want to use it, perform the keyboard reset again with no keys pressed.

●7 Mouse Control

When the mouse control signal (signal name: MSCTRL) changes from High to Low, the mouse sends 3 bytes of data: status, X-direction data, and Y-direction data. The mouse control timing specifications, etc., are shown in Fig. 21 on page 368.

The mouse data signal (signal name: MSDATA) is simply connected in common to the mouse connector on the main unit and the one on the keyboard, but the MSCTRL signal is controlled by the SCC on the main unit side and by the CPU inside the keyboard on the keyboard side. Therefore, in order to allow the mouse to be connected to either side, both the SCC and the keyboard must operate the MSCTRL signal.

The X-direction data and Y-direction data sent from the mouse are signed binary numbers, where $80 represents -128 and $7F represents +127. This data indicates the relative movement from the point at which the previous data was sent.

The format of the status data, which is the first byte of the mouse data, is as shown in Fig. 22 on page 368. The upper 4 bits detect underflow in the Y-direction and X-direction respectively (the amount of movement became -129 or less, so it could no longer be expressed by the movement data) or overflow (the amount of movement became +128 or more, so it could no longer be expressed by the movement data); when such is detected, the corresponding bit...

Figure 21 Mouse Data Transmission Timing

- MSCTRL

```
- High
- Low
- 500µs or more
- 700µs or more
- 500µs or more
```

- MSDATA

```
- High
- Low
- Status
- X Direction Data
- Y Direction Data
- Transmission Speed: 4800bps
- bit 0, 1, 2, 3, 4, 5, 6, 7
- Start Bit (1 bit)
- Data Bit (8 bits)
- Stop Bit (2 bits)
```

Figure 22 Mouse Status Data

- Y UNFL, Y OVFL, X UNFL, X OVFL, '0', SW-L, SW-R
- bit 7, 6, 5, 4, 3, 2, 1, 0
- Y Overflow Flag:

```
- 1: Y direction overflow (movement amount in Y direction is +128 or more)
- 0: No overflow
```

- Y Underflow Flag:

```
- 1: Y direction underflow (movement amount in Y direction is -129 or less)
- 0: No underflow
```

- X Overflow Flag:

```
- 1: X direction overflow (movement amount in X direction is +128 or more)
- 0: No overflow
```

- X Underflow Flag:

- 1: X direction underflow (movement amount in X direction is -129 or less)
- 0: No underflow

• SW-L (Left Switch)

- 1: Left switch ON
- 0: Left switch OFF

• SW-R (Right Switch)

- 1: Right switch ON
- 0: Right switch OFF

Page 368

Keyboard/Mouse

becomes '1'.

bit 1 and bit 0 indicate the state of the left and right switches of the mouse. '1' indicates that the switch is pressed, and '0' indicates that it is released.

(blank page)

Printer

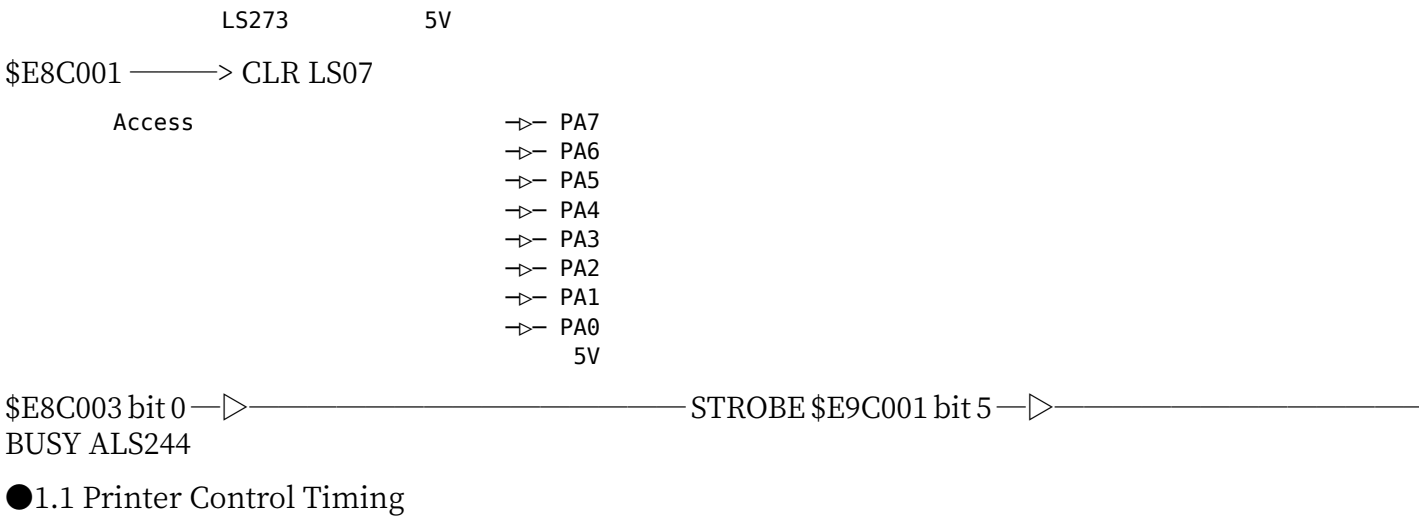
The printing interface is summarized in a small 14-pin connector and is compatible with the X1. Here, we will explain the components of the printing interface and the interface signals.

1 Overview of Printing Interface

In the X68000, the printing interface can be converted to a Centronics interface to connect to a printer. A block diagram of the X68000's printing interface is shown on page 372. PA0~PA7 are 8-bit data lines, and STROBE signals the printer to fetch the data. BUSY indicates whether the printer can receive more data. The X68000 reads the BUSY signal on the I/O controller. If the BUSY signal changes from an idle state (indicating that the next data transfer can be performed), an interrupt will be generated.

Most printers that use the Centronics interface output signals such as the status of the SELECT switch or printer errors, and read data. However, the X68000 does not use these signals. The X68000 will check the BUSY signal to see if the printer is busy (data cannot be fetched) or ready (data can be fetched). If the BUSY signal does not change within a certain period, it will be processed as a timeout error.

●Figure 1 Printer Interface Block Diagram



●1.1 Printer Control Timing

An example of printer control timing is shown in Figure 2. Assume that the STROBE signal is '1' and the BUSY signal from the printer is '1'. Note that '1' and '0' here refer to the data written to or read from the port.

First, confirm that the BUSY signal is '1' (the printer is in the ready state), then set the data you want to send to the printer on PA0~PA7. Next, when you set the STROBE signal to '0', the printer takes in the data, so BUSY goes to '0'; seeing this, return STROBE to '1'. When the data has been taken in by the printer and it is ready to take in the next data, BUSY returns to '1' (if necessary, an interrupt can be generated at this point). The X68000 side sees that BUSY has returned to '0' and then sends out the next data. The period during which the BUSY signal is '1' is determined by the printer's circumstances, and it is not known how long it will be. Also, in practice it seems common to use a method where, without checking the BUSY signal, STROBE is changed continuously between '0' and '1' and then one waits for BUSY to become '1'.

● Fig. 2 Printer Control Timing Example

High STROBE Low High BUSY Low High PA0-PA7

Set data to $E8C001

- State when read by the I/O controller port ($E9C001).

●2 Printer Related Ports

The list of ports related to the printer interface is shown in Fig. 3. The data to be sent to the printer is set to $E8C001 and controlled by the STROBE signal in $E8C003. An interrupt occurs when the BUSY signal from the printer changes from a busy state to a ready state. By reading the upper byte of $E8C001 as status, you can recognize that an interrupt has occurred.

● Fig. 3 Printer Related Ports

| Address | R/W | bit 7 | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 | Remarks | | $E8C001 | W | | | | | | | | Printer Data | | $E8C003 | W | | | | | | | | Printer Strobe | | $E9C001 | R | FDC INT | FDC INT | PRT INT | PRT INT | HDDI EN | HDDI EN | HDDEI EN | HDDEA EN | Printer Status/Interrupt Status | | $E9C003 | W | | | | | | | | Interrupt Mask | | $E9C003 | R | Vect | | | | | | DEVICE | Interrupt Vector |

Note: ”Vect” and ”DEVICE” in the table were kept as they are, possibly being abbreviations or acronyms.

2.1 Printer Data Port

Set the data to be sent to the printer. Please set the data to this port before manipulating the STROBE.

2.2 Printer Strobe Port

The bit layout of the printer strobe port is shown in Figure 4. The lowest bit controls the STROBE signal; when set to '1', the STROBE signal becomes High level, and when set to '0', it becomes Low level.

Figure 4 Printer Strobe Register ($E0C003)

bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
							STRO

1: STROBE signal becomes 'High' level 0: ” ” ” 'Low' level

2.3 Interrupt Signal Status

The bit layout of the interrupt signal status port is shown in Figure 5. The upper 4 bits show whether an interrupt request has occurred, and the lower 4 bits read the value written to the interrupt mask register as is.

Among these, bit 5 shows the interrupt request status for the printer, that is, the status of the BUSY signal. It also shows whether the printer's interrupt is masked by the interrupt mask register.

Printing interrupts occur when the BUSY signal changes from '0' to '1'. When interrupts are disabled by the mask register, bit 5 reflects the state of the BUSY signal, which allows control of the printer without using interrupts.

Printer

●Figure 5 Interrupt Signal Status ($E9C001)

bit	7	6	5	4	3	2	1	0
FDC INT	FDD INT	PRT INT	HDD INT	HDDI EN	FDCI EN	FDDI EN	PRTI EN	

bit 0 (PRTI EN): 1: Printer interrupt enabled 0: Disabled

bit 1 (FDDI EN): 1: FDD interrupt enabled 0: Disabled

bit 2 (FDCI EN): 1: FDC interrupt enabled 0: Disabled

bit 3 (HDDI EN): 1: HDD interrupt enabled 0: Disabled

bit 4 (HDD INT): 1: HDD interrupt has occurred 0: Has not occurred

bit 5 (PRT INT): 1: Printer BUSY signal = 'Low' (READY state) 0: = 'High' (BUSY state)

bit 6 (FDD INT): 1: FDD interrupt has occurred 0: Has not occurred

bit 7 (FDC INT): 1: FDC interrupt has occurred 0: Has not occurred

●2.4 Interrupt Mask

This is a register that determines, for each interrupt source managed by the I/O controller, whether or not an interrupt request is issued to the CPU. The bit layout of this register is shown in Figure 6 on page 376.

Printer interrupt control is performed by bit 0. When this bit is '1', the generation of printer interrupts is enabled, and when it is '0', it is disabled.

●2.5 Interrupt Vector Register

The bit layout of the I/O controller's interrupt vector setting register is as shown in Figure 7 on page 376. This register sets the vector number given to the CPU when an interrupt occurs. The upper 6 bits can be set arbitrarily, but the lower 2 bits change automatically

according to the interrupt source, so settings from the CPU are invalid. When an interrupt from the printer occurs, the lower 2 bits

●Fig.6 Interrupt Mask (5E9C001)

bit 7	6	5	4	3	2	1	bit 0
			HDDI EN	FDCI EN	FDDI EN	PRTI EN	

1: Printer interrupt allowed 0: // Forbidden

1: FDD interrupt allowed 0: // Forbidden

1: FDC interrupt allowed 0: // Forbidden

1: HDD interrupt allowed 0: // Forbidden

●Fig.7 Interrupt Vector (5E9C003)

bit 7	6	5	4	3	2	1	bit 0
Vect	DEVICE						

Arbitrary setting allowed

11: Printer interrupt 10: Hard disk interrupt 01: FDD interrupt 00: FDC interrupt

The vector number that became '11' is sent to the CPU.

Joystick

The standard joystick interface follows Atari's specifications. On the X68000, this interface is also used for purposes such as connecting a cyber stick and as a general-purpose digital I/O.

1. Overview of the Joystick Interface

Figure 1 on page 378 shows a block diagram of the X68000's joystick interface. The joystick interface uses an 8-bit parallel port (called ports A, B, and C in this document) with an LSI µPD8255. Among these, the C port only uses four bits (PC0 to PC3) and is used for selecting ADPCM playback control or sample clock control, and for joystick output. The X68000 has two joystick connectors, but ports A (PC4, PC5, PC6, PC7) are allocated to joystick 1, and ports B to PC5 are allocated to joystick 2.

Port A/B show the conditions such as joystick level conditions and push-button states and can be read. PC4 and PC5 are used for joystick operations and activation checking. When the joystick #1 (High level) is active, the joystick push-button states will be notified.

Joystick #1's optional functions from PC6, PC7 are available on the joystick.

● Diagram ..... 1 Joystick Interface Block Diagram

µPD8255 DTC114ES

PC7 PC6 Port A PA0 PA1 PA2 PA3 PA4 PA5 PA6 PA7

PC4

5V 15KΩ

F

B

L

R

TRG-A TRG-B

```
1  > 5V
2  > Joystick #1
3  >
4  > Joystick
5  >
6  >
7  >
```

PC5 Port B PB0 PB1 PB2 PB3 PB4 PB5 PB6 PB7

PC0 PC1 PC2 PC3

ADPCM Volume Control

ADPCM Sampling Clock Selection

Although it corresponds to, it does not require this function as a typical 4-direction + 2-trigger joystick type, so it can be used with either joystick connector.

Page 378

#2 Joystick Related Ports

Below is a list of ports related to the joystick shown in Table 2. The joystick is controlled by connecting it to a single µPD 8255, and since it does not have an interrupt function, all ports related to the joystick are assigned to the µPD 8255.

Table 1. Joystick Related Ports

Port	Address	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1
8255 Port A	$E9A001	TRG	TRG	RIGHT	LEFT	BACK	FORWARD	Joystick #1
8255 Port B	$E9A003	TRG	TRG	RIGHT	LEFT	BACK	FORWARD	Joystick #2
8255 Port C	$E9A005	OC6	OC5	OC4		PCM RATE	PCM PAN	Joystick Contr
8255 Port D	$E9A007	Work	Mode Control Bit Operation					

◆2-1 Joystick #1 / #2

Ports A and B are ports that read the status of the joystick. The bit arrangement is as shown in Figure 3 of page 380. Bits 4 and below indicate the direction of the joystick. If the joystick is tilted in that direction, the switch attached to that direction will turn ON and a '0' will be read. Further, bit 5 and bit 6 correspond respectively to the trigger button A and trigger button B. When the button is pressed, '0' will be read. Bits 4 and 7 represent the block shown in Figure 4, and since the connector is not connected, and only a resistor is pull-up, a '1' will be read.

◆2-2 Joystick Control

Port C is used to enable/disable the operation of the joystick and for band control of AD-PCM. The bit arrangement is as shown in Figure 4. Bits related to joystick operation control are bits 4, 5, 6, and 7. Writing '1' to these bits will enable operation.

■ Fig. ..... 3 Joystick #1/#2

$E9A001 | bit 7 6 5 4 3 2 1 0

```
          | TRG B  TRG A  RIGHT  LEFT  BACK  FORWARD
          | Joystick 1
```

$E9A003 | bit 7 6 5 4 3 2 1 0

```
          | TRG B  TRG A  RIGHT  LEFT  BACK  FORWARD
          | Joystick 2
```

Direction the stick is pointing 1: Switch in direction pressed 0: Switch in direction not pressed

1: Trigger A button not pressed 0: Trigger A button pressed

1: Trigger B button not pressed 0: Trigger B button pressed

■ Fig. ..... 4 Joystick Control ($E9A005)

$E9A005 | bit 7 6 5 4 3 2 1 0 | IOC7 IOC6 IOC5 IOC4 Sampling Rate PCM PAN

ADPCM Sampling Rate Switching ADPCM Output Switching

1: Joystick #1 operation disabled 0: Normal operation

1: Joystick #2 operation disabled 0: Normal operation

1: Joystick #1 Option Function A operation 0: Normal operation

1: Joystick #1 Option Function B operation 0: Normal operation

The operation status of the connected joystick is input.

Bits 6 and 7 are the option feature bits for Joystick #1, used to output trigger signals. When writing '1' to these bits, the signal level for the trigger button becomes '0' (Low level). Normally, please set these bits to '0'.

2-3 Control Word

The control word register is used to set the initial value and control the bit set/reset function of the µPD 8255. The µPD 8255 has an input/output port function and an operating mode to support parallel data transfer. The X68000 uses the µPD 8255 as a joystick interface, so it can be confusing as to whether the lower bits are used for ADPCM or data transfer, but I will explain briefly here.

2-3-1 Bit Set/Reset Mode

In the control word, if the most significant bit of the written data is '0', the µPD 8255 will interpret it as a bit set/reset command. In the bit set/reset command, you can set the desired bit, whether to operate as an output port, to '1' or '0'. This command format is shown in Figure 5. By setting bit 7 to '0', you specify whether to operate bit 7 to 0 with the operation method specified in the bit and then setting the value.

Figure 5: Control Word (Bit Set/Reset) ([$E9A007])

(Explanation of diagram content) bit 7 bit 6 bit 5 bit 4 bit 3 bit 2 bit 1 bit 0

```
 '0'   BITSEL            DATA
```

BITSEL:

111: PC7 110: PC6 101: PC5 100: PC4 011: PC3 010: PC2 001: PC1 000: PC0

DATA: Set data

2-3-2 Mode Setting Command

When the upper bit of the data written into the control word is '1', it becomes the operation mode setting command of μPD 8255. This format is shown in Figure 6.

μPD 8255 has three ports and is largely divided into two groups. Port A and the upper part of Port C are called Group A, and Port B and the lower part of Port C are called Group B. In this group structure, you can select one of the three modes for Group A and one of the two modes for Group B.

In the control word, bits 0 to 2 are setting the mode for Group B, and bits 3 to 6 are for setting the mode for Group A.

1 Mode 0

Mode 0 is the simplest and programs ports as basic I/O ports. In the case of X 68000, Group A and Group B ports are usually used in this mode. In this mode, ports A, B, and C are used as described below.

Figure 6: Control Word (Mode Setting) ($E9A007) bit 7 6 5 4 3 2 1 0 '1' Group A Mode PORT A IN/OUT PORT C(High) IN/OUT Group B Mode PORT B IN/OUT PORT C(Low) IN/OUT

1: Port C (lower) as Input 0: // as Output

1: Port B as Input 0: // as Output

Group B (Port B and lower part of Port C) operating modes: 1: Mode 1 0: Mode 0

1: Port C (upper) as Input 0: // as Output

1: Port A as Input 0: // as Output

Group A (Port A and upper part of Port C) operating modes: 11: Mode 2 10: Mode 2 01: Mode 1 00: Mode 0

Page 382

Joystick

Port C's top 4 bits, port C's bottom 4 bits can each be individually set to input or output.

The X68000 normally uses ports A and B as inputs, and port C can be set as input or output. As for the control word, you write 392 for standard use.

In the X68000 quincunx course, since ports A, B, PC 4, and PC 5 are directly connected, when connecting a peripheral designed for a joystick board, you can set port A and port B as output, and the top of port C (PC 4 and PC 5) as input to use it.

2 Mode 1

Mode 1 is a mode like a printer interface that supports parallel transmission. It is set to use ports C and D for control signals, so for the X68000, if you only use group A, it will be convenient. Therefore, here we will explain based on group A.

In Mode 1, it operates as input or output depending on whether the control word specifies it. (Port A, port B as input/output)

An example of input behavior: When used as input, when PC 4 requests data from externally, PC 5 fills the buffer with data indicating whether the data is in or not (IBF: Input Buffer Full), and sends an interrupt signal (INTR) to the CPU. In the X68000, since PC 3 is set to ADPCM, interrupts do not occur, but it can work with status check.

When data from the outside is set to PA 0-P7 and set to STB (Low), µPD 8255 stores the data and at the same time IBF will be High. When STB is High, it becomes INTR and µPD 8255 is High. The CPU reads ports and INTR and automatically shifts to Low level, making sure data is stored in the CPU.

This timing is very similar to the work done by a printer.

● Figure 7 Mode 1 (Input Mode)

(Description of the diagram)

Data Input

STB (PC4/PC2) IBF (PC5/PC1) INTR (ADPCM) STB IBF

of the CPU Port A Read

Data Acquisition Timing

In this mode, PC3's interrupt request signal (INTR) is sent to the CPU, and PC7 becomes the output data set complete status signal (OBF: Output Buffer Full). When data is read by the other party, PC6 becomes the acknowledgment signal (ACK). If the program is set to output mode, PC6 becomes the input signal from the other party, but since it is only used as an output in X68000, this mode is not used.

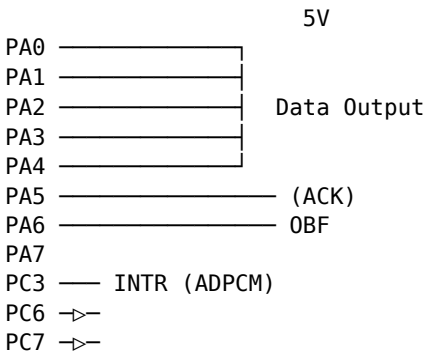
When data is written to Port A, INTR becomes 'Low,' and OBF also becomes 'Low,' indicating that the output data is enabled. The other party receives this and returns an ACK, after which OBF goes 'High' again. INTR also goes 'High,' and it transitions to the next interrupt request to the CPU.

3 Mode 2

This mode is only available in Group A. This mode allows bidirectional data output.

Joystick

●Figure 8 Mode 1 (Output Mode)



(Timing chart) WR INTR (PC3/PC0) OBF (PC7/PC1) ACK (PC6/PC2) ↑CPU writes data ↑Other party takes in data

- On the X68000, PC6 is output-only, so this mode cannot be used.

The operation in this mode is shown in Figure 9 on page 386. It is an operation mode that is, in effect, a combination of the input mode and output mode of Mode 1.

In output operation, writing to Port A causes both INTR and OBF to become 'Low', and when an ACK is received from the other party, OBF returns to 'High'.

Conversely, when STB is received from the other party, IBF, together with INTR, becomes 'High', indicating that data has been taken in. When the CPU reads Port A, IBF returns to 'Low'.

Figure 9: Mode 2

5V

Data Output

OBF ACK

STB IBF

Data Output Operation

Data Input Operation

- On X68000, since PC6 is used as output, data output operation is not possible.

● Floppy Disk ● Drive

Many of you may be seeing for the first time the FDD with the lever removed and the auto-eject mechanism, introduced with the X68000. Here, we will explain the features of the FDD, including the disk read/write functions.

■ 1 Overview of the FDD Interface

The block diagram of the X68000's floppy disk drive interface is shown on page 388. The LSI (FDC: Floppy Disk Controller) that controls the disk read/write uses the μPD72065. Another IC, the $ED9420AC (VFO), separates the data and clock read from the disk using a phase-locked loop (PLL) circuit and is called a data separator.

In general, an FDD (floppy disk drive) only requires interfacing with an FDC and VFO, but the X68000 uses drives that also have an auto-loading mechanism for automatically inserting disks, software-controlled auto-eject mechanisms, and added functionalities to support these controls by monitoring FDD state changes with the I/O controller (LSI manufactured by Sharp for the X68000). Additionally, there are precautions like forcing the I/O controller's drive select signal to '1', OPs (FM sound ICs) using CT2 for output, and forcibly putting the FDC READY terminal to '1'.

Note: LSI stands for Large Scale Integration, and PLL stands for Phase-Locked Loop.

Figure 1 FDD Peripheral Block Diagram

I/O Controller

Op. SEL 0 Option Select 0

```
// 1            // 1
// 2            // 2
// 3            // 3
```

EJECT EJECT EJECT MASK EJECT MASK LED BLINK LED BLINK

TRK0 TRK0 FDDINIT FDDINIT ERRDISK ERRDISK DISKIN DISKIN WPROTECT WRITE PROTECT

DRIVE SEL 0 DRIVE SELECT 0

```
// 1            // 1
// 2            // 2
// 3            // 3
```

RW/SEEK RW/SEEK WPRT/2SIDE WPRT/2SIDE FLT/TRK0 FLT/TRK0

OPM CT2

READY READY INDEX INDEX FLT/TRK0 WPRT/2SIDE RW/SEEK RW/SEEK LCT/DIR

FLTR/STEP TOUT_RDATA WDATA SIDE US0 US0 US1 US1 ISO ISO HOLD HOLD OSC (16MHz)

MIN/STD TRIG

READ_DATA READ DATA

- FDC µPD72065 * µPD72065 (FDC)
- VFO $ED9420AC * $ED9420AC (VFO)

µPD 72065 can use signals called US0 and US1 to output disk selection signals to up to four FDDs. If you specify the drive number in the command during execution, this signal will execute the drive selection. However, the X68000 doesn't use this signal and uses an I/O controller instead. When accessing the disk, the command is written in the FDC, and the I/O controller outputs it before writing the access command.

Floppy Disk Drive

In order to access a specific drive using a register, it is necessary to select the drive in advance. The FDC's READY terminal is usually connected to the FDD's READY signal, and when the FDD becomes accessible (the disk is clamped and the motor rotation is stable), it becomes an input signal of '1'. This function, which forces '1' with the OPM's CT2, is designed to be used to check if the FDD is connected, and is not normally used for regular access.

2. FDD Specifications

The specifications of the FDD built into the X68000 are shown in Fig. 2 on page 390. The X68000's FDD interface supports 2 HD, 2 DD/2D, and formats for 8-inch and 5-inch (initially used for such). Despite supporting these, if the internal FDD can only board up to 2 HD and single-density format, the format cannot be changed to 2 DD and 2D.

Also, for the X68000's FDD, the disk stays clamped and the head stays on the disk, so there is no need to consider head loading/unloading (pressing the head against the disk or releasing it). As shown in Fig. 1, the head load command signal (HDLD) remains open as it is on the FDC.

3. FDD Interface Ports

The list of ports related to the FDD interface is shown in Fig. 3 on page 390. The FDD interface is created using the FDC and I/O controller via OPM (CT2 terminal). The FDC is used for basic FDD operations such as disk reading/writing, head movement, and the I/O controller is used for additional option features such as inserting/disembarking disks and controlling LEDs. The CT2 terminal of the OPM is used to check the connection status of the disk.

Fig. 2: Specifications of the Built-in FDD

Item	Value	Notes
Unformatted Capacity	1667KB	
Formatted Capacity	1056KB	IBM format, High-density mode = 256 bytes x 29, 26 sectors/track
Capacity per Track	10.42KB	
Data Transfer Rate	500Kbit/s	
Track-to-Track Seek Time	3ms	Track movement time
Average Seek Time	15ms	Track movement time + Seek waiting time
Average Access Time	95ms	Average track movement time + Seek waiting time

Item	Value	Notes
Media Rotation Speed	360rpm	
Spindle Motor Start Time	0.5s	
Number of Tracks	TRACK/SIDE: 77 TRACK/DRIVE: 154	
Track Density	96 TPI (Tracks/Inch)	
Heads	2	
Encoding Method	MFM	FM method also possible
Others	Auto Clamp Auto Eject Auto Retry LED	

Fig. 3: FDD Interface Related Port Addresses

Device	Address	Read/Write	Bit	Notes
FDC (\uPD72065)	$E94001	R		FDC Status Register
	$E94001	W	bit 7	FDC Command Register *1
	$E94003	R		FDC Data Register
	$E94003	W	bit 6	FDC Command Register
	$E94005	R		Drive Status
I/O Controller	$E94005	W	DISK IN, ERR DISK	Drive Status Signal Control
	$E94007	R	LED CTRL, EJECT	Access Drive Selector, etc.
	$E94007	W	MASK ON/OFF	
	$E9C001	R	'0', FDC INT,	Interrupt Signal Number Status
	$E9C001	W	'0'	Interrupt Signal Mask
	$E9C003	R	Vect	Interrupt Vector Number
	$E9C003	W	DEVICE	
OPM	$E90003	W	CT1, CT2	Register S18 ≠ S18 ($E9001 ≠ S18 usage rule states to ac

* 1: SET STANDBY(S35), RESET STANDBY(S34), SOFTWARE RESET(S36) cannot be used with the following commands:

Floppy Disk Drive

3-1 I/O Controller FDD Related Ports

3-1-1 Drive Status Register

The bit distribution of the Drive Status Port is shown in Figure 4. This register indicates the disk insertion status in the FDD. bit 7 indicates whether the disk is inserted, and bit 6 indicates whether the disk is being accessed or read.

By reading these status bits, it is possible to determine whether the disk insertion or reading is being performed correctly or if there is any ambiguous input causing errors.

In the X68000, the presence of a disk is determined, and if a disk ejection occurs, an interrupt is generated. By reading the state of each drive in sequence, it is possible to determine whether a drive has changed state. If a disk ejection occurs, the disk is automatically ejected upon receiving an interrupt (bit 7 changes to '0'). This ejection generates an interrupt, leading to the handling of two consecutive interrupts in case of errors.

Figure 4 - Drive Status SE 94005

bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
DISK IN	ERROR DISK	'0'	DISK				

1: Disk inserted (also setting bit 7 to '1') 0: Disk not inserted

1: Disk insertion state 0: Disk non-insertion state

3-1-2 Drive Control Register

The Drive Control Register controls optional features such as the LED or ejection mechanisms.

5 Drive Control 5E 94005

bit	7	6	5	4	3	2	1	0
	LED CTRL	EJECT MASK	EJECT ON/OFF	'0'	DRIVE #3	DRIVE #2	DRIVE #1	DRIVE #0

- When '1'-'0' changes, the selected option function works

```
- 1: Eject media
- 0: Do not eject
- 1: Disable eject button (LED on eject button is off)
- 0: Enable eject button (LED on eject button is on)
- 1: FD access lamp lights up (effective when media is not inserted)
- 0: Access lamp is off
```

The bit arrangement is as shown in Figure 5.

Bits 7 to 5 specify option functionalities. Bits 0 to 3 specify the drive number to which the option functionality is applied (internal drives are from 0 to 1). Each option functionality is designed to work when the drive selection bit changes to '1'. Drive selection can be done simultaneously to multiple drives.

For example, if you write $23 to this register and then write $20, drives 0 and 1 will eject simultaneously.

3-13 Access Drive Select Register

The bit arrangement of this register is shown in Figure 6. This register selects the drive number and media type to access FDD.

When the bit is set to '1', the FDD motor starts rotating, and bits 0 and 1 specify signals to the selected drive, changing the drive access lamp (LED) from green to red. Setting to '0' will keep it green until certain conditions are met, and it will stop after a while. Ensure to set bit 0 to '1' and specify the drive number before accessing.

bit 4 is used for toggling between 2 HD and 2 DD (used only for external drives with 2 HD or 2 DD capabilities). This bit is usually not needed and can be left as '0'.

●■.....6 Access Drive Select $E94007

Floppy Disk Drive

bit 7 6 5 4 3 2 1 0 MOTOR | ON | '0' | '0' | 2HD/2DD | '0' | '0' | ACCESS DRIVE

```
                1                               1    2DD/2D Disk
                                                0    2HD Disk
            0                               2
            0                               1
            0                               0
                                        |
                        Access Drive Number
```

```
11: Drive #3
10: #2
01: #1
00: #0
```

1: Enable Drive Select & Motor ON (Access lamp turns red) 0: Disable Drive Select & Motor OFF (access lamp turns green)

3-14 Interrupt Status Register

The bit layout is shown in FIG. 7 on page 394. The I/O controller has interfaces such as SASI (hard disk), floppy disk, and printer I/O ports, making it possible to manage interrupts with the I/O controller. This register indicates the interrupt status managed by the I/O controller.

The upper bit shows the interrupt request generation state, and the lower 4 bits mirror the contents of the interrupt mask register. This shows whether each interrupt request has been generated.

If the interrupt is not allowed (when '0'), even if an interrupt request is generated, it will not be sent to the CPU. However, you can read the lower 4 bits to infer the occurrence of interrupt requests.

3-15 Interrupt Signal Mask Register

The bit layout is shown in FIG. 8 on page 394. The I/O controller manages interruption permission/prohibition. When each bit is set to '1', the interrupt request is permitted; when set to '0', it is prohibited.

● Fig. ... 7 Interrupt Signal Status $E9C001

bit	7	6	5	4	3	2	1	bit 0
	FDC INT	FDD INT	PRT INT	HDDI INT	HDDI EN	FDCI EN	FDDI EN	PRTI EN
-----	-------	-------	------	-------	-----	---	---	---------

```
1 : Printer interrupt allowed
0 :       //        prohibited

1 : FDD interrupt allowed (occurs for media operation)
0 :       //        prohibited

1 : FDC interrupt allowed
0 :       //        prohibited

1 : SASI disk interrupt (occurs at command completion) allowed
0 :       //        prohibited

1 : SASI disk interrupt is occurring
0 :       //        not occurring

1 : Printer BUSY signal is 'Low' (READY state)
0 :       //          'High' (BUSY state)

1 : FDD interrupt is occurring
0 :       //        not occurring

1 : FDC interrupt is occurring
0 :       //        not occurring
```

● Fig. ... 8 Interrupt Signal Mask $E9C001

bit	7	6	5	4	3	2	1	bit 0
	'0'	'0'	'0'	HDDI EN	FDCI EN	FDDI EN	PRTI EN	
-----	-----	-----	----	------	-----	-----	-----	-------

```
1 : Printer interrupt allowed
0 :       //        prohibited

1 : FDD interrupt allowed (occurs for media operation)
0 :       //        prohibited

1 : FDC interrupt allowed
0 :       //        prohibited

1 : SASI disk interrupt (occurs at command completion) allowed
0 :       //        prohibited
```

Page 394

(3-1) 6 Interrupt Vector Setting Register

The bit allocation is as shown in the figure below. In this register, set the number of the interrupt vector to be output by the I/O controller. The most significant 6 bits are valid in the setting, and the number of the interrupt request is automatically changed by the I/O controller and supplied to the CPU.

■Figure 9: Interrupt Vector Setting $E9C003

bit 7	6	5	4	3	2	1	bit 0
Vect (1 to 63 settings)			DEVICE				

Assignable settings

11: Printer interrupt 10: Hard disk interrupt 01: FDD interrupt 00: FDC interrupt

(3-2) OPM (YM2151) FDD Related Port

The bit allocation of register $1B which controls terminal C of the OPM is shown in Figure 10 on page 396. When bit 6 ”1” is set, the FDC FDD READY terminal is forcibly set to ready state (regardless of whether a disk is inserted or not, it is considered in the inserted state).

This function is used to check the disk connection status. When this bit is set to '1' and the RECALIBRATE command is issued, the FDC attempts to move the head to track 0 under the assumption that the drive is in the ready state. If a disk is not inserted, the FDC cannot detect track 0 even if a move pulse is sent to the head, and an error is detected if an abnormal number of pulses are sent (if this bit is not set to '0' and RECALIBRATE command is issued without a disk inserted, the head does not move and an error is immediately detected). This bit should normally be set to '0' except when checking the drive connection.

Page 395

●Fig. 10 OPM Register $1B ($E90003)

```
 bit 7 6 5 4 3 2 1 bit 0
 ---------------------------------
```

| CT1 | CT2 | W |

```
 ---------------------------------
```

```
CT1:
 1: Forcibly sets the FDC READY terminal to the Ready state.
 0: Normal operation

CT2:
 1: Sets the basic clock of ADPCM to 4MHz
 0: "              "  8MHz
```

LFO output waveform selection

#4 FDC

The FDC ports are assigned to $E94001 and $E94003. Access to the FDC is performed by reading and writing commands and data at the $E94003 address and reading the status at the $E94001 address. Writing to $E94001 is limited to commands used during non-initialization conditions, such as FDC initialization.

#4-1 FDC Status Register

The content of the FDC status register is shown in Fig. 11. bit 7 indicates whether the CPU is ready to receive data (commands) from the FDC, with '1' indicating that the FDC is ready to transfer data and '0' indicating that the CPU is not yet ready.

bit 4 is used in programming to specify, using the SPECIFY command, the timing for E-PHASE (refer to page 397 of the manual) or DMA disk read/write operations. When this bit is '1', it indicates the CPU will select data transfer timing. C-PHASE (refer to page 397) and R-PHASE (refer to page 397) are typically used for data transfer by the CPU, so having '0' means it will not maintain a hold signal.

1–11 FDC Status Register

bit 7 6 5 4 3 2 1 bit 0 RQM DIO NDM CB D3B D2B D1B D0B

FDn Busy 1: Drive is in seek / seek complete / reserved state 0: Not

FDC Busy 1: FDC is busy (seek / recalibrate sets E-PHASE) 0: Not

Non-DMA Mode 1: Data transfer in non-DMA mode 0: Not (C-PHASE, R-PHASE is always ‘0’)

Data Input/Output 1: Transfer from FDC to Host 0: Transfer from Host to FDC

Request For Master 1: FDC is ready for data transfer 0: Not

2 FDC Phase Transition

FDC's operation status can be broadly classified into the command phase (abbreviated as C-PHASE), where parameters for command execution are received from the CPU, the execution phase (E-PHASE), where the command is executed and status is set, and the result phase (R-PHASE), where the result is read by the CPU. These three phases perform their functions sequentially. Please refer to the diagram on page 398 for detailed phase transitions for each command.

When it comes to commands in the seek series or disk read/write commands, the E-PHASE completes while being reserved. The aforementioned Idle state is considered hypothetical. In practice, the FDC waits for subsequent commands from the CPU, thus clearly distinguishing the Idle state is unnatural. Therefore, although phase transitions are time-consuming, the diagram does not depict the Idle phase to keep it simple.

For operations like disk read/write, the data transfer may take place in the E-PHASE, using the CPU and DMAC for data transfer. An FDD performs data transfers similarly to a hard disk like SASI or SCSI.

Figure 12: FDC Phase Transition

(Idle)

Command Phase

- Set Strobe
- Software Reset
- Specify
- Sense Status
- Reset Strobe
- INVALID

Execution Phase

- (Seek System)
  - Occurrence of Interrupt
- (Read/Write System)
  - Occurrence of Interrupt

Result Phase

Figure 13: Transfer Time per Byte | Media Type | Recording Method | FM | MFM | |----------|------------------|----------|----------| | 2HD1 | | 32µs | 16µs | | 2DD/2D1 | | 64µs | 32µs |

*Note: *1: 2HD/2DD will be MFM recording

Since we do not have a large buffer, it is necessary to always service transfer requests from the FDC within a specified time. For this reason, data transfer is typically done using DMAC. The table in figure 13 shows the transfer time per byte of data for each media type and recording method. On the X68000, typically a 2HD format is used, so data must be fetched (during read) or completed (during write) within 16µs.

Floppy Disk Drive

4.3 Result Status

Of the status returned in the R-PHASE, the ones returned by many commands such as read/write commands are the three status bytes ST 0, ST 1, and ST 2. Their data bit arrangements are as shown in Figures 14, 15, and 16.

The state transition mentioned in the explanation when the upper 2 bits of ST 0 are '11' refers to insertion/removal of a disk, but in the X 68000, this change is detected by the I/O controller.

● Figure 14 Result Status 0 (ST 0)

bit 7 6 5 4 3 2 1 0 IC SE EC NR HD US1 US0

Unit Select n (Drive number at time of interrupt request) 11: Drive #3 10: " #2 01: " #1 00: " #0

Head Address (Head number at time of interrupt request) 1: Side 1 (back) 0: Side 0 (front)

Not Ready 1: Specified device is not ready 0: Normal operation

Equipment Check 1: Fault signal received from drive / RECALIBRATE command execution error (Track 0 cannot be found) 0: Normal operation

Seek End 1: SEEK/RECALIBRATE command execution finished 0: Normal operation

Interrupt Code (Cause of interrupt request) 11: Device state transition occurred (AI: Attention Interrupt) 10: Received command was invalid (IC: Invalid Command) 01: Command

ended abnormally (AT: Abnormal Terminate) 00: Command ended normally (NT: Normal Terminate)

Fig. 15 Result Status 1 (ST1)

bit 7	6	5	4	3	2	1	bit 0
EN	DE	OR	ND	NW	MA		

- Missing Address Mark

```
- 1: Address mark cannot be found
- 0: Normal operation
```

- Not Writable

```
- 1: Write prohibited
- 0: Normal operation
```

- No Data

```
- 1: Cannot find the specified sector
- 0: Normal operation
```

- Over Run

```
- 1: Data transmission time out, host did not respond within the specified time
- 0: Normal operation
```

- Data Error

```
- 1: Detected CRC error in disk ID or data (excluding READ ID command)
- 0: Normal operation
```

- End of Cylinder

```
- 1: Attempted to read/write beyond the last sector set by the EOT parameter in the command
- 0: Normal operation
```

(Page 400 in the bottom left corner)

●x□::::16 Result Status 2 (ST 2)

bit

6 5 4 3 2 1 bit 0

CM DD NC SH SN BC MD

```
Missing Address Mark in Data Field
  1: Data address mark could not be found
  0: Normal operation

Bad Cylinder
  1: When ST1 ND bit is '1' and the ID C byte is $FF
     (READ DIAGNOSTIC command is excluded)
  0: Normal operation

Scan Not Satisfied
  1: Condition not met after the last sector in the SCAN command
  0: Normal operation

Scan Equal Hit
  1: Equal condition established in the SCAN command
  0: Normal operation

No Cylinder
  1: When ST1 ND bit is '1', ID C byte is not $FF
```

```
    0: Normal operation

  Data Error in Data Field
    1: Detected a CRC error in the data
    0: Normal operation

  Control Mark
    1: Detected DAM in the READ DATA/READ DIAGNOSTIC/SCAN commands
    0: Detected DAM in the READ DELETED DATA execution command
       Normal operation
```

Track Format

µPD72065 handles the track format details of the disk, as shown on page 402 in Diagram 17. The signal written as INDEX is a pulse signal that occurs once per disk revolution. It is detected by the 5-inch floppy disk's index hole. After the index pulse, it is followed by Gap 4a, SYNC (synchronization pattern), IAM (Index Address Mark), Gap 1, and then each sector's information in order.

Figure 17 Track Format

Index Signal

Book: Diagram showing the format of one track.

FM Format	MFM Format
Gap 4a	$FF x 40 x 6
SYNC	$00 x 12
IAM	$FC x 1
Gap 1	x 26

FM Format	MFM Format
SYNC	IDAM
$00 x 6	$FE x 1
$00 x 12	$A1 x 3

1 Track

Further, after the last sector, Gap 4b follows, indicating the end of one track.

The beginning of each sector contains SYNC, IDAM (ID Address Mark), followed by C (Cylinder), H (Head), R (Sector), N (Sector Length), and CRC data. C, H, R, and N indicate which cylinder (track), which head surface, and which sector the data belongs to. Think of these as like the addresses of the sectors.

Following the CRC check code in this header, Gap 2, SYNC, and then DAM (Data Address Mark) or DDAM (Deleted Data Address Mark) are written.

Usually, DAM is written in this area. If DDAM is written, the CM bit of ST2 is set to '1' when accessing with the usual READ DATA/WRITE DATA commands.

Following DAM/DDAM, the actual read/write data continues, and finally, these CRC check codes and Gap 3 continue.

5 FDC Commands

FDC commands have many parameters, and each parameter is referred to by an abbreviation. The table below lists the abbreviations used in the diagrams of FDC commands

and their corresponding meanings for your reference.

Figure No. 18: List of Abbreviations in Commands and Their Meanings

Abbreviation	Name	Content
MT	Multi track	Operate across multiple tracks
MF	MFM Mode	Perform double density recording, set to '1' (`X68000` usually sets this to '1')
SK	Skip	To skip DDAM/DAM sectors, set to '1'
HD	Head	Specify head address, usually '0' or '1'
US0, US1	Unit Select 0/1	Specify drive number (`X68000` ignores this)
C	Cylinder Number	Cylinder (Track) number of the disk ID information
H	Head Number	// Head number
R	Record Number	// Sector number
N	Record Length	Sector length code
EOT	End Of Track	Last sector number
GPL	Gap Length	Number of bytes for gap 3
GSL	Gap Skip Length	Number of bytes for gap 3 in skipped sectors
DTL	DaTa Length	Number of bytes to be processed per sector (ignored if N=0)
ST0, 1, 2	Status	Result status
SC	Sector	Specify number of sectors per track during WRITE ID command
D	Data	Specify data to be written during WRITE ID command
STP	Step	Process next sector during SCAN command, process sector 'S0' and then process 'S
NCN	Next Cylinder Number	Specify the next cylinder number during SEEK command
SRT	Step Rate Time	Specify the interval of the step pulse (head movement pulse)
HUT	Head Unload Time	Time from when the head load signal (HOLD) is turned off until the head disengag
HLT	Head Load Time	Time from when the head load signal goes ON until the head stabilizes
ND	Non-DMA Mode	Set to '1' when transferring data without using DMA

Page 403

5•1 READ DATA Command

5•01 Command Phase (C-PHASE)

The format of the READ DATA command is shown in Figure 19. In the C-PHASE, 9-byte command parameters are given. The meanings of each parameter (refer to Figure 19) are as follows:

● Figure 19 READ DATA Command Phase | READ/WRITE | bit 7 6 5 4 3 2 1 bit 0

```
| MT  MF  SK | '0' '0' '1' '1' '0'
| -----------+-------------------------------
```

C-PHASE W |

```
| SK: Skip
| DDAM
--------------------------+--------------------
| HD  US1  US0
--------------------------+--------------------
| C     | Execution Start Sector ID Information
--------------------------+--------------------
| H
--------------------------+--------------------
| I
--------------------------+--------------------
| R
--------------------------+--------------------
| N
--------------------------+--------------------
| EOT
--------------------------+--------------------
| GSL
```

```
                -------------------------+-------------------
                | DTL
----------------------+---------------------- E-PHASE R | Data Transfer Operation ----------------------
---+---------------------- R-PHASE R |
                | STO
                -------------------------+-------------------
                | ST1
                -------------------------    * At normal end time
                | ST2
                -------------------------    * Ending Sector's next ID when execution ends
                | C
                -------------------------    * When ending abnormally
                | H
                -------------------------    * The ID of the Sector at End time
                | I
                -------------------------+-------------------
                | R
                -------------------------+-------------------
                | N
```

Floppy Disk Drive

MT: Multi Track When accessing data continuously on the disk, this bit is set to '1'.

MF: MFM Mode When using the MFM method (2 HD or 2 DD), it is set to '1'. On the X 68000, it is usually set to '1'.

SK: Skip DDAM When set to '1', if a sector marked as DDAM (Deleted Data Address Mark) is encountered during access, the access to that sector is skipped, and the next sector is accessed. Once the end of the sector's access is completed, the command's termination status is indicated. At this point, the ST2 CM (Control Mark) bit becomes '1'.

HD: Head Specifies the head number to be used for access. '0' for front face, '1' for back face.

USB0, USB1: Unit Select 0/1 Specifies the drive number to access. On the X 68000, it is selected with I/O controllers, so this designation is for drive selection and has no other meaning. For commands like SEEK or RECALIBRATE, each drive's unique number provided by the FDC is used, so make sure to set the actual drive number you intend to access.

C.H. R. N: Cylinder/Head/Record/Length Specifies the cylinder number, head number, sector number, and sector length when accessing a sector. The FDC writes these parameters to each sector's address mark on the disk, and the FDC reads these marks back during access. If these match, access begins. Usually, the actual physical positional data of the head, such as cylinder number or head number, matches the specified values here.

Additionally, cylinder and head numbers here are compared with the values written in sector addresses for validation. For example, if an unintended writing scenario occurs, a logical disk could be created where every sector is readable and writable as sector 0.

EOT: End of Track Sets the last sector number of the track.

GSL: Gap Skip Length When performing a multi-sector access for contiguous sectors, this prevents reading and writing the gap portion between data parts and the next sector’s ID.

DTL: Data Length If the value of N is 00, this data specifies the sector length for accessing data on this sector.

Page 405

$00 or anything else, this data has no meaning.

Please refer to Fig. 20 for the settings values for each format used according to its parameters. Human 68K adopts the format of 1024 bytes/sector in MFM.

Fig. 20 ... Sector Format and Parameters

Format	Sector Size	FDC Parameter	Notes
	(Bytes/Sector)	EOT, SC, GSL, GPL	
FM	128	$00, $1A, $07, $1B	(IBM Disk 1)
	256	$01, $0F, $0E, $2A	(IBM Disk 2)
	512	$02, $08, $1B, $3A	
	1024	$03, $04, Unspecified, Unspecified	
	2048	$04, $05, Unspecified, Unspecified	
	4096	$05, $01, Unspecified, Unspecified	
MFM	128	$00, $1A, $0E, $36	(IBM Disk 2D)
	256	$01, $0F, $0B, $54	
	512	$02, $08, $15, $74	X68000 Standard Format
	1024	$03, $08, $35, $74	(IBM Disk 2D)
	2048	$04, $05, Unspecified, Unspecified	
	4096	$05, $02, Unspecified, Unspecified	
	8192	$06, $01, Unspecified, Unspecified	

(Note: Names of the format in 8-inch FD are described within parentheses.)

5-2 Execution Phase (E-PHASE)

Data transfer is performed. If the data transfer is done via DMA, you should finish setting up DMA before giving the command.

At the end of access, it is conveyed to the terminal count (TC) terminal. Whether TC is completed is notified, after which FDC will continue to read and conduct sector chaining. When it is $00 or $80, each sector eat into the preceding DTL bytes in case it falls short of 128.

Page 406

Floppy Disk Drive

5.1.3 Result Phase (R-PHASE)

In R-PHASE, the three result statuses (ST 0, ST 1, ST 2) and the C, H, R, N values at completion are returned. On normal completion, C, H, R, N return the ID of the sector following the last accessed sector; on abnormal completion, the ID of the sector at the time of termination is returned.

5.2 READ DELETED DATA Command

The command format is shown in Figure 21. This command reads a sector whose data is preceded by a DDAM (Deleted Data Address Mark) in the sector header. The operation is as follows:

● Figure 21 READ DELETED DATA Command

Phase	READ/WRITE	bit 7 6 5 4 3 2 1 bit 0	Remarks
		MT MF SK '0' '1' '1' '0' '0'	SK: Skip DAM
		HD US1 US0	
		C	
		H	
C-PHASE	W	R	Start sector ID information
		N	
		EOT	

Phase	READ/WRITE	bit 7 6 5 4 3 2 1 bit 0	Remarks
		GSL	
		DTL	
E-PHASE	R		Data transfer operation
		ST0	
		ST1	
		ST2	/Normal completion
R-PHASE	R	C	ID of sector following completion sector
		H	/Abnormal completion
		R	ID of sector at termination
		N	

READ DATA commands: In the explanation, DAM is referred to as DDAM and DDAM as DAM.

For example, in the READ DATA command, DDAM sectors are read out, setting bit 1 of ST 2 to 1, but in this command, when reading a DAM sector, it will become 1.

5.3 READ ID Command

The command format is shown in Figure 22. After receiving the command, it first locates the ID information (C, H, R, N) of the first normal sector (one without errors). This command only reads IDs from the FDC Disk in the E-PHASE and does not transfer any data between the host (CPU/DMA).

Figure 22 Read ID Command | Phase | | | |---------|---|---| | C-PHASE | W | ... | | E-PHASE | - | ... | | R-PHASE | R | ... |

5.4 WRITE ID Command

This is the format for one track. The command format is as shown in Figure 23. For SC and GPL settings, please refer to the READ DATA illustrations. The D byte is... (continued)

ID Command Diagram - 23

Phase	READ/WRITE	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
		'0'	MF	'0'	'0'	'1'	'1' HD	'0' US1	'1' US0
...									
C-PHASE	R		N						
E-PHASE	W		1 Track's ID information transfer						
R-PHASE	R		ST0						
...									

Key: '0' - Value 0 MF - ... HD - ... US1 - ...

Write the data to the sector part. What is given in E-PHASE is the ID for each sector (C, H, R, N). That is, the WRITE ID command in E-PHASE data transferred to the FDC will be 4x (number of sectors per track).

Among the values returned in R-PHASE, C, H, and R have no meaning. The value for N byte is returned as the value given in C-PHASE.

5:5 WRITE DATA Command

This command format is indicated in diagram 24 on page 410. Write data to the specified sector. There is no difference from READ DATA command except that the data transfer direction in E-PHASE is reversed.

Note:

Figure 24 WRITE DATA Command

Phase	READ/WRITE	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0	Remarks
C-PHASE	W	MT	MF	'0'	'0'	'1'	'0'	'1'	'1'	Implementation start sector ID inform
C										
H										
R										
N										
EOT										
GSL										Data Transfer
DTL										
E-PHASE	W									
R-PHASE	R	ST0	ST1	ST2		HD	US1	US0		End of normal completion
		C								Next Sector ID of the completed secto
		H								ID of the sector at abnormal terminat
		R								ID of the sector at termination
		N								

5.6 WRITE DELETED DATA Command

The command format is shown in Figure 25. Writing DDAM into the sector header instead of DAM is similar to the WRITE DATA command.

WRITE DELETED DATA Command

Phase	READ/WRITE	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0	Remarks
MT		MF	'0'	'1'	'0'	'0'	'0'	'1'	'1'	
C-PHASE	W									

```
| | C |
| | H |
| | R | Execution start sector ID information
| | N |
| | EOT |
| | GSL |
| | DTL |
```

E-PHASE | W | Data transmission R-PHASE | R |

```
| | ST0 |
| | ST1 |
| | ST2 |
| | | Normal completion time
| | | Actual end of the next sector ID
| | | Error occurrence time
| | | Execution start sector ID
```

5.7 READ DIAGNOSTIC Command

The command format is as shown on page 412 in Diagram 26. It is very similar to the READ DATA command, but there are points where it reads forcedly from the sector at the back of the INDEX signal regardless of the presence or absence of errors. In the C, H, R, N of the command, even if the C, H, R, N of the sector do not match, by making the ND

(No Data) bit of ST1 '1', it only continues processing and completes normally. If there is an ID or data CRC error, making the DE (Data Error) bit of ST1 or the DD (Data Error in Data Field) bit of ST2 '1' completes it normally. Detecting DDAM sets the CM (Control Mark) bit of ST2 to '1', but processing continues.

◉ Figure: 26 READ DIAGNOSTIC Command

Phase	READ/WRITE	bit 7	bit 6	bit 5	bit
C-PHASE	W	'0'	MF	'0'	'0'
		C	H	R	N
E-PHASE	R	Data Transfer			
		ST0	ST1	ST2	
R-PHASE	R				
		ST0	ST1	ST2	
		Normal termination when ID of the sector next to the execution end sector			
		C			
		ID of the sector next to the execution end sector			

Error status read at the end of the command is handled as the logical sum of all error conditions that occurred during the procedure.

5.8 SCAN EQUAL/SCAN LOW OR EQUAL/SCAN HIGH OR EQUAL Command

Each command format is shown in Figures 27, 28, and 29. The specified sector contents are compared with the data written from the host to the FDC. The data is compared sequentially from the leading data of the sector. However, if the data written from the host to the FDC is 8F FF, the comparison of that byte is not performed and is treated as ”equal”.

Fig 27 - SCAN EQUAL Command

Phase				Notes
		bit 7 6 5 4 3 2 1		
		MT MF SK '1' '0' '0' '1'	HD US1 US0	
-------	------	------------------	-------------------------	------------
C	READ	C		
		H		Perform ID information for start sector execution
	W	R		
-		N		
PHASE		EOT		
		GSL		
		STP		
-------	------	------------------	-------------------------	------------
PHASE	W			Compare Data
-------	------	------------------	-------------------------	------------
E		ST0		
		ST1		
		ST2		
-------	------	------------------	-------------------------	------------
R	READ	C		Compare End Sector
		H		
		R		
		N		

SCAN EQUAL checks if the contents of the compare sector are all equal. SCAN LOW OR EQUAL checks if they are all equal or if the initial data was smaller, checking when the data read from the disk did not match initially. SCAN HIGH OR EQUAL checks if they are

all equal or if the initial data was larger, ending in success if the data read from the disk is larger when it did not initially match.

Figure....28 SCAN LOW OR EQUAL Command

Phase	READ/WRITE	bit 7	6	5	4	3	2	1	bit 0	Remarks
C-PHASE	W					'1'	'1'	'0'	'0'	'1'
E-PHASE	W									
R-PHASE	R		C		H		R		N	

Floppy Disk Drive

Fig.....29 SCAN HIGH OR EQUAL COMMAND

| Phase | READ/WRITE | bit 7 | 6 | 5 | 4 | 3 | 2 | 1 | bit 0 | | ----- | --------- | ---- | - |- | - | - | - | - | - | ---------- | | C-PHASE | READ | MT | MF | SK | '1'| '1'| '1'| '0' | '1'| | HD | US1 | US0 | | | WRITE | C | | | | | | | | | | E-PHASE | WRITE | | | | Data comparison | | R-PHASE | RETURN | C | H | R | N | | | EOT | GSL | STP | Last compared sector ID information |

5-9 SEEK COMMAND

The command format is as shown in figure 30 on page 416. Moves the head to the specified cylinder. The FDC remembers the previous head position, and by managing the head movement direction, it autonomously controls the shortest head movement and moves the head so that the cylinder number given by the CPU command is reached.

For some reason, if the cylinder number managed by the FDC and the actual head position do not match, the data may not match when read/write is performed with the ID on the sector ID. In such cases, use the RECALIBRATE command described earlier to initialize the cylinder number managed by the FDC and the actual drive head position to zero.

● Figure 30 SEEK Command

Phase	READ/WRITE	bit 7 6 5 4 3 2 1 bit 0	Remarks
C-PHASE	W	'0' '0' '0' '1' '1' '1' US1 US0 NCN	
E-PHASE	—		Seek operation execution

The completion of the SEEK operation is notified by an interrupt. When an interrupt occurs, after confirming that the DIO bit of the FDC status register is '0', you send the SENSE INTERRUPT STATUS command and read out ST 0.

5.10 RECALIBRATE Command

The command format is as shown in Figure 31. It sets both the cylinder number managed internally by the FDC and the actual FDD head position to 0. It is similar to the SEEK command, but whereas the SEEK command performs head movement by the difference between the cylinder number managed internally by the FDC and the given cylinder number, the RECALIBRATE command uses the signal (TRK 00) output by the 0-track detection mechanism that the FDD has in hardware, and moves the head until this signal is output. These points differ.

It is used when beginning to use the FDD, or when a disk read error occurs, to initialize the head position and match it with the cylinder position managed by the FDC.

This command only accesses the FDD and does not involve any access to the disk, so as mentioned earlier, in the X 68000 it is used to check whether an FDD is connected.

The completion of the RECALIBRATE command operation is also notified by an interrupt. When an interrupt occurs,

● Figure 31 RECALIBRATE Command

Phase	READ/WRITE	bit 7 6 5 4 3 2 1 bit 0	Remarks
C-PHASE	W	'0' '0' '0' '0' '0' '1' US1 US0 NCN	
E-PHASE	—		Recalibrate operation

When the DIO bit of the FDC Status Register is '0', send the SENSE INTERRUPT STATUS command and retrieve ST0.

5.11 SENSE INTERRUPT STATUS Command

The command format is as shown in diagram 32. When an interrupt occurs and the DIO bit of the FDC Status Register is checked to be '0', issue this command to retrieve the cylinder position at the end of the command, along with ST0.

●Diagram.....32 SENSE INTERRUPT STATUS Command

Phase	READ/WRITE	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
C-PHASE	W	'0'	'0'	'0'	'0'	'1'	'0'	'0'	'0'
E-PHASE	R								PCN

5.12 SENSE DEVICE STATUS Command

The command format is as shown in diagram 33. By using this command, the status of the status line input to the FDC can be read. The content of the result status (ST3) retrieved is as shown in diagram 34 on page 418.

●Diagram.....33 SENSE DEVICE STATUS Command

Phase	READ/WRITE	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
C-PHASE	W	'0'	'0'	'0'	'0'	'0'	'1'	'0'	'0'
R-PHASE	R							ST3	

The text provides details on two command functions for a floppy disk drive (FDD): ”SENSE INTERRUPT STATUS” and ”SENSE DEVICE STATUS.”

Figure.....34 Result Status 3 (ST 3)

bit 7	6	5	4	3	2	1	bit 0
FT	WP	RY	TO	TS	HD	US1	US0

- Value specified by the command phase as US1, US0
- Value specified by the command phase as HD
- Status signal of Two Side for drive
- Status signal of Track 0 for drive
- Status signal of Ready for drive

- Status signal of Write Protect for drive
- Status signal of Fault for drive

5-13 SPECIFY Command

To set the parameters necessary for each type of disk drive, the command format is indicated as Figure 35.

- SRT (STEP RATE TIME): Sets the interval of the step pulse (head movement signal). For the internal drives of the X 68000, it is 3 ms.
- HUT (HEAD UNLOAD TIME): Sets the time from the end of the disk read/write command until the head reaches the unloaded state (separated from the disk). If there is no access command in the interval until return, it allows the time specified by HLT after the next read/write command is issued and starts.
- HLT (HEAD LOAD TIME): Sets the waiting time from the start of the execution of the read/write command until the head connects to the load state (attached and stabilized to the disk).

Figure……35 SPECIFY Command

Phase	READ/WRITE	7	6	5	4	3	2	1	bit 0	Remarks
C-PHASE	W	'0'	'0'	'0'	SRT	HUT	HLT	ND		

Floppy Disk Drive

With 8-inch type FDDs, and with the thick type of 5-inch FDDs, the head physically contacts/separates from the disk for each access, and was, as the name implies, load/unload. However, in the X 68000 internal drive the head is always in contact with the disk, so these parameters are nominal only. By setting HUT longer, you can make continuous accesses while leaving short gaps, so the HLT-portion time becomes unnecessary, and access becomes slightly faster.

The ND bit sets whether the E-PHASE data transfer of read/write-type commands is performed by the CPU or by DMA. Setting it to '1' means transfer by the CPU, and setting it to '0' means transfer by DMA. With the X 68000, unless there is a particular reason, it is normal to use DMA mode.

The relationship between the setting values of the SRT, HUT, and HLT parameters and their respective times is as shown in Figure 36.

● Figure 36 SRT, HUT, HLT Setting Values and Times

Unit: ms

| Setting Value | 2HD (SRT\|HUT\|HLT) | 2DD/2D (SRT\|HUT\|HLT) | |---|---|---| | $00 | 16\|Prohibited\|Prohibited | 32\|Prohibited\|Prohibited | | $01 | 15\|16\|2 | 30\|32\|4 | | $02 | 14\|32\|4 | 28\|64\|8 | | $03 | 13\|48\|6 | 26\|96\|12 | | $04 | 12\|64\|8 | 24\|128\|16 | | $05 | 11\|80\|10 | 22\|160\|20 | | $06 | 10\|96\|12 | 20\|192\|24 | | $07 | 9\|112\|14 | 18\|224\|28 | | $08 | 8\|128\|16 | 16\|256\|32 | | $09 | 7\|144\|18 | 14\|288\|36 | | $0A | 6\|160\|20 | 12\|320\|40 | | $0B | 5\|176\|22 | 10\|352\|44 | | $0C | 4\|192\|24 | 8\|384\|48 | | $0D | 3\|208\|26 | 6\|416\|52 | | $0E | 2\|224\|28 | 4\|448\|56 | | $0F | 1\|240\|30 | 2\|480\|60 | | $10 | -\|-\|32 | -\|-\|64 | | $11 | -\|-\|34 | -\|-\|68 | | ☒ | ☒ | ☒ | | $7E | -\|-\|252 | -\|-\|504 | | $7F | -\|-\|254 | -\|-\|508 |

14 SET STANDBY Command

The command is shown in Figure 37.

The SET STANDBY command stops the internal clock of the FDC and puts it in standby mode. This command writes with E-PHASE and R-PHASE, and transitions to standby mode about 3 μs after writing. In this state, the internal state of the FDC and the state of the input/output terminals are maintained. Stopping the clock reduces the power consumption, but for devices like the X68000 that run on AC power, this command does not have much significance.

Figure 37 SET STANDBY Command

Phase	READ/WRITE	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0	Remarks
C-PHASE	W	'0'	'0'	'1'	'1'	'0'	'1'	'0'	'1'	

15 RESET STANDBY Command

This cancels the standby mode. The command is shown in Figure 38. This command is treated as an INVALID command internally by the FDC and must be retrieved as ST 0 in R-PHASE.

Figure 38 RESET STANDBY Command

Phase	READ/WRITE	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0	Remarks
C-PHASE	W	'0'	'0'	'1'	'1'	'0'	'0'	'1'	'0'	
R-PHASE	R	STA 0	$80 returns							

16. SOFTWARE RESET Command

The command is shown in Figure 39. This command resets (initializes) the FDC, putting it in the same state as after a hardware reset. This command can be issued at any timing.

Figure 39 SOFTWARE RESET Command

Phase	Read/Write	bit 7	6	5	4	3	2	1	0	Remarks
C-PHASE	W	'0'	'0'	'1'	'1'	'1'	'0'	'1'	'0'	

17. FDC Parameters and Status List

The parameters attached to the FDC command and the list of statuses received by the R-PHASE are summarized in Figure 40 on page 422. The parameters are sent from left to right, following the order with the mark "○", and the statuses are received from left to right, following the order with the mark "○".

Page 421

● Figure 40 FDC Parameter/Status List

(○ indicates the parameter/status is present for that command)

Command Name	C-PHASE Parameters: C	H	R	N	EOT	SC	GSL	GPL	DTL	D	STP	NCN
READ DATA	○	○	○	○	○	○	○	○	○			
READ DELETED DATA	○	○	○	○	○	○	○	○	○			
READ ID												
WRITE DATA	○	○	○	○	○	○	○	○	○			
WRITE DELETED DATA	○	○	○	○	○	○	○	○	○			
READ DIAGNOSTIC	○	○	○	○	○	○	○	○	○			

Command Name	C-PHASE Parameters: C	H	R	N	EOT	SC	GSL	GPL	DTL	D	STP	NCN
SCAN EQUAL	○	○	○	○	○	○	○	○			○	
SCAN LOW OR EQUAL	○	○	○	○	○	○	○	○			○	
SCAN HIGH OR EQUAL	○	○	○	○	○	○	○	○			○	
SEEK												○
RECALIBRATE												○
SENSE INTERRUPT STATUS												
SENSE DEVICE STATUS												
SPECIFY												
SET STANDBY												
RESET STANDBY												
SOFTWARE RESET												

Floppy Disk Drive Sample Program

6 Sample Program

As a sample program for FDC access, I created a program that reads a floppy disk. The program takes the block number as a parameter, converts the block number to track (cylinder), head, and sector numbers, and reads them. When starting access, because the rotation of the engine stops when the command to start the monitor is given and it cannot be in the drive ready state, this program repeats the SENSE DRIVE STATUS command until the drive's READY signal becomes '1'.

Additionally, since this program does not use interrupts, FDC interrupts are prohibited. If an interrupt occurs, the interrupt processing of Human68K will be executed, and it will no longer be possible to proceed.

List 1 Floppy Disk Read

/*

- FDC Access Test
- Since volatile is not supported in XC,
- Please invalid volatile by putting the following line
- #define volatile */

#include <doslib.h>

struct DMAREG {

```
    unsigned char csr;
    unsigned char cer;
    unsigned short spare1;
    unsigned char dcr;
    unsigned char ocr;
    unsigned char scr;
    unsigned char ccr;
    unsigned short spare2;
    unsigned short mtc;
    unsigned char *mar;
    unsigned long spare3;
    unsigned char *dar;
    unsigned short spare4;
```

};

Here's a transcribed version of the code:

unsigned short btc; unsigned char *bar; unsigned long spare5; unsigned char spare6; unsigned char niv; unsigned char spare7; unsigned char eiv; unsigned char spare8; unsigned

char mfc; unsigned short spare9; unsigned char spare10; unsigned char cpr; unsigned short spare11; unsigned char spare12; unsigned char dfc; unsigned long spare13; unsigned short spare14; unsigned char spare15; unsigned char bfc; unsigned long spare16; unsigned char spare17; unsigned char gcr; };

volatile struct DMAREG *dma;

volatile unsigned char *fdc_stat = (unsigned char *)0xe94001; volatile unsigned char *fdc_data = (unsigned char *)0xe94003; volatile unsigned char *fdd_sel = (unsigned char *)0xe94007; volatile unsigned char *int_stat = (unsigned char *)0xe9c001;

#define BUFSIZE 0x400 unsigned char diskbuf[BUFSIZE];

void main(); void fd_wait_ready(); void fd_seek(); void motor_on(); void motor_off(); unsigned int fdc_sense_int_stat(); void fdc_int_mask(); void fdc_send_command(); void fdc_read_status(); void fdc_send(); unsigned int fdc_read();

"Floppy Disk Drive"

1. `printf("\n");`
    - This line prints a newline character to the console.
2. `motor_off();`
    - This calls the function to turn off the motor.
3. `fdc_int_umask();`
    - This calls the function to unmask the floppy disk controller interrupt.
4. `void fd_wait_ready()`
    - This defines a function `fd_wait_ready` with no return type and no parameters.

```
do {
   fdc_send(0x04);
   fdc_send(0x00);
} while((fdc_read() & 0x20) == 0);
- This is a do-while loop that sends commands to the floppy disk controller and waits
 ↪  until the controller is ready by checking a specific bit in the status register.
```

6. `void fd_seek(track)`
    - This defines a function `fd_seek` that takes an unsigned integer `track` as a parameter.

```
unsigned int track;
fdc_send(0x0f);
fdc_send(0x00);
fdc_send(track);
fdc_sense_int_stat();
- These lines send seek commands to the floppy disk controller specifying the track and
 ↪  get the interrupt status.
```

8. `unsigned int fdc_sense_int_stat()`
    - This defines a function `fdc_sense_int_stat` that returns an unsigned integer.

```
unsigned int stat;
while(!(*int_stat & 0x80))
    ;
fdc_send(0x08);
printf("Interrupt Status = ");
printf("%02X", stat = fdc_read());
printf("%02X\n", fdc_read());
```

```
return(stat);
- This function reads the interrupt status from the floppy disk controller, prints it to
↪  the console, and returns the status.
```

10. `void motor_on()`
    - This defines a function `motor_on` with no return type and no parameters.

```
*fdd_sel = 0x80;
- This line sets a specific bit to turn the floppy disk drive motor on.
```

11. `void motor_off()`
    - This defines a function `motor_off` with no return type and no parameters.

```
*fdd_sel = 0x00;
- This line resets the specific bit to turn the floppy disk drive motor off.
```

12. `426`
    - This could be a page number or a reference.

Floppy Disk Drive

void fdc_int_mask() { *int_stat &= 0xfb; }

fdc_int_umask() { *int_stat |= 0x4; }

void fdc_send_command(trk, head, sect) unsigned int trk, head, sect; {

```
    fdc_send(0x46);      /* Command            */
    fdc_send(head <<2);  /* HD/US1/US0         */
    fdc_send(trk);       /* Cylinder           */
    fdc_send(head);      /* Head               */
    fdc_send(sect);      /* Record(Sector)     */
    fdc_send(0x03);      /* Num(Block Length)  */
    fdc_send(0x08);      /* EOT                */
    fdc_send(0x35);      /* GSL                */
    fdc_send(0x00);      /* DTL(Not Used)      */
```

}

void fdc_read_status() {

```
    unsigned int   i;
    for (i = 0; i<0x7; i++)
        printf(" %02X", fdc_read());
    printf("\n");
```

}

void fdc_send(dat) unsigned int dat; {

```
    unsigned int   stat;
    printf("Send:-%02X\n", dat);
    while((*fdc_stat & 0xc0) != 0x80)
        ;
    *fdc_data = dat;
```

}

unsigned int fdc_read() {

```
    while((*fdc_stat & 0xc0) != 0xc0)
        ;
```

return(*fdc_data); }

void dma_setup() {

```
    dma->dcr = 0x80;
    dma->ocr = 0xb2;
    dma->scr = 0x04;
    dma->ccr = 0x00;
    dma->cpr = 0x08;
    dma->mfc = 0x05;
    dma->dfc = 0x05;

    dma->mtc = BUFSIZE;
    dma->mar = diskbuf;
    dma->dar = (unsigned char *)fdc_data;
}
void dma_start() { dma->ccr |= 0x80; }
void wait_complete() {
    while(!(dma->csr & 0x90))
        ;
}
void clear_flag() { dma->csr = 0xff; }
```

This is already in English and is source code for Direct Memory Access (DMA) setup and control.

SASI

Since the first generation, the SASI interface has been adopted as a hard disk interface and has become relatively simple without requiring a dedicated LSI. Here, we will explain how to handle the SASI bus and commands to the SASI disk.

1. Overview of the SASI Bus

The SASI (Shugart Associates System Interface) was designed by Shugart Associates in the United States as a hard disk interface. It has been used as an interface for hard disks attached to personal computers, but recently, as SASI has become standardized by ANSI (American National Standard Institute) into the SCSI bus, it has been replaced. Even in the X68000 series, the initial models used the SASI interface for external hard disk interfaces, but models like SUPER and XVI have been changed to SCSI.

1.1 Structure of SASI Disks

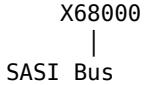
On the SASI bus, a maximum of 8 controllers can be connected to the bus, each with a fixed number (ID). Also, in SASI commands, separate from each controller ID...

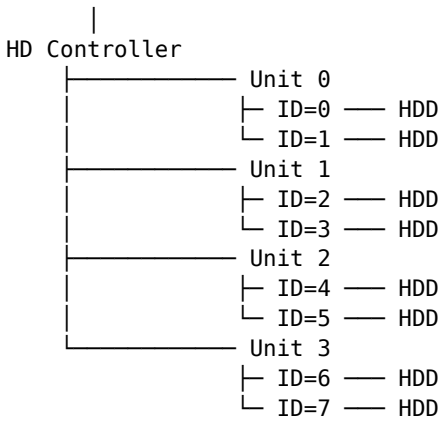
(Page 429)

The logical unit number arrangement allows for up to 8 devices to be connected under each controller, but theoretically up to 64 devices can be connected. However, most controllers available on the market support only logical unit numbers 0 to 7. Even with the Human 68K, the number of devices that can actually be used is limited to 16.

This is illustrated in Figure 1 for SASI hard disk connection. Additionally, since the internal disk uses one ID internally, 14 devices can be connected externally. Thus, a total of 15 disks can be used both internally and externally.

■Figure 1 Connection of SASI hard disks





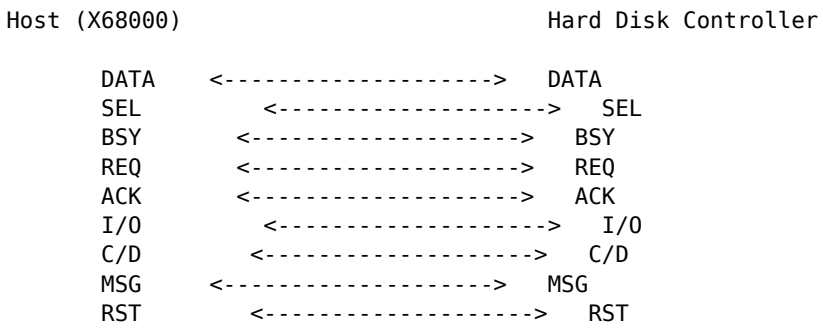
SASI

In access via the SASI bus, the host (in this case, the X68000) reads/writes to the device in units of blocks of a certain size. The block number, assigned in sequential order from the beginning of the disk, indicates which block the access is to be performed. As the block number is converted to sector numbers or track numbers, etc., on the hard disk controller, the host simply specifies the block number to be accessed without needing to worry about how the data is recorded on the actual disk. The block size for the X68000 SASI device is 256 bytes.

○2 SASI Bus Signals

The SASI bus signals are shown in Figure 2. The SASI bus is comprised of an 8-bit data bus and 9 control signals. The SASI bus signals are active-low signals, meaning that when the X68000 side sets to '1', the bus is at 'Low' level, and when set to '0', the bus is at 'High' level. The meanings of each signal are as follows:

SASI Bus Signals



'H' Level: 2 - 5.25 V 'L' Level: 0 - 0.8 V

The labels and diagram indicate the functions and connections of the control signals between the host (X68000) and the hard disk controller.

1-2-1 DATA (Data Bus)

The 8-bit DATA lines are used to perform commands, data, status exchanges, etc., between the host and the controller.

1-2-2 SEL (Select)

The host uses this to determine which controller among the ones connected will be accessed.

1-2-3 BSY (BUSY)

A signal indicating that the SASI bus is currently in use, output by the controller side. The selected controller by the host sets the BSY signal to '1' (Low level), and it continues the BUSY state until the command processing completes.

1-2-4 REQ (Request)

A signal that the controller sends to request data transfer from the host. Usually '0' (High level), but becomes '1' (Low level) upon request. The host responds with data reads/writes after the REQ signal.

1-2-5 ACK (Acknowledge)

A signal the host uses to indicate acknowledgment of the REQ signal. If the request is for a read operation, after pulling data from the DATA line, or for a write, after setting data on the DATA line, the host sets 'ACK' to '1' (Low level). Then, the controller sets REQ to '0', ending the data transfer session. This method is also referred to as REQ-ACK handshaking.

Page 432

1-2-6 I/O (Input/Output)

Used to indicate the direction of the DATA line (whether it is requesting data retrieval or data writing) from the controller to the host. When '1' (Low level), it indicates direction from the controller to the host (Input). When '0' (High level), it indicates direction from the host to the controller (Output).

1-2-7 C/D (Command/Data)

Indicates whether the content of the DATA line is data or a command/status. When '0' (High level), it indicates data.

1-2-8 MSG (Message)

Combined with I/O and C/D lines, it indicates that the content of the DATA line is a message byte. The message byte is transmitted at the end of the bus operation. Therefore, this signal of '1' (Low level) can be considered to indicate the last cycle of the bus operation.

1-2-9 RST (Reset)

A signal used by the host to initialize the SASI bus. When this signal is '1' (Low level), the controller on the SASI bus is reset entirely. Even during data transfer, it is forcibly reset, so please be very careful when using it.

1-3 SASI Bus Phase Transition

The SASI bus operates while transitioning through several bus states. Each of these states is called a phase. The basic phase transitions of SASI are as shown in the diagram on page 434.

Page 433

● Figure 3 SASI Bus Transition Diagram

RESET ——→

[Bus Free Phase] I/O = 0 C/D = 0

MSG = 0
  ↓

[Selection Phase] I/O = 0 C/D = 0

```
MSG = 0
  ↓
```

[Command Phase] I/O = 0 C/D = 1

```
MSG = 0
  ↓   (When data transfer is not required → goes to Status Phase)
```

[Data Transfer Phase] I/O = 0 (WRITE) / 1 (READ) C/D = 0

```
MSG = 0
  ↓
```

[Status Phase] I/O = 1 C/D = 1

```
MSG = 0
  ↓
```

[Message Phase] I/O = 1 C/D = 1

```
MSG = 1
  ↓ (returns to Bus Free Phase)
```

1-3-1 Bus Free Phase

This is the state where the bus is not in use, and bus operations start from here. After a reset, the bus will be in this state.

1-3-2 Selection Phase

This is the phase in which the host (X68000) decides which of the eight controllers on SASI to use.

1-3-3 Command Phase

This is the phase where the controller selected in the selection phase is instructed on what actions to take. For data transfer operations, such as reading/writing the status of the controller or the disk, if necessary, it will transition to the data transfer phase or the status phase.

1-3-4 Data Transfer Phase

This is the phase in which data transfer between the host and the controller takes place. The amount of data transferred is determined by the command and parameters given in the command phase.

1-3-5 Status Phase

This is where the controller informs the host of the execution result of the command. One byte of data is returned. If the operation was successful, $00 is returned, and if there was some kind of error, data other than $00 (usually $02) is returned. If the host receives something other than $00, it will retrieve the status sense using the REQUEST SENSE STATUS command.

Page: 435

1-3-6 Message Phase

The final phase of the transfer cycle. In the Message-In phase, 1 byte of data called Message Byte is sent. In general SASI devices respond with $00 (Command Complete Message) in this phase.

1-4 SASI Bus Operation

We explained the outline of the SASI bus operation, and now we will look at the actual operation of the SASI bus signals. From figure 14, the SASI bus free phase starts and, this represents an example of the SASI bus operation until it returns to the bus free phase. This figure shows the state of actual buses, showing when a signal line is '0' (bus is 'Low' level) and '1' (bus is 'High' level). Of these signals, the signals executed by the host (X68000), are SEL and ACK, and the remaining signals are executed by the peripheral controller (hard disk). Data lines are not shown.

1-4-1 Bus Free Phase

In the Bus Free Phase, all signals are '0'. The host ensures that the bus is in this state before starting the Selection Phase.

1-4-2 Selection Phase

The host sets the ID number on the bus and sets SEL signal to '1'. An ID number from 0 to 7 corresponds to one bit of the data line, and the selected controller ID number's corresponding bit is set to '1' and output to the bus. For example, if ID 0's controller is selected, a value of $01 is output to the bus, and if ID 3, a value of $08 is output. When the selection is successful, the selected controller responds by setting the BSY signal to '1'. At that point, the host returns the SEL signal to '0', completing the selection phase. The BSY signal is held until the final message phase is completed. This indicates that a current operation over the SASI bus is underway.

(Page 436)

SASI

● Figure 4 SASI Bus Operation Example

Top timing diagram phases (left to right): Bus Free Phase | Selection Phase | Command Phase (6 bytes) | Data Transfer Phase

Signals: SEL, ACK, REQ, BSY, I/O, C/D, MSG I/O during Data Transfer Phase: high = (READ), low = (WRITE)

Bottom timing diagram phases (left to right): Data Transfer Phase | Status Phase | Message Phase | Bus Free Phase

Signals: SEL, ACK, REQ, BSY, I/O, C/D, MSG

- All signals on the SASI bus are negated (active-low) logic; when up ('1') it is at the ”Low” level, and when down ('0') it is at the ”High” level.

One can determine whether it is operational or not.

When BSY is always absent, one can assume that the controller does not exist, set SEL to '0' to return, and treat it as a termination error.

In the X68000, there is a port for selection, to which data is written, and the SEL signal will automatically function after putting out the data.

1-4-3 Command Phase

When the selection is completed, the controller requests a command transfer from the controller. I/O, C/D, MSG are set to '0', '1', '0' (Output direction, Command, Not a message), and the REQ signal becomes '1' to request command transfer from the host. The host sets the command to the data line and responds to the controller with ACK. The controller receives the ACK and returns REQ to '0'. The host then clears the data and returns the signal to the original state.

Repeating this REQ-ACK handshake and sending the command's attached parameter together completes the command phase. The thickness of a basic command for SASI disks is 6 bytes, and in most cases, the REQ-ACK handshake is performed in six turns.

In the X6800, writing to the data port will automatically return the ACK signal, so there is no need to operate the ACK signal; you can simply monitor the REQ signal.

1-4-4 Data Transfer Phase

In the data transfer phase, the controller returns C/D to '0', sets I/O to '0' for writing from the host, and '1' for reading, while maintaining MSG as '0'. With the signal settings intact, the required amount of REQ-ACK handshakes are repeated to transfer the data. Like in the command phase, access to the data port will automatically carry out the handshake.

Data transfers can be done not only one byte at a time by the CPU but also via DMA. The sample program presented later performs data transfers in the data transfer phase using DMA, so please refer to it.

5 Status Phase

The status phase becomes C/D, I/O, and 1 respectively and sends 1 byte of status byte. The host retrieves this data using the REQ-ACK handshake.

6 Message Phase

The controller indicates the end of the status phase by setting the MSG signal to 1, which signifies the start of the message phase and requests the retrieval of the message byte. The host retrieves this data using the REQ-ACK handshake. This indicates the end of the series of bus operations. The controller combines BSY as 0, sets all signals to 0, and returns to the bus-free phase.

7 SASI Interface Port List

The I/O port for controlling the SASI bus is shown in Figure 5.

$E96001 is the SASI data read/write port. When performing access to this port, the automatic REQ-ACK handshake is performed. This can save time by only checking the REQ signal while the ACK signal is operating.

In the command phase transfer phase, it uses the DMA channel to perform transfer.

When the DMA request signal becomes REQ=1 (active), it returns when the ACK signal becomes ACK=1 (acknowledged).

Figure 5: List of SASI Interface Port Addresses

Address		bit 7	bit 6	bit 5	bit 4	bit 3	bit 2
$E96001	R/W				DATA	SASI Data Output	
$E96005	R/W			MSG C/D I/O	BSY REQ	SASI Status Output	SEL signal is not asserted (Data output onl
$E96006	W				DATA		
$E96007	W						

Note: The content retains the technical computer terminology and phrases related to SASI interface operations and instructions as they are crucial for accurate understanding and application in a technical context.

You can confirm the control number status of SASI for $E96003 number with reading. By reading this port, you can know the current phase.

The $E96003 and $E96007 numbered writing ports are ports created for selection phase use, and when data is written into them and the data is output to SASI bus, it is always set so that SEL signal is '0.' For $E96003 numbered writing port data, it is usually 00H.

For $E96005 numbered writing port, it waits for a certain period of a SASI bus RESET signal and writes to this port, which resets the hard disk connected to the SASI. Since the read/write light of the disk takes strenuous effort to reset, please be careful when using it.

0.6 SASI Commands

Commands used with SASI disks are of 6 bytes of data. These 6 bytes of data are serially sent to the controller in order.

The first byte indicates the operation code to indicate what kind of work the command should perform. The operation code is divided into the upper and lower 3 bits, with the upper 3 bits indicating the generic usage of the command and whether it is a manufacturer-specific command. Regular commands are designed with Class 0 and the upper 3 bits are '0'.

The next 3-bit in the upper half of the byte is the logic unit number. With 3 bits, you can handle up to 8 units using one ID, but it is usually used as a logic unit number. The upper 2 bits are '0.'

Figure ... 6 General format of SASI command | Transfer Order | bit 6 | bit 5 | bit 4 | bit 3 | bit 2 | bit 1 | bit 0 | Remarks | | ---------------|------|------|------|------|------|------|------|----------| | 0 | Command Class | Command Code | Operation Code | | 1 | Logical Unit Number (Upper) | Logical Address (Upper) | Unit Number/ Start address | | 2 | Logical Address (Middle) | | 3 | Logical Address (Lower) | | 4 | Sector Block Count | Access Block Count | | 5 | Control Byte | usually '00' OK |

Note that the table was converted into text format for clarity.

The number of logical addresses and sector blocks is effective when reading or writing data, specifying the block number to start reading or writing, and the number of blocks to read or write. The size of one block is 68000, which is 256 bytes.

The last control byte is used by SCSI for link commands to execute multiple commands consecutively, but since this varies with manufacturers, it is not used in SASI, and it is fine to leave it as $00.

1.7 Major Commands in SASI

SASI disk commands, aside from basic commands such as reading and writing disks, also have several manufacturer-specific commands and flags added uniquely. These proprietary commands have been omitted here since they are only known by each respective manufacturer, but we will explain the main commands that should be known. Illustrated in the table below are the main commands:

In diagrams 8 and later, bits marked with OFF or shaded in black are mostly unused; manufacturers may have added their own unique functions (e.g., using them to decide on the block number to start formatting from), so leave everything as '0'.

- Figure 7 Major Commands of SASI

Operation Code	Command Class	Command Code	Command Name	Notes
$00	0	$0	TEST DRIVE READY	Check if the drive is ready
$01	0	$1	RECALIBRATE	Move the head to the track 0
$03	0	$3	REQUEST SENSE STATUS	Retrieve error status data
$04	0	$4	FORMAT DRIVE	Perform physical formatting of the dri
$08	0	$8	READ	Retrieve data
$0A	0	$A	WRITE	Write data

1-7-1 TEST DRIVE READY Command

The format of the command is as shown in Figure 8. This command checks whether the disk is in a ready state (operable state). Since there is no data transfer, the command is sent and then transitions to the status phase. If the disk is ready, $00 is returned in the status phase (the value returned when the disk is not in a ready state varies depending on the manufacturer).

Figure 8: TEST DRIVE READY Command

Transmission Order	bit 7	6	5	4	3	2	1	bit 0	Remarks
0	0	0	0	0	0	0	0	0	Operation Code: $00
Logical Unit Number									

1-7-2 RECALIBRATE Command

This command returns the head of the disk to track 0. The format of this command is as shown in Figure 9. In older hard disks, head positioning at track 0 was hardware-defined, with a bit of mechanical movement enforced when moving the head to any regular track position. However, if the head position becomes misaligned even slightly after some operational wear and tear, it leads to errors. This command allows for the periodic return of the head to track 0 for re-calibration, which helps mitigate such positioning errors. Modern hard disks automatically adjust the head position, hence the practical significance of this command has diminished. It is now used mainly for moving the head to track 0.

Figure 9 RE-CALIBRATE Command

Transfer Order	bit 7	6	5	4	3	2	1	bit 0	Remarks
0	0	0	0	0	0	0	0	1	Operation Code: S01
1	Key Unit Number								
2									
3									
4									
5									

1.7.3 REQUEST SENSE STATUS Command

The command format is as shown in Figure 10. When an error occurs (when bit 1 of the status phase data becomes '1'), the host sends this command and retrieves 4 bytes of status from the data phase. After an error occurs and this command is issued, data that is stuck in the status phase without being reset until the error is cleared will normally be returned when frozen. The sense byte format is shown in Figure 11 on page 444. Usually, at the forefront, it is required to judge whether the error content and processing address content are effective or not, but the content of this $00 indicates no error, while anything other than that varies from manufacturer to manufacturer.

Figure 10 REQUEST SENSE STATUS Command

Transfer Order	bit 7	6	5	4	3	2	1	bit 0	Remarks
0	0	0	0	0	0	0	1	1	Operation Code: S03
1	Key Unit Number								
2									
3									
4									
5									

Page 443

FORMAT DRIVE Command

The FORMAT DRIVE command formats the disk physically. The format of the command is shown in Figure 12. After issuing the command, all format processing is done on the controller side, so there is nothing to do until the host transfers to the status phase.

Figure 12. FORMAT DRIVE Command

Transfer Sequence	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0	Remarks
0	0	0	0	0	0	1	0	0	Operation Code: $04
1	Logical Unit Number								
2									
3									
4									
5									

READ Command

The READ command reads data from the disk. It specifies the block number and number of blocks to start reading. On the X 68000, the block size is 256 bytes. The format of the command is as follows (see Figure 13).

Figure 13. Read Command Format

Transfer Sequence	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0	Remarks
0	Logical Unit Number								
1									
2									

Figure 13 READ Command

Transfer Order	bit 7	6	5	4	3	2	1	bit
0	'0'	'0'	'0'	'0'	'0'	'0'	'1'	'0'
1	Logical Unit Number	Logical Address (Upper)	Read Start Block Number					
2	Logical Address (Middle)	Read Start Block Number						
3	Logical Address (Lower)	Read Start Block Number						
4	Sector Block Count	Read Block Count						
5	'0'	'0'						

1.6 WRITE Command

The command format is as shown in Figure 14. By specifying the leading block number and block count, we write to the disk. The controller does not perform disk writing until the data for at least one sector is complete, so there are no issues even if data transfer is delayed.

Figure 14 WRITE Command

Transfer Order	bit 7	6	5	4	3	2	1	b
0	'0'	'0'	'0'	'0'	'1'	'0'	'1'	'0
1	Logical Unit Number	Logical Address (Upper)	Write Start Block Number					
2	Logical Address (Middle)	Write Start Block Number						
3	Logical Address (Lower)	Write Start Block Number						
4	Sector Block Count	Write Block Count						
5	'0'	'0'						

●2 Sample Program

Here is a sample program for reading a SASI disk. Please refer to it.

When you specify a block number with a parameter at startup, the contents of that block (256 bytes) will be displayed. If you specify a block number that is too large, the program will stop.

In this sample, the data transfer phase is done via DMA transfer, but the DMA transfer at this time is set so that the transfer starts with only the first byte (dummy external data transfer request). We thought it might be better to use an external data transfer request, but when we actually did it, the transfer often stopped, so we changed the mode. It's transferred without taking into account the REQ signal's status, so we make sure the CPU confirms that the REQ signal is ”1” before starting the DMA. List 1 SASI Disk Readout

```
/*
 * SASI Hard Disk Access Test
 *
 * Because XC does not support volatile,
 * please disable volatile in the next line.
 * #define volatile
 */

#include <doslib.h>

struct DMAREG {
    unsigned char  csr;
    unsigned char  cer;
    unsigned short spare1;
    unsigned char  dcr;
    unsigned char  ocr;
    unsigned char  scr;
    unsigned char  ccr;
    unsigned short spare2;
};

unsigned short mtc; unsigned char *mar; unsigned long spare3; unsigned char *dar; unsigned short spare4; unsigned short btc; unsigned char *bar; unsigned long spare5; unsigned char spare6; unsigned char niv; unsigned char spare7; unsigned char eiv; unsigned char spare8: unsigned char mfc; unsigned short spare9; unsigned char spare10;
```

unsigned char cpr; unsigned short spare11; unsigned char spare12; unsigned char dfc; unsigned long spare13; unsigned short spare14; unsigned char spare15; unsigned char bfc; unsigned long spare16; unsigned char spare17; unsigned char gcr; }; volatile struct DMAREG *dma; volatile unsigned char *sasi_data; volatile unsigned char *sasi_status; volatile unsigned char *sasi_sel_off; volatile unsigned char *sasi_reset; volatile unsigned char *sasi_sel_on; #define BUFSIZE 0x100 unsigned char diskbuf[BUFSIZE]; #define BUSFREE_PHASE 0x00 #define SELECTION_PHASE 0x02

#define COMMAND_PHASE 0x0a #define DATA_READ_PHASE 0x06 #define STATUS_PHASE 0x0e #define MESSAGE_PHASE 0x1e

#define REQ_BIT 0x01

void main(); void sasi_select(); void sasi_send_command(); void sasi_send_a_byte(); unsigned int sasi_get_status(); unsigned int sasi_get_message(); void wait_sasi_status(); void dma_setup(); void dma_start(); void dma_stop(); void wait_complete(); void clear_flag();

void main(argc, argv) int argc; char *argv[]; {

```
  unsigned int    i, j, id, blk_no, blk_h, blk_m, blk_l;
  unsigned char   c;
  if (argc >= 2)
    blk_no = atoi(argv[1]);
  else  blk_no = 0;
  if (argc >= 3)
    id = atoi(argv[2]);
  else id = 0;
  blk_l = blk_no & 0xff;
  blk_m = (blk_no >> 8) & 0xff;
  blk_h = (blk_no >> 16) & 0x1f;
  printf("Block# = %d(%06X){%02X:%02X:%02X} Drive = %d\n",
    blk_no, blk_no, blk_h, blk_m, blk_l, id);
  for (i=0; i<BUFISIZE; i++)
    diskbuf[i] = 0;
```

}

SUPER(0):

```
    dma      = (struct DMAREG *)0xe84040;
    sasi_data = (unsigned char *)0xe96001;
    sasi_status = (unsigned char *)0xe96003;
    sasi_sel_off = (unsigned char *)0xe96003;
    sasi_reset = (unsigned char *)0xe96005;
    sasi_sel_on = (unsigned char *)0xe96007;
```

clear_flag(); dma_setup(); sasi_select(id); sasi_send_command(8,blk_h,blk_m,blk_l,1,0); wait_sasi_status(DATA_READ_PHASE | REQ_BIT); dma_start(); wait_complete(); clear_flag(); printf("STATUS = "); printf("%02X\n",sasi_get_status()); printf("MESSAGE = "); printf("%02X\n",sasi_get_message()); for(i=0;i<BUFSIZE;i+=0x10) {

```
    for(j=0;j<0x10;j++)
        printf("%02X ",diskbuf[i+j]);
    for(j=0;j<0x10;j++) {
        c = diskbuf[i+j];
        if((c < 0x20) ||(c >= 0xe0)||((c >= 0x80) && (c < 0xa0)))
            printf(".");
        else printf("%c",diskbuf[i+j]);
    }
    printf("\n");
```

}

void sasi_select(id) unsigned int id; {

```
    unsigned int stat;
    if(stat = *sasi_status) {
        printf("SASI stat = %d\n",stat);
        exit(1);
    }
    *sasi_sel_on = 1 << id;
}
wait_sasi_status(SELECTION_PHASE); *sasi_sel_off = 0; }
void sasi_send_command(p1,p2,p3,p4,p5,p6) unsigned int p1,p2,p3,p4,p5,p6; {
    sasi_send_a_byte(p1);
    sasi_send_a_byte(p2);
    sasi_send_a_byte(p3);
    sasi_send_a_byte(p4);
    sasi_send_a_byte(p5);
    sasi_send_a_byte(p6);
}
void sasi_send_a_byte(dat) unsigned int dat; {
    wait_sasi_status(COMMAND_PHASE | REQ_BIT);
    *sasi_data = dat;
}
unsigned int sasi_get_status() {
    wait_sasi_status(STATUS_PHASE | REQ_BIT);
    return((unsigned int)*sasi_data);
}
unsigned int sasi_get_message() {
    wait_sasi_status(MESSAGE_PHASE | REQ_BIT);
    return((unsigned int)*sasi_data);
}
void wait_sasi_status(dat) unsigned int dat; {
    while(*sasi_status != dat)
        ;
}
void dma_setup() {
    dma->dcr = 0x80;
    dma->ocr = 0xb3;
    dma->scr = 0x04;
    dma->ccr = 0x00;
    dma->cpr = 0x08;
    dma->mfc = 0x05;
    dma->dfc = 0x05;
    dma->mtc = BUFSIZE;
    dma->mar = diskbuf;
    dma->dar = (unsigned char *)sasi_data;
}
void dma_start() { dma->ccr |= 0x80; }
void wait_complete() {
```

```
    while(!(dma->csr & 0x90))
    ;
}
```

void clear_flag() { dma->csr = 0xff; }

(Blank page in the original.)

SCSI

The SCSI interface is expected to support new devices such as CD-ROMs, which are included in the SUPER series. Here, the handling of the SCSI controller LSI and commands for SCSI disks will be explained.

1 Overview of SCSI

The SCSI interface is an ANSI-standardized version of SASI, which supports multiple host environments and command functionality enhancements. The main differences between SCSI and SASI are as follows:

- Supports configurations with multiple hosts
- Added the function to disconnect (disconnect) the bus during long command processing and reconnect it later
- Added an arbitration phase to determine bus usage and a selection phase for reconnection
- Added the ability to send messages from the initiator to the target
- Standardized the values sent in the message phase and status phase
- Added the command linking function to indicate the continuation of multiple commands
- Extended format for Read/Write commands and Sense commands

These changes have led to several modifications in terminology. The major changes include renaming the host and controller from SASI to initiator and target, and the fact that message transmission is now bidirectional. Consequently, the message transmission phase from the initiator to the target is now called the message-out phase, and the previous SASI message-in phase has been renamed to message-in phase.

0.1 Composition of the SCSI Bus

The SCSI bus, like SASI, can connect a maximum of 8 devices. However, with SCSI, an initiator (such as the X68000 main unit) requires an ID, and thus only 7 IDs remain available for connecting to the bus. IDs from 0 to 7 are used on the SCSI bus, with the X68000 main unit typically using ID #7.

SCSI uses SASI-like IDs but includes a logical unit number (LUN) system, allowing each ID to manage up to 8 system units. Additionally, extensions like the EXTENDED IDENTIFY message can expand each ID to handle up to 2048 units, although typical SCSI drivers are generally designed to deal with fewer units due to practical constraints (only the command code and unit number are handled), limiting the number of devices connected to the SCSI bus to 7.

An example of the SCSI bus disk connection is shown in Figure 1. The block size for accessing has remained fixed at 256 bytes for SASI and has been somewhat standardized to 256, 512, or 1024 bytes for SCSI drives.

0.2 SCSI Bus Signals

The SCSI bus signals are illustrated in Figure 2 on page 456. In addition to the signals provided by SASI, there are additional signals such as ATN (Attention) and DP (data parity).

While SCSI devices may use parity for data transfers, the X68000's SCSI driver, for instance, implements a controller parity table that allows the user to choose whether or not to use parity when programming. Thus, devices connected to the bus do not necessarily have to use parity.

Diagram 1: Connection of SCSI Disks

[Diagram showing the connection of various HDDs to the X68000 via HD controller, with assigned IDs]

Title: ATN Signal

The ATN signal, added in SCSI, is used by the initiator (generally the X68000) to notify the target (such as disks) of data transfer errors or to request mode settings for operation. If the initiator sets the ATN signal, it indicates to the target that it wants to communicate certain contents. The target receives this notification and executes phase matching when in a message out phase. If the ATN signal is "1" (low level), it shows that there is something the initiator wants to notify the target. The phase that actually sends the notification content is called the message-out phase. When the target is matched, it moves to the message-out phase to receive notifications from the initiator.

Page 455.

Fig ..... 2 SCSI Bus Signals

Initiator Target |------->-----| DATA |-----<------| |------->-----| DP |-----<------| |------->-----| SEL |-----<------| |------->-----| BSY |-----<------| |------->-----| REQ |-----<------| |------->-----| ACK |-----<------| |------->-----| I/O |-----<------| |------->-----| C/D |-----<------| |------->-----| MSG |-----<------| |------->-----| ATN |-----<------| |------->-----| RST |-----<------|

1.3 SCSI Bus Phase Transition

The phases identified during the transition from SASI to SCSI are: the arbitration phase which determines bus usage rights, the message out phase activated by the ATN signal, and the three selection phases that require initiator re-selection from the target; these three phases are relatively straightforward to organize, thus they can generally be represented in forms such as in Figure 3 below.

Page 456

Diagram 3: SCSI Bus Phase Transition Diagram (from the specification)

- Reset
- Bus Free Phase
- Arbitration Phase
- Selection Phase / Reselection Phase
- Information Transfer Phase

```
- Command Phase
- Data In Phase
- Data Out Phase
- Status Phase
- Message In Phase
- Message Out Phase
```

Because it is hard to understand with just this, I will provide an example of specific phase transitions with disk reading as shown in Figure 4 on page 458. Let's try to explain the bus transitions a little simply with this as an example. In this example, after receiving a command, the bus is once cut off and reconnected when data to be read out is identified, realizing a somewhat SCSI-like operation.

Page 457

● Diagram 4 SCSI Bus Operation Example (Disk Read)

Bus-Free Phase

Arbitration Phase Obtain bus usage rights for initiator

Selection Phase Initiator selects target (sets ATN to “1”)

Message Out Phase Send IDENTIFY message

Command Phase Send READ command

Message In Phase Target sends DISCONNECT message and releases the bus

Bus-Free Phase

Arbitration Phase Target acquires bus usage rights

Reselection Phase Target reselects initiator

Message In Phase Target sends IDENTIFY message to initiator

Data In Phase Initiator receives data

Status Phase Receive status of command execution results

Message In Phase Receive command completion message

SCSI

1-3-1 Bus Free Phase

The bus is in an unused state. Bus operations start here.

1-3-2 Arbitration Phase

The use rights of the bus are adjusted. Those who want to use the bus assert their ID numbers onto the data bus, and the one with the highest priority (the higher the number, usually either ID 7 or 68000 body) obtains the use right of the bus. Those that lose go back to the Bus Free Phase until the next opportunity.

1-3-3 Selection Phase

The initiator that gained the bus use rights selects the target during the Selection Phase. Although this is similar to the SASI Selection Phase, the difference is that on the data bus, not only is the target ID but also the initiator's ID set to '1'. This allows the target to know who it will be connected to later during the Selection Phase.

Additionally, in this example, ATN (Attention) is set to '1' during the Selection Phase. Setting ATN to '1' is optional and can be left at '0', but in this example, we use ATN to signal the target to transition to the next phase known as the Message Out Phase to inform the target that we will perform a Disk Request operation.

1-3-4 Message Out Phase

When ATN was set to '1' during the Selection Phase and a target is selected, the target switches to the Message Out Phase before transitioning to the Command Phase and receives the initiator’s message.

The initiator sends an IDENTIFY message here, which makes the Disk Request operation effective.

1-3-5 Command Phase

After receiving a message from the initiator, the target then moves to the command phase and receives a command from the initiator. The operation in this phase is the same as in SASI.

1-3-6 Messaging Phase

In SASI, the target remains BUSY (BSY signal is 1) until the actual data transfer begins. However, in practice, the target moves to the messaging phase after receiving a READ command and before the data is fully gathered, then disconnects and moves to the bus free phase.

In the messaging phase (called the messaging phase in SASI), the target sends a DISCONNECT message to the initiator, notifying a temporary disconnect. Afterwards, the initiator and target release the bus usage rights, moving from the SCSI bus to the bus free phase.

1-3-7 Bus Free Phase

There is no difference from the initial bus free phase. Internally, the initiator waits for a reselect from the target, and the target reads data and prepares to transfer it to the initiator.

1-3-8 Arbitration Phase

The target that has prepared the data participates in the arbitration phase to acquire bus usage rights. If it fails, it waits until the next bus free phase.

1-3-9 Selection Phase

The target that has acquired the bus usage rights enters the selection phase and reconnects with the initiator (if you enter the selection phase instead of reselection, you would end up acting as the initiator). The ID the initiator passed in the initial selection phase is necessary in this phase.

1-3-10 Message In Phase

The target sends an IDENTIFY message to the initiator. By this identifier number in the message, the initiator can know which logical unit's processing result to receive.

1-3-11 Data In Phase

The target sends the data read from the disk to the initiator. The data transfer is conducted in the same way as SASI, with REQ-ACK handshaking.

1-3-12 Status Phase

As with SASI, the status of the command execution result is received. Apart from regular statuses like $00, which were handled arbitrarily by manufacturers, SCSI specifies status numbers and content.

1-3-13 Message In Phase

This phase is the same as the message-in phase in SASI. Normally, message $00 (COMMAND COMPLETE) is sent. This ends command processing, and the SCSI bus returns to the bus free phase.

Note: The numbers such as 1-3-9, 1-3-10, etc., seem to be section headers for different phases in the SCSI protocol described in the document.

Compared to the time of SASI, it may seem more cumbersome, but this is because it uses the Disconnect/Reconnect function. With SCSI, it is possible to perform actions using this, and aside from the Arbitration Phase before the Selection, the operations are almost

the same as the time of SASI. In the sample programs to be introduced later, the Disconnect/Reconnect function is not used for simplicity.

1.4 SCSI Bus Operation

The movements of the signals from phase transition have been traced in Figures 5 and 6. Almost identical to SASI, here you should look at the respective phases of the added Arbitration, Message In, and Selection.

Note: SASI (Shugart Associates System Interface) is a precursor to SCSI (Small Computer System Interface).

SCSI

● Figure 5 SCSI Bus Operation Example (Part 1)

Top timing diagram phases (left to right): Bus Free Phase | Arbitration Phase (2.2 μs) | Selection Phase | Message Out Phase (IDENTIFY)

Note (pointing at BSY transition): All participants in the arbitration lower BSY.

Bottom timing diagram phases (left to right): Message Out Phase | Command Phase | Message In Phase (DISCONNECT) | Bus Free Phase

Signals: ATN, SEL, ACK, REQ, BSY, I/O, C/D, MSG

Figure 6 SCSI Bus Operation Example (Part 2)

Pass-Free Phase → Arbitration Phase → Selection Phase → Message-In Phase

Message-In Phase → Data-In Phase → Status Phase → Message-In Phase (Command Complete) → Pass-Free Phase

1.4 Arbitration Phase

The arbitration phase starts when the BSY signal is '1' (low). The device that wants to participate in arbitration sets its own ID bit on the data line and starts the arbitration. After the BSY signal becomes '1', within 1.8 microseconds, the device sets its ID bit to '1'. After 2.2 microseconds from when BSY becomes '1', the data line is read. The ID with higher priority than its own ID...

(Page 464)

When the high-order bit is not set to '1', the device obtains the use right of the bus. The priority is fixed, with a higher ID #7 and a lower ID #0. The default value of the SCSI driver ID of X 68000 is set to #7.

1.4.2 Message Out Phase

I/O, C/D, and MSG each become '0', '1', '1'. When in the Message In Phase, I/O becomes the opposite. The handling of message data is performed by the REQ-ACK handshake, which does not change from SASI.

1.4.3 Reselection Phase

In the reselection phase, I/O, C/D, and MSG each become '0', '0', '0'. During the reselection phase, the direction of the data bus goes from the target to the initiator, so the state of I/O becomes '1', '0', and the SEL signal becomes activated.

2 Overview of X68000 SCSI Interface

The X68000 SCSI interface has two types: one that corresponds with an optional board and one that is built-in as standard. These use LSI (SPC: SCSI Protocol Controller), but the

port address and encoding are different, so it is possible to attach an interface board to the SCSI built-in model.

Moreover, in order to accommodate SCSI, there is additional information added for SRAM, which was previously unused. This section describes the differences in functions, new information, and more.

2-1 SCSI Related Ports and Allocation

The port address and allocation for the SCSI interface of the X 68000 are as shown in Figure 7. Among them, the SCSI-ROM refers to the ROM for booting from SCSI. The label recognizing the SCSI-ROM denotes the string indicating that it is the SCSI-ROM at that address. For the internal SCSI type, SFC0024 to 5 bytes before have the string ”SCSIIN”, and for the CZ-6BS1, the 5 bytes from $EA0044 have the string ”SCSISEX”.

2-2 Content of IPL-ROM

For X 68000 models without an internal SCSI, of the 256 KB IPL-ROM area, the first half's $FC0000 - $FDFFFF's 128 KB range is not used. The remaining $FE0000 - $FFFFFF is handled similarly as before. In models with internal SCSI, $FC0000 - $FC1FFF's 8KB is used for the IPL program area for SCSI, and the remaining $FC2000 - $FDFFFF is mostly filled with $FF. Additionally, the internal SCSI memory default value is set to 2 MB, so the content at $FF079B within the IPL-ROM is changed from $10 to $20.

2-3 SRAM Content

For SCSI compatibility, SRAMs at $ED006F, $ED0070, $ED0071 are also used. The content for this is shown in the next section.

Figure 8: Additional Information for SRAM

bit 7	
$ED006F	
$ED0070	
$ED0071	
SASI Flag: Set the bit corresponding to the ID of the SASI device to '1' when connecting the SASI disk to the SCSI Bus and re	

The $ED006F address contains a flag to indicate whether the contents of $ED0070 and $ED0071 are initialized for the first time and if they are valid, 'V' (ASCII code 0x56) is written. If 'V' is not written, the SCSI loader program will write 'V' and set $ED0070 to $07 and $ED0071 to $00.

bit 3 of $ED0070 is a flag that indicates which SCSI controller to use: Internal SCSI type (with or without an SCSI option board). '0' means internal SCSI, '1' means using an SCSI option board. Bits 0-2 of $ED0070 represent their own ID number. The behavior when loading SCSI indicates that only the internal ID number is used, and the ID position will be '0'.

The $ED0071 address is intended to consider whether a SASI disk is connected to the SCSI interface. When a SASI disk is connected, the bit corresponding to the SASI disk's ID number is set to '1'. All initial values when loading SCSI are '0', meaning no SASI disk is connected.

4 SCSI Device Media Byte

SCSI can connect not only hard disks but also optical disks and various types of devices. In response to this, the following four types are currently reserved as media bytes for SCSI

devices.

- SF7 Hard Disk

(Page 467)

/$F6 Magneto-optical disk /$F5 CD-ROM /$F4 DAT

Formal support for CD-ROM and DAT has not yet been announced (as of February 1992), but the numbers are reserved in anticipation of future support.

2.5 SCSI Device Parameters

In the first sector of a SCSI device, 16 bytes of data called the device parameters, which collect information such as whether the device can eject or not, are written. Its contents are as shown in Figure 9.

● Figure 9 Contents of the SCSI Device Parameters

Offset from start	Content	Remarks
$00	$58	
$01	$36	
$02	$38	
$03	$53	Character string ”X68SCSI1”
$04	$43	
$05	$53	
$06	$49	
$07	$31	
$08	BLEN (Upper)	Number of bytes in one sector
$09	BLEN (Lower)	
$0A	BLOCK Num (Upper)	Number of usable logical blocks
$0B	"	
$0C	"	
$0D	" (Lower)	
$0E	RW	Other than $00: SCSI extended read/write commands allowed; $00: not allowed
$0F	EJ	Other than $00: EJECT (media exchange) allowed; $00: not allowed

6. Management Information of SCSI Hard Disks

In SCSI hard disks, device parameters and partition information are written from the beginning. The contents are as shown in Table 10. In Human 68K, since the disk management unit is fixed at 1K bytes, the sector value of the table is also in units of 1K bytes. In the case of SCSI drives, when the block size of the disk is 256 bytes or 512 bytes, it is handled by summarizing 4 or 2 units respectively into 1K byte units.

Diagram 10: Management Information of SCSI Disks

Sector Number | Content S00 | SCSI Device Parameter S01 | First IPL S02 | Hard Disk Management Information S03 - S1F | SCSI Driver Reserved Area S20 | Second IPL S21 | First FAT (varies depending on the capacity) ? | Second FAT ? | Root Directory ? | Data Area

*: OS management sector (1 sector = 1K byte) varies depending on the logical format

7. SCSI Controller and DMA

Data and command transfers with SPC can use DMA, but in SCSI drives, DMA transfer is done using channel #1 of DMAC. This channel is shared with the DMA channel of the former SASI functionality, so when using both SASI and SCSI, you must be careful with DMA settings.

With the SASI interface, the SASI REQ signal is recognized by DMAC as a DREQ (DMA transfer request), DMAC then sends a DREQ signal to the CPU, which hands over the bus

usage rights and begins the transfer. This is the conventional method. However, the method used by the SCSI interface is slightly different.

SPC maintains DMA transfer requests, but for the 68000 series, it continues to connect to DMAC and DMAC programs normally in memory areas. Therefore, DMA leaves the transfer request area unchanged, but may pull unintended data, resulting in a mishap. Hence, in the SCSI interface for the 68000 series, DMA of SPC is...

The data transfer request signal is created by using the DTACK (Data Transfer Acknowledge) signal. By doing this, DMA transmission requests are kept waiting for the DMA.

The DTACK signal refers to the signal waiting state where the first side to gain access completes the transfer. In this case, typically the peripheral device accesses the CPU to prepare for the DMA. In other words, this signal is used to prevent the CPU from being forced into waiting for the DMA.

In SCSI interfaces, the SPC DMA request signal uses the DTACK signal. Once the DMA request signals have been accessed, it means they will have created waiting positions until the request arrives. However, if the CPU does not properly process the DMA request signal, around 8 microseconds (μs) will be wasted from the SPC without the DMA request signal being transmitted, causing a bus error and transmission will be interrupted.

In this way, even if the DMA has not accessed within 8 microseconds, the data transfer will be stopped if there is no data. In other words, even if just held in place, a bus error will force the stopping eventually.

3. SPC (SCSI Protocol Controller)

Due to the dedicated nature of SASI LSIs, they do not fit well with interface I/O ports. However, SCSI is ANSI standardized and the dedicated LSIs have been developed in accordance with these standards. On the X68000, the SCSI controller LSI 89352 (SPC: SCSI Protocol Controller) from Fujitsu MB is used. Such LSIs implement necessary hardware functions, reducing software bus management time and effort.

3.1 SPC Register List

The SPC register address arrangement is shown in Figure 11 on page 471. The approximate functions of these registers are as follows.

SCSI Controller (SPC, MB89352) Register List

Registers occupy odd byte offsets from the SPC base. Base address: SCSI Interface Board (CZ-6BS1) = $EA0000; internal model = $E96020.

Offset	R/W	Register	Bit layout (bit 7 → bit 0)
+$01	R/W	BDID (Bus Device ID)	ID7 ID6 ID5 ID4 ID3 ID2 ID1 ID0
+$03	R/W	SCTL (SPC Control)	Reset&Disable · Control Reset · Diag Mode · Arbitration Enable · Parity Ena
+$05	R/W	SCMD (SPC Command)	Command Code (b7–b5) · RST Out (b4) · Intercept Transfer (b3) · Program T
+$09	R	INTS (Interrupt Sense)	Selected · Reselected · Disconnected · Command Complete · Service Requir
+$0B	R	PSNS (Phase Sense)	REQ · ACK · ATN · SEL · BSY · MSG · C/D · I/O
+$0B	W	SDGC (SPC Diag Control)	Diag REQ · Diag ACK · Xfer Enable · Diag BSY · Diag MSG · Diag C/D · Diag I
+$0D	R	SSTS (SPC Status)	Connected-INIT (b7) · Connected-TARG (b6) · SPC Busy (b5) · Transfer In Pr
+$0D	W	SERR (SCSI Error Status)	Data Error (b7–b6) · — · TC Parity Error (b3) · — · Short Transfer Period (b1)
+$11	R/W	PCTL (Phase Control)	Busfree-INT Enable (b7) · unused (b6–b3) · MSG (b2) · C/D (b1) · I/O (b0)
+$13	R	MBC (Modified Byte Counter)	byte count, bits 3–0
+$15	R/W	DREG (Data Register)	8-byte FIFO
+$17	R/W	TEMP (Temporary Register)	temporary data
+$19	R/W	TCH (Transfer Counter High)	transfer counter, upper byte
+$1B	R/W	TCM (Transfer Counter Mid)	transfer counter, middle byte
+$1D	R/W	TCL (Transfer Counter Low)	transfer counter, lower byte

```
* BDID Register
  Sets/reads the ID number of the device.

* SCTL Register
  Selects the SPC operation mode and its associated functions.

* SCMD Register
  Selects operation commands or the data-transfer mode for the SPC.
```

- INTS Register
  Performs the identification of SPC interrupts and the reset of interrupt causes.
- PSNS Register
  Reads the state of the SCSI bus control signal.
- SDGC Register
  For SPC's self-diagnosis. Normally not used.
- SSTS Register
  Reads the connection status between SPC and SCSI bus and the state of SPC's internal buffer.
- SERR Register
  A status register reporting the parity error or hardware anomaly that happened in SPC.
- PCTL Register
  Indicates in which phase SPC will operate next regarding the CPU.
- MBC Register
  A counter that controls data transfer quantity between SPC's internal buffer and CPU. The initial value is set by the TCL register and is reset when the value is below four.
- DREG Register
  When transmitting data by DMA, this register is used to manage data transfer. This register is an 8-byte FIFO (First In First Out) buffer.
- TEMP Register
  Used for manual data transfer operations while SPC performs automatic data transfer by LSI. Mainly used for manual data transfer control during SCSI handshaking signals check.
  Additionally, it is used as a register to set the ID output during arbitration/selection.
- TCH/TCM/TCL (Transfer Byte Counter) Registers
  A 3-byte (24-bit) transfer byte counter. This counter increments via transfers on SCSI during hardware (manual) data transfer or SCSI transfers. Also functions as a time-out and timer setting register when there is a remainder of bytes to transfer.

Next, let's take a closer look at the details of each register.

3.2 BDID Register

The bit configuration is as shown in Figure 12. When writing, set the ID number from 0 to 7 in the lower 3 bits of the register. When reading, the data read corresponds to the ID value set, with only the bit matching the ID value being "1". For instance, if you write ID 2 as $07, the bit corresponding to ID 2 will be "1", thus reading the data $80. The data read from this register is the same as the data output to the bus during the arbitration phase.

Figure 12 BDID Register (Base Address + $01)

Bits	7	6	5	4	3	2	1	0
READ	ID #7	ID #6	ID #5	ID #4	ID #3	ID #2	ID #1	ID #
					Only the corresponding bit is "1" when the defined ID is set			
	// 4	// 5			// 1			// 2
	// 7			// 3		// 6		

| |-------|-------|-------|-------|-------|-------|-------|-------| | | ID | | WRITE | | | | | | | | | | 111: My ID is 7 | | | | | | | | | 110: // 6 | | | | | | | | | | 101: // 5 | | | | | | | | | | 100: // 4 | | | | | | | | | | 011: // 3 | | | | | | | | | | 010: // 2 | | | | | | | | | | 001: // 1 | | | | | | | | | | 000: // 0 | | | | | | | |

3.3 SCTL Register

The bit layout is as shown in Figure 13 on page 474.

Figure 13: SCTL Register (Base Address +$03)

bit	7	6	5	4	3	2	1
Read/Write	Reset & Disable	Control Reset	Diag Mode	Arbitration Enable	Parity Enable	Select Enable	Resele

- SPC reset control

```
- 1: Allow interrupt generation
- 0: Disable
```

- Reselection response control

```
- 1: Respond to Reselection Phase
- 0: Do not respond
```

- Selection response control

```
- 1: Respond to Selection Phase
- 0: Do not respond
```

- Parity check of SCSI bus data line

```
- 1: Enable parity check
- 0: Do not check
```

- Control execution of arbitration phase

```
- 1: Execute arbitration phase after selection/reselection phase
- 0: No execution of arbitration phase
```

- Set SPC to self-diagnosis mode

```
- 1: Self-diagnosis mode
- 0: Normal operation
```

- Reset SPC internal data transfer control circuit

```
- 1: Reset control circuit
- 0: Normal operation
```

- Reset SPC internal bus and control circuit

```
- 1: Reset
- 0: Normal operation
```

The meanings of each bit are as follows:

bit 7: Reset & Disable

- Corresponds to SPC full reset signal. Writing '1' performs a reset. When the hardware reset (when the RESET switch on the main body is pressed), this bit is also set to '1'. When SPC is in a state completely separated from the SCSI bus, it will not respond to external selections etc. After resetting, you must set this bit to '0' before using SPC.

bit 6: Control Reset

- Resets only the data transfer control circuit within SPC. Writing '1' performs reset and writing '0' returns to normal operation.

SCSI

... left as is. Even if you set this bit to '1', the connection to SCSI will not change. bit 5: Diag Mode SPC will enter self-diagnosis mode if you set this bit to '1'. In this mode, SPC will be completely disconnected from SCSI and instead operate as if it were responding to SDGC register accesses in SCSI bus status. bit 4: Arbitration Enable Decides whether the arbitration phase will be executed before the selection/reselection phase. If set to '1', the arbitration phase executes. If set to '0', the arbitration phase is skipped and the selection phase is executed. bit 3: Parity Enable Choose whether to use parity checking for data transfers on the SCSI bus. This setting is for checking if SPC is outputting data correctly. Parity generation for SPC data output is unconditional and does not depend on this bit setting. For X68000, it will be set to '0'. bit 2: Select Enable Decides whether to respond to target selection in the selection phase. If set to '1', it responds in the selection phase; if set to '0', it is ignored. This bit may be considered to decide whether to act as a target. Normally, the X68000 operates as an initiator, so this bit is set to '0'. bit 1: Reselect Enable Decides whether to respond in the reselection phase. If set to '1', it responds in the reselection phase, if set to '0', it is ignored. In systems using the disk reconnect/redisconnect feature over the SCSI phase transition, set this bit to '1'. In the X68000 SCSI interface system, initiators act mainly and this bit is set to '0' after startup. bit 0: Interrupt Enable SPC controls interrupt generation/disablement. If set to '1', interrupts are enabled. If set to '0', they are disabled. When set to '0', if the SCSI reset condition (RST signal going low) is detected, an interrupt occurs. Even if this bit is '0', interrupt causes are reflected in the INTS register.

3.4 SCMD Register

The bit arrangement is as shown in Figure 14.

■ Figure ..... 14 SCMD Register (Base Address + $05)

bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
Command Code	RST Out	Intercept Transfer	Transfer Modifier				

Commands to SPC:

- 111: Set ACK/REQ
- 110: Reset ACK/REQ
- 101: Transfer Pause
- 100: Transfer
- 011: Set ATN
- 010: Reset ATN
- 001: Select
- 000: Bus Release

Termination Mode:

- During Initiator:

```
- 1: Byte Count = 0 but Respond to REQ
- 0: Byte Count = 0 Stop Transmission
```

- During Target:

```
- 1: Parity Error Occurs Stop Transmission
- 0: Byte Count = 0 Continue Transmission if Requested
```

Otherwise, it becomes ”0”.

Program Transfer:

- 1: Transfer by CPU (DREQ not issued)
- 0: DMA

Data Buffer Retention During Manual Transmission:

- 1: Buffer contents retained
- 0: Not retained

SCSI Bus RST (Reset) Signal Control:

- 1: RST becomes '1' (Low level)
- 0: '0' (High) (Normal operation)

Each bit's meaning is as follows:

bit 7, 6, 5 : Command Code Indicates the command execution to SPC. Details of each command's operation will be explained later.

bit 4 : RST Out When '1' is written, the SCSI bus's RST signal becomes '1' and resets the SCSI bus. When the SCTL register is '1', this bit's operation is invalid.

bit 3: Intercept Transfer

When this bit is set to '1', a manual transfer (mode where the CPU controls REQ/ACK signals) is performed and the contents of the 8-byte FIFO buffer in the SPC internal memory are saved.

bit 2: Program Transfer

When set to '1', it operates in a mode that outputs DREQ (DMA Request) signals. For manual transfer, you should set this bit to '1'. As mentioned before, in the X68000, the DREQ signal is utilized in place of the DTACK signal. If you set this bit to '1' and are unable to access DREQ (such as when a bus error occurs), you must ensure that the DREQ signal is generated when performing hardware transfers.

bit 1: (Unused)

It is not used. Please set it to '0'.

bit 0: Termination Mode

When operating as an initiator, this means it operates as a target. When operating as an initiator and this bit is set to '0', the transfer counter terminates sending. The transfer count becomes effective. When '0', if the transfer counter's value is '0', an ack reply without REQ signal is received from the target. When the data direction is from the target to the initiator, the data read out from the target side is discarded and $00 is returned from the initiator to the target. This operation is called Padding.

When the transfer counter is set to '0', if it operates, an initial transfer also become padding transfer. In this case, write $00 to the TEMP register before issuing the Transfer command.

When operating as a target and this bit is set to '1', if a parity error is detected during transfer, the transfer will end. If set to '0', transfer will continue until a parity error is detected.

3.5 INTS Register

The bit arrangement is as shown in Figure 15 on page 478. To enable or disable permissions/prohibitions for conditions leading to an interrupt request, set bit 0 (SCTL register bit 0). Regarding this, the corresponding bit in the INTS register is set by the CPU. When writing to this register, writing '1' will clear the bit but writing '0' has no effect. The register's corresponding conditions are released.

INTS Register (Base Address + $09)

bit	7	6	5	4	3	2	1
READ/WRITE	Selected	Reselected	Disconnected	Command Complete	Service Required	Time Out	SPC Hard

- bit 7: Selected

```
- 1: Indicates that the SPC is connected to another initiator in the Selection phase. From
↪  the time of this connection until a Bus Release command is issued, or until the SCSI
↪  bus is reset, the SPC will continue operating as a target.
- 0: Normal operation
```

- bit 6: Reselected

```
- 1: Indicates that the SPC is reconnected to the controller in the Reselection phase. If
↪  a Disconnect interruption occurs or the SCSI bus is reset, the SPC will assume an
↪  initiator role.
- 0: Normal operation
```

- bit 5: Disconnected

```
- 1: Indicates that the SPC has been disconnected by the Bus Release command, or due to
↪  re-selection failure.
- 0: Normal operation
```

- bit 4: Command Complete

```
- 1: The execution of the Select command or the Transfer command has completed.
- 0: Normal operation
```

- bit 3: Service Required

```
- 1: Indicates that a SCSI interface phase is detected (when bit 7 of SCTL Register is set
↪  to “1”).
- 0: Normal operation
```

- bit 2: Time Out

```
- 1: Indicates that a Timeout error has occurred due to no response from the counterpart
↪  within a certain period after the execution of Selection or Reselection.
- 0: Normal operation
```

- bit 1: SPC Hard Error

```
- 1: Indicates an error displayed in the SERR register such as TC Parity Error, or Short
↪  Transfer Period.
- 0: Normal operation
```

- bit 0: Reset Condition

```
- 1: SCSI bus has been reset
- 0: Normal operation
```

The write factors are as follows:

bit 7: Selected

- Indicates that the SPC is connected to another initiator in the Selection phase. From this interruption until a Bus Release command is issued, or until the SCSI bus is reset, the SPC remains set and continues operating as it is.

bit 6: Reselected

- Indicates that the SPC is reconnected to the controller in the Reselection phase. If a Disconnect interruption occurs or the SCSI bus is reset, the SPC will assume an initiator role.

SCSI

The following is a description of the different bits and their functionalities:

bit 5: Disconnect When bit 7 of the PCTL register on the SPC side is set to 1, it indicates that the release phase of the bus is detected. This bit being 1 means that SPC has not performed SCSI bus control, so SCSI bus must be reset before using it.

bit 4: Command Complete Indicates that a Select command or Transfer command has been processed. SPC sets this bit to 1 when it operates as a target, and also in the case of a parity error that resulted in termination.

bit 3: Service Required During operation as an initiator, if there is a mismatch between the phase on the SCSI bus and the phase executed by the lower 3 bits of the PCTL register, data transfer will be interrupted (when the target phase is switched, for example). If the interface does not match, data transfer is interrupted and the CPU evaluates the situation to decide whether an appropriate recovery operation is needed. If transmission is interrupted and the interface does not match, data transfer on the SCSI bus is paused until conditions align. While the transmission is paused, the internal buffer within SPC is retained. The sequence of receiving data until it finishes will terminate the transfer operation. If this bit becomes 1, check the SST Status register to verify the transfer status of the SPC.

bit 2: Time Out Indicates that there was no response within a certain time period while the select/reselection phase was executed. This is called a selection timeout. If there is a selection timeout, SPC keeps SEL signal as 1. You can resolve it by writing 00 to the TEMP register. To release the bus after a selection timeout, follow the procedures similar to the above instructions.

bit 1: SPC Hard Error Indicates a detection of TC parity error or Short Transfer Period error (both are captured by the SERR register). When this error occurs, SPC does not stop operation.

bit 0: Reset Condition detected Indicates that a reset has been detected on the SCSI bus (RST signal has become 1). There are no rules for the transition of the RST signal, so reset recovery is required when this bit is set to 1.

(SSTS register bit 3 becomes '0') needs to be confirmed before proceeding. When the SCSI bus is reset, all actions in progress are terminated, and the bus becomes free. The internal state of the SPC is also reset, but the content of each Simplex register such as BDID, SCTL, SCMD, PCTL, transfer byte counter remains unchanged.

3.6 PSNS Register

The status of the SCSI bus can be read out. The bit assignment is as shown in Figure 16. This register allows reading the state of the SPC or the state related to the SCSI bus. The data read out is low level on the SCSI bus when '1' in the SASI interface supports chart.

[Figure] 16 PSNS Register (Base Address $0B)

| bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | | | REQ | ACK | ATN | SEL | BSY | MSG | C/D | I/O | | Read | | | | | | | | |

1: I/O signal is '1' (Low level) 0: I/O signal is '0' (High level)

1: C/D signal is '1' (Low level) 0: C/D signal is '0' (High level)

1: MSG signal is '1' (Low level) 0: MSG signal is '0' (High level)

1: BSY signal is '1' (Low level) 0: BSY signal is '0' (High level)

1: SEL signal is '1' (Low level) 0: SEL signal is '0' (High level)

1: ATN signal is '1' (Low level) 0: ATN signal is '0' (High level)

1: ACK signal is '1' (Low level) 0: ACK signal is '0' (High level)

1: REQ signal is '1' (Low level) 0: REQ signal is '0' (High level)

3.7 SDGC Register

The bit configuration is as shown in Diagram 17. bit 5 is a bit that selects whether or not to issue a data transfer request interrupt when a transfer is executed. When it's '1', the interrupt is issued.

Bits 5 and below are used for SPC self-diagnosis. When SPC is in self-diagnosis mode (when bit 5 of the SCTL register is '1'), the SPC SCSI bus interface signal is disconnected from the SCSI bus, and the value set in the SDGC register determines the state of the SCSI bus. This allows checking of SPC operations. In this way, the arbitration test in self-diagnosis mode is successfully completed.

Diagram 17: SDGC Register (Base Address + $0B)

bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
Diag REQ	Diag ACK	Xfer Enable	Diag BSY	Diag MSG	Diag C/D	Diag I/O	Diag

Explanation:

- In diagnosis mode, sets the status of the SCSI bus.
- 1: Issues an interrupt instead of Data Request during program transfer
- 0: Does not issue

3.8 SSTS Register

The bit configuration is as shown in Diagram 18 on page 482. This register is unrelated to SPC operations and can be read at any time. The meanings of each bit are as follows:

bit 7, 6: Connected Indicates the connection state with the SCSI bus. bit 7 represents the state as an initiator and bit 6 as a target.

bit 5: SPC Busy Indicates whether SPC is in a busy or execution waiting state.

bit 4: Transfer In Progress

Figure 18: SST Register (Page Register + $0D)

READ

bit 7	6	5	4	3	2	1	0
Connected	INIT	TARG	SPC Busy	Transfer in progress	SCSI Reset In	TC=0	DREG Full

Legend:

- Connected (bit 7)

```
- 10: Internal lineup and connection
- 01: Connected as target
- 00: Not connected
```

- INIT and TARG (bits 6, 5)

```
- 11: (Not Used)
- 10: Connected as initiator
- 01: Connected as target
- 00: Not connected
```

- SPC Busy (bit 4)

```
- 1: Holding SPC command
- 0: Not holding SPC command
```

- Transfer in progress (bit 3)

```
- 1: Transfer in progress
- 0: No transfer in progress
```

- SCSI Reset In (bit 2)

```
- 1: RST signal is '1' (Low Level)
- 0: RST signal is '0' (High Level)
```

- TC=0 (bit 1)

```
- 1: Transfer byte counter is zero
- 0: Transfer byte counter is not zero
```

- DREG Full (bit 0)

```
- 1: There is data in the DREG
- 0: Data in DREG is empty
```

Status

bit 7 | bit 6 | bit 5 | bit 4 | Operation State

bit 7	6	5	4	State of Operation
0	0	0	0	Not connected with SCSI, SPC is not holding command (Idle/Arbitration)
0	0	0	1	Not connected with SCSI, Select Command Holding (Pause/Arbitration)
0	0	1	0	Operating as target (No operation on SCSI)/ Manual transfer
0	1	1	0	Execute selection phase on SCSI
0	1	1	1	Operating as target (Hard Transfer Communication)
1	0	0	1	Operating as initiator (Pause/Manual transfer)
1	0	0	1	Operating as initiator (Not holding REQ signal, No transfer in progress)
1	0	1	0	Execute selection phase on SCSI
1	0	1	1	Operating as initiator (Hard Transfer Communication)

This is a technical document explaining the bit representation and status conditions for the SST registers used in SCSI systems.

Whether a hard transfer is in progress or a SCSI transfer phase is being requested.

bit 3: SCSI Reset In Indicates the status of the SCSI reset signal (RST).

bit 2: TC=0 Indicates that the value of the transfer byte counter (TCH, TCM, TCL registers) has become 0.

bit 1, 0: DREG Status Indicates the status of the FIFO buffer inside the SPC. The FIFO buffer inside the SPC is 8 bytes, and bit 0 is set if there is no data (empty) and bit 1 is set if it is full (8 bytes of data). If both are '0', it means it is neither full nor empty, implying it contains data between 1 to 7 bytes.

●

```
3.9 SERR Register

 The bit configuration is shown in Figure 19.

 ● Figure 19 SERR Register (Base Address + $0F)

 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
 |---|---|---|---|---|---|---|---|
 | Data Error | SCSI | SPC | Xfer Out | '0' | TC Parity Error | '0' | Short Transfer Period
 ↪  | '0' |

 - 1: REQ or ACK signals could not be exchanged within the SPC's timeout period.
   0: Normal

 - 1: Parity error occurred during decrementing transfer byte count.
   0: Normal

 - 1: Data Request (use when bit 5 of SDGC register = '1')
   0: Normal Operation

 - 11: During input operation, parity error detected in data received from SCSI.
   10: (Not used)
   01: During output operation, parity error detected in the data attempted to be output to
    ↪  the SCSI bus.
   00: No parity error detected.
```

If any of bits 3 or 1 in the error indicated by this register is ”1,” it becomes an SPC Hardware Error (INTS register bit 1), and an interrupt occurs. The meanings of each error are as follows.

bit 7, 6: Data Error Indicates that a parity error was detected on the SCSI. As shown in Figure 29, the combination of bits 6 and 7 indicates, but in summary, bit 6 sets to ”1” when an SPC parity error is detected on reading and to ”0” when detected on input from the host.

bit 3: TC Parity Error Indicates that a parity error was detected when the SPC was performing byte count decrement operation.

bit 1: Short Transfer Period Indicates that an REQ or ACK signal was input at a period significantly different from the SPC stipulated period, shown in Figure 20. This timing sees whether the clock frequency given to the SPC exceeds the input clock frequency. When actually implemented (when CZ-6BS1 is connected to an initial model), it is 5 MHz, so t_CLF = 200 ns.

■ Figure:

20 Limit on REQ/ACK Signal Period

t_cur: SPC Clock Period
(Example on X68000 (initial model) with CZ-6BS1, 200ns)

■ 3•10 PCTL Register

Bit assignment is as shown in Figure 21 on page 485. Bits 6 to 3 are not used, but please write '0'. The meaning of each bit is as follows.

Page 484

Figure 21 PCTL Register (Base Address + $11)

bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0
Busfree INT Enable	'0'	'0'	'0'	Transfer Phase	MSG	C/D	I/O

1: Generates an interrupt if Busfree phase is detected and Disconnected operation.* 0: Normal operation

- Select command issued, and Disconnected reset to '0' to prevent unnecessary interrupts.

bit 7: Busfree INT Enable

Select whether to generate a Disconnected interrupt on detection of Busfree phase. A setting of '1' will result in generation of Disconnected interrupt on detection. When issuing the Select command, reset Disconnected interrupt bit to '0' to prevent unnecessary interrupts.

bits 2, 1, 0: Transfer Phase

Designate the phase executed when connected to an SCSI bus as an initiator, or the phase executed in SCSI when connected as a target. When performing handover as an initiator, data transfer will not be performed if the currently specified phase in this register does not match the bus phase. Ensure matching to avoid issues.

When issuing the Select command, if bit 0 of this register is '0', it indicates Selection phase, and if '1', it indicates Reselection phase.

4 SPC Transfer Mode

SPC-specific transfer modes are outlined in the diagram on page 486. In manual transfers, the CPU controls everything, including the REQ-ACK handshake.

● Figure 22 Transfer Modes of the SPC

Transfer Mode	(sub)	Data Access	DREQ Signal	Register the CPU uses for transfer control	Remark
Manual transfer		TEMP register	Not output	PSNS register	
Hard transfer	Program transfer	DREG register	Not output	SSTS register (or interrupt)	Cannot
Hard transfer	DMA transfer	DREG register	Output		

You may think of it as something similar to the SASI port. In this mode, access to the data lines of the SCSI bus is done through the TEMP register. Hard transfer is a mode in which most of such troublesome control is performed by the SPC itself. In this mode, access to

the data lines of the SCSI bus is done with the DREG register, and an 8-byte FIFO buffer becomes effective.

Furthermore, the hard transfer mode can be classified into two modes, DMA transfer mode and program transfer mode, depending on whether the DREQ (DMA transfer request) signal is output or not. However, in the SCSI interface of the X 68000, the SPC's DREQ signal is used to create the DTACK signal, so if you select program transfer mode, access to the DREG register becomes impossible (everything results in a bus error).

5 SPC Commands

The commands written to the upper 3 bits of the SCMD register and their operations are as follows.

5.1 Bus Release Command

This is used when transitioning to the bus free phase while operating as a target. When transitioning from during a data transfer, first stop the data transfer with the Transfer Pause command, then perform this. This command, after the Select command is issued, when in the state of waiting for the bus free phase,

SCSI

It can also be used to cancel the Select command.

5.2 Select Command

This is the request command to start the Selection/Re-selection phase. When Arbitration Enable is set (bit 4 of the SCTL register is '1'), the Arbitration phase is automatically executed before the Selection/Re-selection phase. If it loses in the Arbitration phase, execution of this command ends.

If it wins in the Arbitration, or if it is not Arbitration Enable, the Selection/Re-selection phase is executed.

When the Select command fails (a Selection Time Out occurs), it sets the Time Out bit (bit 2 of the INTS register) to '1', and when the selection succeeds it generates an interrupt by setting Command Complete (bit 4 of the INTS register) to '1'.

The Select command requires the following settings before the command is issued.

PCTL register bit 0 Selects whether to execute the Selection phase or the Re-selection phase. When '0' the Selection phase is executed, and when '1' the Re-selection phase is executed.

Issuing the Set ATN command When you want to execute the Message Out phase after the selection, issue the Set ATN command prior to the Select command, to instruct the SPC to set the ATN signal to '1'.

TEMP register Set the data in which the bits corresponding to the values to be output to the data lines during the Selection/Re-selection phase (your own ID and the other party's ID) are set to '1'.

TCH/TCM register Set the time to wait for a response from the other party during the Selection/Re-selection phase (the time until the BSY signal becomes '1'). With this time T, taking the value indicated by TCH/TCM as X,

```
T = (X×256+15)×t_CLF×2
```

is expressed. Here, t_CLF is the period of the clock given to the SPC (200 in the X 68000

5.3 Set ATN Command

Sets the ATN line of the SCSI bus to '1'. Effective only from the time SPC becomes an initiator. When issued before the Select command, the ATN line will be '1' during the execution of the Select command. However, if Selected or Reselected occurs before phase assertion, the Set ATN command will be invalidated.

5.4 Reset ATN Command

Resets the ATN line output to the SCSI bus to '0'. However, if a parity error is detected during data transfer by SPC, the ATN line of the SCSI bus is automatically set to '1' and the Reset ATN command should not reset ATN until the Transfer command in progress completes. When performing manual transfer, the ATN signal should be reset before setting the ACK signal to '1'.

In the following cases, SPC automatically resets the ATN signal to '0':

- When a disconnection occurs
- When executing a hard reset mode with the message or interface and sending the final byte
- When BSY signal response becomes '0' after detecting selection time out and the selection time out detection becomes invalid
- When BSY signal response does not become '1' even if the selection time out period is infinitely long, writing '1' to bit 2 of the INTS register resets and returns to an invalid selection time out 상태

5.5 Transfer Commands

These are commands that indicate the start of data transfer (hardware transfer) for each phase like data in/out, status, command, message in/out, etc. Before executing these commands, the following settings need to be made:

- Set the number of bytes to be transferred in the transfer byte counter (TCH/TCM/TCL)
- Set the phase to be executed in the lower 3 bits of the PCTL register

When functioning as a target, the conditions for terminating the execution of the command are as follows:

- The transfer of the number of bytes set in the transfer byte counter has completed
- The Transfer Pause command is executed
- A data parity error was detected on the data line during SCMD input operations when the bit 0 of the SCMD register was set to '1'

When functioning as an initiator, the execution of the command will be terminated under the following conditions:

- Not in padding transfer mode and the transfer of the number of bytes set in the transfer byte counter has completed
- The target shifts to a phase other than the one specified by PCTL
- A disconnection interrupt occurred

At the start of transfer, if the phase specified by the PCTL register does not match the phase on the SCSI bus, a Service Required interrupt will occur. Additionally, when the initiator performs a hardware transfer, set the value of the transfer byte counter to 2 or more.

Page 489

5.6 Transfer Pause Command

This is a command that interrupts the hard transfer operation while operating as a target. It cannot be used while operating as an initiator. During output operations, issuing this command does not perform any write access to the DREG register.

5.7 Set ACK/REQ Command

This is used to set the SCSI bus ACK/REQ signal to '1' during manual transfer. When operating as an initiator, the ACK signal becomes '1', and when operating as a target, the REQ signal becomes '1'. At this time, set the phase to be executed in the lower 3 bits of the PCTL register.

5.8 Reset ACK/REQ Command

This is used to set the ACK/REQ signal on the SCSI bus to '0' during manual transfer. When operating as an initiator, the ACK signal becomes '0', and when operating as a target, the REQ signal becomes '0'. If necessary, you can issue the Set ATN command before this command to output the ATN signal. If the SPC has just sent a hard transfer as part of the message-in phase and the last byte is fetched, the ACK signal will remain '1' and transfer will be completed, so this command is required to restore the ACK signal to '0'.

6 Major SCSI Commands

When standardizing SCSI, commands that can be used with the SCSI interface were also organized, but from the perspective that this alone is still insufficient, the voice of manufacturers is strong, thus ANSI also ...

We are working on standardization. These commands are called CCS (Common Command Set).

It is impractical to explain all of CCS here, so let me limit the explanation to the commands used mainly by the Human68K SCSI drive, etc.

6-1 General Format of SCSI Commands

The general format of SCSI command is shown in Figure 23 on page 492. The SCSI commands include 6-byte commands similar to SASI, as well as 10-byte and 12-byte commands. Among these, the high 3-bit of the first byte in a 10-byte command is either '001', while in a 12-byte command it is '0011'. The formats of the 6-byte commands are similar to SASI, with no substantial changes in nomenclature.

Human68K SCSI drive, etc., use commands with group code 0 (group code '000'), and some group 1 commands like Read Capacity. Except for this, there are no other group 5 commands.

The control byte (the last byte of each command) includes the Link bit, which is a flag used to chain multiple command executions together. If you want to execute commands continuously, set this bit to '1'. If this is supported by the target, it returns to the INTERMEDIATE status phase after executing the command and then transitions to the message-in phase, and the command continues.

The Flag bit is valid only when the Link bit is set to '1'. If the Link bit is '0', this bit must be '0' as well. When this bit is '1', after the command completes normally, the target sends a LINKED COMMAND COMPLETE WITH FLAG message, and when it is '0', the target sends a LINKED COMMAND COMPLETE message. Generally, this flag is used to mark within a chain of commands to identify when a specific command execution finishes (depending on which message is returned, it is possible to identify whether the marked command was executed or not).

Figure...23 General Form of SCSI Command

6-Byte Command (Group 0)

Transfer Sequence	bit 7	6	5	4	3
0	Group Code	Command Code		Operation Code	
1	LUN (Logical Unit Number)	Logical Block Address (Upper)			
2	Logical Block Address				
3	Logical Block Address (Lower)				
4	Transfer Length				
5	V	Reserved	Flag	Link	Control B

10-Byte Command (Group 1) / 12-Byte Command (Group 5)

Transfer Sequence	10 byte length	12 byte length	bit 7	6
0			Group Code	Command Code
1			LUN (Logical Unit Number)	(Reserved)
2			Logical Block Address (Upper)	
3			Logical Block Address	
4	5		Logical Block Address (Lower)	
5	6		Reserved	(Reserved for future expansion)
5			//	
7	8		Transfer Length (Upper)	
8	9		Transfer Length (Upper)	
9	10		Transfer Length (Lower)	
9	11		V	Reserved

Rel Adr: The logical block address here is relative to the location accessed previously (*2-byte field)
V: Free usage per vendor

The text and tables explain the structure of SCSI commands, specifically the 6-byte and 10/12-byte command formats, and detail the bit allocation and significance of each field within these formats.

6.2 SCSI Command Codes

The table below (Figure 24) shows the main command codes used for the X68000.

Figure 24: Main SCSI Commands

Operation Code	Group Code	Command Code	Command Name	Notes
S 00	0	S 0	Test Unit Ready	Check if the unit is ready for use
S 01	0	S 1	Rezero Unit	Perform head movement of the cylinder
S 03	0	S 3	Request Sense	Acquire sensor data
S 04	0	S 4	Format Unit	Format media
S 08	0	S 8	Read	Read data
S 0A	0	S A	Write	Write data
S 12	0	S 12	Inquiry	Acquire target or unit property information
S 1A	0	S 1A	Mode Sense	Acquire media or unit parameters
S 25	1	S 25	Read Capacity	Acquire the number of unit blocks or block size
S 28	1	S 28	Read	Extended READ (includes block address transmissi
S 2A	1	S 2A	Write	Extended WRITE (same as above)
S 18	0	S 18	Copy	Copy between units logically / same-unit copy
S 39	0	S 39	Compare	Data comparison
S 3A	1	S 1A	Copy And Verify	Copy and verify

The commands listed from top to bottom, up to 9, are the minimum required commands that SCSI drivers must support. To connect a SCSI disk to the X68000, these commands must be supported at the very least.

Commands $28 and $2A in the bottom two rows are 6-byte commands. These are the extended READ/WRITE commands, which extend the block address and the number of blocks.

In the X68000, whether these extended READ/WRITE commands for reading/writing device buffers' starting blocks, or meters can be used is uncertain.

Last three commands, Copy, Compare, and Copy And Verify, are not frequently used. However, to explain this clearly, these operational command names are listed here.

6.3 Details of Main SCSI Commands

Each of the SCSI commands, $00, $01, $03, $04, $08, $0A are described in SAS1.

Since they are the same as what was explained earlier, they are omitted; here we will explain the commands $12 (INQUIRY), $1A (MODE SENSE), $25 (READ CAPACITY), $28 (extended READ), and $2A (extended WRITE).

6.3 INQUIRY Command (Operation Code $12)

The format of the INQUIRY command is shown in Figure 25.

● Figure 25 INQUIRY Command

Transfer Order	bit 7	6	5	4	3	2	1	bit 0	Remarks
0	'0'	'0'	'0'	'1'	'0'	'0'	'1'	'0'	Operation Code: $12
1	LUN (Logical Unit Number)	Reserved							
2	Reserved								
3	Reserved								
4	Allocation Length								Length in bytes of the buffer t
5	V	Reserved					Flag	Link	

This command reads out attribute information from the target, such as what kind of device the unit (device) connected under it is, and whether it is removable. The format of the data obtained by this command is shown in Figure 26.

The first byte indicates the type of device — whether the connected device is a direct-access (random-access) device such as an HDD, or a sequential-access device. In the X 68000, currently direct-access devices are supported, but in the future support for sequential-access devices such as CD-ROM and DAT will likely be provided.

The RMB bit indicates whether the device is removable (can be taken out). For devices that cannot be removed, such as an HD, the RMB bit becomes '0', and for removable devices such as magneto-optical disks the RMB bit becomes '1'.

The conformance standard is used to judge which standard the device conforms to. The lower 3 bits indicate conformance to ANSI standards, and the other bits indicate conformance to SCSI standards stipulated by ISO, ECMA, etc. Naturally, the ANSI standard document defines only the ANSI bits. It seems that general SCSI-compatible hard disks set the ISO and ECMA bits all to '0'.

Figure: 26 INQUIRY Data

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |----|----------------------------|---|---|---|---|---|---|---|---------------------------------| | Peripheral Device Type | Device Type | | 1 | RMB | Device-Type Qualifier | Lower bit is parameter removable (e.g., DIP switch state) | | 2 | ISO Version | ECMA Version | ANSI-Approved Version | | 3 | (Reserved) | | 4 | Additional Length | Additional data length | | 5 ~ n+4 | Vendor Unique Parameter Bytes | Additional data |

- Peripheral Device Type

Value	Description
$00	Direct Access Device (such as HDD)
$01	Sequential Access Device (such as MT)
$02	Printer Device
$03	Processor Device
$04	WORM (Write Once Read Many) Device
$05	CD-ROM Device
$06 ~ $7E	Reserved for future expansion
$7F	Communication Device
$80 ~ $FF	Vendor-specific usage

- ANSI-Approved Version

Value	Description
0	Complies with ANSI X3.131-1986 (most common SCSI standard)
1	Reserved for future expansion
2	Conforms to ANSI X3T9.2/86-109 (SCSI-2)
3 ~ 7	Reserved for future expansion

If the ANSI bit is '1', it means the device conforms to ANSI X3.131-1986 which is expected to be the most common standard. If the bit is '2', it indicates conformance to ANSI X3T9.2/86-109 (SCSI-2). If it predates X3.131-1986, it should return '0'.

6.3.2 MODE SENSE Command (Operation Code $1A)

The MODE SENSE command reports media and peripheral device parameters. The command format is illustrated in Figure 27 on page 496.

When the target receives this command, it sends data in the format shown in Figure 28 on page 498. Particularly important in this format is the WP (Write Protect) bit. When the WP bit is '1', it indicates that the media is write-protected.

● Figure 27 MODE SENSE Command

Transfer Order	bit 7	6	5	4	3	2	1	bit 0	Remarks
0	'0'	'0'	'0'	'1'	'1'	'0'	'1'	'0'	Operation Code: $1A
1	LUN (Logical Unit Number)	(Reserved)							
2	(Reserved) PC	(Reserved)	Page Code						Reserved in ANSI X3
3	(Reserved)								
4	Allocation Length								Length in bytes of th
5	V	(Reserved)					Flag	Link	

PC	Content
'00'	Current Value
'01'	Changeable Value

PC	Content
'10'	Default Value
'11'	Saved Value

Page Code	Content
0	Page descriptors are not transferred
1	Read/Write Error Recovery Parameters
2	Disconnect/Reconnect Parameters
3	Format Parameters
4	Drive Parameters
7	Verify Error Recovery Parameters
8	Caching Parameters
$21	Additional Error Recovery Parameters
$22	Reconnection Timing Parameters
$3F	All Parameters

● Figure 28 MODE SENSE Data

Transfer Order	bit 7 6 5 4 3 2 1 bit 0	Remarks
0	Sense Data Length	Sense Data Length (does not include itself)
1	Medium Type	Medium Type
2	WP (Reserved)	WP: Write Protect (writing prohibited when '1')
3	Block Descriptor Length	Block Descriptor Length (multiple of 8)
0	Density Code	Density Code
1	Number of Blocks (MSB)	
2	Number of Blocks	Number of Blocks
3	Number of Blocks (LSB)	Block Descriptor (there may be more than one)
4	(Reserved)	(Reserved for future extension)
5	Block Length (MSB)	
6	Block Length	Block Length
7	Block Length (LSB)	
0 ～ n	Vendor Unique Parameter Bytes	Freely usable per vendor (manufacturer)

Density Code

Code	Content
S00	Default (support only for single density)
S01	Standard density floppy disk
S02	High density floppy disk
S03~S7F	(Reserved for future use)
S80~SFF	Vendor (Manufacturer) specific, free to use

The media type considers parameters such as floppy disk and MT (Magnetic Tape), and it seems that HD is only described as S00. The contents of the media type are shown in Table 29.

In the Allocation Length in the command, the initiator specifies the number of bytes of MODE SENSE data it wants to receive. The target will not send more than the number of bytes specified here in MODE SENSE data.

Also, the data of Transfer Descriptor 2 in the command is defined by ANSI X3.131-1986, but there seems to be a subsequent standard job data called PC and Page Code. Unfortunately, since my hand is not technical, if you look at the manuals issued by the disk manufacturers, you might find it helpful.

Value—29

Media Type

Value	Media Type
S00	Default media (currently mounted medium type)
S01	Single-sided floppy disk (unspecified medium)
S02	Double-sided //

Media Types of Floppy Disk

Value	Size	Bit Density Bits/Radian	Track Density mm/(inch)	Side	Reference Standard
S05	8 inch	6 631	1.9 (48)	1	ANSI X3.73-1980
S06	//	6 631	//	2	EMCA 59
S09	//	13 262	//	1	None
S0A	//	13 262	//	2	ANSI X3.121-1984
S0D	5.25 inch	3 979	//	1	ANSI X3.82-1980
S12	//	7 958	//	2	ANSI X3.125-1985
S16	//	7 958	3.8 (96)	2	ANSI X3.126-1986
S1A	//	13 262	//	2	ISO DIS 8630-1985
S1E	3.5 inch	7 958	5.3 (135)	2	ANSI X3.137

Direct Access MT

Value	Width (mm)	Track Count	Density ftpi (ftpi)	Reference Standard
S40	6.3	12	394 (10000)	ANSI X3B5/85-151
S44	6.3	24	394 (10000)	//

For the MODE SENSE data, among the Vendor Unique Parameter Bytes, there is information such as error recovery count and retry count, deferred block swap processing method, cylinder number, and head count. PCs and pagers use some of these parameters, and it's possible that some of these might be changed with a MODE SELECT command (operation code $15). Since the actual content of these parameters is extremely complicated and not typically used on a daily basis, they will be omitted from the explanation. As a reference, the specifics on PC pagers and the MODE SELECT will be included in fig. 27, so refer to those if needed.

6•3 READ CAPACITY Command (Operation Code $25)

The format of this command is shown in fig. 30. This command reports the number of blocks and the block length to the drive. The format of the response data for this command is shown in fig. 31. In the X68000 SCSI drives, bytes 256, 512, and 1024 are supported, so the number of blocks and block length are specified at the start of the disk's SCSI parameter area.

■Fig...:30 READ CAPACITY Command

Bit order	7	6	5	4	3	2	1	0	Notes
0	'0'	'0'	'0'	'1'	'0'	'0'	'1'	'1'	Operation Code: $25
1	LUN (Logical Unit Number)	(Reserved)	Rel Adr						
2	Logical Block Address (MSB)								
3	Logical Block Address								
4	Logical Block Address								
5	Logical Block Address (LSB)								

Bit order	7	6	5	4	3	2	1	0	Notes
6	(Reserved)								
7	(Reserved)								
8	V (Reserved)	PMI							
9	V (Reserved)	Flag	Link						

PMI (Partial Medium Indicator) '0': If not performing subsequent block access, the block length information will be the unit's basic block information.

Note: If Logical Block Address is all '0's.

'1': When performing subsequent block access, Logical Block Address becomes the address of the maximum block that can actually be used, not including the specified Logical Block Address.

SCSI

● Figure 31 READ CAPACITY Data

Transfer Order	bit 7 6 5 4 3 2 1 bit 0	Remarks
0	Logical Block Address (MSB)	Logical Block Address
1	"	
2	"	
3	" (LSB)	
4	Block Length (MSB)	Block Length (in bytes)
5	"	
6	"	
7	" (LSB)	

6.3.4 Extended READ Command (Operation Code $28)

The command format is shown in Figure 32. This extends the READ command so that larger block numbers and transfer block counts can be specified.

The Rel Adr of transfer order 1 is a flag indicating that the value is a relative value from the last accessed block number; set it to '1' to make it a relative value.

● Figure 32 Extended READ Command

Transfer Order	bit 7 6 5 4 3 2 1 bit 0	Remarks
0	'0' '0' '1' '0' '1' '0' '0' '0'	Operation Code: $28
1	LUN (Logical Unit Number) (Reserved) Rel Adr	
2	Logical Block Address (MSB)	Block address to start reading
3	Logical Block Address	
4	Logical Block Address	
5	Logical Block Address (LSB)	
6	(Reserved)	(For future extension)
7	Transfer Length (MSB)	Number of blocks to read
8	Transfer Length (LSB)	
9	V (Reserved) Flag Link	

6.3.5 Extended WRITE Command (Operation Code: $2A)

The command format is as shown in Figure 33. Similar to the extended READ command, it specifies a large block number and the number of blocks to be written.

● Figure 33 Extended WRITE Command

Transmission Order	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2
0	0	1	0	1	0	1
1	LUN	(Reserved)	Rel Adr			
2	Logical Block Address (MSB)					
3	Logical Block Address	Address of the block to be written				
4	Logical Block Address					
5	Logical Block Address (LSB)					
6	(Reserved)	(Future expansion)				
7	Transfer Length (MSB)	Number of blocks to be written				
8	Transfer Length (LSB)					
9	V	(Reserved)	Flag Link			

● 7 Status Byte

In SASI, the status byte (data passed in the status phase) was always $00 unless otherwise specified, but in SCSI, it is used frequently, and the code splitting is defined. Figure 34 on page 501 shows the contents of the status byte. The status byte code specified in SCSI for bits 1 to 4 are as follows:

(Note: LUN typically stands for Logical Unit Number, MSB for Most Significant Byte, LSB for Least Significant Byte, Rel Adr for Relative Address, and V might indicate Valid.)

Fig. 34 Status Byte Format

bit 7 | 6 | 5 | 4 | 3 | 2 | 1 | bit 0 R | V1 | V2 | Status | V3

Future Expansion Free to use

Status Byte Code 0000: Good 0001: Check Condition 0010: Condition Met/Good 0100: Busy 1000: Intermediate/Good 1010: Intermediate/Condition Met/Good 1100: Reservation Conflict

Good The target has completed the command processing normally.

Check Condition An error that reflects in the sense data, such as an anomaly or abnormal condition, has occurred. When receiving this status, the initiator must obtain the sense data using the REQUEST SENSE command.

Condition Met This is returned when data is found by a search command. The logical block address indicates the sense data (response data when the REQUEST SENSE command is issued).

Busy The target is busy. The initiator may have to wait and then reissue the command.

Intermediate This is answered using the chaining function (setting the Link flag in the controller byte of the command) with a command processing completion message.

Reservation Conflict This is returned when the designated drive is reserved by another initiator and cannot be used until it is released (normally it is very rare for an initiator other than X68000 to become an initiator in a SCSI system for X68000, so you usually don't get this back).

When Check Condition is returned, it is necessary to always issue the REQUEST SENSE command immediately afterwards. Most SCSI devices will not accept further commands until the REQUEST SENSE data is received.

commands other than that will no longer be executed. Also, when the drive is reset, or when the power is turned ON/OFF, the response to the first command always becomes Check Condition.

8 Sense Data

The sense data returned in response to the REQUEST SENSE command, since the SASI-equivalent 4-byte data provides somewhat insufficient information, has been newly defined in SCSI as an extended sense data format. The format of the extended sense data is shown in Figure 35. When the lower 7 bits of the first byte are $70, it indicates extended sense data.

● Figure 35 Extended Sense Data Format

Transfer Order	bit 7 6 5 4 3 2 1 bit 0	Remarks
0	V '1' '1' '1' '0' '0' '0' '0'	Indicates that this is extended sense data
1	Segment Number	On abnormal termination of Copy, Compare, Copy & Verify, indicates the
2	FM EOM ILI (R) Sense Key	
3	Information Byte (Upper)	
4	"	Information Byte
5	"	
6	Information Byte (Lower)	
7	Additional Sense Data Length	
8 ～ n+7	Additional Sense Data	

V (Valid): Indicates that the contents of the Information Byte are valid when '1' FM (File Mark): For sequential-access devices, indicates that a file mark has been detected EOM (End of Medium): Indicates end of medium ILI (Incorrect Length Indicator): Indicates that a mismatch in data block length has been detected (R): Reserved for future extension

Sense Key and Details The lower 4 bits of the 2-byte transfer packet of the sense data are called the Sense Key. They indicate what kind of sense data it is. The Sense Key values and their details correspond with Table 36. Also, in Tables 37 and 38, errors indicated by READ and WRITE commands are shown with the corresponding Sense Keys for reference.

Table 36: Sense Key and Details | Sense Key | Name | Details | |-----------|----------------|---| | $0 | No Sense | No specific sense | | $1 | Recovered Error| The last command was successfully completed after recovery action | | $2 | Not Ready | The specified unit cannot be accessed | | $3 | Medium Error | A recoverable error in data written on the media | | $4 | Hardware Error | A hardware error occurred (controller, device malfunction, etc.) | | $5 | Illegal Request| An invalid value in the command parameter was detected | | $6 | Unit Attention | The device was reset or removed/replaced | | $7 | Data Protect | Attempted to read/write on a protected area | | $8 | Blank Check | A blank was encountered during the read check* | | $9 | Vendor Unique | Used by the vendor at their discretion | | $A | Copy Aborted | Copy, Compare, Copy & Verify commands were canceled due to device irregularities | | $B | Aborted Command| The target abnormally terminated the command execution | | $C | Equal | Search command found a match in the data | | $D | Volume Overflow| Data remained in the buffer, causing device block overrun | | $E | Miscompare | Data read from source and media do not match | | $F | (Reserved) | (Reserved for future expansion) |

*1: When using a sequential device *2: When the device is tape or another similar device

Table 37: Sense Key Values for READ Command | State | Sense Key in Sense Data | |-----------|----------| | Specified an unusable block address | Illegal Request (specified as block address) | | Target was reset or media was swapped | Unit Attention | | Recoverable medium read error | Medium Error | | Recoverable read error | Recovered Error | | Error from overlapping retry | Aborted Command |

●Figure 38: Sense Key Values for WRITE Command

Situation	Sense Key in Sense Data
Specified an invalid block address	Illegal Request (Invalid or abnormal logical block address specified)
Target was reset, or a power on reset occurred	Unit Attention
Error that can be recovered through overlap/ retry	Aborted Command

●9 Message Data

SASI messages played no more roles than being the last data sent during command processing, but in SCSI the meaning of messages has been standardized, and in SCSI major messages are coded with standard regulations. Figure 39 shows a list of message data specified in SCSI.

●Figure 39: Message Data

Code	Mandatory (M) Optional (O)	Name	I/O	Remarks
$00	M	Command Complete	I	Command Execution Completed
$01	O	Extended Message	I/O	Byte Count for Extended Message
$02	O	Save Data Pointer	I	Request to Save Current Data Poi
$03	O	Restore Pointers	I	Restore the Previously Saved Data
$04	O	Disconnect	I/O	Notification of Bus Separation by
$05	O	Initiator Detected Error	O	Initiator Detected Error
$06	O	Abort	I/O	Clear the Operation in Progress
$07	O	Message Reject	I/O	Message Received is Not Supporte
$08	O	No Operation	O	No Operation Message is Reserve
$09	O	Message Parity Error	O	Parity Error Detected in Message
$0A	O	Linked Command Complete	I	Successfully Completed Linked C
$0B	O	Linked Command Complete (with Flag)	I	Flag 'I'
$0C	O	Bus Device Reset	I/O	Clear All In-progress Operations a
$0D-$7F	O	(Reserved Codes)		(Reserved for Future Use)
$80-$FF	O	Identify	I/O	Set Input/Output roles between In

Page 504

9.1 IDENTIFY Message

The IDENTIFY message is divided by bits as shown in Figure 40. After the Selection phase and during the Message Out phase, it is used to determine whether the target will perform disconnect processing or not.

...40 IDENTIFY Message

| bit 7 | 6 | 5 | 4 | 3 | 2 | 1 | bit 0 | | '1' | D | Reserved | LUN |

Disconnect Privilege
1: Target may perform disconnect processing
0: Does not perform

Logical Unit Number

9.2 EXTENDED Message

Code S01 of the EXTENDED MESSAGE indicates that it is an extended message spanning multiple bytes. The format of the extended message is shown in Figure 41.

...41 Format of Extended Message

Transfer Order	Value	Remarks
0	S01	Indicates it is an extended message

Transfer Order	Value	Remarks
1	Extended message length	
2	Extended message code	
3~N+1	Extended message arguments	

Following the S01, the data indicates the byte count after the extended message code in the transfer order 2. The extended message code distinguishes the type of extended message, and the following bytes are arguments associated with the extended message. The extended message codes are listed on page 506, as shown in Figure 42.

[Fig].....42 Extended Message Codes

Code	Name	I/O	Remarks
S00	Modify Data Pointer	I	Increases the value of the current data pointer
S01	Synchronous Data Transfer Request	I/O	Sets parameters for synchronous data transfer
S02	Extended Identify	I/O	Extends the LUN of the Identify message
S03 - S7F	(Reserved)	-	(Reserved for future use)
S80 - SFF	(Vendor Unique)	-	Defined freely by each device

9·2·1 MODIFY DATA POINTER Message The format of the message is as shown in [Fig] 43. It is a message to notify the initiator to increment the value of the current data pointer for the target. The increment is a compensation value shown in the appended section by the initiator.

The pointer can specify three types: command pointer, data pointer, and status pointer. The command pointer specifies the beginning of the command, data pointer and status pointer indicate the storage position of data and status.

[Fig].....43 MODIFY DATA POINTER Message

Transfer Order	Data	Remarks
0	S01	Indicates it is an extended message
1	S05	Extended message length
2	S00	Modify Data Pointer message
3	Most significant byte of data pointer	
4	Move of Data Pointer	
5	2 consecutive bytes' worth of count	
6	Least significant byte of data pointer	

9·2·2 SYNCHRONOUS DATA TRANSFER (Synchronous Data Transfer Request) Message

In SCSI, in addition to the REQ-ACK handshake procedure like SASI transfer, synchronous data transfer that pre-sends and transmits data by continuously issuing REQ signals without waiting for an ACK has been defined (this functionality is optional). This greatly improves the data transfer speed.

It is possible. Unfortunately, the SCSI controller used in X 68000, MB89352, does not support synchronous data transfer, so this message cannot be used. The message that designates synchronous data transfer mode when transferring data is the synchronous data transfer request message. The message format is shown in Figure 44. By designating the number of transfers during transmission, this message specifies the number of bytes that

can be sent in advance for transfer. For instance, if this value is four, the data transfer (4 bytes) can be carried out four times even if ACK is not returned.

●Fig. ... 44 Synchronous Data Transfer Request Message

Transfer Order	Data	Notes
0	$01	Indicates that it is an extended message
1	$03	Message Length = 3 bytes
2	$02	Extended Identify Message
3	x	Logical Unit Number

9. 2. 3 EXTENDED IDENTIFY Message

Usually, the logical unit numbers that can be designated by command range from 0 to 7, but this message extends it to be able to handle up to 2048 logical units by combining three bits of the command with the 8-bit sub-logical unit number assigned by this message. This extends the range so that up to 2048 logical units can be designated.

●Fig. ... 45 EXTENDED IDENTIFY Message

Transfer Order	Data	Notes
0	$01	Indicates that it is an extended message
1	$03	Extended Message Length
2	$01	Synchronous Data Transfer Request Message
3	x	Transfer Period (4 x {n} ns)
4	x	REQ/ACK Offset

Human68K SCSI drivers do not use logical unit numbers, so this message is never used.

#10 Sample Program

We have created a sample program that reads a specified block from a SCSI disk. In the first argument, specify the block number to access, and in the second argument, specify the SCSI disk ID number to access. If fewer arguments are specified, 1 will be used by default. In this sample, the block size is 512 bytes, and the SCSI interface assumes CZ-6BS1. If the block size is other than 512 bytes, adjust the value of BUFSIZE appropriately. Likewise, for SCSI type devices, adjust the value of spc accordingly.

- List * SCSI Disk Specified Block Readout

/*

- SCSI Hard Disk Access Test
-
- Since 'volatile' is not supported in XC,
- please insert the following line and disable 'volatile'
- #define volatile */

#include <doslib.h>

struct DMAREG {

```
    unsigned char csr;
    unsigned char cer;
    unsigned short spare1;
    unsigned char dcr;
    unsigned char ocr;
    unsigned char scr;
    unsigned char ccr;
```

```
    unsigned short spare2;
    unsigned short mtc;
    unsigned char *mar;
    unsigned long spare3;
```

};

unsigned char *dar; unsigned short spare4; unsigned short btc; unsigned char *bar; unsigned long spare5; unsigned char spare6; unsigned char niv; unsigned char spare7; unsigned char eiv; unsigned char spare8; unsigned char mfc; unsigned short spare9; unsigned char spare10; unsigned char cpr; unsigned short spare11; unsigned char spare12; unsigned char dfc; unsigned long spare13; unsigned short spare14; unsigned char spare15; unsigned char bfc; unsigned long spare16; unsigned char spare17; unsigned char gcr; } ; volatile struct DMAREG *dma;

struct SPCREG {

```
    unsigned char DUMMY0;
    unsigned char bdid;
    unsigned char DUMMY1;
    unsigned char sctl;
    unsigned char DUMMY2;
    unsigned char scmd;
    unsigned short DUMMY4;
    unsigned char DUMMY3;
    unsigned char ints;
    unsigned char DUMMY5;
    unsigned char psns;
    unsigned char DUMMY6;
    unsigned char ssts;
    unsigned char DUMMY7;
    unsigned char serr;
```

} ;

```
    unsigned char DUMMY8;
    unsigned char pct1;
    unsigned char DUMMY9;
    unsigned char mbc;
    unsigned char DUMMY10;
    unsigned char dreg;
    unsigned char DUMMY11;
    unsigned char temp;
    unsigned char DUMMY12;
    unsigned char tch;
    unsigned char DUMMY13;
    unsigned char tcm;
    unsigned char DUMMY14;
    unsigned char tcl;
```

};

volatile struct SPCREG *spc;

#define BUFSIZE 0x200 unsigned char diskbuf[BUFSIZE];

#define PSNS_REQ 0x80 #define PSNS_ACK 0x40 #define PSNS_BUSFREE 0x00 #define PSNS_STATUS 0x0b #define PSNS_MESSAGE 0x0f #define PSNS_COMMAND 0x0a

#define PCTL_DATA_IN 0x1 #define PCTL_COMMAND 0x2 #define PCTL_STATUS 0x3 #define PCTL_MESSAGE 0x7

#define SCMD_SELECT 0x20 #define SCMD_SET_ACK 0xe4 #define SCMD_RESET_ACK 0xc4 #define SCMD_TRANSFER 0x80

#define INTS_DISCONNECT 0x20 #define INTS_COMPLETE 0x10

#define SSTS_DREG_EMPTY 0x01

void main(); void scsi_busfree();

void scsi_ints_wait(); void scsi_phase_wait(); void scsi_select(); void scsi_send_command(); void scsi_send_a_byte(); void scsi_data_transfer(); void scsi_buffer_wait(); unsigned int scsi_get_status(); unsigned int scsi_get_message(); unsigned int scsi_get_a_byte(); void dma_setup(); void dma_start(); void dma_stop(); void wait_complete(); void clear_flag();

void main(argc, argv)

```
    int argc;
    char *argv[];
```

{

```
    unsigned int i, j, id, blk_no, blk_h, blk_m, blk_l;
    unsigned char c;
    if (argc >= 2)
        blk_no = atoi(argv[1]);
    else
        blk_no = 0;
    if (argc >= 3)
        id = atoi(argv[2]);
    else
        id = 0;
    SUPER(0);
    spc = (struct SPCREG *)0xea0000;
    dma = (struct DMAREG *)0xe84040;
    spc->bdid = 0x7;
    spc->sctl = 0x10;
    blk_l = blk_no & 0xff;
    blk_m = (blk_no >> 8) & 0xff;
    blk_h = (blk_no >> 16) & 0xff;
    printf("Block# = %d(%06X)[%02X:%02X:%02X] Drive = %d\n",
        blk_no, blk_no, blk_h, blk_m, blk_l, id);
    printf("Bus Free\n");
    scsi_busfree();
    printf("Select\n");
    scsi_select(id);
    printf("Command\n");
```

}

The comments represent typical function operations in SCSI and DMA communication in a C program. It initializes some variables, processes command-line arguments, sets up and manipulates the SCSI (Small Computer System Interface) and DMA (Direct Memory Access) registers, prints status messages, and performs necessary operations like bus free and selection.

scsi_send_command(8,blk_h,blk_m,blk_l,1,0); printf(”Data In\n”); scsi_data_transfer(); printf(”Status = %02X\n”,scsi_get_status()); printf(”Message= %02X\n”,scsi_get_message()); for (i=0; i<BUFSIZE; i+=0x10) {

```
    for (j=0; j<0x10; j++)
        printf("%02X ",diskbuf[i+j]);
    for (j=0; j<0x10; j++) {
        c = diskbuf[i+j];
        if ((c < 0x20) || (c >= 0xe0) || ((c >= 0x80) && (c < 0xa0)))
            printf(".");
        else printf("%c",diskbuf[i+j]);
    }
    printf("\n");
```

}}

void scsi_busfree() {

```
    scsi_phase_wait(PSNS_BUSFREE);
    if (spc->ints & INTS_DISCONNECT)
        spc->ints = INTS_DISCONNECT;
```

}

void scsi_ints_wait(dat) unsigned int dat; {

```
    while(!(spc->ints & dat))
        ;
    spc->ints = dat;
```

}

void scsi_phase_wait(phase) unsigned char phase; {

```
    while(spc->psns != phase)
        ;
```

}

void scsi_select(id) unsigned int id; { spc->temp = (1 « id) | (spc->bdid); }

sc->tch = 0; spc->tcm = 0; spc->tcl = 3; spc->pctl = 0; spc->scmd = SCMD_SELECT; scsi_ints_wait(INTS_COMPLETE); }

void scsi_send_command(p1, p2, p3, p4, p5, p6) unsigned int p1, p2, p3, p4, p5, p6; {

```
    unsigned char param[6];
    param[0] = p1;
    param[1] = p2;
    param[2] = p3;
    param[3] = p4;
    param[4] = p5;
    param[5] = p6;
    clear_flag();
    dma_setup(0, param, &(spc->dreg), 6);
    spc->tch = 0;
    spc->tcm = 0;
    spc->tcl = 6;
    spc->pctl = PCTL_COMMAND;
    spc->scmd = SCMD_TRANSFER;
    scsi_phase_wait(PSNS_COMMAND | PSNS_REQ);
    dma_start();
    wait_complete();
    scsi_ints_wait(INTS_COMPLETE);
```

}

void scsi_data_transfer() {

```
    unsigned int    i;
    clear_flag();
    dma_setup(1, diskbuf, &(spc->dreg), BUFSIZE);
    spc->tch = (BUFSIZE >> 16) & 0xff;
    spc->tcm = (BUFSIZE >> 8) & 0xff;
    spc->tcl = BUFSIZE & 0xff;
    spc->pctl = PCTL_DATA_IN;
    spc->scmd = SCMD_TRANSFER;
    scsi_buffer_wait();
    dma_start();
```

}

This code snippet includes three functions that likely pertain to handling SCSI commands for a device: `scsi_send_command` for sending commands, `scsi_data_transfer` for transferring data, and some initialization and waiting functions for setting up and managing SCSI operations.

wait_complete(); scsi_ints_wait(INTS_COMPLETE); }

void scsi_buffer_wait() {

```
    while(spc->ssts & SSTS_DREG_EMPTY)
        ;
```

}

unsigned int scsi_get_status() {

```
    spc->pctl = PCTL_STATUS;
    scsi_phase_wait(PSNS_STATUS | PSNS_REQ);
    return(scsci_get_a_byte());
```

}

unsigned int scsi_get_message() {

```
    spc->pctl = PCTL_MESSAGE;
    scsi_phase_wait(PSNS_MESSAGE | PSNS_REQ);
    return(scsi_get_a_byte());
```

}

unsigned int scsi_get_a_byte() {

```
    unsigned int dat;
    while(!(spc->psns & PSNS_REQ))
       ;
    dat = spc->temp;
    spc->scmd = SCMD_SET_ACK;
    while(spc->psns & PSNS_REQ)
       ;
    spc->scmd = SCMD_RESET_ACK;
    while(spc->psns & PSNS_ACK)
       ;
    return(dat);
```

}

void dma_setup(dir,ma,da,len)

```
    unsigned int dir,len;
    unsigned char *ma,*da;
```

{

This particular snippet seems to be part of a SCSI (Small Computer System Interface) controller code.

dma->dcr = 0x80; dma->ocr = 0x31 | ((dir & 0x1) « 7); dma->scr = 0x04; dma->ccr = 0x00; dma->cpr = 0x08; dma->mfc = 0x05; dma->dfc = 0x05;

dma->mtc = len; dma->mar = ma; dma->dar = da; }

void dma_start() { dma->ccr |= 0x80; }

void wait_complete() {

```
    while(!(dma->csr & 0x90))
    ;
```

}

void clear_flag() { dma->csr = 0xff; }

(Blank page in the original.)

System Port

System ports gather signals for peripheral control such as display, keyboard, and power-off control. Here, we will explain the contents of each system port and how to operate them.

1 System Port Address Mapping

System ports consist of 6 registers to perform settings like display contrast and power ON/OFF control. Their address mappings are shown in Figure 1 on page 518.

1.1 System Port #1

This adjusts the computer screen contrast. The brightness degree is determined by the lower 4 bits, with $F being the brightest and $0 being the darkest. Normally used for Human 68K, it automatically darkens the screen before shutting down the power to avoid sudden drops.

● Figure 1 System Ports

Register	Address	Function (bit 7 6 5 4 3 2 1 bit 0)	Remarks
1	$E8E001	CONTRAST	Computer screen contrast
2	$E8E003	TV CTRL / 3D-L / 3D-R	Display/3D Scope control
3	$E8E005	Color Image Unit control	
4	$E8E007	KEY CTRL / NMI RESET / HRL	Keyboard/NMI
5	$E8E00D	SRAM Write Enable Control	SRAM write control
6	$E8E00F	Power OFF Control	Main unit power OFF control

1.2 System Port #2

The bit arrangement is shown in Figure 2. The lower 2 bits are used for control of the optional 3D Scope. bit 0 corresponds to the right-eye shutter and bit 1 to the left-eye shutter; when each is '1', the shutter OPENs and the screen becomes visible.

bit 3, when writing, acts as the display control signal, and when reading, acts as the ON/OFF status of the display's power. For details on this bit, refer to the keyboard explanation page.

● Figure 2 System Port #2 ($E8E003)

| bit 7 | 6 | 5 | 4 | 3 | 2 | 1 | bit 0 | | | | | | TV CTRL | | 3D-L | 3D-R |

3D-R: 1: 3D Scope right shutter OPEN 0: " CLOSE

3D-L: 1: 3D Scope left shutter OPEN 0: " CLOSE

TV CTRL: On WRITE 1: Set display control signal to '1' 0: Display control signal becomes the control signal from the keyboard (normal operation)

On READ 1: Display power is OFF 0: " ON

●3 System Port #3

System Port #3 is used for the control of an optional color image unit. When data is written here, it will be output as is to IMAGE IN pins 17 to 21 (17 corresponds to bit 4, and 21 corresponds to bit 0).

●4 System Port #4

The bit layout is as shown in Figure 3. bit 1 is used when switching the dot clock, but it is generally kept at '0'. bit 2: When NMI occurs, when the processing of the NMI is completed, writing '1' to this bit will not cause the next NMI until this bit is written to '1'. bit 3 checks whether the CPU control and keyboard connector of the keyboard are connected. For details on this bit, refer to the description in the keyboard section.

●Figure 3 System Port #4 ($E8E007)

WRITE Operation 1: Key data transmission allowed 0: Not allowed

READ Operation 1: Key jack (keyboard connector) is connected 0: Not connected

bit 0: HRL bit 1: NMI RESET bit 2: --- bit 3: KEY CTRL bit 4: --- bit 5: --- bit 6: --- bit 7: ---

Dot clock switch (normally fixed at '0') 1: Reset NMI 0: No action

Each ”-” (long dash) denotes no specific function or bit usage description.

☒ 5 SYSTEM PORT #5

This port controls the permission/prohibition of write access to SRAM. Since SRAM contains memory areas such as system information, startup devices, and keyboard character display information, it is necessary to ensure that these do not get easily rewritten by programming mistakes. Therefore, this port is designed to protect write access to SRAM. By writing $31 to this port, writing to SRAM is permitted. Writing other data will prohibit writing to SRAM.

☒ 6 SYSTEM PORT #6

This performs the power OFF operation of the unit. When the main power switch is turned OFF and the value $200, $0F sequentially written to this port, the power of the unit can be turned OFF. To prevent unnecessary power interrupts, this port will not function unless this sequence is performed. When the main power switch is turned OFF, an interrupt from MFP's GPIP 2 occurs, so be aware that the unit's power will immediately turn off during this interrupt processing. When experimenting, if you do not disable the GPIP 2 interrupt, the moment the power switch is turned off, the power OFF processing of Human 68K will start, so please be careful.

In this text, I am wondering if I may have taken on a challenging task (You might also say that it is better to have a bad explanation than none at all).

I first touched the hardware of the X68000 in 1987, about a year after the X68000 was released. At that time, I was absolutely sure that there would be numerous guides, but even if I went to the bookstore, I couldn't find any at all. Since there are a considerable number of X68000 users, I thought if I waited patiently, surely new guides would appear, but they never came out. At that time, someone suggested to me, ”Why don't you write a guide on the hardware analysis of the X68000?” I decided to give it a try. Caught up in this, I dug out a kit that had been stored away in the components for the forgotten wireless hobby, put together a TK-80, and examined the ICs one by one... Before I knew it, I had become passionate about publishing a universal manual, and it almost felt like my fate had decided ”I have to do it myself or it won't get done.”

I decided to shed my sense of discomfort about not having LSI manuals especially because until now, I had mostly written soft-centric things during the 8-bit computer era. Now, the bulk of the software I was doing was introduction to LSIs. While this may seem like the best place to explain hardware analysis, covering just CRTs and such and making custom graphics with the memory maps for each part is beyond the scope of what can

be explained here. So I'll keep my explanations brief, and for further details, refer to the respective LSI manuals. I apologize for incomplete explanations.

These kinds of things are emphasized constantly in university research labs or maker's research and development departments, so they may seem out of place. In reality, those in jobs that require proficiency might obtain the LSI manuals, have the office copy printed, or purchase them separately (each manual can cost about 3,000 yen to 10,000 yen). It's still rare for them to be completely unattainable, surprisingly. This time around, the exhibition is ongoing... and I've actually navigated through these issues, brought some sample manuals to the exhibition booth, and have taken the ultimate step of swapping them over.

Going forward, even if you don't understand every single part at first glance, it's for better or worse about making the easiest versions of what are often LSI manuals, and making them easily accessible to many people. May you enjoy reading for the first time and continue for many iterations.

Afterword

I didn't understand what was going on, and often ended up having no clue about what I was doing while creating programs and performing operational checks.

Repeatedly reading the contents of the LSI manual didn't help much; honestly, I didn't understand the manual at all. But if I gave up on understanding it, I knew I would face difficulties writing the programs I was supposed to. Hence, I had no choice but to endure the process of reading it multiple times. However, despite my struggles with the manual, I cannot explain how I managed to muddle through it.

Until now, I believed that the workload of 86 CPUs was quite manageable and treated it lightly. Even with typical 86 PC representatives like the IBM PC or PC-9801, most of the built-in I/O devices didn't rely solely on the CPU, as they were mostly DMA-controlled. With 64K boundary constraints, data transfer was difficult, but CPUs with slow DMA and bank switching were equipped with 16 color video RAMs. But, just like 8-bit PCs, the sound generators included 1/0 ports and waveforms exclusively driven by the CPU.

However, the X68000, which features CRTC, Sprite Controller, Display Controller, DMA, SCC, SPC, OPM, ADPCM... and more LSI, is notably high-performance and vastly superior to ordinary 86 PCs.

For example, the RS-232C commonly adopted the 8530 SCC widely used in non-standard implementations. Some might know several other popular LSIs in this series. The ADPCM chip received significant acclaim for its sample-playback sound. I also learned from other sources that the compression algorithm of ADPCM is beneficial for commercial applications.

Moreover, due to the advanced use of FM sound synthesis within the LSI, examining technical manuals of FM synthesis was equally valuable. Assembling extensive data by combining copied materials and information with FREE/OPENMAN manuals was a daunting task—but crucial for this project. (Recently, to expedite the launch of publications from PC Engine-related matters, I even stayed overnight in a mountain of materials at the office).

In these situations, the diverse functions of these LSIs, along with the complexities within the LSI manuals, created obstacles. While reading through the manuals, I occasionally came across incomprehensible terms like ”additive synthesis” or ”68881 coprocessor,” leaving me baffled, despite finding occasional implementation examples of instructions.

Additionally, standard Japanese technical terms in the second volume of the manual might include words like “SUBSTRUCT (subtraction)” and others written unclearly

(perhaps summarized from multiple add/sub operations into subtraction?). There were times I wondered whether the assembly manual was genuinely correct, albeit written by the LSI manufacturer.

Afterword

I ended up reading the English manuals thoroughly. Probably because it progressed more smoothly when I added English materials...).

About the X68000 main unit, using previously published commentary as a reference, I meticulously checked the operation and found strange behavior here and there. Since early machines were being sold, I wanted to get a grasp of whether there were similar issues, so I created a program for checking, gathered simple applications to investigate the behavior, and a lot of effort was put in.

Since I was also busy with legal and accounting work towards the deadline, I just read, decoded, and attacked the keyboard with a calculator. My daughter, who was born in June last year, was also crying at my feet at times, making for an awkward situation! After two months of delays, I truly apologize for the prolonged delay.

This turned out to be somewhat detailed, honest work with proper verifications carried out, so it might come across as some parts being quite bad. It's not only due to my lack of technical skills, but I think the X68000 has deeply intricate mechanisms contributing.

For graphical aspects or parts where it feels like I plainly explained things in the manual, those represent sections where I felt particularly strong attachment.

Concerning the utility switch, I tried to implement various areas like avoiding dumb decisions like turning off the electric power with the utility switch, figuring out how to reposition the keyboard and adjust the settings for screen contrast to 16 levels. Looking at other brands, it's clear that business computers emphasize usability over catalog specifications. There is no need for anything extra to do the job, not necessarily inferior just because of no FDD auto-eject feature. Actually, the FDD industry only spares devices with high specifications, cost down and specifications exclude auto-eject mechanisms, which are unnecessary high features. It was perfectly reasonable that normal consumer devices don’t have certain hardware features as this would drive up costs. Therefore, it's decided that using devices like this will let users handle them without trouble. With such economic cutting, let alone do-it-yourself considerations, this decision errors on the side of leading business computer designs.

I believe the X68000 probably represents a machine with a presence even among those portions perceived as ordinary by other computers. While it has deeply ingrained unique capabilities, in the personal computer world, this level of detail cannot be considered basic.

Using the X68000 skillfully, we expected it opened up the world of personal computing, but without finishing the book or being solely helped by books, victory can’t be seized.

February 11, 1992 (Tuesday) Teaming up with CZ-600DE during the Winter Olympics Yoshinori Kariya

(Page 523)

● References

X 68000 Technical Data Book ..... ASCII X 68000 Introduction to Best Programming ..... Gijutsu-Hyoron Sha Semiconductor Integrated Circuits Communications LSI ..... Oki Electric Industry Microcomputer Technical Material Z8530SCC .....

Sharp SCSI Board CZ-6BS1 Instruction Manual ..... Sharp 16/32-bit Microprocessor TLCS-68000 Peripheral Devices ..... Toshiba 16-bit Microprocessor TLCS-68000 Microprocessor Edition ..... Toshiba YM2151 Catalog ..... Nippon Gakki Seizo (Yamaha) YM2151 Application Catalog ..... Nippon Gakki Seizo (Yamaha) YM2608 Application Manual ..... Nippon Gakki Seizo (Yamaha) µPD72065/72066 CMOS FDC User Manual ..... NEC DS300B-100/DS500B-100 Disk Controller Functional Specification ..... NEC Multi-chip ..... NEC SEMICONDUCTOR DATA BOOK 8/16-bit Microcomputers ..... Hitachi SCSI Protocol Controller MB89352A User Manual ..... Fujitsu Intelligent Disk Controller OEM Manual ..... Fujitsu RP5C15 User's Manual ..... Ricoh MOS Microprocessor and Peripherals ..... AMD Latest SCSI Manual ..... CQ Publishing M68000 Microprocessor User's Manual ..... CQ Publishing MC68901 MULTI-FUNCTION PERIPHERAL (Advanced Information) ... MOTOROLA MC68881 User's Manual ..... MOTOROLA ANSI X3.131-1986 ..... American National Standards Institute X 68000 Technical Material ..... Sharp

INDEX - Alphabetical Order

INDEX

INDEX (Japanese syllabary order)

Inside X68000

Published Date: First edition, first printing: April 23, 1992 First edition, fourth printing: December 12, 1992

Author: Shigemasa Kuwano Supervisor: Masaaki Sugita

Publisher: Softbank Corporation, Publishing Division Address: 108 Tokyo Minato-ku Takanawa 2-19-13 NS Takanawa Building Sales Department: 03 (5488) 1360 Editing Department: 03 (5488) 1326

Printing: Kita Noh Printing Co., Ltd.

(Blank page in the original.)

INSIDE X68000

This document is a technical data book that documents the operations of the CPU and surrounding LSI devices built into the X68000 main unit meticulously verified by the author alongside widely available existing technical resources.

In its compilation, it is not limited to screen control-related matters but also covers aspects less often or almost never documented in existing resources, such as DMA, numeric processing unit, FM sound source, ADPCM, SASI, SCSI, etc.

Furthermore, the book includes sample programs using gcc (can also be compiled with XC), allowing the reader practical application, making it a highly actionable content.